MATERIAL DE ESTUDO

SEGURANÇA DA INFORMAÇÃO

Guia de Estudo — Edição Ilustrada



Dos fundamentos à engenharia: ameaças, criptografia, identidade, redes, desenvolvimento seguro, operação e resposta a incidentes — com termos técnicos, padrões e estratégias de defesa.

20 MÓDULOS | 750+ TERMOS | OWASP · NIST · ISO | DEFESA EM CAMADAS

EDIÇÃO DE ESTUDO · VERSÃO NOV/2026

ÍNDICE

SUMÁRIO

Do vocabulário e das ameaças à criptografia, à identidade, às redes e ao desenvolvimento seguro — e daí à operação, à resposta a incidentes e a um projeto completo. Cada módulo funciona sozinho.

—	**Como usar este guia** Mapa dos 20 módulos, legenda dos quadros, trilhas de estudo e pré-requisitos	03
01	**Fundamentos da segurança** Tríade CIA, AAA, ameaça × vulnerabilidade × risco, princípios de projeto e defesa em profundidade	05
02	**Ameaças e ataques** Atores, malware, ransomware, engenharia social, ataques a credenciais, DoS e MITRE ATT&CK	11
03	**Riscos e modelagem de ameaças** Gestão de risco, ALE, matriz de risco, STRIDE, diagramas de fluxo, CVSS, EPSS e KEV	20
04	**Normas, leis e conformidade** ISO 27001/27002, NIST CSF 2.0, CIS Controls, PCI DSS, LGPD, Marco Civil e leis penais	27
05	**Criptografia aplicada** Hash, cifras, AEAD, assinaturas, PKI, TLS 1.3, gestão de chaves e criptografia pós-quântica	34
06	**Identidade e autenticação** Senhas e Argon2id, MFA, passkeys/FIDO2, sessões, SSO, OAuth 2.0, OIDC, SAML e JWT	44
07	**Autorização e controle de acesso** DAC, MAC, RBAC, ABAC, ReBAC, menor privilégio, IDOR/BOLA, motores de política e Zero Trust	53
08	**Proteção de dados e privacidade** Classificação, ciclo de vida, cifragem, tokenização, anonimização, backup 3-2-1 e LGPD	59
09	**Segurança de redes** TCP/IP, firewalls, segmentação, DMZ, IDS/IPS, VPN, DNS, SPF/DKIM/DMARC, Wi-Fi e DDoS	67
10	**Sistemas, nuvem e contêineres** Hardening, patching, EDR, permissões no Linux, nuvem (IAM), Docker e Kubernetes	78
11	**Ciclo de vida seguro (SSDLC)** Shift-left, requisitos e ASVS, projeto seguro, NIST SSDF, OWASP SAMM e definição de pronto	90
12	**Segurança web e OWASP Top 10** OWASP Top 10:2025, injeção, XSS, CSRF, SSRF, cabeçalhos, cookies, CSP e CORS	98
13	**APIs, mobile e sistemas de IA** API Security Top 10, BOLA, JWT, rate limiting, MASVS, OWASP Top 10 para LLM e agentes	112
14	**Código seguro e segurança de memória** Validação, erros, concorrência, buffer overflow, use-after-free, CWE Top 25 e memory safety	122
15	**Cadeia de suprimentos e DevSecOps** SCA, SBOM, SLSA, Sigstore, segredos, CI/CD, SAST/DAST/IAST, fuzzing e infraestrutura como código	135
16	**Monitoramento e detecção** Logs, SIEM, EDR/NDR/XDR, SOC, engenharia de detecção (Sigma, YARA) e threat intelligence	144
17	**Resposta a incidentes e continuidade** NIST SP 800-61r3, playbooks, forense, notificação à ANPD, RTO/RPO e resiliência a ransomware	156
18	**Testes de segurança** Avaliação de vulnerabilidades, pentest, red/blue/purple team, bug bounty e limites legais	166
19	**Pessoas, cultura e proteção pessoal** Fator humano, phishing e golpes, higiene digital, cultura de segurança e checklist pessoal	180
20	**Projeto-guia: um sistema seguro** Do requisito à operação: modelagem, arquitetura, controles, verificação e perguntas de revisão	189
—	**Glossário técnico** Termos de A a Z para consulta rápida, com siglas e conceitos usados no guia	200
—	**Fontes e leituras** Padrões, relatórios, leis e materiais para se aprofundar, organizados por tema	225

ANTES DE COMEÇAR

— COMO USAR ESTE GUIA

Um mapa rápido para tirar o máximo do material — dos conceitos ao projeto de um sistema seguro.

Este guia reúne os principais **termos, padrões, técnicas e estratégias de engenharia** da segurança da informação — dos fundamentos ao projeto de um sistema seguro. A ordem vai do "o quê e por quê" ao "como construir e operar", mas cada módulo funciona sozinho.

MAPA DO GUIA

1 · ENTENDER
Fundamentos e contexto

01 Fundamentos da segurança
02 Ameaças e ataques
03 Riscos e modelagem de ameaças
04 Normas, leis e conformidade

2 · PROTEGER
Dados e identidades

05 Criptografia aplicada
06 Identidade e autenticação
07 Autorização e controle de acesso
08 Proteção de dados e privacidade

3 · INFRAESTRUTURA
Redes e ambientes

09 Segurança de redes
10 Sistemas, nuvem e contêineres

4 · DESENVOLVER
Segurança no desenvolvimento

11 Ciclo de vida seguro (SSDLC)
12 Segurança web e OWASP Top 10
13 APIs, mobile e sistemas de IA
14 Código seguro e memória
15 Cadeia de suprimentos e DevSecOps

5 · OPERAR
Detecção, resposta e testes

16 Monitoramento e detecção
17 Resposta a incidentes
18 Testes de segurança

6 · PRATICAR
Consolidação

19 Pessoas e proteção pessoal
20 Projeto-guia: sistema seguro

Seis blocos, do entendimento à prática. Os números dentro de cada bloco são os módulos; o sumário mostra a página de cada um.

LEGENDA DOS QUADROS

Em resumo: a ideia central de uma seção, em uma ou duas frases. Ótimo para revisar.

Boa prática: um controle ou hábito que funciona na prática e vale virar rotina.

Atenção: um erro comum ou um risco real. Leia antes de seguir em frente.

DICA DA CORUJA

o conselho de quem já viu isso dar errado: um atalho, um critério de decisão ou uma regra de bolso.

EXEMPLO RESOLVIDO · ESTUDO DE CASO

Blocos escuros como este trazem **cálculos resolvidos, incidentes reais e exemplos de projeto**. Acompanhe o raciocínio com papel e caneta e refaça com os números do seu cenário.

— COMO USAR ESTE GUIA

CONTINUAÇÃO

TRILHAS DE ESTUDO

SEU PERFIL	COMECE POR	DEPOIS	FIXE COM
Primeira disciplina de segurança	p. 5 → p. 11 → p. 34 → p. 44	p. 53 → p. 67 → p. 98	Projeto-guia (p. 189) e glossário (p. 200)
Quero desenvolver software seguro	p. 90 → p. 98 → p. 122	p. 44 → p. 53 → p. 112 → p. 135	Checklists dos módulos e projeto-guia (p. 189)
Infraestrutura e operações (*blue team*)	p. 67 → p. 78 → p. 144 → p. 156	p. 20 → p. 27 → p. 59	Playbooks e exercícios de p. 156
Segurança ofensiva, de forma ética	p. 11 → p. 98 → p. 166	p. 112 → p. 34 → p. 67	Laboratórios legais indicados em p. 166
Gestão, risco e conformidade (GRC)	p. 5 → p. 20 → p. 27	p. 59 → p. 156 → p. 180	Matriz de risco e quadro de controles
Revisão rápida em 1 hora	Quadros "Em resumo" de cada módulo	Perguntas "Para fixar"	Glossário (p. 200)

COMO LER O TEXTO

Termos e código

Termos em inglês aparecem em itálico na primeira vez. `Comandos`, arquivos e trechos de código ficam em fonte monoespaçada; em blocos de comparação, linhas em vermelho são inseguras e em verde, corretas.

Referências de página

Indicações como **p. 98** levam ao início do módulo citado. No PDF digital, elas, o sumário e as fontes são clicáveis. Cada módulo termina com "Para fixar" e "Aprofunde-se".

Datas e números

Padrões e relatórios mudam (OWASP, NIST, CWE, DBIR). Este guia reflete **outubro e novembro de 2026**; confira a edição vigente nas fontes (**p. 225**).

ÉTICA E LEGALIDADE PRIMEIRO

Atenção: este material é **defensivo e educacional**. Pratique somente em sistemas seus ou em laboratórios feitos para isso. Testar, invadir ou interferir em sistemas de terceiros **sem autorização é crime** no Brasil (Código Penal, art. 154-A, incluído pela Lei nº 12.737/2012 e agravado pela Lei nº 14.155/2021). Detalhes em p. 27 e p. 166.

O QUE VOCÊ PRECISA SABER ANTES

Não é pré-requisito

- Ser especialista em redes, sistemas ou criptografia
- Programar em uma linguagem específica
- Conhecer ferramentas de ataque

Ajuda bastante

- Noções de redes (IP, porta, DNS, HTTP) e de sistemas operacionais (processo, permissão)
- Lógica de programação, SQL básico e Git
- Matemática discreta: módulo, potências e probabilidade (para criptografia e risco)
- Um laboratório pequeno: uma máquina virtual e um alvo de treino como o OWASP Juice Shop

MÓDULO 01 · FUNDAMENTOS

01 FUNDAMENTOS DA SEGURANÇA

O vocabulário e os princípios que sustentam todo o resto: o que se protege, de quem e como se pensa a defesa.

Segurança da informação é proteger a informação — em qualquer meio e em todo o seu ciclo de vida — contra acesso, alteração, destruição ou indisponibilidade indevidos. Este módulo fixa o vocabulário e os princípios de projeto que sustentam todos os outros: sem eles, controles viram uma lista de ferramentas sem critério.

O QUE SE PROTEGE: SI, CIBERSEGURANÇA E VIZINHOS

TERMO	FOCO	EXEMPLO
Segurança da informação (SI)	Proteção da informação em qualquer meio (digital, papel, conversa) e em todo o ciclo de vida	Mesa limpa, backup, controle de acesso a documentos
Cibersegurança	Proteção de sistemas, redes e dados no ambiente digital contra ataques	MFA, EDR, firewall, WAF
Segurança de aplicações e sistemas	Propriedades de segurança do software e da infraestrutura em que ele roda	Validação de entrada, *hardening*, gestão de patches
Privacidade	Uso adequado de dados pessoais e respeito aos direitos do titular (finalidade, consentimento)	LGPD, minimização de dados, direito de acesso
***Safety* (segurança operacional)**	Evitar danos a pessoas e ao ambiente causados por falhas — em português, também se diz "segurança"	Intertravamento de uma máquina industrial

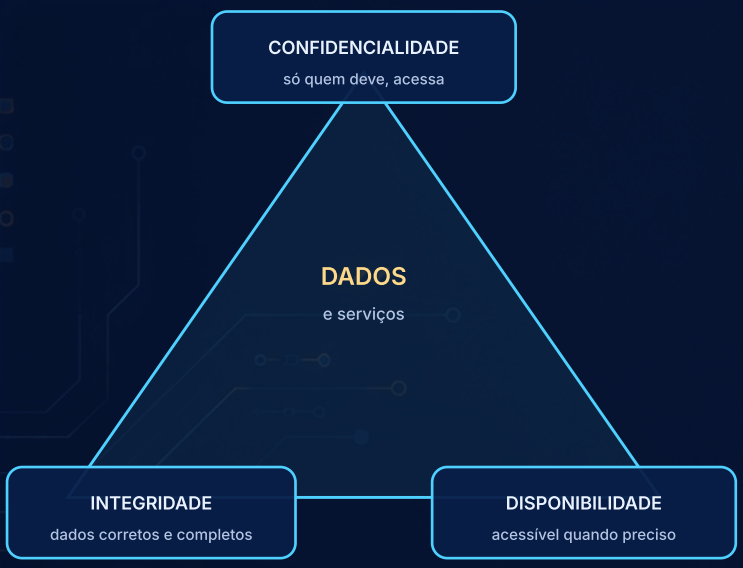
VOCABULÁRIO ESSENCIAL

TERMO	DEFINIÇÃO	EXEMPLO
Ativo (*asset*)	Tudo que tem valor e precisa de proteção: dados, sistemas, pessoas, reputação	Banco de notas dos alunos; servidor de provas
Ameaça (*threat*)	Causa potencial de um incidente indesejado: evento ou ator com capacidade de causar dano	Grupo de ransomware; erro humano; incêndio
Agente de ameaça (*threat actor*)	Quem ou o que concretiza a ameaça	Cibercriminoso, *insider*, Estado, *malware* autônomo
Vulnerabilidade	Fraqueza em um ativo, processo ou controle que uma ameaça pode explorar	Biblioteca sem correção; senha padrão; falta de treinamento
Exploit	Técnica ou código que aproveita uma vulnerabilidade específica	Código que explora o Log4Shell (CVE-2021-44228)
Vetor de ataque	Caminho usado para chegar ao alvo	E-mail de phishing; serviço exposto na internet; USB
Superfície de ataque	Soma de todos os pontos de entrada e saída de dados que um atacante pode alcançar	APIs, portas abertas, contas, dependências, pessoas

01 FUNDAMENTOS DA SEGURANÇA — CONTINUAÇÃO

TERMO	DEFINIÇÃO	EXEMPLO
Risco	Combinação da probabilidade de uma ameaça explorar uma vulnerabilidade com o impacto resultante	Alta chance de vazamento por credencial fraca × multa e perda de confiança
Impacto	Consequência medida: financeira, operacional, legal, reputacional	Três dias parados; multa; perda de clientes
Controle (contramedida)	Medida que reduz a probabilidade ou o impacto de um risco	MFA, backup, política, câmera
Evento × incidente	Evento: ocorrência observável. Incidente: evento (ou série) que compromete a CIA ou viola a política	Login falho (evento); conta de administrador tomada (incidente)

TRÍADE CIA E PROPRIEDADES ESTENDIDAS



PROPRIEDADES ESTENDIDAS

Autenticidade
a origem é comprovada: é mesmo quem diz ser

Não repúdio
o autor não pode negar a ação (assinatura digital)

Responsabilização (accountability)
ações rastreáveis a uma identidade (logs, auditoria)

Privacidade
dado pessoal tratado só para a finalidade informada

Confiabilidade (reliability)
o sistema se comporta como esperado

A tríade CIA e o que a cerca. Todo controle de segurança existe para proteger ao menos uma destas propriedades; as extensões à direita aparecem em normas, leis e arquiteturas de identidade.

PROPRIEDADE	COMO É VIOLADA	CONTROLES TÍPICOS	CASO PARA LEMBRAR
Confidencialidade	Vazamento, interceptação, acesso indevido	Cifragem, controle de acesso, classificação de dados	Equifax (2017): ao menos 145,5 milhões de pessoas expostas (GAO)
Integridade	Alteração não autorizada de dados, código ou configuração	Hash, assinatura digital, controle de versão, logs imutáveis	SolarWinds (2020): atualização legítima adulterada na origem
Disponibilidade	Negação de serviço, ransomware, falha de hardware ou de software, desastre	Redundância, backups testados, mitigação de DDoS, plano de DR	CrowdStrike (2024): milhões de máquinas fora do ar — sem ataque algum

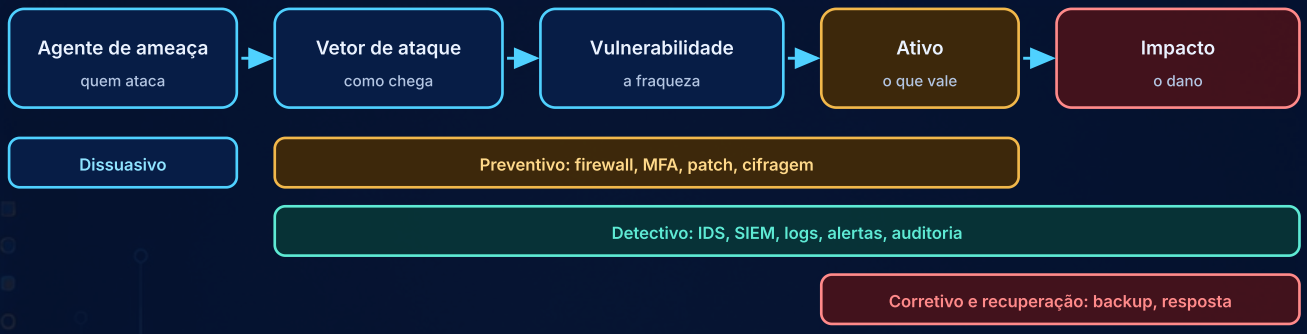
AAA descreve a cadeia que transforma "alguém bate à porta" em uma decisão registrada: a pessoa ou o sistema se **identifica**, **prova** quem é (autenticação), recebe apenas o que pode fazer (autorização) e tem as ações **registradas** (*accounting*, auditoria). Falhar em qualquer elo enfraquece todos os outros.

01 **FUNDAMENTOS DA SEGURANÇA** CONTINUAÇÃO



Em resumo: CIA é um modelo de **objetivos**; os controles são os **meios**. Um mesmo incidente costuma violar várias propriedades — no ransomware com dupla extorsão, a disponibilidade (arquivos cifrados) e a confidencialidade (dados roubados e ameaçados de publicação).

TIPOS DE CONTROLE: FUNÇÃO × NATUREZA



Onde cada tipo de controle atua. Dissuasivos desencorajam o agente; preventivos reduzem a chance de a vulnerabilidade ser explorada; detectivos revelam o que passou; corretivos e de recuperação limitam o impacto. Controles compensatórios substituem um controle que não pode ser aplicado.

FUNÇÃO	ADMINISTRATIVO	TÉCNICO	FÍSICO
Preventivo	Política de acesso, treinamento, separação de funções	Firewall, MFA, cifragem, patches	Cadeado, catraca, controle de acesso ao datacenter
Detectivo	Auditorias, revisão de logs, inspeções	IDS, SIEM, verificação de integridade	Câmeras, sensores de movimento, alarme
Corretivo	Plano de resposta a incidentes, lições aprendidas	Restaurar backup, aplicar correção, isolar host	Extintor, reparo ou troca de hardware
Dissuasivo	Sanções disciplinares, termos de uso	Banner de aviso de monitoramento	Cercas, placas, câmeras visíveis
Compensatório	Revisão dupla quando não há separação de funções	*Virtual patching* em WAF; segmentação extra	Vigilância adicional enquanto a porta é consertada
Recuperação	Plano de continuidade e de recuperação de desastres	Sistemas redundantes, restauração automatizada	Sítio alternativo, gerador de energia

PRINCÍPIOS DE PROJETO: SALTZER E SCHROEDER (1975)

Oito princípios publicados há meio século continuam sendo o vocabulário comum de quem projeta sistemas seguros. A tabela traduz cada um e mostra como aparece hoje.

01 FUNDAMENTOS DA SEGURANÇA

CONTINUAÇÃO

PRINCÍPIO	IDEIA	COMO APLICAR HOJE
Economia de mecanismo (*economy of mechanism*)	Projetos simples e pequenos são mais fáceis de entender, testar e verificar	Menos código, menos dependências, menos funcionalidades expostas
Padrões à prova de falhas (*fail-safe defaults*)	Por padrão, o acesso é negado; permissões são concedidas, não retiradas	*Deny by default* em firewall e em autorização
Mediação completa (*complete mediation*)	Todo acesso a todo objeto é verificado, sempre	Autorização em toda requisição, **no servidor**
Projeto aberto (*open design*)	A segurança não pode depender do segredo do projeto, só das chaves (princípio de Kerckhoffs)	Algoritmos públicos como o AES; código auditável
Separação de privilégios (*separation of privilege*)	Ações críticas exigem mais de uma condição ou pessoa	MFA, aprovação em dupla (*four eyes*)
Menor privilégio (*least privilege*)	Cada entidade recebe o mínimo necessário, pelo menor tempo possível	Papéis enxutos, tokens com escopo, acesso sob demanda (JIT)
Menor mecanismo comum (*least common mechanism*)	Minimizar o que é compartilhado entre usuários, para limitar a propagação de falhas	Isolamento entre clientes (*tenants*), contêineres, contas separadas
Aceitabilidade psicológica (*psychological acceptability*)	Controles difíceis de usar são contornados	SSO, *passkeys*, mensagens de erro claras e fluxos curtos

Princípios modernos

- **Defesa em profundidade:** várias camadas independentes
- **Falhar de forma segura** (*fail securely*): o erro nega, não libera
- **Seguro por projeto e por padrão** (*secure by design / default*)
- **Zero Trust:** nada é confiável só por estar "dentro" da rede
- **Assuma o comprometimento** (*assume breach*): projete para detectar e conter

Um mito a evitar

Segurança por obscuridade — esconder o projeto, a porta ou o endereço — não é um controle, é um atraso. Pode complementar defesas reais, mas nunca substituí-las: se o sistema só é seguro enquanto ninguém souber como ele funciona, ele não é seguro.

SUPERFÍCIE DE ATAQUE E FRONTEIRAS DE CONFIANÇA

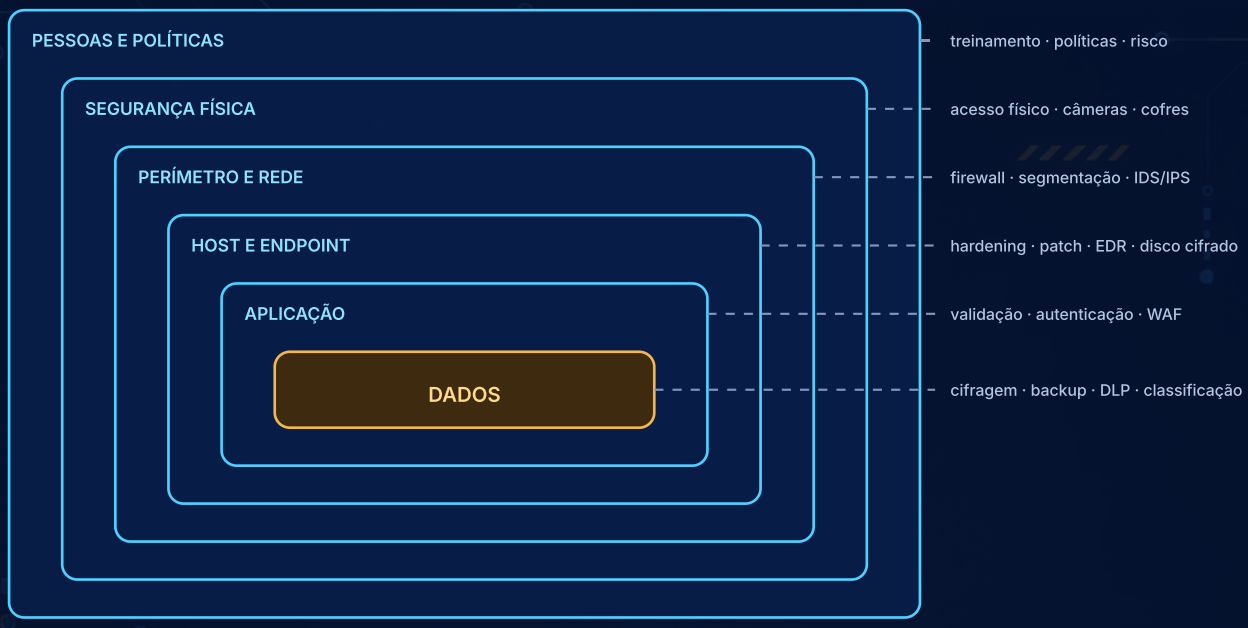
A **superfície de ataque** é tudo o que um adversário pode tocar: portas e serviços, APIs, formulários, contas, dependências de software, dispositivos, fornecedores e as próprias pessoas. Reduzi-la é um dos ganhos mais baratos em segurança: **desligue** o que não usa, **exponha** menos, **valide** mais e **remova** acessos antigos.

Uma **fronteira de confiança** (*trust boundary*) é o ponto em que os dados passam de um nível de confiança a outro — da internet para o servidor web, do servidor para o banco, de um cliente (*tenant*) para outro. É ali que se concentram autenticação, validação e registro. Aprofundamos o desenho de fronteiras em p. 20.

01 FUNDAMENTOS DA SEGURANÇA

CONTINUAÇÃO

DEFESA EM PROFUNDIDADE



Camadas independentes. Se uma falha, a seguinte reduz o dano e dá tempo de detectar. Nenhuma camada sozinha basta — e uma camada fraca não é compensada por repetir controles da mesma natureza.

> **Atenção:** defesa em profundidade **não** é "mais do mesmo". Três firewalls do mesmo fabricante com a mesma regra falham juntos; camadas distintas (rede, host, aplicação, dados, pessoas) falham por motivos diferentes.

TRADE-OFFS: NENHUM SISTEMA É 100% SEGURO

Segurança × usabilidade

Controle que atrapalha o trabalho é contornado: senha em post-it, compartilhamento de conta, *shadow IT*. O caminho seguro precisa ser o mais fácil.

Atacante × defensor

Quem defende precisa cobrir tudo, sempre; quem ataca precisa de uma brecha. Por isso: reduzir superfície, detectar cedo e projetar assumindo que algo falhará.

Custo × risco

Um controle não deve custar mais do que o risco que reduz (veja o cálculo do ALE em **p. 20**). Priorize pelo risco real, não pelo medo nem pela moda.

DICA DA CORUJA

Antes de adotar qualquer ferramenta, responda por escrito: **"protege qual ativo, de qual ameaça, com qual controle e como vou saber que funciona?"** Se não souber responder, ainda não é um controle — é só um produto.

QUEM FAZ O QUÊ: PAPÉIS EM SEGURANÇA

PAPEL	FOCO	ONDE APROFUNDAR
Gestão de segurança (CISO)	Estratégia, orçamento, risco e diálogo com a diretoria	p. 20, p. 27
Analista de SOC e resposta a incidentes	Monitorar, triar alertas, conter e investigar incidentes	p. 144, p. 156

01 FUNDAMENTOS DA SEGURANÇA — CONTINUAÇÃO

PAPEL	FOCO	ONDE APROFUNDAR
Engenharia de segurança de aplicações (AppSec)	Ciclo de vida seguro, revisão de código, ferramentas no pipeline	p. 90, p. 98, p. 135
Pentester e *red team*	Testes de intrusão autorizados e emulação de adversário	p. 166
Arquitetura e engenharia de infraestrutura e nuvem	Rede, identidade, *hardening*, contêineres	p. 67, p. 78
GRC, conformidade e privacidade (encarregado/DPO)	Normas, auditoria, LGPD e relação com a ANPD	p. 27, p. 59

EXEMPLO RESOLVIDO

Cenário. Em um portal acadêmico, um aluno explora uma falha de autorização, altera as próprias notas e, no mesmo acesso, baixa a lista de e-mails da turma.

Classificação CIA. Integridade violada (notas alteradas); confidencialidade violada (e-mails expostos); disponibilidade preservada.

O que faltou. Autorização verificada no servidor em cada operação (princípio da mediação completa — p. 53); registro e alerta de alterações de notas (responsabilização — p. 144); minimização do que a página de turma expõe (p. 59).

PARA FIXAR

Conceitos

- Diferencie ameaça, vulnerabilidade e risco com um exemplo seu.
- Por que "projeto aberto" não significa publicar as chaves?
- Dê um exemplo de controle detectivo técnico e um corretivo administrativo.

Aplicação

- Classifique nos eixos CIA: vazamento de e-mails, preços alterados e site fora do ar na Black Friday — e proponha um controle preventivo para cada.
- Aplique a defesa em profundidade a uma pasta com TCCs: um controle por camada.
- Cite dois controles que violam a aceitabilidade psicológica e como melhorá-los.

APROFUNDE-SE

Leituras e referências

- The Protection of Information in Computer Systems — Saltzer e Schroeder, *Proceedings of the IEEE* (1975): a fonte dos oito princípios.
- NIST Computer Security Resource Center — Glossary — definições oficiais de centenas de termos.
- Security Engineering — Ross Anderson (3ª ed.): capítulos abertos sobre fundamentos e economia da segurança.
- Building Secure and Reliable Systems — Google (livro aberto): princípios de projeto na prática.
- Cartilha de Segurança para Internet — CERT.br/NIC.br: linguagem acessível, em português.

MÓDULO 02 · FUNDAMENTOS

02 AMEAÇAS E ATAQUES

Quem ataca, por que e como: o cenário de ameaças e os modelos que organizam o conhecimento sobre ataques.

Defender bem exige entender **quem ataca, por quê e como**. Este módulo apresenta os agentes de ameaça, as grandes famílias de ataque e os modelos que organizam o raciocínio de quem protege — *Kill Chain*, MITRE ATT&CK e *Pyramid of Pain* — sempre com foco em **detectar, prevenir e interromper**.

PANORAMA: O QUE OS RELATÓRIOS MOSTRAM

31%
dos acessos iniciais vieram da exploração de vulnerabilidades, o vetor mais comum
VERIZON DBIR 2026

62%
das violações envolveram o elemento humano: erro, phishing ou credenciais
VERIZON DBIR 2026

48%
das violações envolveram terceiros (eram 30% no ano anterior)
VERIZON DBIR 2026

US$ 4,99
milhões: custo médio global de uma violação de dados, alta de 12%
IBM · 2026

O abuso de credenciais respondeu por 13% dos acessos iniciais e o ransomware esteve presente em 48% das violações. Um dado revela o problema de fundo: só **26%** das vulnerabilidades críticas do catálogo KEV foram totalmente corrigidas em 2025, e o tempo mediano de correção subiu de 32 para 43 dias. Ou seja, boa parte dos ataques bem-sucedidos usa **falhas conhecidas** e **identidades fracas** — e a resposta é, em grande medida, higiene: correção, MFA, menor privilégio e backup.

Como ler os números: relatórios têm metodologias e amostras próprias: use-os para enxergar **tendências**, não como retrato exato da sua organização. Os valores acima seguem a cobertura especializada das edições de 2026; confira-os nas fontes originais (p. 225).

AGENTES DE AMEAÇA

Um **agente de ameaça** (*threat actor*) se descreve por **motivação**, **capacidade** e **oportunidade**. Saber contra quem você projeta a defesa — o chamado *modelo de adversário*, detalhado em p. 20 — muda quais controles valem o custo.

AGENTE	MOTIVAÇÃO	CAPACIDADE E RECURSOS	ALVOS TÍPICOS
Oportunista (*script kiddie*)	Curiosidade, notoriedade	Baixa: usa ferramentas e scripts prontos	Sistemas expostos com falhas conhecidas
Hacktivista	Causa política ou social	Baixa a média: DDoS, pichação de sites, vazamentos	Governos e empresas controversas
Cibercriminoso (inclui RaaS e corretores de acesso)	Lucro financeiro	Média a alta: mercado organizado e especializado	Qualquer organização com dados ou receita
Crime organizado	Lucro em escala, lavagem	Alta: infraestrutura própria, fraude e extorsão	Bancos, meios de pagamento, varejo

02 AMEAÇAS E ATAQUES

CONTINUAÇÃO

AGENTE	MOTIVAÇÃO	CAPACIDADE E RECURSOS	ALVOS TÍPICOS
Insider (malicioso, negligente ou comprometido)	Dinheiro, vingança, descuido	Variável: já possui acesso legítimo	Dados e sistemas da própria organização
APT / Estado-nação	Espionagem, sabotagem, influência	Muito alta: recursos, paciência e falhas inéditas (*zero-days*)	Governo, defesa, energia, cadeia de suprimentos



DICA DA CORUJA

defina o adversário **antes** de escolher controles: "contra quem, com quais recursos, atrás de quê?". Proteger um blog pessoal de um Estado é desperdício; proteger um banco só com antivírus é negligência.

MALWARE: FAMÍLIAS, COMPORTAMENTO E DEFESA

Malware (*malicious software*) é todo software feito para causar dano ou obter acesso não autorizado. As categorias se sobrepõem — um mesmo programa pode ser *trojan*, *loader* e ransomware ao mesmo tempo. O que se detecta, portanto, é o **comportamento**, mais do que o rótulo.

TIPO	COMO AGE OU SE PROPAGA	SINAIS E DEFESA
Vírus	Anexa-se a arquivos ou programas e precisa que o hospedeiro seja executado	Antimalware/EDR, assinatura de código, *allowlisting*
Worm (verme)	Propaga-se sozinho pela rede explorando falhas (WannaCry usou SMBv1)	Correção de falhas, segmentação, bloqueio de SMB entre estações
Trojan (cavalo de troia)	Disfarça-se de software legítimo e entrega outras cargas	Instalar só de fontes confiáveis, reputação, *sandbox*
Ransomware	Cifra dados e, hoje, quase sempre também os exfiltra, para exigir pagamento	Backups imutáveis, EDR, menor privilégio, segmentação
Spyware / *infostealer*	Coleta senhas, *cookies* de sessão e dados do navegador	MFA resistente a phishing, sessões curtas, EDR (p. 44)
Rootkit / bootkit	Oculta a presença no sistema, no *kernel* ou na inicialização (*boot*)	*Secure Boot*, TPM e atestação, verificação de integridade
Botnet / RAT	Máquinas controladas à distância por um servidor C2; a Mirai (2016) usou IoT com senha padrão	Trocar senhas padrão, filtro de saída, detecção de *beaconing*
Fileless* / *living off the land	Abusa de ferramentas legítimas do sistema (PowerShell, WMI) e da memória	Registro de scripts, EDR comportamental, restringir ferramentas
Wiper	Destrói dados de forma irrecuperável; o NotPetya (2017) imitava ransomware	Backups offline, segmentação, recuperação ensaiada

Atenção: antivírus baseado só em **assinaturas** (hashes) é insuficiente: basta recompilar para o hash mudar (veja a *Pyramid of Pain* adiante). Combine detecção comportamental, *allowlisting*, correção de falhas e backup.

02 **AMEAÇAS E ATAQUES** CONTINUAÇÃO

RANSOMWARE: ECONOMIA E CICLO DE ATAQUE

Ransomware é um negócio, não só um programa. No modelo *Ransomware-as-a-Service* (RaaS), os desenvolvedores alugam a plataforma a **afiliados**, que invadem as vítimas e dividem o resgate. **Corretores de acesso inicial** (*initial access brokers*) vendem credenciais e acessos prontos. A pressão cresceu em camadas: a **dupla extorsão** (cifrar *e* ameaçar vazar os dados) e a **tripla extorsão** (somar DDoS ou pressão sobre clientes e parceiros). Em maio de 2021, o ataque do grupo DarkSide à Colonial Pipeline levou a operação a ser interrompida por precaução (alerta CISA AA21-131A).

O backup comum não basta: os operadores procuram e apagam as cópias **antes** de cifrar. A regra **3-2-1-1-0** — 3 cópias, 2 mídias, 1 fora do local, 1 offline ou imutável e 0 erros na verificação de restauração — e o **teste periódico de restauração** são o que separa um incidente de uma catástrofe.



Ciclo típico de um ataque de ransomware. Cada etapa tem um ponto de interrupção: nas quatro primeiras a vítima ainda opera normalmente e só a detecção percebe; quando a criptografia começa, só a preparação (backup, segmentação, plano de crise) salva.

> **Boa prática:** meça o que importa: **RTO** (tempo máximo para voltar a operar) e **RPO** (quanto dado se aceita perder). Ensaie a restauração completa pelo menos uma vez por ano e guarde, fora do domínio, o procedimento impresso ou offline (p. 156).

ENGENHARIA SOCIAL

A **engenharia social** manipula **pessoas**, não sistemas: explora confiança, pressa e respeito à autoridade. Por isso está na origem de grande parte dos incidentes — o phishing foi o vetor inicial mais comum no relatório da IBM de 2026, pelo quarto ano seguido. As alavancas psicológicas descritas por **Robert Cialdini** explicam por que ela funciona.

TÉCNICA	CANAL E MECANISMO	SINAL DE ALERTA	DEFESA
Phishing	E-mail em massa com link ou anexo falso	Urgência; remetente ou URL quase igual ao legítimo	Filtro de e-mail, DMARC, MFA resistente a phishing
Spear phishing	Mensagem personalizada para uma pessoa ou área	Conhece detalhes internos; pede ação específica	Verificar por outro canal; treinamento

02 AMEAÇAS E ATAQUES

CONTINUAÇÃO

TÉCNICA	CANAL E MECANISMO	SINAL DE ALERTA	DEFESA
***Whaling* / BEC**	Fraude contra executivos e financeiro: pagamento, troca de conta	Pedido incomum de transferência ou sigilo	Dupla aprovação; ligar de volta para número conhecido
Vishing* / *smishing	Ligação de voz e SMS	Pede código, senha ou instalar aplicativo	Nunca informar códigos; desligar e ligar de volta
Quishing	QR Code malicioso em e-mail, cartaz ou adesivo	Código em local inesperado	Pré-visualizar o endereço; filtro de URL móvel
Pretexting	Cenário inventado (TI, RH, fornecedor) para obter dados	Ignora o procedimento padrão	Procedimento fixo de verificação de identidade
***Deepfake* de voz ou vídeo**	Clone da voz ou do rosto de um superior ou parente	Urgência + sigilo + pedido de pagamento	Palavra combinada; confirmar fora do canal
MFA fatigue	Dezenas de pedidos de aprovação até alguém aceitar (caso Uber, 2022)	Notificações que você não iniciou	Correspondência de número, FIDO2, limite de tentativas

Reciprocidade
Um favor cria obrigação: o "técnico" que ajuda de graça e depois pede um acesso.

Compromisso e coerência
Um pequeno "sim" leva a outro: começa com um pedido inocente e escala.

Prova social
"Todo mundo já fez": "seus colegas já atualizaram o cadastro".

Autoridade
Chefe, banco, TI ou polícia: obedece-se a quem parece ter poder.

Afinidade
Simpatia e semelhança: perfil falso amigável, com interesses em comum.

Escassez e urgência
"Só hoje", "sua conta será bloqueada em 1 hora": a pressa desliga o espírito crítico.



DICA DA CORUJA

toda mensagem que junta **urgência + autoridade + pedido incomum** (pagar, informar um código, instalar algo) merece verificação por **outro canal**. A IA generativa barateou textos perfeitos e vozes clonadas, então "erros de português" já não denunciam golpe: confie no **processo**, não na "cara de golpe".

ATAQUES A CREDENCIAIS

Credenciais são a moeda do ataque moderno. Em vez de "quebrar" sistemas, o adversário **entra com uma identidade válida** — e passa despercebido. As defesas completas (hash de senha, MFA, passkeys) estão em **p. 44**; aqui o foco é entender o ataque.

ATAQUE	COMO FUNCIONA	DEFESA PRINCIPAL
Força bruta	Testa combinações até acertar, online ou contra um hash roubado	Senhas longas, limite de tentativas, hash lento (Argon2id)
Dicionário e regras	Testa senhas prováveis e suas mutações (*Verao2026!*)	Checar contra listas de senhas vazadas; sem regras de composição
Credential stuffing	Reaproveita pares usuário/senha vazados de um site em outros	Senha única por serviço, MFA, detecção de bots

02 AMEAÇAS E ATAQUES

CONTINUAÇÃO

ATAQUE	COMO FUNCIONA	DEFESA PRINCIPAL
Password spraying	Poucas senhas comuns contra muitas contas, para fugir do bloqueio	Alertas por origem e por senha, MFA, bloqueio inteligente
Interceptação / AiTM	Captura senha e *cookie* com proxy de phishing (*adversary-in-the-middle*), *sniffing* ou *keylogger*	FIDO2/passkeys, TLS, EDR
Pass-the-hash	Reutiliza o hash NTLM roubado, sem conhecer a senha (ambientes Windows/AD)	Segmentação, restringir administrador local, LAPS, proteção de credenciais
Kerberoasting	Pede tickets de serviço e quebra a senha da conta de serviço offline	Contas de serviço com senha longa ou gMSA, AES, detecção
SIM swap	Transfere o número da vítima para outro chip e recebe SMS e códigos	Evitar SMS como MFA, PIN na operadora, FIDO2

Atenção: bloquear a conta após N tentativas pode virar um **ataque de negação de serviço contra usuários legítimos** (o adversário trava contas alheias de propósito). Prefira atraso progressivo, limites por origem e por dispositivo, e MFA — veja p. 44.

ATAQUES DE REDE E NEGAÇÃO DE SERVIÇO

Muitos protocolos da internet nasceram sem autenticação: quem está "no caminho" pode ler, alterar ou redirecionar o tráfego. Já a **negação de serviço** (DoS) ataca a **disponibilidade**; no formato distribuído (**DDoS**), o tráfego vem de milhares de origens — botnets, muitas vezes de dispositivos IoT, como a Mirai, que em 2016 derrubou o provedor de DNS Dyn. Defesas de rede em p. 67.

ATAQUE	POR QUE FUNCIONA	DEFESA (VER MÓDULO 09)
Sniffing	Tráfego sem criptografia é legível por quem está no caminho (Wi-Fi aberto, hub, porta espelhada)	TLS em tudo, VPN, Wi-Fi com WPA3 ou 802.1X
***Man-in-the-middle* (MITM)**	O adversário se coloca entre as partes e lê ou altera as mensagens	TLS com validação de certificado, HSTS
ARP spoofing	O ARP não tem autenticação: respostas falsas redirecionam o tráfego na rede local	Inspeção dinâmica de ARP, segmentação, 802.1X
Envenenamento de cache DNS	O resolvedor aceita uma resposta falsa e a entrega a todos os clientes	DNSSEC, resolvedores atualizados, DoT/DoH
DDoS volumétrico	Esgota a banda do enlace; a *amplificação/reflexão* multiplica o tráfego (DNS, NTP, memcached)	Absorção em CDN ou *scrubbing*, *anycast*, filtro antifalsificação (BCP 38)
DDoS de protocolo	Esgota tabelas de estado de firewalls e servidores (*SYN flood*)	*SYN cookies*, limites de conexão, equipamentos com estado robusto
DDoS de aplicação (camada 7)	Requisições aparentemente legítimas que consomem CPU e banco (HTTP flood, Slowloris)	WAF, *rate limiting*, cache, desafios, *autoscaling* com teto

02 **AMEAÇAS E ATAQUES** CONTINUAÇÃO

Exemplo: fator de amplificação: se um pedido falsificado de 60 bytes gera uma resposta de 3.000 bytes, o fator é 3.000 ÷ 60 = **50×**. Com 100 Mbit/s de pedidos, a vítima recebe cerca de 5 Gbit/s. O adversário "empresta" a banda de servidores abertos (resolvedores DNS, NTP) e falsifica o IP de origem; por isso o filtro antifalsificação na borda (BCP 38, RFC 2827) e o fechamento de serviços abertos importam.

AMEAÇAS INTERNAS, FÍSICAS E DE TERCEIROS

Ameaça interna

Três perfis: **malicioso** (age de propósito), **negligente** (erra ou burla controle) e **comprometido** (conta roubada). Controles: menor privilégio, separação de funções, desligamento imediato de acessos e monitoramento de comportamento (**p. 53**, **p. 144**).

Ameaça física e ambiental

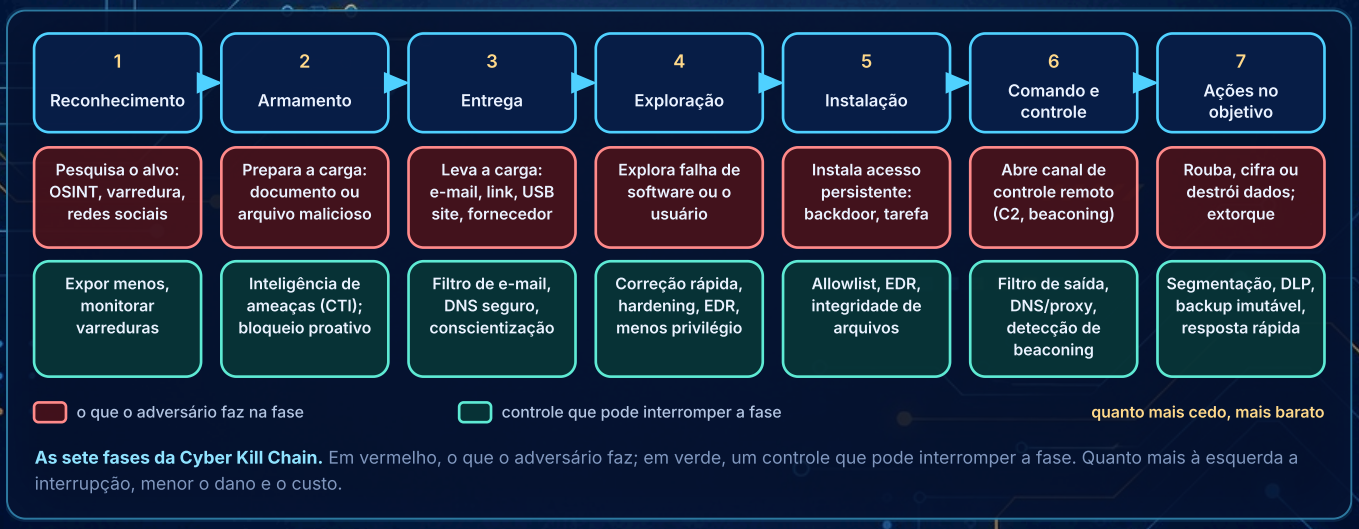
Quem tem acesso físico ao equipamento anula muitos controles lógicos: perda de notebook, *tailgating* (seguir alguém na porta), USB deixado de isca. Também entram incêndio, energia e enchente. Controles: criptografia de disco, travas, crachás, *nobreak*, plano de recuperação.

Terceiros e cadeia de suprimentos

O adversário ataca quem você usa: fornecedores, bibliotecas e atualizações. SolarWinds (2020) e xz (2024) mostram o padrão; o DBIR 2026 aponta terceiros em 48% das violações. Controles: avaliação de fornecedores, SBOM, assinatura e verificação (**p. 135**).

CYBER KILL CHAIN: CADA FASE É UMA CHANCE DE INTERROMPER

A *Cyber Kill Chain* (Lockheed Martin, 2011) descreve a intrusão em **sete fases**. Seu valor didático é a ideia de que a defesa não precisa vencer em todas: **basta interromper uma** — e quanto mais cedo, mais barato. Seus limites: assume um ataque linear vindo de fora e diz pouco sobre credenciais legítimas, movimento lateral e cadeia de suprimentos, e por isso costuma ser usada junto com o ATT&CK.



As sete fases da Cyber Kill Chain. Em vermelho, o que o adversário faz; em verde, um controle que pode interromper a fase. Quanto mais à esquerda a interrupção, menor o dano e o custo.

MITRE ATT&CK: O VOCABULÁRIO COMUM

O **MITRE ATT&CK** é uma base de conhecimento pública, construída a partir de observações reais, que organiza o comportamento do adversário em **táticas** (o *porquê*: objetivos como acesso inicial ou persistência), **técnicas** e **subtécnicas** (o *como*; ex.: T1566 *Phishing* → T1566.001 anexo). Cada técnica se liga a **mitigações** (M####), **grupos** (G####) e **software** (S####) observados. Em outubro de 2026, a v20 reorganizou a matriz Enterprise: a tática *Defense Evasion* foi dividida em **Stealth** (TA0005, misturar-se ao ambiente) e **Defense Impairment** (TA0112, degradar as defesas), chegando a 15 táticas.

02 AMEAÇAS E ATAQUES

CONTINUAÇÃO

Reconnaissance
Reconhecimento
T1595 Varredura ativa
T1589 Dados de identidade
T1593 Busca em sites

Resource Development
Preparar recursos
T1583 Infraestrutura
T1586 Comprometer contas
T1588 Obter capacidades

Initial Access
Acesso inicial
T1566 Phishing
T1190 App exposto
T1078 Contas válidas

Execution
Execução
T1059 Scripts e comandos
T1204 Ação do usuário
T1047 WMI

Persistence
Persistência
T1053 Tarefa agendada
T1547 Execução ao iniciar
T1136 Criar conta

Privilege Escalation
Escalada de privilégio
T1068 Explorar falha local
T1548 Abuso de elevação
T1134 Manipular tokens

Stealth v20
Furtividade · TA0005
T1027 Ofuscação
T1036 Disfarce
T1070 Apagar indícios

Defense Impairment v20
Degradar defesas · TA0112
T1562 Prejudicar defesas
ex.: desligar EDR
ex.: cortar envio de logs

Credential Access
Acesso a credenciais
T1003 Dump de credenciais
T1110 Força bruta
T1558 Tickets Kerberos

Discovery
Descoberta
T1087 Descoberta de contas
T1046 Serviços de rede
T1083 Arquivos e pastas

Lateral Movement
Movimento lateral
T1021 Serviços remotos
T1550 Reuso de credencial
T1570 Ferramentas laterais

Collection
Coleta
T1005 Dados locais
T1560 Arquivar dados
T1213 Repositórios de info.

Command and Control
Comando e controle
T1071 Protocolos de app
T1573 Canal cifrado
T1090 Proxy

Exfiltration
Exfiltração
T1041 Via canal C2
T1567 Via serviço web
T1048 Outro protocolo

Impact
Impacto
T1486 Dados cifrados
T1490 Inibir recuperação
T1498 Negação de serviço

preparar e entrar · permanecer e se esconder · expandir o acesso · cumprir o objetivo

Matriz resumida do ATT&CK Enterprise (15 táticas, com exemplos de técnicas). Nomes de técnicas abreviados ou traduzidos; consulte o nome oficial e as subtécnicas no site. As duas táticas marcadas "v20" substituem a *Defense Evasion*.

Para que serve

- **Linguagem comum** entre SOC, caça a ameaças e gestão
- **Cobertura de detecção**: quais técnicas você enxerga e quais não? (o *ATT&CK Navigator* ajuda a pintar o mapa)
- *Threat-informed defense*: priorizar o que os grupos relevantes ao seu setor realmente fazem
- Planejar **emulação de adversário** e exercícios *purple team* (**p. 166**)

Cuidados

- Não é checklist: ter "verde" em uma técnica não prova que a detecção funciona — **teste**
- Técnica não é severidade: ela descreve comportamento, não risco (**p. 20**)
- Nomes e IDs mudam entre versões; cite sempre a versão consultada

PYRAMID OF PAIN E DIAMOND MODEL

A *Pyramid of Pain* (David Bianco, 2013) classifica os indicadores pelo **incômodo que causam ao adversário quando você os detecta**. Bloquear um hash é trivial de contornar; detectar um **comportamento** (TTPs) o obriga a mudar o jeito de operar. O *Diamond Model* (Caltagirone et al., 2013) lembra que todo evento de intrusão liga quatro vértices — **adversário, capacidade, infraestrutura e vítima** — e permite "pivotar" de um indicador para outro (a mesma infraestrutura aparece em outro ataque?).

NÍVEL (DA BASE AO TOPO)	TROCAR É...	EXEMPLO DE DETECÇÃO
Valores de hash	Trivial	Bloqueio por hash de arquivo (basta mudar um bit)
Endereços IP	Fácil	Listas de IPs maliciosos (trocam-se com facilidade)
Nomes de domínio	Simples	Bloqueio e *sinkhole* de DNS (custa registrar outro)
Artefatos de rede e de host	Incômodo	Padrões de *User-Agent*, chaves de registro, nomes de *mutex*
Ferramentas	Desafiador	Detecção da assinatura comportamental da ferramenta usada
TTPs (táticas, técnicas e procedimentos)	Difícil	Detecção do **modo de operar**: movimento lateral, uso de credenciais

02 **AMEAÇAS E ATAQUES** CONTINUAÇÃO

LINHA DO TEMPO: CASOS QUE ENSINARAM A DEFENDER

2017 · WANNACRY
ransomware com propagação de worm via EternalBlue contra SMBv1 (MS17-010): centenas de milhares de máquinas em cerca de 150 países; paralisou serviços do NHS inglês. Um "kill switch" foi descoberto por pesquisador.

2017 · NOTPETYA
distribuído por uma atualização do software contábil ucraniano M.E.Doc (cadeia de suprimentos); destrutivo, disfarçado de ransomware.

2020 · SOLARWINDS
build do Orion comprometido; atualização adulterada (SUNBURST) enviada a cerca de 18.000 clientes, segundo a empresa; atribuída pelo governo dos EUA a ator russo.

2020 · STJ
ransomware (RansomEXX) derrubou sistemas do Superior Tribunal de Justiça por vários dias.

2021 · COLONIAL PIPELINE
ransomware DarkSide (RaaS); operação interrompida por precaução (CISA AA21-131A).

2021 · LOG4SHELL
CVE-2021-44228 no Log4j 2: consulta JNDI em mensagens de log levava a execução remota de código; CVSS 10,0; exemplo clássico de risco de dependência.

2024 · XZ/LIBLZMA
CVE-2024-3094: backdoor nas versões 5.6.0 e 5.6.1, descoberto por Andres Freund ao notar lentidão no SSH; fruto de engenharia social de longo prazo contra quem mantinha o projeto.

2024 · CROWDSTRIKE (NÃO FOI ATAQUE)
atualização de conteúdo defeituosa derrubou cerca de 8,5 milhões de máquinas Windows (estimativa da Microsoft): lembrete de que a **disponibilidade** também falha por erro interno.

EXEMPLO RESOLVIDO

Cenário. Uma analista do financeiro digita a senha em um site falso recebido por e-mail. Dois dias depois, alguém entra pela VPN (sem MFA), obtém credenciais de administrador, copia arquivos e cifra os servidores.

Mapeamento. Kill Chain: entrega → exploração (o usuário) → instalação → ações no objetivo. ATT&CK: *Initial Access* (T1566 *Phishing*) → *Initial Access* com T1078 *Valid Accounts* → *Credential Access* → *Lateral Movement* → *Exfiltration* → *Impact* (T1486).

Controles que interromperiam a cadeia, do mais cedo ao mais tarde. (1) **FIDO2/passkeys**: o site falso não consegue reaproveitar a prova criptográfica do domínio original; (2) **MFA na VPN** e alerta de login atípico; (3) **menor privilégio** e senha de administrador local única (LAPS); (4) **filtro de saída** e DLP para barrar a cópia; (5) **backup imutável** com restauração testada.

Para refletir. Qual desses controles é o mais barato? Qual interrompe mais cedo? Se só um pudesse ser implantado neste trimestre, qual você escolheria e por quê?

MAPA RÁPIDO: DA AMEAÇA AO MÓDULO DE DEFESA

AMEAÇA	PRIMEIRAS DEFESAS	ONDE ESTUDAR
Phishing e engenharia social	Treinamento, DMARC, MFA resistente a phishing, verificação por outro canal	p. 180, p. 44
Credenciais fracas ou vazadas	Hash lento, MFA, passkeys, detecção de abuso de contas	p. 44
Ransomware	Backup imutável, segmentação, EDR, menor privilégio, plano de resposta	p. 78, p. 144, p. 156
Exploração de vulnerabilidades	Gestão de vulnerabilidades, *hardening*, correção rápida, WAF	p. 20, p. 98
Interceptação e MITM	TLS, HSTS, DNSSEC, validação de certificados	p. 34, p. 67
Negação de serviço (DDoS)	CDN, *rate limiting*, filtro antifalsificação, planejamento de capacidade	p. 67

02 AMEAÇAS E ATAQUES — CONTINUAÇÃO

AMEAÇA	PRIMEIRAS DEFESAS	ONDE ESTUDAR
Ameaça interna	Menor privilégio, separação de funções, monitoramento e desligamento de acessos	p. 53, p. 144
Cadeia de suprimentos	SBOM, assinatura e verificação de artefatos, avaliação de fornecedores	p. 135

PARA FIXAR

Conceitos

- Diferencie dupla e tripla extorsão e explique o papel dos corretores de acesso inicial.
- Por que a *Pyramid of Pain* coloca TTPs no topo e hashes na base?
- Qual a diferença entre *credential stuffing* e *password spraying*?

Aplicação

- Para o incidente do exemplo, escolha uma técnica do ATT&CK em cada tática e proponha uma detecção para cada uma.
- Calcule a banda recebida por uma vítima se 200 Mbit/s de pedidos falsificados tiverem fator de amplificação 40×.
- Monte um plano de backup 3-2-1-1-0 para um laboratório de universidade.

APROFUNDE-SE

Leituras e referências

- MITRE ATT&CK — MITRE: táticas, técnicas, grupos, mitigações e o Navigator.
- #StopRansomware — CISA: guias e alertas sobre ransomware, com listas de verificação.
- Alerta AA21-131A (DarkSide) — CISA/FBI, 2021: caso real com indicadores e mitigações.
- Data Breach Investigations Report — Verizon: estatísticas anuais de incidentes e violações.
- Cartilha de Segurança para Internet — CERT.br/NIC.br: golpes, códigos maliciosos e ataques, em português.

MÓDULO 03 · RISCOS

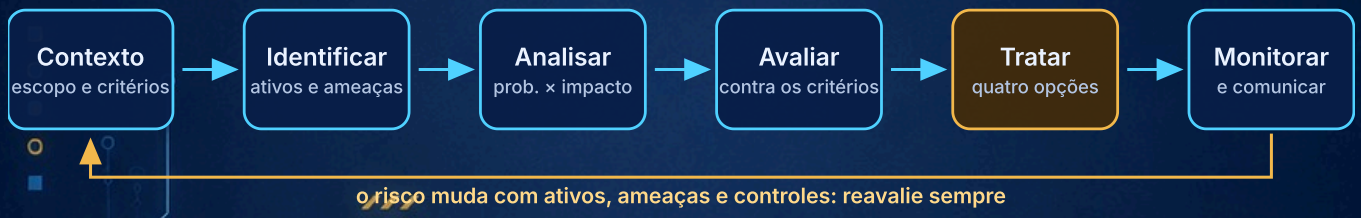
03 RISCOS E MODELAGEM DE AMEAÇAS

Como medir, priorizar e tratar riscos — e como descobrir ameaças ainda no projeto, antes do código.

Risco é o que transforma "segurança" em decisão: o que proteger primeiro, com quanto dinheiro e até que ponto aceitar o que sobra. Este módulo ensina a **estimar e tratar riscos**, a **priorizar vulnerabilidades** (CVSS, EPSS, KEV) e a **modelar ameaças** (DFD e STRIDE) ainda na fase de projeto.

GESTÃO DE RISCO: O CICLO

Um **risco** combina a **probabilidade** de uma ameaça explorar uma vulnerabilidade e o **impacto** resultante; na forma mais simples, *risco = probabilidade × impacto*. A **gestão de riscos** é o processo contínuo de identificar, analisar, avaliar e tratar esses riscos. Ela sustenta normas como a **ISO/IEC 27005** e a **ISO 31000**, e guias como o **NIST SP 800-30 Rev. 1** (avaliação de risco), ligado ao *Risk Management Framework* (NIST SP 800-37 Rev. 2).



Risco inerente × residual

Inerente é o risco antes de qualquer controle; **residual** é o que sobra depois deles. O objetivo não é zerar o residual, e sim levá-lo **abaixo do que a organização tolera**.

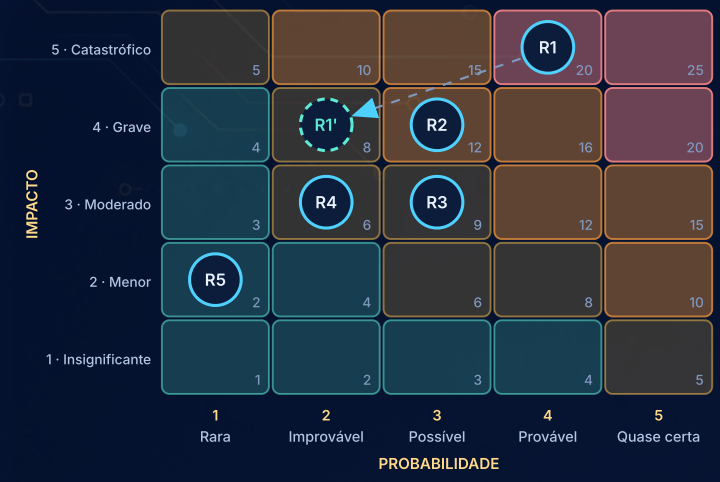
Apetite × tolerância

Apetite é o quanto de risco a organização aceita buscar para atingir seus objetivos; **tolerância** é o desvio máximo aceitável em torno disso. Ambos são decididos pela **gestão**, não pela equipe técnica.

ANÁLISE QUALITATIVA: A MATRIZ 5×5

Quando faltam dados históricos, probabilidade e impacto são classificados em **escalas ordinais** de 1 a 5 e multiplicados. O método é rápido e bom para comparar, mas **subjetivo**: duas pessoas pontuam diferente. Para reduzir o problema, defina cada nível por escrito — por exemplo, impacto 5 = parada superior a 3 dias ou vazamento de dados sensíveis em larga escala; probabilidade 5 = esperado mais de uma vez por ano.

03 RISCOS E MODELAGEM DE AMEAÇAS — CONTINUAÇÃO



REGISTRO DE RISCOS (EXEMPLO: PORTAL ACADÊMICO)

- **R1** Phishing a docentes leva à alteração de notas — P4 × I5 = 20 · crítico
- **R2** Ransomware no servidor de arquivos — P3 × I4 = 12 · alto
- **R3** Matrícula fora do ar (DDoS de aplicação) — P3 × I3 = 9 · médio
- **R4** Perda de notebook com dados de pesquisa — P2 × I3 = 6 · médio
- **R5** Pichação do site institucional — P1 × I2 = 2 · baixo
- **R1'** R1 residual, com FIDO2 e alertas (tracejado) — P2 × I4 = 8 · médio

Matriz de risco 5×5 (mapa de calor). O número no canto de cada célula é probabilidade × impacto. R1' mostra o **risco residual** de R1 depois de controles que reduzem a probabilidade e o impacto.

> **Atenção:** a matriz tem armadilhas conhecidas: multiplicar números ordinais não é rigoroso; um risco raro, mas catastrófico, some em uma célula amarela; e dois riscos com o mesmo nível podem exigir respostas muito diferentes. Use a matriz para **organizar a conversa**, não para decidir sozinha.

ANÁLISE QUANTITATIVA: SLE, ARO E ALE

Com estimativas monetárias, o risco vira uma **perda anual esperada**:

- **AV** (*Asset Value*): valor do ativo;
- **EF** (*Exposure Factor*): fração do valor perdida em um incidente, de 0 a 1;
- **SLE** (*Single Loss Expectancy*) = AV × EF: perda por ocorrência;
- **ARO** (*Annualized Rate of Occurrence*): ocorrências esperadas por ano;
- **ALE** (*Annualized Loss Expectancy*) = SLE × ARO: perda anual esperada.

Um controle compensa, em termos econômicos, se **(ALE antes − ALE depois) > custo anual do controle**. O retorno se mede pelo **ROSI** (*Return on Security Investment*) = (redução do ALE − custo) ÷ custo. O método **FAIR** (*Factor Analysis of Information Risk*, padrão Open FAIR) refina a ideia: separa **frequência** e **magnitude** da perda, usa estimativas calibradas em **faixas** (mínimo, mais provável, máximo) e simula milhares de cenários (Monte Carlo) para obter uma distribuição, não um número único.

EXEMPLO RESOLVIDO

Cenário. A base de matrículas (AV = R$ 2.000.000, somando recuperação, multas e perda de confiança) pode ser exposta por uma falha de injeção SQL. Em um vazamento, estima-se perder **EF = 25%** do valor: **SLE = 2.000.000 × 0,25 = R$ 500.000**. A equipe estima 1 ocorrência a cada 5 anos, ou seja, **ARO = 0,2**. Logo, **ALE = 500.000 × 0,2 = R$ 100.000 por ano**.

Controle. Revisão de código, SAST e WAF custam **R$ 40.000 por ano** e reduzem o ARO para 0,05 (1 a cada 20 anos): novo ALE = 500.000 × 0,05 = R$ 25.000. A redução é de **R$ 75.000**; o valor líquido é 75.000 − 40.000 = **R$ 35.000 por ano**; **ROSI = 35.000 ÷ 40.000 = 87,5%**.

Decisão e ressalvas. Em termos econômicos, o controle se paga. Mas são estimativas: use faixas, registre as hipóteses e compare com o apetite a risco. Além disso, uma obrigação legal (como a LGPD) pode exigir o controle **independentemente do ROSI**.

03 RISCOS E MODELAGEM DE AMEAÇAS

CONTINUAÇÃO

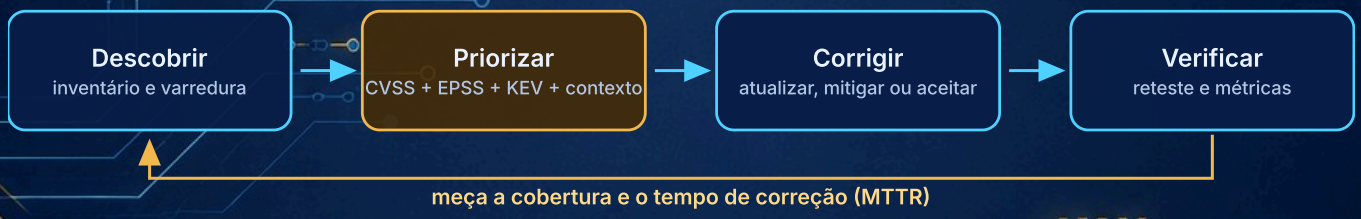
TRATAMENTO DO RISCO E REGISTRO DE RISCOS

OPÇÃO	O QUE É	EXEMPLO	QUANDO ESCOLHER
Mitigar (reduzir)	Aplicar controles que reduzem a probabilidade ou o impacto	MFA, correção de falhas, backup, segmentação	O custo do controle é menor que a redução do risco
Transferir (compartilhar)	Passar parte do impacto financeiro a terceiros	Seguro cibernético, cláusulas contratuais, serviço gerenciado	Impacto alto e probabilidade baixa; responsabilidade legal e reputação **não** se transferem
Evitar (eliminar)	Eliminar a atividade ou o ativo que gera o risco	Não coletar dado desnecessário; desligar sistema legado	O risco supera o benefício da atividade
Aceitar (reter)	Conviver com o risco de forma consciente e documentada	Risco baixo ou tratamento mais caro que a perda	Está dentro do apetite; tem dono, registro e revisão

Em resumo: o **registro de riscos** é o documento vivo da gestão. Campos mínimos: ID · ativo · ameaça · vulnerabilidade · probabilidade · impacto · nível · tratamento · dono · prazo · status · próxima revisão. Todo risco aceito precisa da **assinatura de quem tem autoridade para aceitá-lo**.

GESTÃO DE VULNERABILIDADES

Uma **vulnerabilidade** é uma fraqueza que pode ser explorada. Gerenciá-las é um **ciclo contínuo**, não um projeto com data de término. O volume é enorme — dezenas de milhares de CVEs por ano —, então o problema central é **priorizar**.

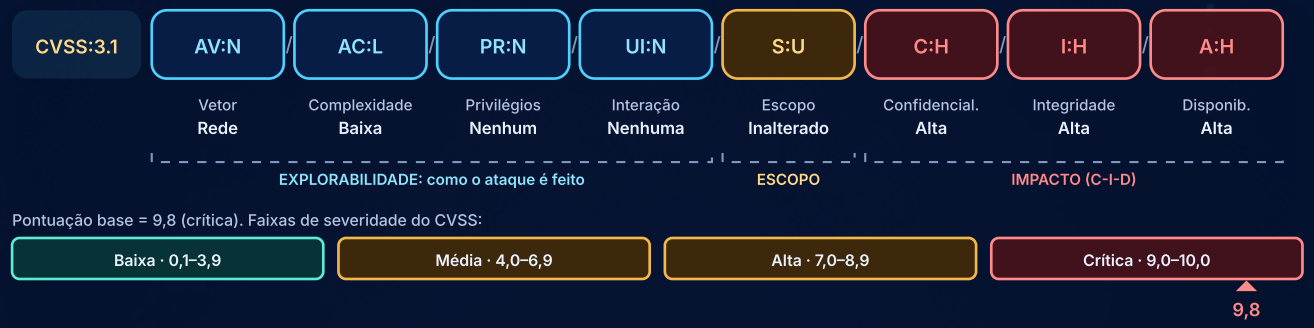


SIGLA	O QUE IDENTIFICA	MANTIDO POR	EXEMPLO
CVE	Uma vulnerabilidade específica em um produto	MITRE / CVE Program	CVE-2021-44228 (Log4Shell)
CWE	Um **tipo** de fraqueza de software ou hardware	MITRE	CWE-89 (injeção SQL)
CAPEC	Padrões de ataque: como uma fraqueza é explorada	MITRE	CAPEC-66 (injeção SQL)
CPE / NVD	Nome padronizado de produto e versão; base do NIST que enriquece CVEs com CVSS, CWE e CPE	NIST	nvd.nist.gov
CVSS	Gravidade técnica, de 0,0 a 10,0	FIRST	9,8 (crítica)
EPSS	Probabilidade de exploração nos próximos 30 dias, de 0 a 1	FIRST	0,91 = 91%
KEV	Catálogo de vulnerabilidades com exploração ativa confirmada	CISA	Está ou não está no catálogo

03 **RISCOS E MODELAGEM DE AMEAÇAS** CONTINUAÇÃO

CVSS: COMO LER UM VETOR

O **CVSS** (*Common Vulnerability Scoring System*, mantido pelo FIRST) descreve a **gravidade técnica** de uma vulnerabilidade com um **vetor** legível e uma nota de 0,0 a 10,0. A versão 3.1 tem três grupos: **Base** (propriedades intrínsecas), **Temporal** (maturidade do exploit, existência de correção) e **Ambiental** (importância do ativo no seu contexto). A **v4.0** (2023) renomeia o temporal para **Threat**, acrescenta os requisitos de ataque (*Attack Requirements*, AT), troca o *Scope* por impactos no sistema vulnerável (VC/VI/VA) e em sistemas subsequentes (SC/SI/SA) e inclui o grupo **Supplemental**.



Anatomia de um vetor CVSS 3.1. Cada par métrica:valor é uma pergunta sobre a vulnerabilidade. Este vetor (rede, baixa complexidade, sem privilégios nem interação, impacto alto em C, I e D) resulta em 9,8. O Log4Shell chega a 10,0 porque o escopo muda (S:C).

> **Atenção:** a nota **Base** mede severidade, **não risco**: ignora se há exploração real, se o serviço está exposto e quanto o ativo vale. Usar só o CVSS gera uma fila de "críticos" que ninguém consegue corrigir — e deixa passar o que realmente está sendo explorado.

Por isso somam-se sinais de contexto. O **EPSS** (FIRST) estima a probabilidade de exploração em 30 dias a partir de dados observados; o **KEV** (CISA) lista o que já está sendo explorado de fato. Juntos respondem à pergunta que o CVSS não responde: *"isso está sendo usado contra alguém agora?"*. O **SSVC** (CISA e Carnegie Mellon) vai além: é uma árvore de decisão que combina pontos como exploração, automatização, impacto técnico e impacto na missão, e termina em ações como acompanhar, atender ou agir.

VULNERABILIDADE	CVSS E EPSS	KEV, EXPOSIÇÃO E ATIVO	DECISÃO
A · execução remota em biblioteca interna	CVSS 9,8 · EPSS 0,02	Fora do KEV; rede interna isolada; ativo de baixo valor	P3: agendar no ciclo normal
B · falha de autenticação na VPN	CVSS 7,5 · EPSS 0,91	No KEV; exposta à internet; ativo crítico	P1: corrigir ou mitigar em 24 a 72 h
C · XSS no portal do aluno	CVSS 6,1 · EPSS 0,10	Fora do KEV; exposto; ativo médio	P2: corrigir em 15 a 30 dias

EXEMPLO RESOLVIDO

Leitura. B não tem a maior nota (A tem 9,8), mas é a mais urgente: exploração ativa (KEV), 91% de chance de exploração em 30 dias, borda da rede e ativo crítico. A, com a maior nota, fica por último: sem exploração conhecida e sem exposição. A ordem **B → C → A** inverte a ordem do CVSS.

Regra de bolso. Prioridade = exploração real (KEV, EPSS) + exposição + criticidade do ativo e, só então, a nota CVSS. Prazos são política de cada organização; como exemplo: explorada ativamente, em dias; crítica, 7 a 15 dias; alta, 30; média, 90.

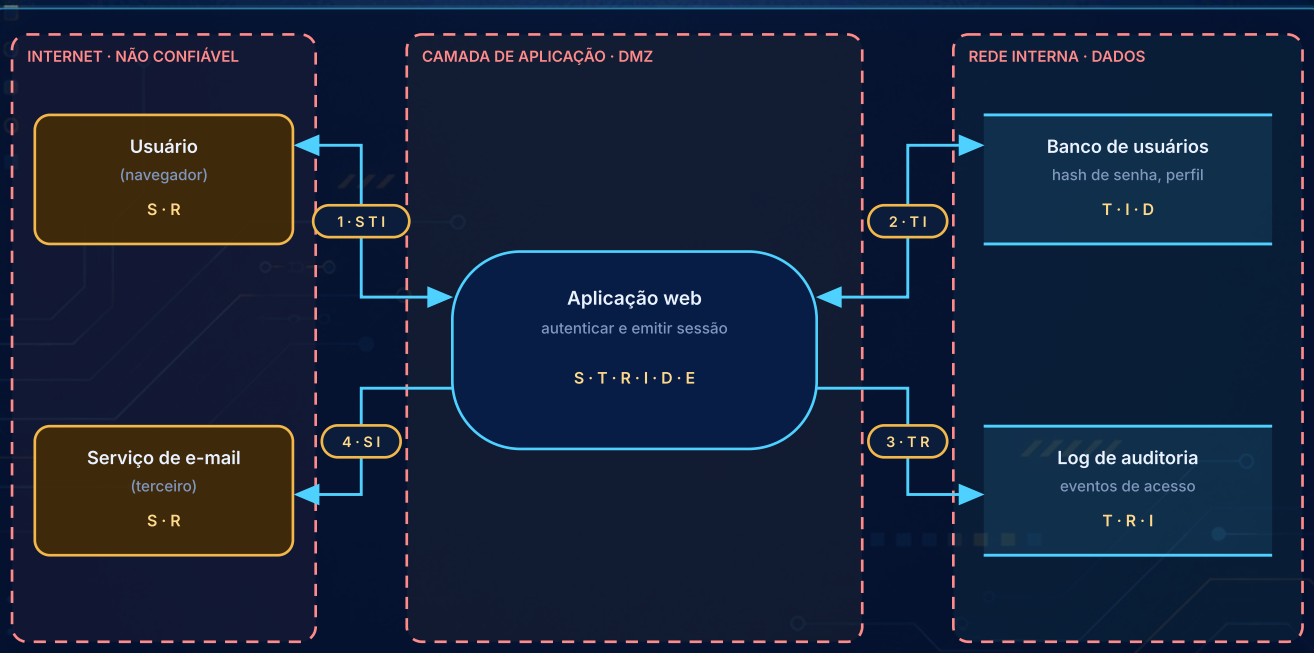
03 **RISCOS E MODELAGEM DE AMEAÇAS** CONTINUAÇÃO

MODELAGEM DE AMEAÇAS

Modelar ameaças é descobrir, ainda no projeto, **o que pode dar errado** e decidir o que fazer a respeito. É o controle mais barato do ciclo: corrigir uma falha de desenho em um diagrama custa horas; corrigi-la em produção custa semanas — e, às vezes, dados. Adam Shostack resume o método em **quatro perguntas**:

1. **O que estamos construindo?** Diagrama de fluxo de dados, ativos e fronteiras de confiança.
2. **O que pode dar errado?** STRIDE, árvores de ataque e casos de abuso.
3. **O que faremos a respeito?** Mitigar, transferir, evitar ou aceitar cada ameaça.
4. **Fizemos um bom trabalho?** Validar com testes, revisão por pares e métricas.

O **diagrama de fluxo de dados** (DFD) usa quatro elementos: **entidade externa** (quem está fora do seu controle), **processo** (código que transforma dados), **repositório** (onde os dados repousam) e **fluxo** (dados em trânsito). Sobre ele se desenham as **fronteiras de confiança**, linhas que separam níveis de confiança, e a regra de ouro é: **todo fluxo que cruza uma fronteira merece análise** — autenticação, validação, criptografia e registro.



DFD de um login web com fronteiras de confiança. As letras mostram as ameaças STRIDE aplicáveis a cada elemento (nas caixas) e a cada fluxo (nas pílulas numeradas): 1 credenciais e sessão, 2 consulta ao banco, 3 eventos de auditoria, 4 e-mail de redefinição de senha.

AMEAÇA	PROPRIEDADE VIOLADA	EXEMPLO NO LOGIN WEB	MITIGAÇÃO TÍPICA
Spoofing (falsificação de identidade)	Autenticidade	*Credential stuffing*; sessão roubada e reutilizada	MFA resistente a phishing, cookies `Secure; HttpOnly`, limite de tentativas
Tampering (adulteração)	Integridade	Alterar um parâmetro da requisição; alterar o banco ou o log	Validação no servidor, consultas parametrizadas, TLS, logs imutáveis

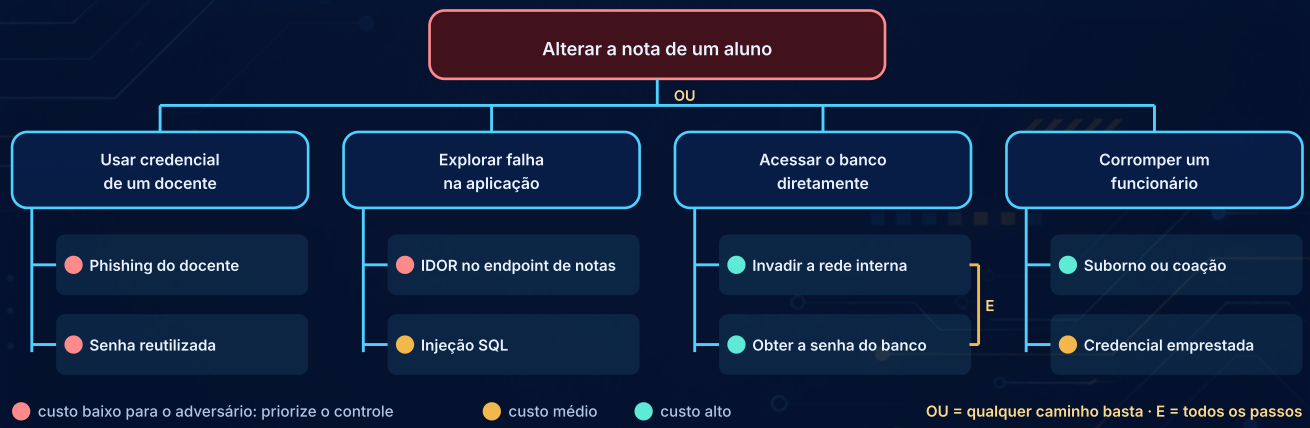
03 RISCOS E MODELAGEM DE AMEAÇAS — CONTINUAÇÃO

AMEAÇA	PROPRIEDADE VIOLADA	EXEMPLO NO LOGIN WEB	MITIGAÇÃO TÍPICA
Repudiation (repúdio)	Não repúdio	"Não fui eu que alterei a nota", sem registro confiável	Log de auditoria com identidade, hora (UTC) e integridade
Information disclosure (vazamento)	Confidencialidade	Erro que revela se o usuário existe; hash exposto em um *dump*	Mensagens genéricas, hash lento com sal, criptografia em repouso
Denial of service (negação de serviço)	Disponibilidade	Inundação de tentativas de login esgota o processo	*Rate limiting*, filas, limites por origem, *autoscaling* com teto
Elevation of privilege (elevação)	Autorização	Trocar `role=aluno` por `role=admin` e ser aceito	Autorização no servidor em cada operação, menor privilégio

STRIDE por elemento: nem toda ameaça se aplica a todo elemento do DFD. **Entidades externas**: S e R. **Processos**: as seis. **Repositórios**: T, I e D (e R, se guardam logs). **Fluxos**: T, I e D. As letras nas caixas e nas pílulas do diagrama mostram o que cada elemento pede.

ÁRVORES DE ATAQUE E OUTRAS ABORDAGENS

O STRIDE olha **cada elemento** do sistema; a **árvore de ataque** (Bruce Schneier, 1999) olha **o objetivo do adversário** e desce até os passos concretos. A raiz é o objetivo; os nós intermediários são caminhos alternativos (**OU**) ou passos obrigatórios em conjunto (**E**); as folhas recebem uma estimativa de **custo, habilidade ou probabilidade**. O ganho é prático: o caminho **mais barato** para o adversário é onde o controle rende mais.



Árvore de ataque: objetivo "alterar a nota de um aluno". O objetivo fica na raiz; cada ramo é um caminho alternativo (OU); o ramo do banco exige todos os passos (E). Os pontos vermelhos mostram os passos mais baratos para o adversário, onde o controle rende mais.

MÉTODO	FOCO	QUANDO USAR
Árvores de ataque (Schneier, 1999)	Objetivo do adversário na raiz; caminhos alternativos como ramos, com custo e probabilidade	Analisar um objetivo crítico (ex.: alterar notas)
PASTA	Processo de sete etapas orientado ao risco de negócio e à simulação de ataques	Quando há forte ligação com risco e negócio

03 RISCOS E MODELAGEM DE AMEAÇAS — CONTINUAÇÃO

MÉTODO	FOCO	QUANDO USAR
LINDDUN	Ameaças à privacidade: vinculação, identificação, não repúdio, detecção, divulgação, desconhecimento e não conformidade	Sistemas com dados pessoais (LGPD)
Casos de abuso (*abuse cases*)	Histórias de usuário "ao contrário": "como atacante, quero..."	Levantamento de requisitos de segurança
DREAD	Pontuação de dano, reprodutibilidade, explorabilidade, usuários afetados e descoberta	Obsoleto: notas subjetivas e inconsistentes; prefira risco e CVSS

Boa prática: trate o modelo como **artefato vivo**: guarde-o versionado junto ao código (por exemplo, com OWASP *Threat Dragon* ou *pytm*, que descreve o modelo como código), revise a cada mudança de arquitetura ou integração e ligue cada ameaça a um **requisito** e a um **teste**. O *Threat Modeling Manifesto* (2020) valoriza **pessoas e colaboração** acima de processos e ferramentas, e **fazer** a modelagem acima de falar sobre ela.



DICA DA CORUJA

comece pequeno: um fluxo por vez (login, pagamento, redefinição de senha). Um DFD que cabe em uma folha e gera dez perguntas incômodas já valeu a pena; o perfeito, que ninguém termina, não protege nada.

PARA FIXAR

Conceitos

- Diferencie apetite e tolerância ao risco, e risco inerente de residual.
- Por que o CVSS Base não basta para priorizar correções? O que EPSS e KEV acrescentam?
- Por que "todo fluxo que cruza uma fronteira de confiança merece análise"?

Aplicação

- Uma loja virtual tem AV = R$ 800.000, EF = 40% e ARO = 0,5. Calcule SLE e ALE; um controle de R$ 60.000 por ano reduz o ARO para 0,1. Vale a pena?
- Desenhe o DFD do fluxo "esqueci minha senha" e aplique STRIDE a cada elemento.
- Plote em uma matriz 5×5 quatro riscos do seu laboratório e proponha o tratamento de cada um.

APROFUNDE-SE

Leituras e referências

- NIST SP 800-30 Rev. 1 — NIST: guia para conduzir avaliações de risco.
- CVSS — FIRST: especificação, calculadoras e guia do usuário das versões 3.1 e 4.0.
- EPSS — FIRST: modelo, dados diários e documentação.
- Known Exploited Vulnerabilities Catalog — CISA: lista de vulnerabilidades com exploração ativa.
- Threat Modeling Manifesto — valores e princípios da modelagem de ameaças.
- OWASP Threat Dragon — OWASP: ferramenta de código aberto para diagramar e modelar ameaças.

MÓDULO 04 · GOVERNANÇA

04 NORMAS, LEIS E CONFORMIDADE

Frameworks, padrões e leis que dão estrutura à segurança — e a diferença entre estar conforme e estar seguro.

Segurança sem governança vira esforço individual: normas e leis dão **estrutura, linguagem comum e obrigações** ao que a equipe técnica faz. Este módulo mostra como as organizações governam a segurança da informação, os principais *frameworks* (ISO/IEC 27001, NIST CSF 2.0, CIS Controls, PCI DSS), o arcabouço legal brasileiro e uma distinção essencial: **estar em conformidade não é estar seguro**.

GOVERNANÇA, GESTÃO E OPERAÇÃO

Três níveis se complementam. A **governança** define direção e responsabilidade: a alta direção decide o apetite a risco, aprova a política e cobra resultados. A **gestão** planeja, organiza e mede: o CISO, o comitê de segurança e os donos de ativos transformam a direção em programa, orçamento e indicadores. A **operação** executa e monitora: TI, desenvolvimento, SOC e suporte aplicam os controles no dia a dia. Sem governança, ninguém é cobrado; sem gestão, não há priorização; sem operação, só existe papel.

DOCUMENTO	NATUREZA	RESPONDE A	EXEMPLO
Política	Obrigatória; curta e estável; aprovada pela direção	O quê e por quê	Política de Segurança da Informação: princípios, papéis e sanções
Norma	Obrigatória; requisitos objetivos e verificáveis	Quanto e quais	Norma de senhas e de MFA; norma de classificação da informação
Procedimento	Obrigatório; passo a passo operacional	Como	Revogação de acessos no desligamento de um funcionário
Diretriz (guia)	Recomendada; boas práticas e orientação	Como fazer melhor	Guia de trabalho remoto seguro; guia de desenvolvimento seguro



DICA DA CORUJA

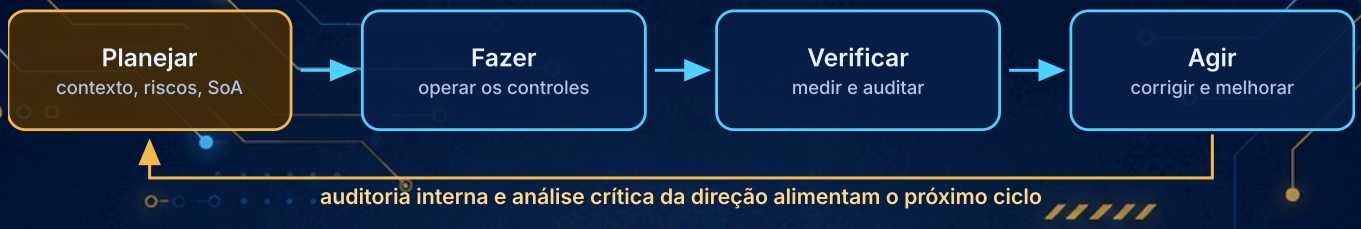
uma política que ninguém consegue cumprir é pior do que nenhuma: vira "letra morta" e ensina a equipe a ignorar regras. Escreva política **curta**, norma **verificável** e procedimento **testado por quem vai usá-lo**; revise tudo uma vez por ano e depois de cada incidente relevante.

ISO/IEC 27001:2022 E A FAMÍLIA 27000

A **ISO/IEC 27001** especifica os requisitos de um **Sistema de Gestão de Segurança da Informação** (SGSI, *ISMS*). Não é uma lista de ferramentas: é um **sistema de gestão** que decide o que proteger, avalia riscos, escolhe controles, mede e melhora continuamente — o ciclo **PDCA** (*Plan-Do-Check-Act*). É a norma **certificável** do tema: um organismo de certificação acreditado audita em duas etapas (documentação e implementação), emite um certificado com validade de três anos e realiza auditorias de manutenção anuais. A transição da edição de 2013 para a de 2022 terminou em **31 de outubro de 2025**.

04 NORMAS, LEIS E CONFORMIDADE

CONTINUAÇÃO



CLÁUSULA	PDCA	O QUE EXIGE
4 · Contexto da organização	Planejar	Entender o contexto e as partes interessadas; definir o **escopo** do SGSI
5 · Liderança	Planejar	Compromisso da direção; **política** de segurança; papéis e responsabilidades
6 · Planejamento	Planejar	Avaliar e tratar **riscos**; produzir a **Declaração de Aplicabilidade**; fixar objetivos
7 · Apoio	Fazer	Recursos, competência, conscientização, comunicação e informação documentada
8 · Operação	Fazer	Executar o plano de tratamento de riscos e as avaliações de risco planejadas
9 · Avaliação de desempenho	Verificar	Monitoramento e medição, **auditoria interna** e análise crítica pela direção
10 · Melhoria	Agir	Tratar não conformidades, executar ações corretivas e melhorar continuamente

Em resumo: a **Declaração de Aplicabilidade** (*Statement of Applicability*, SoA) lista cada controle do Anexo A, diz se é aplicável, justifica inclusões e exclusões e informa o estado de implementação. É o documento que o auditor lê primeiro, e ela só faz sentido se nascer da **avaliação de riscos** — controle não se escolhe por moda.

TEMA DO ANEXO A	Nº	COBRE	EXEMPLOS
Organizacionais (A.5)	37	Políticas, papéis, ativos, acesso, fornecedores, incidentes, continuidade, requisitos legais	Inteligência de ameaças; segurança de serviços em nuvem; prontidão de TIC para continuidade
Pessoas (A.6)	8	Antes, durante e depois do vínculo com a organização	Triagem; termos de contratação; conscientização; trabalho remoto; reporte de eventos
Físicos (A.7)	14	Perímetros, áreas seguras, equipamentos e mídias	Monitoramento da segurança física; mesa e tela limpas; descarte seguro
Tecnológicos (A.8)	34	Dispositivos, identidades, redes, criptografia e desenvolvimento	Gestão de configuração; mascaramento de dados; DLP; monitoramento; filtragem web; codificação segura

A edição de 2022 reduziu os controles de 114 (em 14 domínios) para **93 em 4 temas** e acrescentou **11 novos**: inteligência de ameaças, segurança de nuvem, prontidão de TIC para continuidade, monitoramento da segurança física, gestão de configuração, exclusão de informação, mascaramento de dados, prevenção de vazamento de dados, monitoramento de atividades, filtragem web e codificação segura. Em volta dela orbita uma **família**: a **ISO/IEC 27002** (orientação de implementação dos controles), a **27005** (riscos), a **27017** e a **27018** (nuvem e dados pessoais em nuvem pública), a **27701** (extensão de privacidade), a **27035** (incidentes) e a **ISO 22301** (continuidade de negócios).

04 NORMAS, LEIS E CONFORMIDADE — CONTINUAÇÃO

NIST CSF 2.0, SP 800-53 E CIS CONTROLS

O **NIST Cybersecurity Framework 2.0** (fevereiro de 2024) é um vocabulário de **resultados** em seis funções, com 22 categorias e 106 subcategorias. Não é certificável nem prescreve ferramentas: ajuda a descrever o que se quer alcançar e a conversar com a diretoria. A novidade é a função **Govern**, que coloca estratégia, papéis e gestão de riscos no centro. Duas ideias o tornam prático: os **Perfis** (*Profiles*: situação atual × situação-alvo, com o plano para fechar a diferença) e os **Níveis** (*Tiers*: parcial, ciente de risco, repetível e adaptativo).

GV · GOVERN · GOVERNAR
Estratégia, papéis, políticas, gestão de riscos e da cadeia de suprimentos.

ID · IDENTIFY · IDENTIFICAR
Ativos, riscos e oportunidades de melhoria que a organização precisa entender.

PR · PROTECT · PROTEGER
Salvaguardas: identidade e acesso, treinamento, segurança de dados e de plataformas.

DE · DETECT · DETECTAR
Monitoramento contínuo e análise de eventos adversos para achar ataques.

RS · RESPOND · RESPONDER
Gestão, análise, comunicação e mitigação dos incidentes detectados.

RC · RECOVER · RECUPERAR
Restaurar ativos e operações afetados e comunicar durante a recuperação.

No nível dos **controles**, o **NIST SP 800-53 Rev. 5** é o catálogo mais granular (20 famílias, como AC — controle de acesso, AU — auditoria e IR — resposta a incidentes), base de autorizações do governo dos EUA; o **SP 800-171** protege informações não classificadas controladas (CUI) em sistemas de fornecedores. Os **CIS Controls v8.1** (junho de 2024) trazem **18 controles** priorizados com base em dados de ataques reais, em três **Grupos de Implementação**: IG1 (higiene cibernética essencial, para equipes pequenas), IG2 e IG3 (organizações maiores e mais expostas). Os **CIS Benchmarks** completam o quadro com configurações seguras por produto (sistemas operacionais, nuvem, navegadores).

COMPARANDO FRAMEWORKS E ESCOLHENDO POR ONDE COMEÇAR



Mapa dos frameworks: cada um responde a uma pergunta diferente. Eles se complementam, não competem: obrigações legais definem o piso; o CSF organiza resultados; a ISO 27001 sustenta o sistema de gestão; os catálogos detalham os controles; os modelos de maturidade medem a evolução.

04 NORMAS, LEIS E CONFORMIDADE

CONTINUAÇÃO

FRAMEWORK	FINALIDADE	CERTIFICÁVEL?	QUANDO USAR
ISO/IEC 27001:2022	Requisitos de um SGSI e controles do Anexo A (93)	Sim, por organismo acreditado	Demonstrar gestão madura a clientes e reguladores
NIST CSF 2.0	Resultados em seis funções; perfis e níveis	Não (autoavaliação)	Estruturar o programa e falar com a diretoria
NIST SP 800-53 Rev. 5	Catálogo detalhado de controles (20 famílias)	Não; base de autorizações do governo dos EUA	Alta granularidade; sistemas governamentais
CIS Controls v8.1	18 controles priorizados; grupos IG1 a IG3	Não	Começar pelo essencial com orçamento limitado
PCI DSS v4.0.1	12 requisitos para dados de cartões de pagamento	Validação exigida por contrato	Quem armazena, processa ou transmite dados de cartão
SOC 2 (AICPA)	Relatório de auditor independente sobre os controles de um serviço	Atestação (relatório Tipo I ou II)	Fornecedores de SaaS que precisam provar controles

SOC 2 e PCI DSS em uma linha: o **SOC 2** avalia controles segundo cinco critérios de confiança — segurança (obrigatório), disponibilidade, integridade de processamento, confidencialidade e privacidade; o **Tipo I** atesta o desenho em uma data e o **Tipo II** atesta a eficácia ao longo de um período. O **PCI DSS** tem 12 requisitos; os requisitos "futuros" da versão 4.0 tornaram-se obrigatórios em 31 de março de 2025.

1 Comece pela obrigação
Leis (LGPD), regulação do setor e cláusulas de contrato definem o **piso inegociável**.

2 Estruture com resultados
Use o CSF 2.0 para descrever a situação **atual e a alvo** e comunicar com a gestão.

3 Escolha o catálogo de controles
CIS IG1 para começar; ISO/IEC 27002 ou NIST SP 800-53 para detalhar.

4 Prove e meça
ISO 27001 ou SOC 2 para **demonstrar a terceiros**; SAMM, BSIMM ou Tiers para medir a maturidade.

Maturidade: o **CMMI** descreve níveis de maturidade de processos; o **OWASP SAMM v2** é um modelo *prescritivo* de segurança de software (5 funções de negócio, 15 práticas, 3 níveis de maturidade); o **BSIMM** é *descritivo*: mede o que dezenas de empresas realmente fazem. Use-os para comparar-se e planejar, não para colecionar notas.

BRASIL: O QUE A LEI E OS REGULADORES EXIGEM

NORMA	TRATA DE	O QUE EXIGE EM SEGURANÇA
LGPD · Lei nº 13.709/2018	Tratamento de dados pessoais	Segurança como princípio (art. 6º, VII); medidas técnicas e administrativas (arts. 46 e 47); comunicação de incidentes (art. 48); sanções (art. 52)
Resolução CD/ANPD nº 15/2024	Comunicação de incidente de segurança	Comunicar a ANPD e os titulares em até **3 dias úteis** quando houver risco ou dano relevante; manter o registro por, no mínimo, 5 anos
Marco Civil · Lei nº 12.965/2014	Direitos e deveres na internet	Guarda de registros de conexão por 1 ano (art. 13) e de acesso a aplicações por 6 meses (art. 15), para quem se enquadra

04 NORMAS, LEIS E CONFORMIDADE

CONTINUAÇÃO

NORMA	TRATA DE	O QUE EXIGE EM SEGURANÇA
Código Penal, art. 154-A (Leis nº 12.737/2012 e 14.155/2021)	Invasão de dispositivo informático	Crime: reclusão de 1 a 4 anos e multa; 2 a 5 anos nas hipóteses do §3º; fraude eletrônica (art. 171, §2º-A): 4 a 8 anos
Resoluções CMN nº 4.893/2021 e BCB nº 85/2021	Segurança cibernética no setor financeiro	Política de segurança cibernética, plano de resposta a incidentes e requisitos para contratar serviços em nuvem e terceiros

O Decreto nº 11.856/2023 instituiu a **Política Nacional de Cibersegurança** (PNCiber). Na prática do dia a dia, o **CERT.br/NIC.br** (notificação e orientação; autor da *Cartilha de Segurança para Internet*) e o **CTIR Gov** (resposta a incidentes do governo federal) são as referências de coordenação. No meio acadêmico, vale conhecer também a **RNP**, que mantém o seu próprio centro de resposta.



Atores da LGPD e como se relacionam. O titular exerce direitos perante o controlador; o controlador instrui o operador e comunica incidentes; o encarregado é o canal de comunicação; a ANPD regula, fiscaliza e sanciona.

Na linguagem da LGPD, controlador e operador são os **agentes de tratamento**. O controlador responde pelas decisões e pelo cumprimento da lei; o operador, pelas instruções recebidas e pelas medidas de segurança. Bases legais, direitos do titular, relatório de impacto e anonimização são detalhados em p. 59; a resposta a incidentes com dados pessoais, em p. 156. Fora do Brasil, três regimes aparecem com frequência em contratos e questionários:

NORMA	ESCOPO	PONTOS DE ATENÇÃO
GDPR · Regulamento (UE) 2016/679	Proteção de dados pessoais na UE; alcança quem oferece bens ou serviços a pessoas na UE	Multas de até € 20 milhões ou 4% do faturamento global; notificação à autoridade em até 72 h
NIS2 · Diretiva (UE) 2022/2555	Segurança de redes e sistemas de entidades essenciais e importantes	Gestão de riscos; notificação de incidentes significativos (alerta inicial em 24 h e notificação em 72 h)
CRA · Regulamento (UE) 2024/2847	Requisitos de segurança para produtos com elementos digitais	Em vigor desde dez/2024; notificação de vulnerabilidades exploradas a partir de set/2026 e obrigações principais a partir de dez/2027, segundo o calendário publicado

04 NORMAS, LEIS E CONFORMIDADE

CONTINUAÇÃO

Atenção: este guia é material de estudo, **não aconselhamento jurídico**. Prazos e multas mudam; a interpretação de uma lei em um caso concreto cabe à assessoria jurídica e ao encarregado.

CONFORMIDADE NÃO É SEGURANÇA

Conformidade

- Responde a **requisitos externos** (lei, norma, contrato)
- Olha o **passado**: auditoria por amostragem, em um instante
- É binária: está ou não está conforme
- Produz **evidências**: políticas, registros, relatórios

Segurança

- Responde a **adversários e riscos reais**
- Olha o **presente e o futuro**: detecção e resposta contínuas
- É gradual: sempre dá para melhorar, e nada é 100%
- Produz **resultados**: menos incidentes, detecção mais rápida

A **conformidade é o piso, não o teto**: cumprir um requisito de "ter MFA" não diz se ele cobre as contas privilegiadas nem se resiste a *phishing*. O vício a evitar é o *checkbox compliance*, marcar caixinhas para passar na auditoria. Trate a auditoria como um **teste do sistema**: a **interna** (de primeira parte) corrige antes que apareça lá fora; a de **segunda parte** é a que o cliente faz no fornecedor; a de **terceira parte** (certificação ou relatório independente) dá confiança a quem não pode auditar. Quando um requisito não puder ser cumprido, use **controle compensatório** documentado; e gerencie **exceções** com dono, justificativa, risco aceito, prazo e data de revisão — exceção sem prazo vira regra.

INDICADOR	TIPO	O QUE MOSTRA
% de ativos críticos com EDR e inventário atualizado	KPI	Cobertura dos controles essenciais
Tempo mediano de correção de vulnerabilidades críticas	KPI	Eficiência da gestão de vulnerabilidades
% de contas privilegiadas com MFA resistente a phishing	KPI	Proteção das identidades de maior risco
Tempo médio de detecção e de resposta (MTTD e MTTR)	KPI	Maturidade do monitoramento e da resposta
Vulnerabilidades do KEV ainda abertas	KRI	Exposição a ataques em curso
Exceções vencidas e riscos aceitos sem revisão	KRI	Governança frágil e dívida de segurança

Em resumo: **KPI** (*Key Performance Indicator*) mede o desempenho de um controle; **KRI** (*Key Risk Indicator*) avisa que o risco está subindo. Bons indicadores são **mensuráveis, comparáveis no tempo e ligados a uma decisão**: se ninguém age quando o número piora, ele é decoração.

04 NORMAS, LEIS E CONFORMIDADE

CONTINUAÇÃO

ESTUDO DE CASO

Cenário. Uma startup de provas online (SaaS) fecha contrato com uma universidade brasileira. Guarda notas e dados cadastrais de alunos, alguns adolescentes (dados de crianças e adolescentes pedem cuidado adicional, art. 14 da LGPD). O contrato prevê **questionário de segurança** e **aviso de incidentes em 48 horas**; em breve, um cliente europeu pedirá certificação.

Papéis. A universidade é **controladora**; a startup é **operadora**, segue as instruções do contrato e responde pela segurança do que processa. Quem comunica à ANPD e aos titulares é a controladora, então o contrato precisa obrigar a operadora a avisar **rápido**.

Roteiro em quatro fases. (1) **Obrigações**: contrato de tratamento, inventário de dados, encarregado e plano de resposta que caiba nas 48 h do contrato e nos 3 dias úteis da ANPD. (2) **Controles essenciais**: CIS IG1 — inventário, MFA, backup, correção de falhas, logs. (3) **Programa**: CSF 2.0 com perfil atual e alvo. (4) **Prova a terceiros**: SOC 2 (Tipo I e depois II) ou ISO/IEC 27001, sempre com evidência de que os controles **funcionam**, não só existem.

PARA FIXAR

Conceitos

- Diferencie política, norma, procedimento e diretriz com um exemplo de cada.
- O que é a Declaração de Aplicabilidade e por que o auditor a lê primeiro?
- Por que "estar em conformidade" não implica "estar seguro"?

Aplicação

- Uma clínica com 20 funcionários guarda prontuários. Quais obrigações brasileiras se aplicam e por onde você começaria (CIS IG1, CSF, ISO 27001)?
- Monte um quadro com cinco indicadores (três KPIs e dois KRIs) para um programa de gestão de vulnerabilidades.
- Um vazamento de dados de alunos foi descoberto na segunda-feira. Que prazos correm e quem precisa ser notificado?

APROFUNDE-SE

Leituras e referências

- ISO/IEC 27001 — ISO: página oficial da norma e de seu histórico de edições.
- NIST Cybersecurity Framework — NIST: CSF 2.0, guias rápidos e exemplos de implementação.
- CIS Critical Security Controls — CIS: controles, grupos de implementação e mapeamentos.
- Lei nº 13.709/2018 (LGPD) — Planalto: texto integral e atualizado.
- Autoridade Nacional de Proteção de Dados — gov.br: regulamentos, guias e a Resolução CD/ANPD nº 15/2024.
- PCI Security Standards Council — PCI SSC: padrões e documentos de apoio ao PCI DSS.

MÓDULO 05 · CRIPTOGRAFIA

05 CRIPTOGRAFIA APLICADA

Primitivas, protocolos e erros clássicos: como usar criptografia sem inventar a própria.

A criptografia transforma **confiança em matemática**: protege sigilo, integridade e autenticidade mesmo quando o canal, o disco ou o provedor não merecem confiança. Este módulo percorre as primitivas (cifras, *hashes*, MACs e assinaturas), mostra como combiná-las em construções e protocolos (AEAD, *envelope encryption*, TLS 1.3), explica a infraestrutura de chaves públicas e lista os erros que quebram sistemas reais: quase nunca é o algoritmo, é o **uso**.

PRIMITIVAS, CONSTRUÇÕES E PROTOCOLOS

A criptografia moderna tem três camadas. As **primitivas** são blocos com propriedades matemáticas bem definidas (cifra de bloco, função de *hash*, troca de chaves). As **construções** combinam primitivas de forma padronizada (cifragem autenticada, HMAC, assinatura com preenchimento). Os **protocolos** orquestram tudo em uma conversa completa (TLS, SSH, Signal). O **princípio de Kerckhoffs** (1883) diz que a segurança deve depender **somente da chave**, nunca do segredo do algoritmo; por isso se usam algoritmos públicos, analisados por anos em concursos abertos (AES, SHA-3, criptografia pós-quântica). E vale a regra mais repetida da área: **não invente a sua própria criptografia**. Segundo a *lei de Schneier*, qualquer pessoa consegue criar um algoritmo que ela mesma não consegue quebrar.

PRIMITIVA	GARANTE	NÃO GARANTE	EXEMPLO
Cifra simétrica (AEAD)	Sigilo e integridade, com chave compartilhada	Quem enviou, se várias partes têm a chave	AES-256-GCM
Função de *hash*	Resumo de tamanho fixo; detecta alteração	Autenticidade: qualquer um recalcula o resumo	SHA-256
MAC / HMAC	Integridade e autenticidade entre quem divide a chave	Não repúdio: os dois lados geram a etiqueta	HMAC-SHA-256
Assinatura digital	Integridade, autenticidade e **não repúdio**	Sigilo: a mensagem segue legível	Ed25519, RSA-PSS
Acordo de chaves	Segredo compartilhado sobre canal aberto	Identidade do par (exige certificado ou assinatura)	X25519 (ECDH)
Cifra de chave pública / KEM	Sigilo para quem detém a chave privada	Eficiência com dados grandes (use o modo híbrido)	RSA-OAEP, ML-KEM

Atenção: cifrar não é o mesmo que proteger. Um AES-256 impecável, com a chave no repositório, não protege nada. A segurança de um sistema criptográfico é a do seu **elo mais fraco**, e esse elo costuma ser a gestão de chaves, a aleatoriedade ou a validação de certificados, não o algoritmo.

05 **CRIPTOGRAFIA APLICADA** CONTINUAÇÃO

CRIPTOGRAFIA SIMÉTRICA E CIFRAGEM AUTENTICADA (AEAD)

Cifras simétricas usam a **mesma chave** para cifrar e decifrar. As **cifras de bloco**, como o **AES** (FIPS 197; bloco de 128 bits e chaves de 128, 192 ou 256 bits), processam blocos fixos e precisam de um **modo de operação** para mensagens maiores. As **cifras de fluxo**, como o **ChaCha20**, geram uma sequência pseudoaleatória combinada por XOR com o texto. O ponto decisivo: **cifrar sem autenticar é perigoso**. Sem etiqueta de integridade, o atacante altera o texto cifrado de modo previsível (*maleabilidade*) ou explora as respostas de erro do servidor (*padding oracle*, contra o CBC).

A solução moderna é a **cifragem autenticada com dados associados** (*Authenticated Encryption with Associated Data*, **AEAD**). A operação `Enc(chave, nonce, claro, AAD)` devolve o texto cifrado e uma **etiqueta de autenticação**; a decifração **verifica a etiqueta antes de liberar qualquer byte**. O **AAD** (dados associados) é autenticado, mas não cifrado: amarra o dado ao seu contexto (ID do registro, nome do campo, versão do esquema) e impede trocar o cifrado de um registro pelo de outro. O **nonce** (*number used once*) não é segredo, mas **não pode se repetir** com a mesma chave.

MODO	GARANTE	ARMADILHA	VEREDITO
ECB	Só sigilo, bloco a bloco	Blocos iguais geram cifrados iguais: o padrão da imagem aparece	Nunca usar
CBC	Sigilo, se o IV for imprevisível	Sem integridade; *padding oracle*; IV previsível (CWE-329)	Legado; só com MAC
CTR	Sigilo (modo de fluxo)	Sem integridade: inverter um bit do cifrado inverte o mesmo bit do claro	Prefira AEAD
AES-GCM	Sigilo + integridade + AAD	Nonce de 96 bits **nunca** pode repetir com a mesma chave	Padrão
ChaCha20-Poly1305	Idem (RFC 8439); rápido sem aceleração de AES	Mesma regra do nonce; XChaCha20 oferece nonce de 192 bits	Padrão
AES-GCM-SIV	AEAD resistente a reuso de nonce (RFC 8452)	Mais lento; um reuso só revela que duas mensagens são iguais	Nonce difícil de gerir

AES-256-GCM COM A BIBLIOTECA CRYPTOGRAPHY (PYTHON)

```
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.exceptions import InvalidTag

def cifrar(chave: bytes, claro: bytes, aad: bytes) -> bytes:
    nonce = os.urandom(12)                    # 96 bits, novo a cada mensagem
    return nonce + AESGCM(chave).encrypt(nonce, claro, aad)   # cifrado + etiqueta (16 B)

def decifrar(chave: bytes, blob: bytes, aad: bytes) -> bytes:
    nonce, cifrado = blob[:12], blob[12:]
    try:
        return AESGCM(chave).decrypt(nonce, cifrado, aad)
    except InvalidTag:                        # chave errada, dado adulterado ou AAD distinto
        raise ValueError("falha de integridade")   # não detalhe o motivo ao cliente

chave = AESGCM.generate_key(bit_length=256)   # em produção: vem do KMS, nunca do código
aad = b"aluno:42|campo:cpf|v1"                # amarra o dado ao registro e ao campo
blob = cifrar(chave, b"123.456.789-09", aad)
assert decifrar(chave, blob, aad) == b"123.456.789-09"
# decifrar(chave, blob, b"aluno:43|campo:cpf|v1")  ->  ValueError: o AAD não confere
```

05 CRIPTOGRAFIA APLICADA — CONTINUAÇÃO

Nonce repetido quebra o GCM: com a mesma chave e o mesmo nonce, o atacante obtém o **XOR dos dois textos claros** e, a partir de duas mensagens autenticadas, consegue recuperar a chave de autenticação e **forjar etiquetas**. Com nonces aleatórios de 96 bits, o NIST (SP 800-38D) limita o uso a 2^{32} **mensagens por chave**. Acima disso: uma chave por registro (veja a cifragem em envelope), XChaCha20-Poly1305 ou AES-GCM-SIV.

HASH, MAC E DERIVAÇÃO DE CHAVES

Uma **função de *hash* criptográfica** mapeia dados de qualquer tamanho a um resumo de tamanho fixo e deve oferecer **resistência à pré-imagem** (dado h, não achar m com $H(m) = h$), **à segunda pré-imagem** (dado m, não achar outro m' com o mesmo resumo) e **a colisões** (não achar quaisquer $m \neq m'$ com o mesmo resumo). Pelo paradoxo do aniversário, achar uma colisão custa cerca de $2^{n/2}$: o SHA-256 oferece ~128 bits de resistência a colisões. **MD5** (colisões desde 2004; certificado falso de AC em 2008) e **SHA-1** (colisão prática, *SHAttered*, 2017) **não servem** para assinaturas, certificados nem integridade diante de um adversário. O padrão geral é a família **SHA-2** (FIPS 180-4); o **SHA-3** (FIPS 202, Keccak) tem construção diferente; **BLAKE2** e **BLAKE3** são rápidas em software.

Um *hash* simples **não autentica**: qualquer um o recalcula. Para isso existe o **MAC** (*Message Authentication Code*). E `SHA256(chave + mensagem)` **não é um MAC seguro**: os *hashes* da família Merkle–Damgård (MD5, SHA-1, SHA-2) sofrem **extensão de comprimento** (dado o resumo de *segredo* ‖ *m*, calcula-se o de *segredo* ‖ *m* ‖ *preenchimento* ‖ *x* sem conhecer o segredo). O **HMAC** (RFC 2104, FIPS 198-1), definido como $H((K \oplus opad) \parallel H((K \oplus ipad) \parallel m))$, elimina o problema. Para **derivar chaves** de um segredo de alta entropia, como o resultado de um ECDH, use o **HKDF** (RFC 5869): a etapa *extract* concentra a entropia e a etapa *expand* gera uma subchave por finalidade, separadas pelo parâmetro `info`. Senhas humanas têm pouca entropia: para elas servem KDFs **lentos e com uso de memória** (Argon2id, scrypt, bcrypt, PBKDF2), em p. 44.

NECESSIDADE	USE	EVITE
Conferir integridade de um download	SHA-256 publicado em canal autenticado, de preferência com assinatura	MD5 ou SHA-1; resumo servido pelo mesmo site sem HTTPS
Autenticar webhook, cookie ou token	HMAC-SHA-256, chave ≥ 32 bytes, comparação em tempo constante	`SHA256(chave + msg)`; comparação com `==`
Derivar subchaves	HKDF-SHA-256, com `info` diferente por finalidade	Usar a mesma chave para fins distintos
Guardar senhas	Argon2id, com sal único (p. 44)	SHA-256 simples, MD5, bcrypt sem custo mínimo
Assinar documentos e certificados	*Hash* SHA-256 ou superior dentro de RSA-PSS, ECDSA ou Ed25519	Assinar com SHA-1; assinar sem *hash*

VERIFICAÇÃO DE WEBHOOK COM HMAC E COMPARAÇÃO EM TEMPO CONSTANTE

```
import hashlib, hmac, os, time

SEGREDO = bytes.fromhex(os.environ["WEBHOOK_SECRET_HEX"])    # 32 bytes aleatórios ou mais

def assinatura_valida(corpo: bytes, assinatura_hex: str, timestamp: str) -> bool:
    if not timestamp.isdigit() or abs(time.time() - int(timestamp)) > 300:
        return False                                  # janela de 5 minutos contra replay
    msg = timestamp.encode() + b"." + corpo           # o tempo entra na parte autenticada
    esperado = hmac.new(SEGREDO, msg, hashlib.sha256).hexdigest()
    return hmac.compare_digest(esperado, assinatura_hex)   # tempo constante; nunca use ==
```

05 **CRIPTOGRAFIA APLICADA** CONTINUAÇÃO

CSPRNG: de onde vêm os bytes aleatórios

Tokens, IDs de sessão, nonces e chaves exigem um **gerador criptograficamente seguro**, com **≥ 128 bits** de entropia:

- Python: `secrets`, `os.urandom`
- Node.js: `crypto.randomBytes`, `crypto.randomUUID`
- Java: `SecureRandom`; Go: `crypto/rand`
- Linux: `getrandom(2)` e `/dev/urandom`

Nunca `random`, `Math.random()` ou `rand()` (CWE-338): são previsíveis.

Comparação em tempo constante

Comparar segredos com `==` termina no primeiro byte diferente, e o tempo de resposta vaza quantos bytes acertaram (*timing side channel*, CWE-208). Use:

- Python: `hmac.compare_digest`
- Node.js: `crypto.timingSafeEqual`
- Java: `MessageDigest.isEqual`
- Go: `subtle.ConstantTimeCompare`
- PHP: `hash_equals`

CRIPTOGRAFIA ASSIMÉTRICA, TROCA DE CHAVES E ASSINATURAS

Na criptografia de **chave pública** cada parte tem um par: a chave **pública** (divulgável) e a **privada** (que nunca sai do dono). Dois problemas difíceis sustentam quase tudo: a **fatoração** de inteiros (RSA) e o **logaritmo discreto**, em campos finitos (Diffie–Hellman) ou em **curvas elípticas** (ECDH, ECDSA, EdDSA). As curvas elípticas dão o mesmo nível de segurança com chaves bem menores: ~128 bits de segurança equivalem a RSA de 3072 bits ou a uma curva de 256 bits (NIST SP 800-57 Parte 1).

Acordo de chaves. No **Diffie–Hellman** duas partes derivam um segredo compartilhado a partir de chaves públicas trocadas em canal aberto. Com chaves **efêmeras** (ECDHE), cada sessão usa um par novo, descartado ao final: se a chave privada de longo prazo vazar depois, as sessões gravadas **continuam ilegíveis**. Essa propriedade é o **sigilo perfeito de encaminhamento** (*forward secrecy*). O DH sozinho não autentica ninguém: um atacante no meio troca chaves com cada lado (*man-in-the-middle*); a identidade vem da **assinatura** ou do **certificado** do servidor.

Assinaturas. Assinar é aplicar a chave privada ao *hash* da mensagem; qualquer um verifica com a pública. Os padrões atuais são **RSA-PSS**, **ECDSA** (P-256) e **Ed25519** (EdDSA, RFC 8032). O **ECDSA exige um número aleatório *k* único por assinatura**: repetir ou prever *k* revela a chave privada (foi assim que se recuperou a chave de assinatura do PlayStation 3, em 2010). O Ed25519 deriva *k* da chave e da mensagem, o que elimina essa classe de erro. **Cifrar com RSA** só vale para cargas pequenas, como uma chave simétrica, e com **OAEP**; o preenchimento PKCS#1 v1.5 sofre ataques de oráculo desde Bleichenbacher (1998). Na prática usa-se a **cifragem híbrida**: uma chave simétrica aleatória cifra os dados, e a chave pública protege só essa chave. Esse é o princípio do **KEM** (*Key Encapsulation Mechanism*), base do HPKE (RFC 9180) e do ML-KEM.

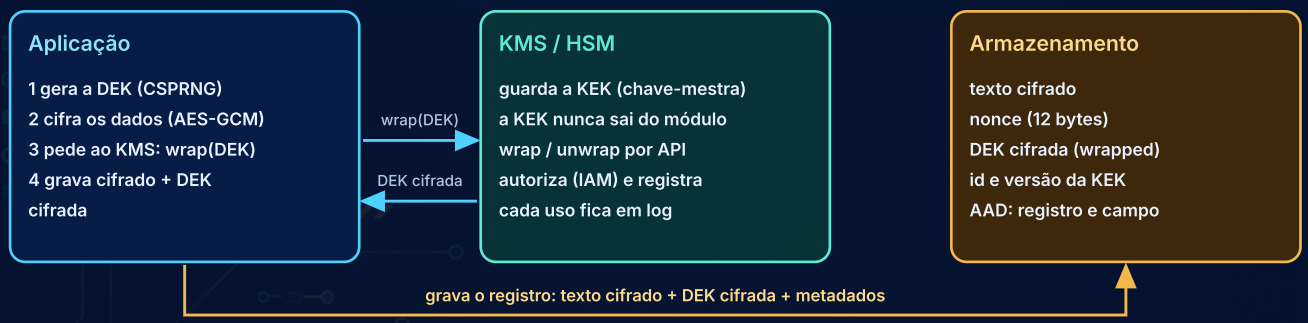
ALGORITMO	USO	PARÂMETRO RECOMENDADO	SITUAÇÃO
AES-GCM	Dados em repouso e em trânsito	Chave de 128 ou 256 bits; nonce de 96 bits único	Recomendado
ChaCha20-Poly1305	Idem, sem aceleração de AES	Chave de 256 bits; nonce de 96 bits (RFC 8439)	Recomendado
SHA-256, 384, 512 e SHA-3	Resumo, integridade, assinatura	Saída ≥ 256 bits para novos projetos	Recomendado
HMAC-SHA-256	Autenticação de mensagens	Chave ≥ 32 bytes aleatórios	Recomendado
ECDH e ECDSA	Acordo de chaves e assinatura	Curvas P-256 ou P-384	Recomendado
X25519 e Ed25519	Acordo de chaves e assinatura	Curvas 25519 (RFC 7748 e 8032)	Recomendado
RSA	Assinatura (PSS) e transporte de chave (OAEP)	≥ 2048 bits; 3072 para uso além de ~2030	Aceito; planejar migração

05 CRIPTOGRAFIA APLICADA — CONTINUAÇÃO

ALGORITMO	USO	PARÂMETRO RECOMENDADO	SITUAÇÃO
ML-KEM e ML-DSA	KEM e assinatura pós-quânticos	FIPS 203 e 204 (2024); em modo híbrido	Em adoção
MD5, SHA-1, DES, 3DES, RC4, RSA-1024	—	—	Proibido ou legado

CIFRAGEM EM ENVELOPE E GESTÃO DE CHAVES

Cifrar é fácil; **gerir chaves é o difícil**. O padrão da indústria é a **cifragem em envelope**: os dados são cifrados com uma **chave de dados** (DEK, uma por registro, objeto ou arquivo) e a DEK é cifrada por uma **chave de criptografia de chaves** (KEK) que vive em um **KMS** (*Key Management Service*) ou **HSM** (*Hardware Security Module*), de onde nunca sai em claro. As vantagens são concretas: a KEK protege milhões de DEKs, mas só as DEKs pequenas viajam até o KMS; a **rotação é barata** (troca-se a KEK e re-cifram-se as DEKs, sem tocar nos dados); todo uso passa por IAM e fica em log; e a **destruição criptográfica** (*crypto-shredding*) apaga dados de forma lógica ao destruir a chave, o que ajuda a LGPD e os backups (p. 59).



LER
lê o registro, envia a DEK cifrada ao KMS (unwrap) e decifra os dados em memória com a DEK

ROTACIONAR
nova versão da KEK: re-cifra (re-wrap) só as DEKs; os dados não são tocados

Cifragem em envelope (envelope encryption). Cada registro é cifrado com uma DEK própria; o KMS cifra (*wrap*) a DEK com a KEK, que nunca sai do módulo. No armazenamento ficam o texto cifrado, o nonce, a DEK cifrada e metadados. Rotacionar a KEK só exige re-cifrar as DEKs.

FASE DA CHAVE	O QUE FAZER	ARMADILHA COMUM
Geração	CSPRNG ou HSM/KMS; uma chave por finalidade	Derivar de senha fraca; reutilizar a chave em outro contexto
Armazenamento	KMS, HSM ou cofre; acesso mínimo; nunca em claro em disco	Chave no repositório, em `.env` versionado ou na imagem do contêiner (CWE-321)
Uso e distribuição	Chamadas à API do KMS; separação de funções; log de cada uso	Baixar a chave para o servidor e deixá-la numa variável de ambiente sem controle
Rotação	Periódica e após suspeita; chaves versionadas (`kid`)	Rotação nunca testada: dados ilegíveis depois da troca
Revogação e destruição	Desabilitar, agendar exclusão, registrar; *crypto-shredding*	Backups com chaves antigas esquecidos; ninguém sabe o que a chave protegia

05 CRIPTOGRAFIA APLICADA

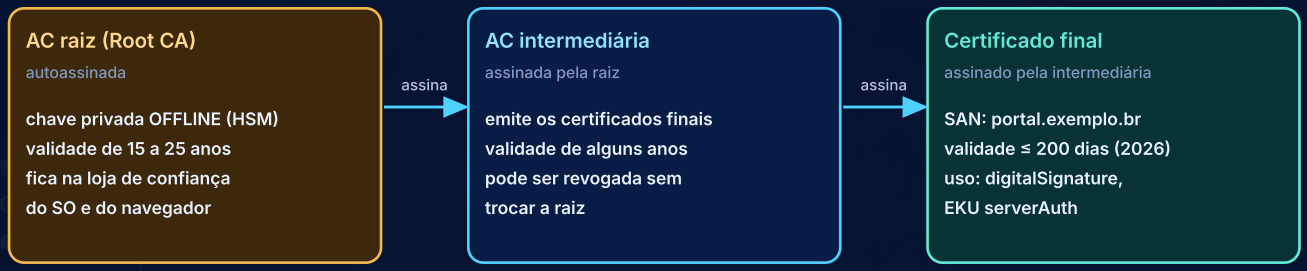
CONTINUAÇÃO

DICA DA CORUJA

faça um **simulado de rotação**: cifre um registro, gire a KEK, restaure um backup antigo e decifre. Se nunca testou, você não tem rotação, tem esperança. O NIST SP 800-57 chama o prazo de uso autorizado de uma chave de **criptoperíodo**; guarde também a versão da chave em cada registro cifrado.

PKI E CERTIFICADOS X.509

Uma chave pública sozinha não diz **de quem** ela é. A **PKI** (*Public Key Infrastructure*) resolve isso com **certificados X.509** (RFC 5280): documentos assinados por uma **autoridade certificadora** (AC, *CA*) que vinculam uma chave pública a uma identidade. No TLS, essa identidade é um **nome DNS** no campo *Subject Alternative Name* (SAN); os navegadores atuais ignoram o *Common Name*. O cliente confia em um conjunto de **raízes** (a loja de confiança do sistema operacional ou do navegador) e valida a **cadeia**: folha → intermediária → raiz.



COMO O CLIENTE VALIDA (DA FOLHA PARA A RAIZ)

1	2	3	4	5	6	7
assinaturas	datas	hostname	uso da chave	revogação	CT (SCT)	raiz confiável

Cadeia de confiança de uma PKI. A raiz fica offline e só assina intermediárias; as intermediárias emitem os certificados finais. O cliente valida a cadeia da folha até uma raiz que já esteja em sua loja de confiança e confere nome, validade, uso, revogação e prova de Certificate Transparency.

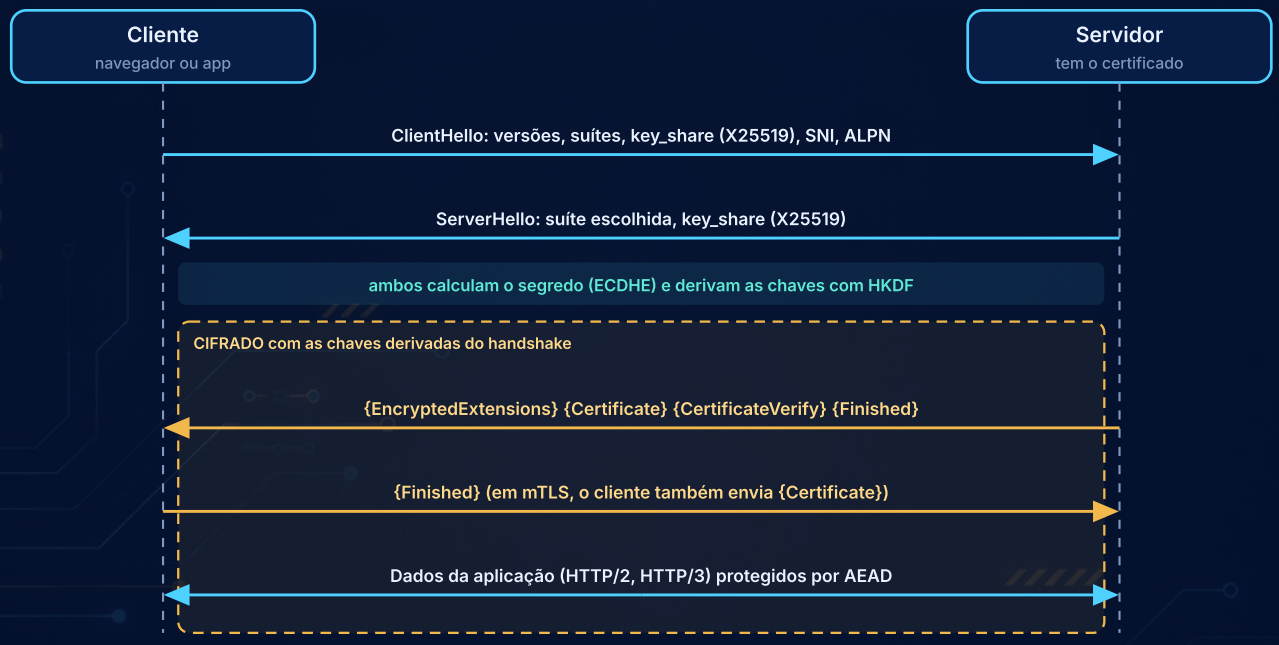
MECANISMO	PROBLEMA QUE RESOLVE	COMO FUNCIONA
CRL (RFC 5280)	Revogar certificado comprometido	Lista assinada de seriais revogados, publicada pela AC e baixada periodicamente
OCSP e *stapling* (RFC 6960, 6066)	Consultar o status em tempo real	Um respondedor assina "válido" ou "revogado"; no *stapling*, o servidor anexa a resposta ao handshake e o cliente não precisa consultar a AC
Certificate Transparency (RFC 6962, 9162)	Detectar emissão indevida	Logs públicos e auditáveis; o certificado traz provas de inclusão (SCT); donos monitoram os próprios domínios
CAA (RFC 8659)	Limitar quem pode emitir	Registro DNS, por exemplo `exemplo.br. CAA 0 issue "letsencrypt.org"`
ACME (RFC 8555)	Emitir e renovar sem humanos	O cliente prova o controle do domínio (desafios HTTP-01 ou DNS-01) e recebe o certificado

05 **CRIPTOGRAFIA APLICADA** CONTINUAÇÃO

A revogação sempre foi o ponto fraco da PKI (o OCSP tem problemas de privacidade e de disponibilidade), e o setor responde com **certificados de vida curta**. Segundo o CA/Browser Forum (ballot SC-081v3), a validade máxima dos certificados TLS públicos caiu para **200 dias em 15/03/2026**, cai para **100 dias em 2027** e para **47 dias em 2029**. Conclusão prática: **automatize com ACME** e monitore a expiração. Certificado vencido é uma das causas mais comuns de indisponibilidade, e sem automação a renovação a cada 47 dias é inviável. No **mTLS** (TLS mútuo) o cliente também apresenta certificado: é a forma comum de autenticar serviços entre si e APIs entre empresas (p. 67).

TLS 1.3 NA PRÁTICA

O **TLS 1.3** (RFC 8446, 2018) simplificou e endureceu o protocolo. Removeu a troca de chaves por RSA estático, os modos CBC, o RC4, o 3DES, a compressão e a renegociação; **exige AEAD** (restam só cinco suítes) e **sigilo perfeito** (todo acordo é efêmero); e reduziu o handshake a **1 RTT**. Tudo após o *ServerHello* é cifrado, inclusive o certificado; o nome do servidor (SNI) segue visível, a menos que se use o ECH (*Encrypted Client Hello*), ainda em implantação. O **0-RTT** na retomada de sessão permite enviar dados no primeiro voo, mas **não protege contra replay**: só para requisições idempotentes.



Handshake do TLS 1.3 (RFC 8446): 1 RTT e tudo cifrado após o ServerHello. Cliente e servidor trocam *key shares* efêmeros (ECDHE), derivam as chaves com HKDF e o servidor se autentica com certificado e assinatura (*CertificateVerify*) sobre a transcrição do handshake. As chaves efêmeras garantem sigilo perfeito de encaminhamento.

ASPECTO	TLS 1.2 (RFC 5246)	TLS 1.3 (RFC 8446)
Handshake completo	2 RTT	1 RTT; 0-RTT na retomada, com risco de replay
Troca de chaves	RSA estático ou (EC)DHE, conforme a suíte	Somente (EC)DHE efêmero: sigilo perfeito obrigatório
Cifras	CBC, RC4, 3DES e AEAD	Somente AEAD (cinco suítes)
Certificado do servidor	Trafega em claro	Cifrado com as chaves do handshake
Recursos removidos	—	Compressão, renegociação e troca de chaves por RSA
Recomendação	Aceitar só com ECDHE + AEAD	Preferir sempre; manter o 1.2 por compatibilidade

05 CRIPTOGRAFIA APLICADA

CONTINUAÇÃO

NGINX: TLS 1.2 E 1.3 COM HSTS (DERIVADO DAS RECOMENDAÇÕES DA MOZILLA E DA RFC 9325)

```
server {
    listen 443 ssl;
    http2 on;
    server_name portal.exemplo.br;

    ssl_certificate     /etc/ssl/portal/fullchain.pem;   # folha + intermediária
    ssl_certificate_key /etc/ssl/portal/privkey.pem;     # modo 0600; nunca no repositório

    ssl_protocols TLSv1.2 TLSv1.3;                       # 1.0 e 1.1 aposentados (RFC 8996)
    ssl_ciphers   ECDHE+AESGCM:ECDHE+CHACHA20;           # só AEAD com ECDHE (sigilo perfeito)
    ssl_prefer_server_ciphers off;                       # o cliente escolhe a melhor suíte
    ssl_session_tickets off;

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
}
server { listen 80; server_name portal.exemplo.br; return 301 https://$host$request_uri; }
```

Boa prática: gere a configuração no gerador da Mozilla (*SSL Configuration Generator*) e **teste**: `openssl s_client -connect portal.exemplo.br:443 -servername portal.exemplo.br -tls1_3` mostra protocolo, suíte e cadeia; `testssl.sh` ou o teste do SSL Labs apontam suítes fracas e problemas de certificado. O **HSTS** (RFC 6797) impede o *downgrade* para HTTP; antes de ativar `includeSubDomains` ou `preload`, confira que **todos** os subdomínios atendem por HTTPS, pois desfazer é lento.

ERROS COMUNS DE CRIPTOGRAFIA

ERRO	CONSEQUÊNCIA	CORREÇÃO
Chave fixa no código ou no repositório (CWE-321)	Quem lê o código decifra tudo; trocar a chave exige novo deploy	Cofre ou KMS; injeção em tempo de execução; varredura de segredos (p. 135)
ECB, ou CBC e CTR sem autenticação (CWE-327)	Padrões vazam; texto cifrado maleável; *padding oracle*	AEAD: AES-GCM ou ChaCha20-Poly1305
Nonce ou IV repetido ou previsível (CWE-323, CWE-329)	Vaza o XOR dos textos; etiquetas forjadas	Nonce aleatório (CSPRNG) ou contador seguro; uma chave por registro
`random` ou `Math.random` para tokens e chaves (CWE-338)	Valores previsíveis; sequestro de sessão e de redefinição de senha	`secrets`, `crypto.randomBytes`, `SecureRandom`
***Hash* rápido para senha (CWE-916); `SHA256(chave + msg)` como MAC**	Quebra por força bruta em GPU; extensão de comprimento	Argon2id (p. 44); HMAC
Comparar MAC ou token com `==` (CWE-208)	Vazamento por tempo de resposta	`compare_digest` e equivalentes
Desligar a validação de certificado: `verify=False`, `InsecureSkipVerify` (CWE-295)	Interceptação trivial; o TLS vira enfeite	Instalar a AC interna na loja de confiança; *pinning* só com plano de rotação

05 CRIPTOGRAFIA APLICADA — CONTINUAÇÃO

ERRO	CONSEQUÊNCIA	CORREÇÃO
Algoritmos ou tamanhos obsoletos: MD5, SHA-1, DES, RC4, RSA-1024 (CWE-327, CWE-326)	Colisões e quebras práticas	Tabela de algoritmos desta seção; inventário criptográfico
"Criptografar" com Base64, XOR fixo ou ofuscação	Nenhuma proteção: é só codificação	Cifrar de verdade com AEAD; Base64 serve apenas para transportar bytes

CRIPTOGRAFIA PÓS-QUÂNTICA E O QUE VEM A SEGUIR

Um computador quântico suficientemente grande executaria o **algoritmo de Shor** (1994), que fatora inteiros e resolve logaritmos discretos em tempo polinomial: RSA, Diffie–Hellman, ECDH, ECDSA e EdDSA deixariam de proteger. O **algoritmo de Grover** dá só um ganho quadrático na busca por chaves, e por isso AES-256 e *hashes* de ≥ 256 bits mantêm margem. Ninguém sabe quando essa máquina chegará; o risco imediato é o **"colha agora, decifre depois"** (*harvest now, decrypt later*): um adversário grava hoje o tráfego cifrado e o decifra no futuro, o que preocupa dados de **longa vida** (saúde, identidade, segredos de Estado).

1994 · SHOR
Algoritmo quântico para fatoração e logaritmo discreto.

2016 · NIST
Chamada pública por algoritmos pós-quânticos; seleção de Kyber, Dilithium, Falcon e SPHINCS+ em 2022.

AGO/2024 · PADRÕES
FIPS 203 (ML-KEM), FIPS 204 (ML-DSA) e FIPS 205 (SLH-DSA) finalizados; o FIPS 206 (FN-DSA) segue em rascunho.

MAR/2025 · HQC
Escolhido como KEM de reserva, com outra base matemática (códigos corretores).

2030 E 2035 · PRAZOS
O rascunho NIST IR 8547 propõe depreciar o RSA e o ECC de 112 bits até 2030 e proibir todo RSA e ECC vulneráveis até 2035.

Os novos algoritmos custam **espaço**: uma chave pública ML-KEM-768 tem cerca de 1.184 bytes (X25519: 32), e uma assinatura ML-DSA-65, cerca de 3.309 bytes (Ed25519: 64), o que pesa em handshakes, certificados e dispositivos limitados. A migração começa pelos **híbridos**: o acordo **X25519 + ML-KEM** já é oferecido por navegadores e grandes provedores no TLS 1.3, de modo que a sessão só cai se **ambos** os algoritmos forem quebrados. A parte do projetista é a **cripto-agilidade**: identificar o algoritmo nos dados (campos `alg` e `kid`), concentrar as chamadas em uma biblioteca e manter um **inventário criptográfico** (*CBOM*, o equivalente do SBOM para criptografia) para saber onde há RSA e ECC.

Fronteiras da área: três linhas de pesquisa já aparecem em produtos: a **criptografia de ponta a ponta** (E2EE; no Signal, o *Double Ratchet* renova as chaves a cada mensagem, dando sigilo futuro e recuperação após comprometimento), as **provas de conhecimento zero** (provar uma afirmação sem revelar o dado) e a **criptografia homomórfica** (calcular sobre dados cifrados). Entenda-as como **ferramentas de arquitetura**, caras e especializadas, não como substitutas das primitivas deste módulo.

05 CRIPTOGRAFIA APLICADA — CONTINUAÇÃO

EXEMPLO RESOLVIDO

Problema. O portal cifra cada resposta de formulário com AES-256-GCM e nonce aleatório de 96 bits, sempre com a **mesma chave**. Qual a chance de repetir um nonce depois de n mensagens?

Cálculo (paradoxo do aniversário). $P \approx n^2 \div 2^{97}$.

- Para $n = 2^{32}$ (cerca de 4,3 bilhões): $P \approx 2^{64} \div 2^{97} = 2^{-33}$, um em 8,6 bilhões. É o limite adotado pelo NIST (SP 800-38D).
- Para $n = 2^{40}$ (cerca de 1 trilhão): $P \approx 2^{80} \div 2^{97} = 2^{-17}$, um em 131 mil. Inaceitável: cada repetição vaza o XOR de dois textos e compromete a autenticação.

Decisão. Com 50 milhões de mensagens por mês, a cota de 2^{32} dura cerca de 86 meses, mas ela vale **por chave**: duas instâncias com a mesma chave dividem o mesmo orçamento. A saída robusta é a **cifragem em envelope**, com uma DEK por registro. Cada chave cifra uma única mensagem e o risco de nonce repetido desaparece.

PARA FIXAR

Conceitos

- Por que a segurança de um sistema criptográfico não deve depender do segredo do algoritmo (princípio de Kerckhoffs)?
- Diferencie *hash*, MAC e assinatura digital: o que cada um garante e o que não garante?
- Por que `SHA256(chave + mensagem)` não é um MAC seguro e o que o HMAC muda?

Aplicação

- Projete a cifragem do campo CPF do portal com envelope encryption: o que vai ao banco, o que fica no KMS, qual o AAD e como rotacionar a KEK?
- A chave privada de um servidor TLS 1.3 vazou em 2026. As sessões gravadas antes do vazamento podem ser decifradas? Justifique e diga o que fazer com o certificado.
- Calcule a probabilidade de colisão de nonces de 96 bits com $n = 2^{36}$ mensagens.

APROFUNDE-SE

Leituras e referências

- NIST SP 800-57 Parte 1 Rev. 5 — NIST: gestão de chaves, criptoperíodos e equivalência de níveis de segurança.
- RFC 8446 (TLS 1.3) — IETF: o protocolo completo; leia as seções do handshake e do cronograma de chaves.
- RFC 9325 — IETF: recomendações atuais para uso seguro do TLS e do DTLS.
- Cryptographic Storage Cheat Sheet — OWASP: como cifrar dados em repouso sem erros clássicos.
- SSL Configuration Generator — Mozilla: configurações de TLS prontas para nginx, Apache e outros.
- Post-Quantum Cryptography — NIST: padrões, rascunhos e cronograma da migração.

MÓDULO 06 · IDENTIDADE

06 IDENTIDADE E AUTENTICAÇÃO

Provar quem você é: fatores, armazenamento de credenciais, federação e os erros mais comuns em login.

Autenticar é provar quem se é, e é a porta de entrada de grande parte das violações: senhas roubadas, reutilizadas ou aprovadas por engano estão no começo de muitos incidentes graves. Este módulo cobre identidade e credenciais, o armazenamento correto de senhas, MFA e *passkeys*, sessões, tokens (JWT), OAuth 2.0, OpenID Connect e SAML, além das identidades de máquina e do acesso privilegiado.

IDENTIDADE, CREDENCIAL E AUTENTICADOR

Identidade é o conjunto de atributos que representa uma pessoa, um serviço ou um dispositivo em um sistema (o *subject*). **Identificação** é afirmar quem se é (um nome de usuário); **autenticação** (*AuthN*) é provar a afirmação; **autorização** (*AuthZ*, p. 53) é decidir o que a identidade pode fazer; a **contabilização** (*accounting*) registra o que foi feito. Juntas formam o **AAA**. A **credencial** é o dado ou objeto usado como prova (senha, certificado, chave física); o **autenticador** é o mecanismo que produz a prova (um app de códigos, uma chave FIDO2). O **provedor de identidade** (*IdP*, *Identity Provider*) autentica e emite asserções; a **parte confiante** (*relying party*, RP) é a aplicação que confia nelas.

O **ciclo de vida da identidade** (*Joiner–Mover–Leaver*, JML) é onde muitos incidentes nascem: na **entrada**, provisionar só o necessário; na **mudança de função**, retirar o que não cabe mais (senão se acumulam privilégios); na **saída**, desativar no mesmo dia, pois contas órfãs são portas abertas. Automatize com **SCIM** (RFC 7644) e revisões periódicas de acesso.

FATOR	EXEMPLOS	ATAQUES TÍPICOS	OBSERVAÇÃO
Conhecimento (algo que você sabe)	Senha, PIN	Phishing, *credential stuffing*, força bruta offline	Copiável e reutilizável
Posse (algo que você tem)	Chave FIDO2, app de códigos, celular	Roubo, *SIM swap*, malware, fadiga de MFA	Melhor quando há prova criptográfica
Inerência (algo que você é)	Digital, face	Falsificação; vazamento irrevogável	Use para desbloquear o autenticador localmente
Contexto e risco	Dispositivo, local, horário, comportamento	Evasão com VPN residencial; falsos positivos	Complementa os fatores, não os substitui

MFA de verdade: a **autenticação multifator** (MFA) exige ao menos dois fatores de **categorias diferentes**. Duas senhas não são MFA, e um código SMS enviado ao mesmo celular em que se digita a senha enfraquece a separação entre os fatores.

ARMAZENAMENTO DE SENHAS: ASSUMA QUE O BANCO VAI VAZAR

Todo banco de dados pode vazar (injeção SQL, backup exposto, funcionário mal-intencionado). O objetivo é que, **depois do vazamento**, cada tentativa de adivinhar uma senha custe caro. Por isso **nunca se guarda a senha**, nem cifrada com uma chave que o sistema conheça: guarda-se o resultado de uma **função de derivação de chave lenta e que consome memória**, calculada com um **sal**. O **Argon2id** (RFC 9106), vencedor da *Password Hashing Competition* (2015), é a escolha atual: o uso obrigatório de memória encarece o ataque com GPUs e hardware dedicado.

06 IDENTIDADE E AUTENTICAÇÃO — CONTINUAÇÃO

Sal (salt)

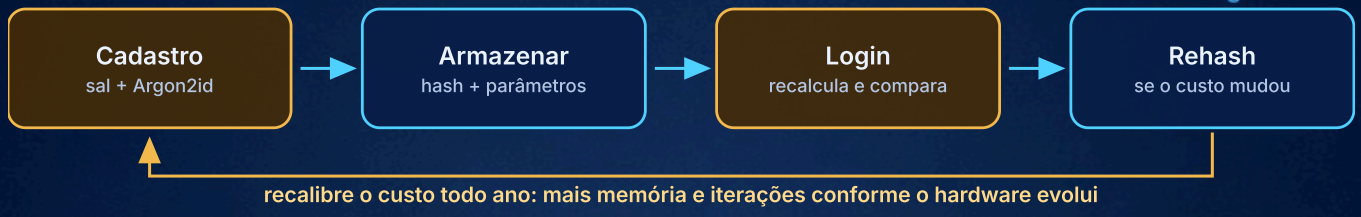
Valor **aleatório e único por senha** (≥ 16 bytes de um CSPRNG), guardado junto do hash.

- Duas senhas iguais geram hashes diferentes
- Inutiliza tabelas pré-calculadas (*rainbow tables*)
- Não é segredo

Pepper

Segredo **comum a todas as senhas**, guardado **fora do banco** (KMS, HSM ou variável protegida).

- Aplicado antes do KDF, por exemplo com HMAC
- Um dump do banco, sozinho, não basta para o ataque offline
- Complica a rotação: planeje versões



HASH E VERIFICAÇÃO DE SENHA COM ARGON2-CFFI (PYTHON)

```
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError

# Argon2id; m = 19 MiB (19456 KiB), t = 2, p = 1: mínimo recomendado pela OWASP
ph = PasswordHasher(time_cost=2, memory_cost=19456, parallelism=1, hash_len=32, salt_len=16)

def cadastrar(senha: str) -> str:
    return ph.hash(senha)          # o sal vai embutido: "$argon2id$v=19$m=19456,t=2,p=1$..."

def autenticar(hash_salvo: str, senha: str) -> bool:
    try:
        ph.verify(hash_salvo, senha)            # usa o sal e os parâmetros do próprio hash
    except VerifyMismatchError:
        return False
    if ph.check_needs_rehash(hash_salvo):       # parâmetros antigos? regrave no login
        regravar_hash(ph.hash(senha))
    return True
```

ALGORITMO	PARÂMETROS MÍNIMOS	QUANDO USAR	OBSERVAÇÃO
Argon2id (RFC 9106)	m = 19 MiB, t = 2, p = 1 (ou m = 46 MiB, t = 1, p = 1)	Padrão em sistemas novos	Resiste a GPU por exigir memória
scrypt (RFC 7914)	$N = 2^{17}$, r = 8, p = 1	Quando não há Argon2id	Também exige memória
bcrypt	Custo ≥ 10	Sistemas legados	Limite de 72 bytes na senha
PBKDF2-HMAC-SHA-256 (RFC 8018)	≥ 600.000 iterações	Quando FIPS é exigido	Barato em GPU: use só se obrigatório

06 **IDENTIDADE E AUTENTICAÇÃO** CONTINUAÇÃO

EXEMPLO RESOLVIDO

Por que o hash lento importa (e o que ele não resolve). A entropia de uma senha escolhida ao acaso é $\mathbf{L \times \log_2(N)}$ bits, com L caracteres de um alfabeto de N símbolos. Para 8 caracteres alfanuméricos (N = 62): $8 \times 5{,}95 \approx$ **47,6 bits**, ou $62^8 \approx 2{,}2 \times 10^{14}$ combinações.

Hipótese didática (os valores reais variam com o hardware): um hash rápido permite 10^{10} tentativas por segundo; o Argon2id calibrado, 10^4.

- Hash rápido: $2{,}2 \times 10^{14} \div 10^{10} \approx 2{,}2 \times 10^4$ s, cerca de **6 horas**
- Argon2id: $2{,}2 \times 10^{14} \div 10^4 \approx 2{,}2 \times 10^{10}$ s, cerca de **690 anos**

O custo do atacante cresce um milhão de vezes. Mas senhas **escolhidas por pessoas** caem em listas de poucos bilhões de candidatos (senhas vazadas mais regras de variação): com 10^9 candidatos, o Argon2id leva cerca de 28 horas, não séculos. Conclusão: hash lento **e** bloqueio de senhas comuns **e** MFA, nunca um só.

POLÍTICA DE SENHAS MODERNA (NIST SP 800-63B-4)

As regras de composição e de troca periódica envelheceram mal: estimulam "Senha@2026" e "Senha@2027". O **NIST SP 800-63B-4** (2025) descreve o que funciona. A verificação contra senhas vazadas pode usar uma lista local ou a API *Pwned Passwords* do *Have I Been Pwned*, que aplica **k-anonimato**: o cliente calcula o SHA-1 da senha, envia só os **5 primeiros caracteres hexadecimais** e recebe os sufixos que começam igual, com contagens; a comparação final é local, e a senha nunca sai do servidor.

TEMA	FAÇA	EVITE
Tamanho	Mínimo de 15 caracteres se a senha for o único fator (8 se fizer parte de MFA); aceitar 64 ou mais	Tetos baixos (16 a 20) e truncamento silencioso
Composição	Qualquer caractere Unicode, inclusive espaços; incentivar frases-senha	Regras como "1 maiúscula e 1 símbolo"
Verificação	Rejeitar senhas vazadas e comuns, no cadastro e na troca	Aceitar "123456", o nome do site ou do usuário
Ergonomia	Permitir colar e usar gerenciadores de senha	Bloquear o colar; teclado virtual obrigatório
Troca	Trocar somente diante de indício de comprometimento	Expiração periódica obrigatória
Recuperação	Link de uso único e MFA; sem perguntas secretas (KBA)	"Nome da mãe", "cidade natal"

MFA, FIDO2 E PASSKEYS

O **TOTP** (RFC 6238) calcula um código de 6 dígitos a partir de uma **semente** compartilhada e do tempo: `HMAC-SHA-1(semente, ⌊t ÷ 30⌋)`, truncado (o **HOTP**, RFC 4226, usa um contador no lugar do tempo). É simples e funciona offline, mas o código **pode ser reencaminhado em tempo real** por uma página falsa. O **push** tem o problema da **fadiga de MFA**: o atacante dispara pedidos até o usuário aprovar por cansaço ou engano; a correção é a **correspondência de número** (o usuário digita na tela o número exibido no login). O **SMS** sofre com *SIM swap* e interceptação: é o último recurso.

O **FIDO2/WebAuthn** muda a lógica. No cadastro, o autenticador gera um **par de chaves por site** (por *relying party ID*) e entrega ao servidor só a chave pública. No login, assina um **desafio** aleatório do servidor, e a assinatura cobre a **origem** (*origin*) vista pelo navegador. Em um site falso, o navegador não oferece a credencial e uma assinatura de outra origem seria rejeitada: **não existe segredo que o usuário possa entregar**. As **passkeys** são credenciais FIDO *descobríveis* (dispensam digitar o usuário), que podem ser **sincronizadas** pelo gerenciador da plataforma ou **presas ao dispositivo**, como nas chaves de hardware; o NIST SP 800-63B-4 aceita autenticadores sincronizáveis no nível de garantia AAL2.

06 IDENTIDADE E AUTENTICAÇÃO

CONTINUAÇÃO



Escada de força da autenticação. Cada degrau reduz uma classe de ataque; só o FIDO2/WebAuthn resiste ao phishing em tempo real, porque a assinatura da chave está presa à origem (domínio) legítima. Vermelho: evitar; azul: aceitável com ressalvas; verde: recomendado.

MÉTODO	ANTI-PHISHING	FRAQUEZA PRINCIPAL	OBSERVAÇÃO
SMS ou voz	Não	*SIM swap*, interceptação, engenharia social na operadora	Último recurso
Push simples	Não	Fadiga de MFA: aprovação por cansaço ou engano	Evitar
Push com número	Parcial	Phishing em tempo real: o usuário digita o número na página falsa	Aceitável
TOTP (RFC 6238)	Não	Código reencaminhado em tempo real; semente roubada	Bom como reserva
FIDO2 / passkey	**Sim**	Perda do dispositivo (exige recuperação); suporte em cada app	Recomendado

Atenção: MFA não é à prova de tudo. Proxies reversos de phishing (*adversary-in-the-middle*, AiTM) capturam a senha, o código **e o cookie de sessão** emitido depois do MFA; *infostealers* roubam cookies direto do navegador. Defesas: FIDO2, sessões curtas e vinculadas ao dispositivo, detecção de anomalias (p. 144) e revogação rápida.

ATAQUES ÀS CREDENCIAIS E RECUPERAÇÃO DE CONTA

ATAQUE	COMO FUNCIONA	DEFESA
Credential stuffing	Testa pares usuário/senha vazados em outros sites, aproveitando o reuso	MFA; checagem de senha vazada; limite de taxa por conta, IP e dispositivo; detecção de bots
Password spraying	Poucas senhas comuns contra muitas contas, para fugir do bloqueio por conta	Lista de senhas proibidas; alerta sobre padrão distribuído; MFA
Força bruta online	Tentativas sequenciais no formulário de login	Atraso progressivo (em vez de bloqueio permanente, que vira negação de serviço); limites

06 IDENTIDADE E AUTENTICAÇÃO — CONTINUAÇÃO

ATAQUE	COMO FUNCIONA	DEFESA
Enumeração de usuários	Mensagens ou tempos de resposta diferentes revelam se a conta existe (CWE-204)	Resposta e tempo uniformes: "usuário ou senha inválidos"
Fadiga de MFA	Pedidos de push repetidos até o usuário aprovar	Correspondência de número; limite de pedidos; FIDO2
Phishing e AiTM	Página falsa ou proxy captura senha, código e cookie	FIDO2 e passkeys; sessões curtas e vinculadas ao dispositivo

A **recuperação de conta** é o elo mais fraco: quem controla o e-mail ou o telefone de recuperação controla a conta. Use um token de redefinição **aleatório** (≥ 128 bits), de **uso único**, com validade curta (15 a 60 minutos) e guardado só como *hash*; devolva a **mesma resposta** quer o e-mail exista ou não; ao concluir, **encerre todas as sessões**, avise o usuário e exija MFA quando houver. Guarde os **códigos de recuperação** do MFA offline. No suporte humano, siga um procedimento de **verificação de identidade**: a engenharia social contra a central de atendimento é comum.

SESSÕES E COOKIES

O HTTP não guarda estado. Depois do login, o servidor emite um **identificador de sessão** (≥ 128 bits de um CSPRNG) que o navegador devolve a cada requisição, em um *cookie*. Quem roubar o ID **passa a ser o usuário** até a sessão expirar: o cookie de sessão é o segredo mais valioso do navegador. O ID deve ser **opaco** (sem dados dentro), guardado no servidor e **renovado no login** e em toda mudança de privilégio, o que impede a **fixação de sessão** (o atacante define um ID conhecido antes do login e herda a sessão depois). Defina **expiração ociosa** (por exemplo, 15 a 30 minutos) **e absoluta** (8 a 12 horas), e faça o *logout* invalidar a sessão **no servidor**, não só apagar o cookie.

ATRIBUTO	EFEITO	SEM ELE
`Secure`	O cookie só trafega por HTTPS	Pode vazar em HTTP, por *sniffing* ou *downgrade*
`HttpOnly`	JavaScript não consegue ler o cookie	Um XSS rouba a sessão com `document.cookie`
`SameSite=Lax` (ou `Strict`)	Não envia o cookie em requisições entre sites que alteram estado	CSRF fica mais fácil (p. 98)
Prefixo `__Host-`	Exige `Secure`, `Path=/` e nenhum `Domain`; prende o cookie ao host exato	Subdomínios ou HTTP podem sobrescrever o cookie
`Max-Age` ou `Expires`	Limita a vida do cookie no navegador	Sessão "eterna": combine com expiração no servidor

INÍCIO DE SESSÃO (PSEUDOCÓDIGO EM PYTHON)

```
import secrets, time

def iniciar_sessao(req, resp, usuario):
    if req.sid:
        store.delete(req.sid)           # descarta o ID antigo: evita fixação de sessão
    sid = secrets.token_urlsafe(32)     # 256 bits de um CSPRNG; o cookie leva só o ID
    dados = {"uid": usuario.id, "criada": time.time()}
    store.set(sid, dados, ttl=12 * 3600)        # limite absoluto de 12 h
    resp.set_cookie("__Host-sid", sid, path="/", secure=True, httponly=True,
                    samesite="Lax", max_age=1800)   # ocioso: 30 min, renovado a cada uso
```

06 IDENTIDADE E AUTENTICAÇÃO

CONTINUAÇÃO

TOKENS E JWT

Há duas famílias de token. O **opaco** é uma string aleatória sem significado: o servidor consulta um armazenamento para saber a quem pertence (simples e revogável). O **autocontido** carrega os dados e uma assinatura: o **JWT** (*JSON Web Token*, RFC 7519) tem três partes em base64url, `cabeçalho.carga.assinatura`; a carga traz *claims* como `iss` (emissor), `sub` (sujeito), `aud` (destinatário), `exp` (expiração), `nbf`, `iat` e `jti`. A API valida a assinatura **sem consultar o emissor**, o que escala bem entre serviços, mas custa a **revogação imediata**: um JWT roubado vale até o `exp`. A carga é apenas **codificada** (JWS assinado, não cifrado): **não coloque segredos nela**. Para o login de um site tradicional, prefira **sessão no servidor**; reserve o JWT para federação e comunicação entre serviços, com **vida curta** (minutos). O guia de boas práticas é a RFC 8725.

FALHA	COMO OCORRE	DEFESA
`alg: none`	A biblioteca aceita um token sem assinatura	Lista fixa de algoritmos no verificador; rejeitar `none`
Confusão de algoritmo	Token RS256 reenviado como HS256 e verificado com a chave pública usada como segredo HMAC	Fixar o algoritmo por chave; nunca deixar o token escolher
Segredo HS256 fraco	Segredo curto cai em força bruta offline	≥ 256 bits aleatórios, ou use RS256, ES256 ou EdDSA
***Claims* ignoradas**	`iss`, `aud` e `exp` sem validação: o token de outro serviço é aceito	Validar `iss`, `aud`, `exp` e `nbf`; exigir as *claims*
`kid` manipulável	O identificador da chave vira caminho de arquivo ou consulta SQL (CWE-22, CWE-89)	Buscar a chave em um conjunto (JWKS) de IDs conhecidos
Armazenamento no navegador	`localStorage` é lido por XSS; cookie sem proteção sofre CSRF	Cookie `HttpOnly; Secure; SameSite`, ou token só em memória

VERIFICAÇÃO DE JWT COM ALGORITMO FIXADO (PYJWT)

```
import jwt   # PyJWT

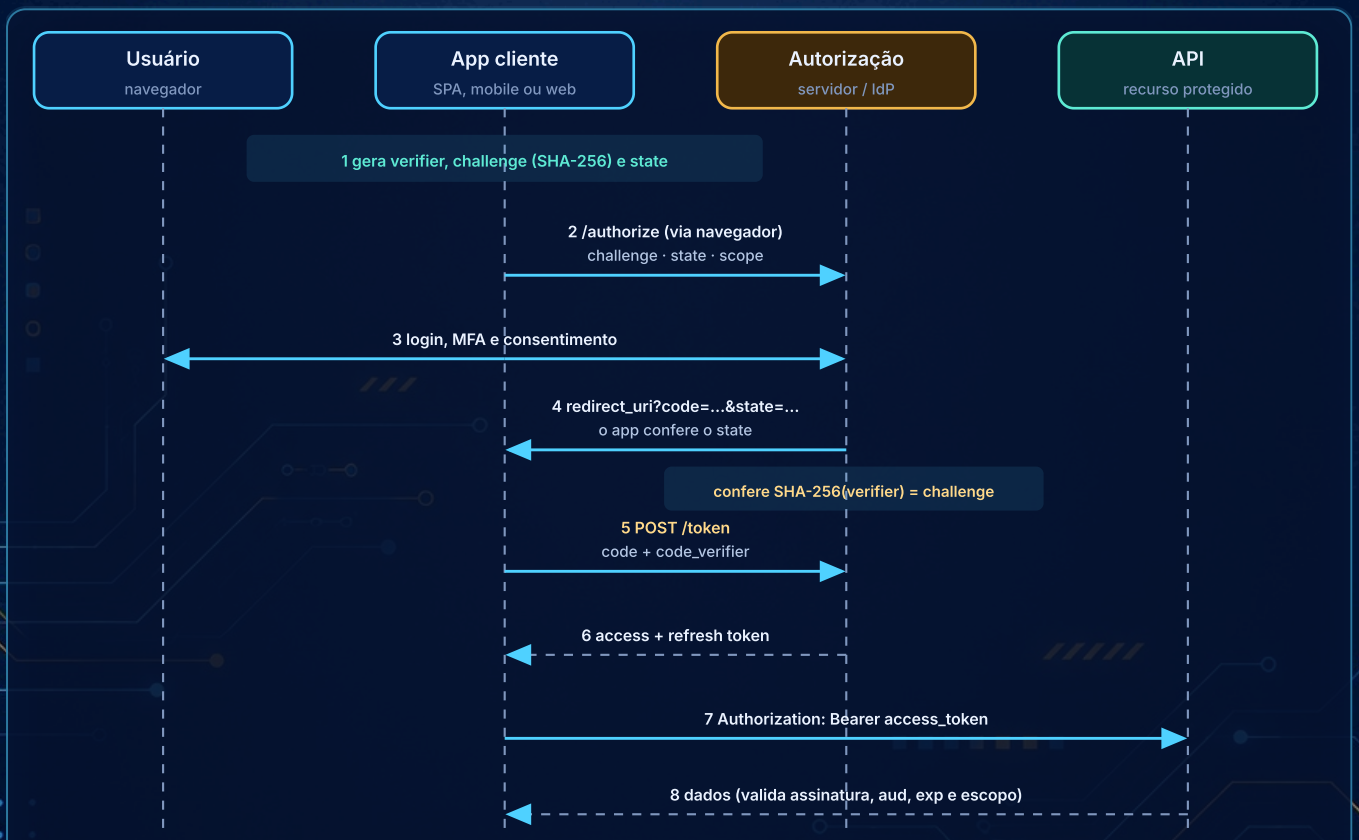
def validar(token: str, chave_publica_pem: str) -> dict:
    return jwt.decode(
        token,
        chave_publica_pem,
        algorithms=["RS256"],              # lista fixa: nunca derive do cabeçalho do token
        audience="portal-api",             # aud esperado
        issuer="https://id.exemplo.br",    # iss esperado
        options={"require": ["exp", "iat", "sub"]},   # claims obrigatórias
    )                                      # qualquer falha lança exceção: trate como 401
```

06 **IDENTIDADE E AUTENTICAÇÃO** CONTINUAÇÃO

OAUTH 2.0, OPENID CONNECT E SAML

O **OAuth 2.0** (RFC 6749) é um protocolo de **delegação de acesso**: permite que um aplicativo use uma API **em nome do usuário** sem conhecer sua senha. Os papéis são o **dono do recurso** (usuário), o **cliente** (app), o **servidor de autorização** (emite tokens) e o **servidor de recursos** (a API). O OAuth **não autentica** o usuário; para isso existe o **OpenID Connect** (OIDC), uma camada de identidade sobre ele que acrescenta o **ID token** (um JWT assinado com `sub`, `iss`, `aud`, `exp` e `nonce`), o endpoint *UserInfo* e a **descoberta** (`/.well-known/openid-configuration`, que publica os endpoints e o JWKS com as chaves públicas).

O fluxo recomendado para web, SPA e mobile é o **Authorization Code com PKCE** (*Proof Key for Code Exchange*, RFC 7636). O app cria um segredo aleatório (*code_verifier*, de 43 a 128 caracteres), envia só o seu **hash** (*code_challenge* = BASE64URL(SHA-256(verifier))) na requisição inicial e revela o verifier ao trocar o código por tokens. Se um atacante interceptar o `code` no redirecionamento (um problema comum em apps móveis), não consegue usá-lo sem o verifier.



OAuth 2.0 Authorization Code com PKCE (RFC 6749, 7636 e 9700). O app cria um segredo (*code_verifier*) e envia só o seu hash (*code_challenge*); o usuário autentica no servidor de autorização, que devolve um *code* de uso único; na troca por tokens o app revela o verifier e o servidor confere. Quem interceptar o *code* não consegue usá-lo.

06 IDENTIDADE E AUTENTICAÇÃO — CONTINUAÇÃO

Fluxos (grant types)

- **Authorization Code + PKCE**: padrão para web, SPA e mobile
- **Client Credentials**: máquina a máquina, sem usuário
- **Device Authorization** (RFC 8628): TVs e linhas de comando
- **Implicit** e **Password**: **não usar** (RFC 9700)

Regras de ouro (RFC 9700)

- `redirect_uri` com correspondência **exata**, sem curingas
- `state` (anti-CSRF) e PKCE em todo cliente; `nonce` no OIDC
- Access token de vida curta; refresh token com **rotação** e detecção de reuso
- Escopos mínimos; nunca token na URL; DPoP (RFC 9449) em alto risco

PROTOCOLO	SERVE PARA	FORMATO	QUANDO USAR
OAuth 2.0	Delegar acesso a uma API	Access token (opaco ou JWT)	Apps e APIs; integrações de terceiros
OpenID Connect	Login e SSO modernos	ID token (JWT)	"Entrar com..."; SSO de apps web e mobile
SAML 2.0	SSO corporativo entre IdP e SaaS	Asserção XML assinada	Aplicações corporativas e legadas
Kerberos	SSO em redes de domínio (Active Directory)	Tickets	Rede interna corporativa (apenas menção)

Atenção: o SAML depende de XML assinado. Falhas de validação, como o **XML signature wrapping** (o atacante move o trecho assinado e injeta outro, que a aplicação lê) e o XXE, já geraram desvio de autenticação em vários produtos. Use bibliotecas mantidas, valide a assinatura do **elemento que será consumido** e desative DTD e entidades externas.

IDENTIDADES DE MÁQUINA E ACESSO PRIVILEGIADO

Em sistemas modernos, **as identidades que mais crescem não são humanas**: contas de serviço, chaves de API, tokens de CI/CD e certificados de carga de trabalho. Raramente têm MFA, quase nunca são revisadas e, quando vazam, dão acesso silencioso. Princípios: **uma identidade por carga de trabalho**; **credenciais efêmeras** no lugar de segredos estáticos (a federação OIDC entre o provedor de CI e a nuvem, por exemplo, troca um token temporário por um papel, sem chave guardada); **mTLS** com certificados de vida curta; padrões de identidade de carga como o **SPIFFE** (cada serviço recebe um ID `spiffe://domínio/serviço` e um documento assinado, o SVID); e segredos em **cofre**, com rotação (p. 135).

CONTROLE DE ACESSO PRIVILEGIADO (PAM)	O QUE FAZ	POR QUÊ
Contas administrativas separadas	O admin usa uma conta nominal distinta da conta do dia a dia	Phishing na conta comum não entrega privilégio
Acesso *just-in-time*	Privilégio concedido por tempo limitado, com aprovação	Elimina privilégios permanentes
Cofre de credenciais	Senhas e chaves privilegiadas entregues sob demanda e rotacionadas	Ninguém conhece a senha de root
Gravação de sessão	Sessões privilegiadas registradas e revisáveis	Detecção e não repúdio
Conta de emergência (*break-glass*)	Conta rara, protegida e monitorada, para falha do IdP	Evita exceções permanentes

06 IDENTIDADE E AUTENTICAÇÃO

CONTINUAÇÃO

ESTUDO DE CASO

Uber, 2022: o push que alguém aprovou. Segundo a empresa e os relatos públicos da época, um invasor obteve credenciais de um **prestador de serviço** e tentou entrar repetidamente; o segundo fator era uma notificação *push* de aprovar ou negar. Diante da insistência dos pedidos (*MFA fatigue*) e de mensagens de alguém que se passava pela equipe de TI, a vítima aprovou um deles, e o invasor entrou na rede.

Controles que mudariam o desfecho.

- MFA **resistente a phishing** (FIDO2) para quem acessa sistemas internos
- Se houver push: **correspondência de número**, limite de pedidos e alerta a cada rajada
- Terceirizados com o mesmo rigor dos funcionários: dispositivo gerido, acesso mínimo e MFA forte
- Tempo para **detectar e revogar** sessões suspeitas (p. 144, p. 156)

PARA FIXAR

Conceitos

- Diferencie identificação, autenticação, autorização e contabilização.
- Por que o sal e o pepper resolvem problemas diferentes?
- Por que uma passkey resiste a phishing e um código TOTP não?

Aplicação

- Projete o cadastro e o login com Argon2id: o que vai ao banco, como migrar hashes antigos e como calibrar o custo?
- Liste as validações que o backend deve fazer em um JWT e explique o risco de aceitar o `alg` do cabeçalho.
- Uma SPA precisa chamar uma API em nome do usuário: escolha o fluxo OAuth e explique o papel do PKCE e do `state`.

APROFUNDE-SE

Leituras e referências

- NIST SP 800-63-4 — NIST: diretrizes de identidade digital, autenticação (63B) e federação (63C).
- Password Storage Cheat Sheet — OWASP: parâmetros atuais de Argon2id, scrypt, bcrypt e PBKDF2.
- Authentication Cheat Sheet — OWASP: login, recuperação, MFA e respostas uniformes.
- RFC 9700 — IETF: boas práticas atuais de segurança do OAuth 2.0.
- RFC 8725 — IETF: boas práticas para JSON Web Tokens.
- Passkeys — FIDO Alliance: como funcionam as passkeys; especificação em W3C WebAuthn.

MÓDULO 07 · ACESSO

07 AUTORIZAÇÃO E CONTROLE DE ACESSO

Decidir o que cada identidade pode fazer — a falha número um das aplicações modernas.

Autenticação responde "quem é você?"; **autorização** responde "o que você pode fazer?". É na segunda pergunta que a maioria das aplicações falha: o **controle de acesso quebrado** lidera o OWASP Top 10:2025 (A01), e as fraquezas de autorização ausente (CWE-862) e incorreta (CWE-863) ocupam o 4º e o 17º lugar do CWE Top 25 de 2025. Este módulo apresenta os princípios, os modelos (DAC, MAC, RBAC, ABAC e ReBAC), a arquitetura de decisão (PEP/PDP), as falhas clássicas e como projetar e testar um controle de acesso sólido.

PRINCÍPIOS QUE ORIENTAM TODA DECISÃO DE ACESSO

Os princípios de projeto de Saltzer e Schroeder (p. 5) viram regras práticas quando se trata de acesso. Antes de escolher um modelo, a equipe precisa concordar em seis ideias.

01 MENOR PRIVILÉGIO

Conceder só o necessário e só pelo tempo necessário. Reduz o estrago de um erro ou de uma invasão.

02 NEGAR POR PADRÃO

Sem regra explícita, a resposta é "não". Falhas e exceções devem **fechar** o acesso (*fail closed*), nunca abri-lo.

03 MEDIAÇÃO COMPLETA

Toda requisição a todo recurso é verificada, sempre. Nada de "já foi checado antes" nem *cache* de permissão sem invalidação.

04 SEPARAÇÃO DE FUNÇÕES

Quem pede não aprova; quem desenvolve não publica sozinho em produção. Ações críticas exigem duas pessoas.

05 NEED-TO-KNOW

Acesso à informação só a quem precisa dela para a tarefa, mesmo entre pessoas de mesmo nível ou cargo.

06 DECIDIR NO SERVIDOR

Esconder um botão não é autorizar. A decisão ocorre no servidor, em cada operação, com dados do servidor.

MODELOS DE CONTROLE DE ACESSO

Um **modelo de controle de acesso** define **como** se decide quem pode fazer o quê com qual recurso. Dois conceitos antecedem os modelos. A **ACL** (*Access Control List*) fica no **recurso** e lista quem pode fazer o quê (como as permissões de arquivos do Unix); a **capability** é um **token infalsificável** que o sujeito carrega e que dá direito a um recurso específico (um *link* assinado, um *file descriptor*). As capabilities evitam o problema do **delegado confuso** (*confused deputy*), em que um serviço com muitos privilégios é induzido a usá-los em nome de quem não os tem.

MODELO	COMO DECIDE	PONTOS FORTES	LIMITES E USO TÍPICO
DAC (discricionário)	O **dono** do recurso concede os acessos (`chmod`, compartilhar um arquivo)	Flexível e simples	Dono descuidado vaza dados; difícil de auditar. Arquivos e colaboração
MAC (obrigatório)	O **sistema** compara rótulos de sujeito e objeto; o dono não decide	Garantia forte; política central	Rígido e caro de administrar. SELinux, AppArmor, ambientes militares

07 AUTORIZAÇÃO E CONTROLE DE ACESSO — CONTINUAÇÃO

MODELO	COMO DECIDE	PONTOS FORTES	LIMITES E USO TÍPICO
RBAC (papéis)	Permissões ligadas a **papéis**; usuários recebem papéis; hierarquia e separação de funções (NIST, ANSI INCITS 359)	Fácil de entender e auditar; combina com o ciclo JML	Explosão de papéis; não expressa contexto. Padrão corporativo
ABAC (atributos)	Regras sobre atributos de sujeito, recurso, ação e ambiente (NIST SP 800-162)	Granular e dinâmico; poucas regras cobrem muito	Política difícil de testar; depende de dados de qualidade. Regras contextuais
ReBAC (relações)	Decide por **relações** em um grafo (dono, membro, editor): modelo Zanzibar, do Google	Natural para compartilhamento e hierarquias	Exige armazenar e consultar o grafo. Documentos, repositórios, SaaS

Bell–LaPadula × Biba: no MAC clássico, o modelo **Bell–LaPadula** protege o **sigilo**: não ler níveis acima (*no read up*) e não escrever em níveis abaixo (*no write down*), para que um segredo não vaze para um nível público. O modelo **Biba** protege a **integridade** e inverte as regras: não ler abaixo e não escrever acima, para que dado pouco confiável não contamine dado confiável.

No **RBAC** do NIST, usuários recebem **papéis**, papéis recebem **permissões** e uma **sessão** ativa um subconjunto de papéis. A **hierarquia** (um papel herda outro) reduz repetição; as restrições de **separação de funções** impedem acúmulos perigosos: a SSD (*static*) proíbe que a mesma pessoa tenha dois papéis conflitantes (solicitante e aprovador de pagamento) e a DSD (*dynamic*) proíbe ativá-los na mesma sessão. No **ReBAC**, a unidade é a **tupla de relação**: `documento:ementa#editor@usuario:ana` diz que Ana edita a ementa, e "quem edita também lê" é derivado (**OpenFGA** e **SpiceDB** implementam o modelo). Sistemas maduros **combinam** modelos: RBAC para o grosso, ABAC para exceções contextuais e ReBAC para compartilhamento.

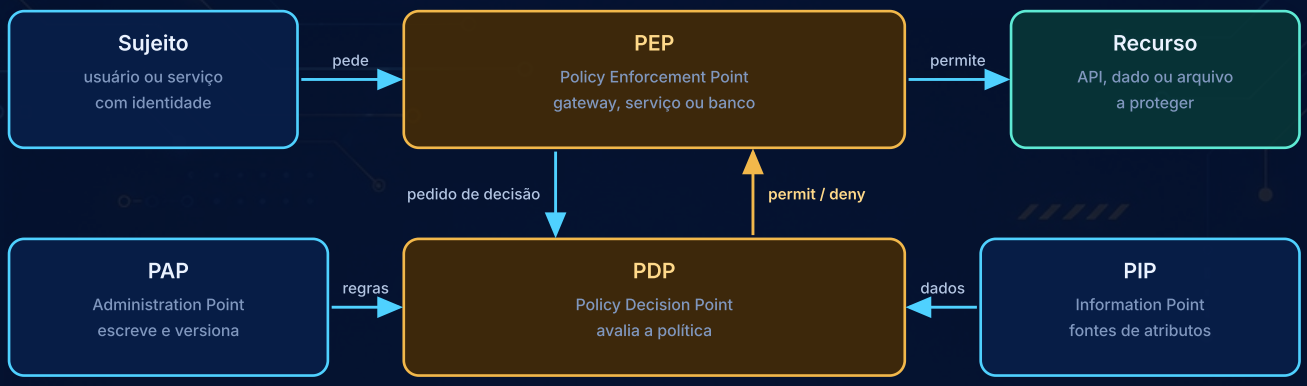
ARQUITETURA DE DECISÃO E POLÍTICAS COMO CÓDIGO

Espalhar `if usuario.admin` pelo código é o caminho para falhas de autorização. A arquitetura de referência (XACML e NIST SP 800-162) **separa a decisão da aplicação**: o **PEP** (*Policy Enforcement Point*) intercepta a requisição e impõe a resposta; o **PDP** (*Policy Decision Point*) avalia a política e responde *permit* ou *deny*; o **PIP** (*Policy Information Point*) fornece atributos (diretório, RH, risco); o **PAP** (*Policy Administration Point*) é onde as políticas são escritas, revisadas e versionadas. Com isso, regras mudam sem alterar código, e todas as decisões podem ser registradas em um só lugar.

Há três lugares para impor a regra, e eles se **somam**: o **gateway** ou *middleware* (autenticação e regras grossas, como "só professores acessam `/turmas`"), o **serviço** (regras de negócio, como "só o professor desta turma lança a nota") e o **banco de dados** (*row-level security*, a última linha de defesa). Mecanismos como o **OPA** (*Open Policy Agent*, linguagem Rego, projeto graduado da CNCF), o **Cedar** (da AWS) e o próprio **XACML** (padrão OASIS) permitem escrever a política como código: versionada, revisada por pares e **testada** em CI.

07 AUTORIZAÇÃO E CONTROLE DE ACESSO

CONTINUAÇÃO



Entradas da decisão: sujeito (papéis, grupos), ação, recurso (dono, classificação) e ambiente (horário, dispositivo, risco).
Toda decisão deve ser registrada em log (quem, o quê, quando, resultado) para auditoria e investigação.

Arquitetura de decisão de acesso (XACML, NIST SP 800-162 e 800-207). O PEP intercepta cada requisição e consulta o PDP; o PDP pede atributos ao PIP e aplica as políticas escritas no PAP; a decisão (permit ou deny) volta ao PEP, que a impõe.

POLÍTICA EM REGO (OPA): NEGAR POR PADRÃO E PERMITIR CASOS EXPLÍCITOS

```
package portal.authz

import rego.v1

default allow := false                          # negar por padrão

allow if {                                      # o aluno lê apenas as próprias notas
    input.action == "ler"
    input.resource.tipo == "nota"
    input.resource.aluno_id == input.user.id
}

allow if {                                      # o professor lança nota de turma que leciona
    input.action == "escrever"
    input.resource.tipo == "nota"
    "professor" in input.user.papeis
    input.resource.turma_id in input.user.turmas
}
```

ROW-LEVEL SECURITY NO POSTGRESQL: O BANCO TAMBÉM NEGA

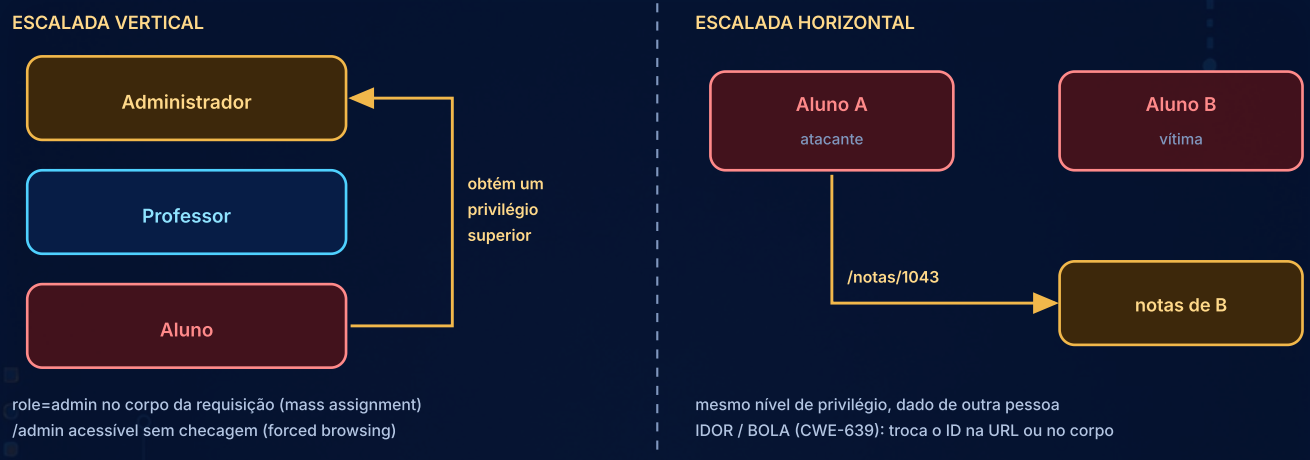
```
ALTER TABLE notas ENABLE ROW LEVEL SECURITY;
ALTER TABLE notas FORCE ROW LEVEL SECURITY;      -- vale também para o dono da tabela

CREATE POLICY aluno_ve_as_proprias ON notas
    FOR SELECT
    USING (aluno_id = current_setting('app.usuario_id')::int);    -- definido pela aplicação a cada requisição
```

07 AUTORIZAÇÃO E CONTROLE DE ACESSO — CONTINUAÇÃO

FALHAS DE AUTORIZAÇÃO

Na **escalada vertical** o atacante sobe de papel; na **horizontal**, mantém o papel, mas alcança dados de outro usuário. A segunda é a mais frequente e tem nome próprio: **IDOR** (*Insecure Direct Object Reference*) em aplicações web e **BOLA** (*Broken Object Level Authorization*, API1 do OWASP API Security Top 10:2023) em APIs, catalogada como CWE-639. A causa é quase sempre a mesma: o servidor confia em um **identificador enviado pelo cliente** (na URL, no corpo ou em um cabeçalho) sem verificar se aquele objeto pertence ao usuário autenticado.



Escalada vertical × horizontal. Na vertical, o atacante sobe de papel (aluno → administrador). Na horizontal, continua com o mesmo papel, mas alcança dados de outro usuário. A horizontal (IDOR/BOLA) é a falha de autorização mais comum em aplicações web e APIs.

FALHA	EXEMPLO	CORREÇÃO
IDOR / BOLA (CWE-639)	`GET /notas/1043` devolve a nota de outro aluno	Consultar já filtrando pelo dono ou checar a propriedade; IDs não sequenciais só **ajudam**, não autorizam
Autorização ausente (CWE-862)	Endpoint administrativo esquecido, sem nenhuma checagem	Autorização central por padrão; teste que falha se houver rota sem política
Autorização incorreta (CWE-863)	Checa o papel, mas não a turma ou o tenant do recurso	Políticas que olham o recurso, não só o papel
***Mass assignment* (CWE-915)**	O cliente envia `{"papel": "admin"}` e o *framework* grava o campo	Lista de permissão de campos editáveis; DTOs separados da entidade
Checagem só no cliente	O botão "Excluir" some, mas a rota continua aberta	Toda decisão no servidor; a interface apenas reflete o resultado
***Forced browsing* (CWE-425)**	Acessar `/admin` ou `/backup.zip` digitando a URL	Negar por padrão; não expor arquivos e rotas internas
Confiança cega em *claims*	Aceitar `role` de cabeçalho ou de token sem validar, ou *claims* já desatualizadas	Validar o token (p. 44); consultar o papel atual quando o risco justificar

07 AUTORIZAÇÃO E CONTROLE DE ACESSO — CONTINUAÇÃO

ENDPOINT VULNERÁVEL × CORRIGIDO (IDOR E MASS ASSIGNMENT)

```
@app.get("/api/notas/{nota_id}")
def ler_nota(nota_id: int, user=Depends(usuario_atual)):
-    return db.query_one("SELECT * FROM notas WHERE id = %s", (nota_id,))
+    nota = db.query_one(
+        "SELECT * FROM notas WHERE id = %s AND aluno_id = %s", (nota_id, user.id))
+    if nota is None:
+        raise HTTPException(404)                # não revela se a nota existe
+    return nota

@app.patch("/api/perfil")
def editar_perfil(corpo: dict, user=Depends(usuario_atual)):
-    user.update(**corpo)                        # {"papel": "admin"} também passa
+    EDITAVEIS = {"nome", "telefone"}            # lista de permissão de campos
+    user.update(**{k: v for k, v in corpo.items() if k in EDITAVEIS})
```

Escopos OAuth não são permissões do usuário. Um token com o escopo `notas:ler` indica o que o **aplicativo** pode pedir em nome do usuário, mas não diz **quais notas** o usuário pode ler: a API continua obrigada a autorizar cada objeto. Em sistemas **multi-inquilino** (*multi-tenant*), isole por `tenant_id` em **toda** consulta e reforce com RLS; para isolamento mais forte, use um esquema ou um banco por cliente, ao custo de operação. Cuidado com *caches* e *buckets* sem o inquilino na chave. E lembre-se: superusuários e papéis com `BYPASSRLS` ignoram a RLS, então a aplicação **não** deve conectar-se com eles.

ZERO TRUST, ACESSO PRIVILEGIADO E REVISÃO CONTÍNUA

O modelo de "castelo e fosso" (confiar em tudo que está dentro da rede) falhou diante de VPNs comprometidas, nuvem e trabalho remoto. O **Zero Trust** (NIST SP 800-207, 2020) parte do oposto: **nenhuma rede, usuário ou dispositivo é confiável por localização**; cada acesso é decidido por política, com base em identidade, estado do dispositivo e contexto. Na arquitetura do NIST, o **mecanismo de política** (o PDP) decide e o PEP aplica, exatamente o desenho da figura anterior.

Princípios (NIST SP 800-207)

- Todo dado e serviço é um **recurso** a proteger
- Toda comunicação é protegida, **qualquer que seja a rede**
- Acesso concedido **por sessão**, ao recurso individual
- Política **dinâmica**: identidade, dispositivo e comportamento
- Postura de segurança dos ativos **monitorada** continuamente
- Autenticação e autorização **estritas** antes de cada acesso
- Coleta contínua de informação para melhorar a política

Na prática

- MFA resistente a phishing e dispositivos gerenciados
- **Microssegmentação** e mTLS entre serviços (**p. 67**)
- Acesso *just-in-time* e contas privilegiadas separadas (**p. 44**)
- **Recertificação**: donos de recurso confirmam a cada trimestre quem deve manter o acesso; o restante é removido
- Logs de decisão, alertas de acesso anômalo e revogação rápida (**p. 144**)
- IAM de nuvem com o mesmo rigor (**p. 78**)

COMO TESTAR AUTORIZAÇÃO

Autorização quebra em silêncio: a tela funciona para quem deve acessar e **ninguém testa quem não deve**. A prática é montar uma **matriz de testes** (perfil × recurso × ação) a partir dos requisitos, executá-la com **contas reais de cada perfil** e incluir **testes negativos** no CI. O **OWASP ASVS** dedica o capítulo V8 à autorização; ferramentas como o ZAP e a extensão *Autorize*, do Burp, repetem requisições de um perfil com a sessão de outro; e as políticas em OPA têm testes unitários próprios (`opa test`).

07 AUTORIZAÇÃO E CONTROLE DE ACESSO

CONTINUAÇÃO

EXEMPLO RESOLVIDO

Matriz de testes de autorização do portal acadêmico. Cada linha da tabela abaixo vira um teste automatizado, executado com contas reais de cada perfil. O resultado esperado inclui o código HTTP: **404** quando o recurso nem deve ser revelado e **403** quando a ação é proibida.

AÇÃO SOBRE O RECURSO	ALUNO	PROFESSOR	COORDENAÇÃO
Ler as próprias notas	Sim	Não se aplica	Não se aplica
Ler notas de outro aluno	**Não** (404)	Só da própria turma	Só do próprio curso
Lançar ou alterar nota	**Não** (403)	Só da turma que leciona	Não (403)
Alterar o papel de um usuário	**Não** (403)	**Não** (403)	**Não** (403)
Exportar todas as notas	**Não** (403)	**Não** (403)	Só com aprovação e log

CENÁRIO

O boletim abre em `/notas/1042`. Um aluno troca o número para `1043` e vê a nota de outra pessoa. **Diagnóstico:** BOLA/IDOR (CWE-639), isto é, autorização ausente no nível do objeto. **Correção:** consultar a nota já filtrando por `aluno_id` do usuário autenticado (ou aplicar uma política que compare dono e usuário) e responder 404. **Teste de regressão:** com as contas de dois alunos, o aluno A deve receber 404 ao pedir qualquer ID do aluno B. **Defesa em profundidade:** RLS no banco, para que um erro na aplicação não exponha dados.

PARA FIXAR

Conceitos

- Diferencie DAC, MAC, RBAC, ABAC e ReBAC e dê um caso de uso de cada.
- Qual a função de cada componente: PEP, PDP, PIP e PAP?
- Por que “negar por padrão” e “mediação completa” previnem classes inteiras de falhas?

Aplicação

- Modele em RBAC os papéis de um portal acadêmico e liste dois conflitos de separação de funções.
- Reescreva em Rego a regra “o professor só lança nota na turma que leciona e nunca depois do fechamento do semestre”.
- Escreva três testes negativos de autorização para o endpoint `PATCH /usuarios/{id}`.

APROFUNDE-SE

Leituras e referências

- NIST SP 800-162 — NIST: definição e considerações sobre controle de acesso baseado em atributos (ABAC).
- NIST SP 800-207 — NIST: arquitetura de Zero Trust.
- Authorization Cheat Sheet — OWASP: práticas de autorização e prevenção de IDOR.
- OWASP API Security Top 10 — OWASP: BOLA, BFLA e autorização em nível de propriedade.
- Open Policy Agent — OPA/CNCF: documentação de Rego e testes de política.
- OpenFGA — CNCF: modelagem de autorização baseada em relações (Zanzibar).

MÓDULO 08 · DADOS

08 PROTEÇÃO DE DADOS E PRIVACIDADE

Proteger a informação em todas as fases — da coleta ao descarte — e respeitar os direitos dos titulares.

Dados são o que os atacantes querem e o que a lei obriga a proteger. Este módulo mostra como **classificar** o que se tem, proteger o dado em cada **estado** e em cada **fase do ciclo de vida**, reduzir o risco com **mascaramento, pseudonimização e anonimização**, garantir a recuperação com **backups** e aplicar a **LGPD** (Lei nº 13.709/2018) com profundidade técnica. A pergunta que guia tudo: *quem precisa deste dado, por quanto tempo e contra quem ele deve ficar protegido?*

CLASSIFICAÇÃO E INVENTÁRIO DE DADOS

Não se protege o que não se conhece. O primeiro passo é o **inventário e mapeamento de dados**: que dados existem, onde estão (bancos, planilhas, *buckets*, logs, backups, SaaS), quem os acessa, por que são tratados e por quanto tempo. A LGPD exige que controlador e operador mantenham o **registro das operações de tratamento** (art. 37, o *RoPA*). Sobre o inventário aplica-se a **classificação**, que define o nível de proteção: **pública** (pode ser divulgada), **interna** (uso da organização), **confidencial** (vazamento causa dano relevante) e **restrita** (o pior caso: segredos, credenciais, dados sensíveis). Cada classe tem regras de acesso, cifragem, retenção e descarte; sem regras, a classificação vira apenas rótulo.

DADO DO PORTAL	CLASSIFICAÇÃO	TRATAMENTO ADEQUADO
CPF	Confidencial (dado pessoal)	Coletar só se necessário; cifrar o campo (envelope, p. 34); mascarar na interface (`***.456.789-**`); registrar quem lê
Notas e histórico	Confidencial	Acesso por aluno e por turma (p. 53); cifragem em repouso; retenção conforme as normas acadêmicas
E-mail institucional	Interna (dado pessoal)	Visível só a quem precisa; sem listas expostas; base legal definida para comunicações externas
Senha	Restrita (segredo)	Nunca guardar a senha: só o *hash* Argon2id com sal (p. 44); jamais em log
Log de acesso (IP, horário)	Interna ou confidencial	Retenção mínima, com os prazos do Marco Civil quando aplicáveis; acesso restrito; integridade protegida
Foto do aluno	Confidencial; sensível se usada para reconhecimento facial	Armazenamento cifrado e acesso restrito; reconhecimento facial só com base legal e relatório de impacto (RIPD)

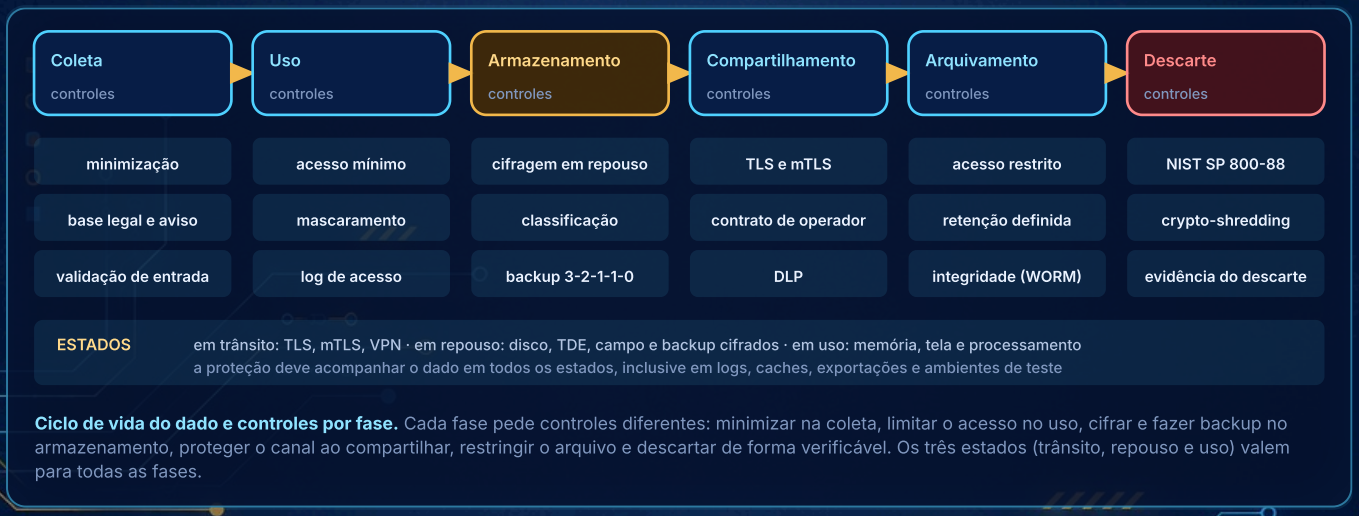
ESTADOS DO DADO E ONDE CIFRAR

O dado existe em três **estados**: **em trânsito** (TLS 1.3, mTLS, VPN: p. 34 e p. 67), **em repouso** (disco, banco, backup) e **em uso** (memória, tela, processamento; aqui entram o controle de acesso e, em casos extremos, o *confidential computing*, que roda o código em enclaves de hardware). Mas "cifrar em repouso" não é uma decisão única: **a camada em que se cifra define contra quem você protege**.

08 PROTEÇÃO DE DADOS E PRIVACIDADE — CONTINUAÇÃO

CAMADA DE CIFRAGEM	PROTEGE CONTRA	NÃO PROTEGE CONTRA
Disco ou volume (LUKS, BitLocker, volumes da nuvem)	Perda ou roubo do disco ou do equipamento	Qualquer processo ou usuário com o sistema montado e em execução
Banco de dados (TDE)	Roubo dos arquivos de dados e dos backups do banco	SQL injection e administradores: o banco decifra para quem consulta
Campo ou aplicação (envelope)	Vazamento do banco, SQL injection e DBA sem a chave	Comprometimento da aplicação, que detém a chave; busca e índices ficam limitados
Ponta a ponta (E2EE)	O próprio provedor do serviço	Dispositivo do usuário comprometido; perda da chave é perda do dado

O dado também percorre um **ciclo de vida**: **coleta → uso → armazenamento → compartilhamento → arquivamento → descarte**. Cada fase pede controles próprios, e a fase mais esquecida é o fim: dados que ninguém mais usa continuam vazando em backups, réplicas, exportações e ambientes de teste.



Ciclo de vida do dado e controles por fase. Cada fase pede controles diferentes: minimizar na coleta, limitar o acesso no uso, cifrar e fazer backup no armazenamento, proteger o canal ao compartilhar, restringir o arquivo e descartar de forma verificável. Os três estados (trânsito, repouso e uso) valem para todas as fases.

Em resumo: use a **combinação** de camadas conforme a ameaça: cifragem de disco e TDE são o piso (baratas e quase transparentes); a cifragem de **campo** é o diferencial para os dados mais sensíveis; e nada disso substitui o controle de acesso e a minimização.

MASCARAMENTO, TOKENIZAÇÃO, PSEUDONIMIZAÇÃO E ANONIMIZAÇÃO

Reduzir o risco do dado é, muitas vezes, melhor do que protegê-lo: o que não existe não vaza. As técnicas abaixo são frequentemente confundidas, e a diferença é **jurídica e técnica**: um dado **pseudonimizado continua sendo dado pessoal** (a LGPD o define no art. 13, §4º, como o dado que só se associa a uma pessoa com uma informação adicional guardada separadamente), enquanto o **anonimizado** deixa de ser dado pessoal, **salvo se o processo puder ser revertido** com esforços próprios ou razoáveis (art. 12).

08 PROTEÇÃO DE DADOS E PRIVACIDADE — CONTINUAÇÃO

TÉCNICA	REVERSÍVEL?	AINDA É DADO PESSOAL?	USO TÍPICO
Mascaramento (*masking*)	Não, na visão exibida; o original existe em outro lugar	Sim, enquanto o original existir	Telas e relatórios (`***.456.789-**`); dados fictícios em ambientes de teste
Tokenização	Sim, por consulta a um cofre de tokens	Sim	Cartões de pagamento (PCI DSS); CPF em sistemas periféricos
Pseudonimização	Sim, com a chave ou tabela guardada à parte	**Sim**	Análises, pesquisa e junção de tabelas sem expor o identificador
Anonimização	Não, por meios razoáveis	**Não**, se for irreversível de fato	Estatísticas e dados abertos

Um erro clássico: **"anonimizar" com hash simples**. Um CPF tem 9 dígitos de base (10^9 combinações possíveis); calcular o SHA-256 de todas leva segundos em uma GPU, de modo que `SHA256(CPF)` é facilmente revertido por **ataque de dicionário**. O mesmo vale para telefones e datas de nascimento. O caminho correto é o **HMAC com uma chave secreta** guardada no KMS, que produz um **pseudônimo estável** (o mesmo CPF gera sempre o mesmo valor, o que permite junções) e continua sendo dado pessoal, mas só é revertido por quem tem a chave.

PSEUDONIMIZAÇÃO COM HMAC E MASCARAMENTO DE CPF (PYTHON)

```python
import hashlib, hmac, os

CHAVE = bytes.fromhex(os.environ["PSEUDONIMO_KEY_HEX"])   # guardada no KMS, nunca no banco

def so_digitos(cpf: str) -> str:
    return "".join(c for c in cpf if c.isdigit())

def pseudonimizar(cpf: str) -> str:
    # mesmo CPF gera o mesmo pseudônimo; sem a chave não dá para recalcular nem testar candidatos
    return hmac.new(CHAVE, so_digitos(cpf).encode(), hashlib.sha256).hexdigest()

def mascarar(cpf: str) -> str:
    d = so_digitos(cpf)
    return f"***.{d[3:6]}.{d[6:9]}-**"      # exibe só o miolo: suficiente para conferência visual
```

Para **anonimizar um conjunto de dados** não basta remover nome e CPF: os **quase-identificadores** (CEP, data de nascimento, sexo, profissão) permitem reidentificar pessoas por cruzamento com outras bases; estudos clássicos (Sweeney, 2000) mostraram que CEP, data de nascimento e sexo bastam para identificar de modo único a maior parte da população. O **k-anonimato** exige que cada registro seja indistinguível de pelo menos $k - 1$ outros nos quase-identificadores, usando **generalização** (idade 23 → faixa 20–29; CEP completo → 5 primeiros dígitos) e **supressão**; ele falha quando os registros do grupo compartilham o mesmo valor sensível (*ataque de homogeneidade*), o que motivou refinamentos como *l-diversity* e *t-closeness*. A **privacidade diferencial** troca a ideia de "esconder registros" por uma garantia matemática: adiciona ruído calibrado aos **resultados** de consultas, de modo que a presença ou ausência de qualquer pessoa altera pouco a saída; o parâmetro **ε** (épsilon, o orçamento de privacidade) mede o compromisso, e quanto menor, mais privacidade e menos precisão. Foi usada, por exemplo, no Censo de 2020 dos EUA.

08 PROTEÇÃO DE DADOS E PRIVACIDADE — CONTINUAÇÃO

ESTUDO DE CASO

AOL (2006) e Netflix (2006–2008): quando o pseudônimo não esconde ninguém. Em agosto de 2006, a AOL publicou para pesquisa cerca de 20 milhões de buscas de mais de 650 mil usuários, trocando os nomes por números. Em poucos dias, o *New York Times* identificou a usuária nº 4417749, uma viúva de 62 anos de Lilburn, na Geórgia, só pelo conteúdo das buscas (nomes, sobrenomes e lugares), cruzado com listas telefônicas. A AOL retirou a base do ar, mas as cópias já circulavam, e seu diretor de tecnologia pediu demissão. No mesmo ano, a Netflix divulgou avaliações de cerca de 500 mil assinantes, sem nomes, para um concurso de algoritmos de recomendação. Em 2008, Narayanan e Shmatikov mostraram que poucas avaliações e datas aproximadas, cruzadas com avaliações públicas no IMDb, bastavam para achar o registro de uma pessoa; a segunda edição do concurso foi cancelada em 2010, em meio a um processo judicial e a questionamentos da FTC.

- **Comportamento identifica:** buscas, avaliações, trajetos e compras formam uma "impressão digital" mesmo sem nome.
- **O risco depende do que o atacante já sabe** (dados auxiliares), e isso só cresce com o tempo.
- **Conjuntos esparsos e de muitas dimensões** são os mais difíceis de anonimizar.
- **Na prática:** trate o conjunto apenas pseudonimizado como dado pessoal, com controle de acesso e contrato, salvo se a anonimização for demonstrada por testes de reidentificação, k-anonimato ou privacidade diferencial.

RETENÇÃO, DESCARTE SEGURO E PREVENÇÃO DE VAZAMENTO

Minimizar é a medida mais barata: colete só o necessário (princípio da necessidade, LGPD art. 6º, III), defina **prazos de retenção** por tipo de dado e **automatize a exclusão** (TTL em bancos e *buckets*, rotinas agendadas). Ao fim do prazo, o **descarte seguro** segue o **NIST SP 800-88** (Rev. 2, de setembro de 2025), que mantém três métodos de saneamento de mídia: **Clear** (sobrescrever por meios lógicos), **Purge** (tornar a recuperação inviável mesmo em laboratório: apagamento criptográfico, *block erase*, desmagnetização) e **Destroy** (destruição física). Na nuvem, onde não se tem acesso ao hardware, o caminho é o **apagamento criptográfico** (*crypto-erase*): destruir a chave que cifra o dado inutiliza também as cópias e os backups. A norma só o aceita como método *Purge* sob condições: dado sempre cifrado desde a gravação, algoritmo de pelo menos 128 bits em módulo criptográfico validado (FIPS) e **zeroização** de todas as cópias da chave; o descarte deve gerar **evidência** (certificado de saneamento).

A **DLP** (*Data Loss Prevention*) monitora e bloqueia saídas indevidas de dados. Ela inspeciona o conteúdo (padrões como CPF e número de cartão com verificação de Luhn, palavras-chave, impressões digitais de documentos) em e-mail, uploads, USB, nuvem e *endpoints*, e aplica ações: alertar, bloquear ou cifrar. Funciona bem quando a **classificação** já foi feita; sem ela, gera apenas ruído.

BACKUP E RESILIÊNCIA DE DADOS

O backup é a última linha de defesa contra erro humano, falha de hardware, corrupção e, sobretudo, **ransomware**, que hoje procura e destrói os backups antes de cifrar a produção. A regra **3-2-1** (três cópias, duas mídias, uma fora do local) evoluiu para o **3-2-1-1-0**: acrescenta **uma cópia offline ou imutável** (fita, *air gap*, ou *Object Lock* e WORM na nuvem, em que nem o administrador consegue apagar antes do prazo) e **zero erros** na verificação de restauração. As credenciais do backup devem ser **distintas** das de produção, com MFA, para que um invasor que controle o domínio não apague as cópias.

08 PROTEÇÃO DE DADOS E PRIVACIDADE

CONTINUAÇÃO



Replicação em tempo real não é backup: ela copia também a exclusão acidental e a criptografia do ransomware.

Regra 3-2-1-1-0. Três cópias, em duas mídias, uma fora do local, uma offline ou imutável (que o ransomware não consegue apagar ou cifrar) e zero erros na verificação de restauração. A última cópia deve usar credenciais separadas das de produção.

MECANISMO	SERVE PARA	GUARDA HISTÓRICO?	PROTEGE DE RANSOMWARE?
Replicação (espelho, réplica)	Alta disponibilidade e continuidade	Não: replica o erro e a exclusão	Não: replica o dado já cifrado
Backup	Voltar a um estado anterior (versões)	Sim	Sim, se imutável ou offline
Arquivamento	Guarda de longo prazo por obrigação legal ou histórica	Sim, por prazo definido	Parcial; foco em retenção e integridade

EXEMPLO RESOLVIDO

RPO e RTO do banco do portal. O **RPO** (*Recovery Point Objective*) é a perda de dados tolerável, medida em tempo; o **RTO** (*Recovery Time Objective*) é o tempo máximo para voltar ao ar. O banco tem 500 GB, backup completo todo dia às 02:00 e **arquivamento dos logs de transação a cada 5 minutos**.

- **RPO** ≈ 5 minutos: no pior caso, perde-se o que ocorreu desde o último log arquivado. Sem os logs, seria de até 24 horas.
- **RTO**: restaurar 500 GB a 200 MB/s leva 500.000 ÷ 200 = 2.500 s ≈ **42 min**; somam-se cerca de 15 min para reaplicar os logs e 10 min de validação: **cerca de 1 h 10 min**.

Se a diretoria exigir RTO de 30 minutos, a conta mostra o que mudar: restaurar de um *snapshot* mais recente, manter uma réplica pronta para promoção ou reduzir o volume restaurado. E é o **teste de restauração** periódico que transforma esses números em fatos.

08 **PROTEÇÃO DE DADOS E PRIVACIDADE** CONTINUAÇÃO

LGPD APROFUNDADA

A **LGPD** (Lei nº 13.709/2018) vale para qualquer tratamento de dados pessoais de pessoas no Brasil, por pessoa física ou jurídica, de direito público ou privado. Papéis (controlador, operador, encarregado) e princípios foram vistos em **p. 27**; aqui estão os detalhes que a engenharia precisa dominar.

Dado pessoal é a informação relacionada a pessoa natural identificada ou **identificável** (art. 5º, I). **Dado pessoal sensível** (art. 5º, II) revela origem racial ou étnica, convicção religiosa, opinião política, filiação sindical ou a organização religiosa, filosófica ou política, ou é dado referente à saúde ou à vida sexual, dado **genético** ou **biométrico**, quando vinculado a uma pessoa natural; ele exige bases legais mais restritas (art. 11). O tratamento de dados de **crianças e adolescentes** deve atender ao seu **melhor interesse** (art. 14), e o de crianças exige consentimento específico e em destaque de um dos pais ou do responsável legal.

BASE LEGAL (ART. 7º)	QUANDO USAR	EXEMPLO NO PORTAL ACADÊMICO
Consentimento (I)	Livre, informado e inequívoco, para finalidade determinada; pode ser revogado	Boletim de eventos; divulgação de foto em redes sociais
Obrigação legal ou regulatória (II)	Uma lei ou norma exige o tratamento	Guarda de histórico escolar e diplomas
Execução de contrato (V)	Necessário ao contrato ou a procedimentos preliminares pedidos pelo titular	Matrícula e gestão do vínculo com o aluno
Legítimo interesse (IX)	Interesse legítimo do controlador ou de terceiro, após teste de balanceamento, sem prevalecer sobre os direitos do titular	Logs de segurança e prevenção a fraudes
Políticas públicas e pesquisa (III e IV)	Execução de políticas públicas; estudos por órgão de pesquisa, com anonimização sempre que possível	Estatísticas institucionais; pesquisa acadêmica

As demais bases são o exercício regular de direitos em processo (VI), a proteção da vida ou da incolumidade física (VII), a tutela da saúde (VIII) e a proteção do crédito (X). Para o titular, o art. 18 garante direitos que viram **requisitos de sistema**; sem inventário de dados, atendê-los dentro do prazo é inviável.

DIREITO DO TITULAR (ART. 18)	O QUE O SISTEMA PRECISA FAZER
Confirmação e acesso (I e II; art. 19)	Localizar os dados do titular em todos os sistemas e entregá-los: de imediato, em formato simplificado, ou em declaração completa em até 15 dias
Correção (III)	Fluxo de edição que propaga a correção a réplicas, caches e sistemas derivados
Anonimização, bloqueio ou eliminação (IV e VI)	Rotina de exclusão que alcance backups (por expiração ou *crypto-shredding*) e registre as exceções legais de retenção
Portabilidade (V)	Exportação em formato estruturado e interoperável (JSON ou CSV)
Revogação do consentimento (IX)	Registro de consentimentos por finalidade, versão e data, com revogação tão simples quanto a concessão
Revisão de decisões automatizadas (art. 20)	Registrar os critérios e procedimentos da decisão, explicá-los quando solicitado e manter canal para o titular pedir a revisão

08 PROTEÇÃO DE DADOS E PRIVACIDADE — CONTINUAÇÃO

O requisito mais difícil de cumprir é a **eliminação** (art. 18, IV e VI) quando o dado também está em backups imutáveis, que por definição não podem ser alterados. A técnica de engenharia mais usada é o **crypto-shredding**: cifrar os dados de cada titular com uma **chave própria** (a DEK, protegida por uma chave-mestra no KMS, como no envelope de **p. 34**) e, para eliminar, **destruir a chave**. As cópias e os backups continuam existindo, mas ilegíveis. Se isso basta para a lei é uma decisão a validar com o encarregado e o jurídico, que podem exigir também a expiração das cópias pelo ciclo normal de retenção; o registro da eliminação serve de evidência.

CRYPTO-SHREDDING POR TITULAR (PYTHON, BIBLIOTECA CRYPTOGRAPHY)

```
# Crypto-shredding: uma chave de dados (DEK) por titular, protegida pela KEK do KMS
def cadastrar_titular(titular_id: str) -> None:
    dek = AESGCM.generate_key(bit_length=256)
    cofre.salvar(titular_id, kms.cifrar(dek))       # só a DEK cifrada é guardada

def gravar(titular_id: str, dado: bytes) -> bytes:
    dek = kms.decifrar(cofre.ler(titular_id))
    nonce = os.urandom(12)                          # novo a cada cifragem
    return nonce + AESGCM(dek).encrypt(nonce, dado, titular_id.encode())   # AAD: titular

def eliminar_titular(titular_id: str) -> None:
    cofre.apagar(titular_id)                        # sem a DEK, cópias ficam ilegíveis
    auditoria.registrar("eliminacao", titular_id)   # evidência do atendimento (art. 18)
```

Segurança e incidentes (arts. 46 a 49)

- Medidas técnicas e administrativas aptas a proteger os dados, **desde a concepção** (art. 46, §2º)
- O dever de segurança continua após o fim do tratamento (art. 47)
- Incidente com risco ou dano relevante: comunicar a ANPD e os titulares em **3 dias úteis** (Res. CD/ANPD nº 15/2024) e guardar o registro por ≥ 5 anos (**p. 156**)
- Sistemas estruturados para cumprir requisitos de segurança e boas práticas (art. 49)

Governança e sanções

- **Encarregado** (DPO) com identidade e contato divulgados (art. 41)
- **RIPD**, o relatório de impacto, para tratamentos de alto risco; a ANPD pode exigi-lo (art. 38)
- **Transferência internacional** (arts. 33 a 36): países com grau de proteção adequado, cláusulas-padrão, normas corporativas globais ou consentimento específico
- **Sanções** (art. 52): advertência; multa de até 2% do faturamento no Brasil, limitada a R$ 50 milhões por infração; multa diária; publicização, bloqueio e eliminação dos dados

PRIVACIDADE DESDE A CONCEPÇÃO

O **privacy by design**, de Ann Cavoukian, tem sete princípios: ser **proativo** e preventivo; **privacidade como padrão** (*by default*, com a máxima proteção sem ação do usuário); privacidade **embutida no projeto**; **funcionalidade total** (soma positiva, sem trocar privacidade por função); segurança **de ponta a ponta** em todo o ciclo de vida; **visibilidade e transparência**; e **respeito ao usuário**. A LGPD o consagra no art. 46, §2º. Na engenharia isso vira prática: minimizar campos e logs (**não registre CPF nem senha em log**), pseudonimizar por padrão, retenção automática, ambientes de teste com dados fictícios (nunca uma cópia da produção), modelagem de ameaças à privacidade com LINDDUN (**p. 20**) e RIPD antes de funcionalidades de alto risco, como biometria e perfilamento.

DICA DA CORUJA

em toda revisão de projeto, pergunte: “esse campo é realmente necessário?”. Cada campo a menos é um dado a menos para cifrar, mascarar, inventariar, entregar a um titular e deixar vazar.

08 PROTEÇÃO DE DADOS E PRIVACIDADE — CONTINUAÇÃO

CHECKLIST

Proteção de dados em um sistema novo (revise a cada entrega):

- Existe inventário atualizado: que dados, onde estão, quem acessa, base legal, finalidade e prazo de retenção.
- Cada campo coletado tem justificativa; o que não for necessário foi removido.
- A classificação define cifragem, acesso, retenção e descarte, e o dado mais sensível tem cifragem de campo com chaves no KMS.
- Logs, caches, exportações e ambientes de teste não contêm dado pessoal real (ou estão mascarados).
- Pseudonimização por HMAC (e não hash simples) onde o identificador não é necessário; anonimização só com teste de reidentificação.
- Backup 3-2-1-1-0 com cópia imutável, credenciais separadas e restauração testada, com RPO e RTO medidos.
- Fluxos prontos para os direitos do titular (acesso, correção, eliminação, portabilidade) e para a revogação do consentimento.
- Encarregado designado, RIPD feito para tratamentos de alto risco e plano de resposta a incidentes com o prazo de 3 dias úteis.

PARA FIXAR

Conceitos

- Diferencie pseudonimização e anonimização. Por que `SHA256(CPF)` não anonimiza?
- Em que camada de cifragem cada ameaça é barrada: disco roubado, SQL injection e DBA mal-intencionado?
- O que cada número de 3-2-1-1-0 significa, e por que replicação não é backup?

Aplicação

- Classifique cinco dados de uma loja virtual e defina cifragem, retenção e descarte de cada um.
- Um backup completo de 2 TB restaura a 250 MB/s e os logs são arquivados a cada 10 minutos. Calcule o RPO e estime o RTO, somando 20 minutos para reaplicar os logs.
- Desenhe o fluxo de um pedido de eliminação de dados (art. 18, VI) em um sistema com réplicas e backups.

APROFUNDE-SE

Leituras e referências

- Lei nº 13.709/2018 (LGPD) — Planalto: o texto integral, com os artigos citados neste módulo.
- ANPD — governo federal: regulamentos, guias orientativos e canal de comunicação de incidentes.
- NIST SP 800-88 Rev. 2 — NIST: diretrizes de saneamento de mídia (*Clear*, *Purge* e *Destroy*) e de apagamento criptográfico.
- #StopRansomware — CISA: guias de prevenção e de resiliência contra ransomware, incluindo backups.
- How To Break Anonymity of the Netflix Prize Dataset — Narayanan e Shmatikov: o caso clássico de reidentificação.
- The Algorithmic Foundations of Differential Privacy — Dwork e Roth: o texto de referência sobre privacidade diferencial.

MÓDULO 09 · REDES

09 SEGURANÇA DE REDES

Controlar o que entra, sai e circula: segmentação, filtragem, canais seguros e proteção de serviços de rede.

A rede é o caminho por onde os ataques chegam e por onde os dados saem. Este módulo mostra como defendê-la: os **controles por camada**, **firewalls** e o desenho de suas regras, **segmentação** (DMZ, VLAN, microssegmentação), **IDS/IPS**, **VPN** e **Zero Trust**, além dos pontos que costumam falhar na prática: **DNS, e-mail, Wi-Fi, DDoS** e redes **OT/IoT**. A pergunta de projeto é sempre a mesma: *quem precisa falar com quem, em qual porta e em que sentido? Todo o resto deve ser negado.*

CAMADAS, ATAQUES E CONTROLES

O modelo **OSI** (sete camadas) e o **TCP/IP** (quatro: enlace, internet, transporte e aplicação) servem de mapa: cada camada tem protocolos, ataques e controles próprios, e uma defesa em uma só camada deixa as outras abertas, daí a **defesa em profundidade** (p. 5). Um firewall de borda não impede que um usuário interno faça *ARP spoofing* no Wi-Fi, nem que uma aplicação vulnerável responda a um *SQL injection* que entrou pela porta 443.

CAMADA (OSI)	ATAQUES TÍPICOS	CONTROLES PRINCIPAIS
Física (1)	Acesso físico, *tap* de cabo, dispositivo USB malicioso, interferência	Controle de acesso físico, portas de rede desativadas, cabeamento protegido, 802.1X nas tomadas
Enlace (2)	*ARP spoofing*, *MAC flooding*, *VLAN hopping*, servidor DHCP falso (*rogue*)	*Port security*, DAI (*Dynamic ARP Inspection*), *DHCP snooping*, VLANs bem configuradas, 802.1X
Rede (3)	*IP spoofing*, ICMP flood, sequestro de rotas (BGP), fragmentação maliciosa	ACLs e firewalls, filtro anti-spoofing (BCP 38), RPKI, IPsec
Transporte (4)	*SYN flood*, varredura de portas, sequestro de sessão TCP, amplificação UDP	*SYN cookies*, firewall *stateful*, *rate limiting*, fechar as portas desnecessárias
Sessão a aplicação (5–7)	*Man-in-the-middle* e *downgrade* de TLS, envenenamento de DNS, XSS, SQL injection, DDoS HTTP, phishing	TLS 1.3 com HSTS (p. 34), DNSSEC, WAF, validação de entrada (p. 98), filtragem de e-mail, proxy

PROTOCOLOS LEGADOS E SEUS SUBSTITUTOS

Muitos protocolos nasceram em redes que se supunham confiáveis e trafegam credenciais e dados em **texto claro**. Qualquer um que veja o tráfego (um Wi-Fi aberto, um comutador comprometido) lê tudo. A primeira tarefa de quem administra uma rede é substituir ou encapsular o que for legado.

LEGADO (INSEGURO)	PROBLEMA	USE NO LUGAR
Telnet (porta 23)	Usuário e senha em texto claro	SSH (22) com chaves
FTP (21)	Credenciais e arquivos em claro	SFTP (sobre SSH) ou FTPS
HTTP (80)	Sem confidencialidade nem integridade	HTTPS (TLS 1.3) com HSTS

09 **SEGURANÇA DE REDES** CONTINUAÇÃO

LEGADO (INSEGURO)	PROBLEMA	USE NO LUGAR
SNMP v1 e v2c	*Community string* em claro, sem autenticação forte	SNMPv3 com autenticação e cifragem
SMBv1	Protocolo antigo explorado pelo EternalBlue e pelo WannaCry (2017)	SMB 3.x com assinatura e cifragem; SMBv1 desativado
DNS em claro (53)	Consultas visíveis e falsificáveis	DNSSEC (autenticidade) e DoT ou DoH (sigilo)
SSLv3, TLS 1.0 e 1.1	Cifras e construções quebradas; aposentados pelo RFC 8996 (2021)	TLS 1.2 como mínimo e TLS 1.3 preferido

A regra prática: **todo protocolo que envie credenciais ou dados sem cifra deve ser trocado ou colocado dentro de um túnel** (SSH, VPN, TLS). Complemente com um **inventário de portas e serviços** e desligue o que ninguém usa: cada porta aberta é superfície de ataque.

FIREWALLS: TIPOS E DESENHO DE REGRAS

O **firewall** aplica a política de rede: decide quais conexões passam entre zonas. As gerações se acumulam: cada tipo vê mais do tráfego, a um custo maior de processamento e de configuração.

TIPO	COMO DECIDE	LIMITES
Filtro de pacotes (sem estado)	Compara IP, porta e protocolo de cada pacote isoladamente (ACL)	Não entende conexões; exige regras para o tráfego de retorno; fácil de enganar com fragmentos
Stateful	Acompanha o estado das conexões em uma tabela e libera automaticamente o retorno do que foi iniciado de dentro	Não enxerga o conteúdo da aplicação
Proxy / gateway de aplicação	Termina a conexão e inspeciona o protocolo (HTTP, SMTP, DNS); pode ser *proxy* direto (saída) ou reverso (entrada)	Latência; em geral um proxy por protocolo
NGFW (*next-generation*)	*Stateful* mais identificação de aplicações e usuários, IPS, inteligência de ameaças e inspeção de TLS	Inspeção de TLS é cara e levanta questões de privacidade; configuração complexa
WAF (*Web Application Firewall*)	Filtra HTTP e HTTPS por regras (por exemplo, OWASP Core Rule Set), com *rate limiting* e controle de bots	Não corrige a aplicação; pode ser contornado; gera falsos positivos

O **desenho das regras** (NIST SP 800-41 Rev. 1) importa mais do que a marca do equipamento.

09 **SEGURANÇA DE REDES** CONTINUAÇÃO

Princípios de desenho

- **Negar por padrão**: política `DROP` e liberação apenas do que está justificado
- **Menor privilégio**: origem, destino, porta e protocolo os mais específicos possíveis; evite `any any`
- **Filtrar também a saída** (*egress*): um servidor comprometido não deve falar com a internet inteira nem com o servidor de comando e controle
- **Bloquear spoofing**: descartar na borda pacotes que chegam da internet com origem interna ou reservada (RFC 1918, BCP 38)

Operação das regras

- **A ordem importa**: a primeira regra que casa vale, então as negações específicas vêm antes das liberações amplas
- **Cada regra tem dono, justificativa e prazo de revisão**; regras "temporárias" sem prazo viram permanentes
- **Registre** negações e acessos a serviços críticos e envie ao SIEM (p. 144)
- **Mude com controle**: gestão de mudanças, teste em modo de apenas registro e plano de volta

SERVIDOR LINUX COM POLÍTICA PADRÃO DE DESCARTE (NFTABLES)

```
#!/usr/sbin/nft -f
flush ruleset

table inet filtro {
  set admins { type ipv4_addr; elements = { 198.51.100.7, 198.51.100.8 } }

  chain entrada {
    type filter hook input priority 0; policy drop;     # negar por padrão
    ct state established,related accept                 # retorno de conexões legítimas
    ct state invalid drop
    iifname "lo" accept
    meta l4proto { icmp, ipv6-icmp } limit rate 10/second accept   # ICMPv6 é necessário
    tcp dport 443 accept                                # serviço público (HTTPS)
    # SSH somente dos administradores, com limite de novas conexões
    tcp dport 22 ip saddr @admins ct state new limit rate 6/minute accept
  }

  chain saida {
    type filter hook output priority 0; policy drop;    # filtro de saída (egress)
    ct state established,related accept
    oifname "lo" accept
    udp dport 53 ip daddr 10.0.0.53 accept              # só o resolvedor interno
    tcp dport { 80, 443 } ip daddr 10.0.0.3 accept      # só o proxy de saída
  }
}
```

Antes de aplicar: valide a sintaxe sem ativar (`nft -c -f arquivo.nft`) e, em acesso remoto, mantenha uma segunda sessão aberta ou agende a reversão automática: uma regra de entrada errada pode trancar o próprio administrador para fora do servidor.

SEGMENTAÇÃO: CONTER O ESTRAGO

Segmentar é dividir a rede em zonas com regras entre elas, de modo que a invasão de uma máquina **não dê acesso à rede inteira**. É a principal defesa contra o **movimento lateral** (p. 11), a fase em que o invasor salta de um host para outro até chegar aos dados. As ferramentas mudam de escala, mas a ideia é a mesma:

09 SEGURANÇA DE REDES — CONTINUAÇÃO

Sub-redes e VLANs

Uma **VLAN** separa domínios de broadcast no enlace; o tráfego entre VLANs passa por um roteador ou firewall, onde as regras são aplicadas. Sem essas regras, segmentar é só organizar.

DMZ

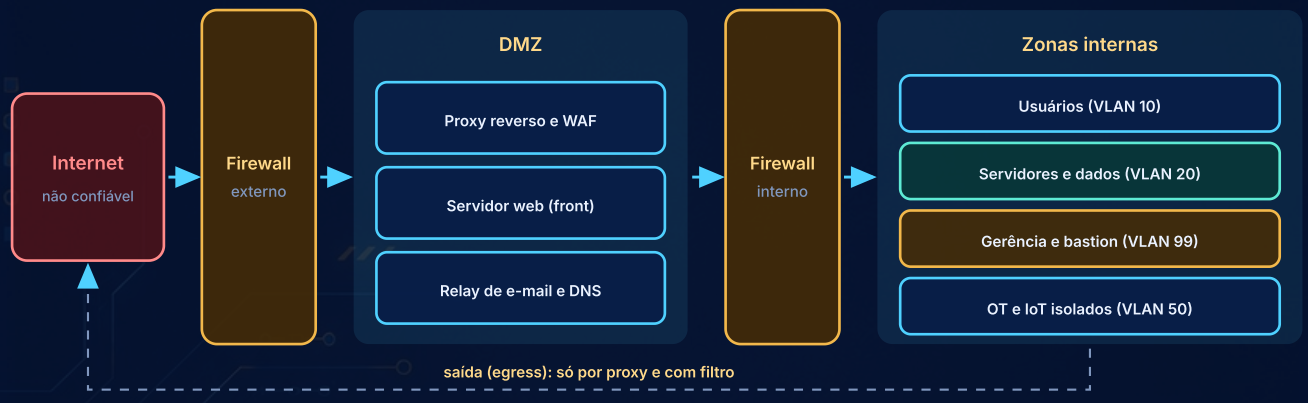
A **zona desmilitarizada** fica entre dois firewalls e abriga os serviços expostos à internet. Se um servidor público for comprometido, o invasor ainda precisa atravessar o firewall interno.

Microssegmentação

Regras por **carga de trabalho** (host firewall, grupos de segurança na nuvem, NetworkPolicy do Kubernetes, p. 78), em vez de por faixa de IP. Cada serviço fala só com os vizinhos de que depende.

Bastion (jump host)

Único ponto de entrada administrativo: endurecido, com MFA e sessões gravadas. Evita expor SSH e RDP de cada servidor e concentra a auditoria.



Internet → DMZ: só 80 e 443 (web) e 25 (e-mail). DMZ → interno: só o necessário (ex.: aplicação → banco, porta 5432).

Interno → internet: por proxy, com filtro de saída. Gerência: só a partir do bastion, com MFA e sessão registrada.

Rede segmentada. Dois firewalls separam três áreas: a internet (não confiável), a DMZ, onde ficam os serviços expostos, e as zonas internas, divididas em VLANs. O que entra e o que sai passa por regras explícitas, e a gerência só ocorre por um bastion; assim, a invasão de um servidor público não alcança os dados.

- Na prática, derive as regras do **diagrama de fluxo de dados** (p. 20): para cada seta do diagrama, uma regra explícita; tudo que não está no diagrama é negado. Um banco de dados, por exemplo, deve aceitar conexões **apenas** dos servidores de aplicação, na porta do SGBD, e nunca da rede dos usuários nem da internet.

ZERO TRUST E ACESSO À REDE

O modelo clássico de "castelo e fosso" trata quem está dentro da rede como confiável. O **Zero Trust** (NIST SP 800-207) rejeita essa premissa: cada requisição é **autenticada, autorizada e avaliada** por identidade, estado do dispositivo e contexto, com acesso mínimo (*nunca confie, sempre verifique*), e a rede interna é tratada como potencialmente hostil. Duas siglas aparecem com frequência: **ZTNA** (*Zero Trust Network Access*) substitui a VPN que concede acesso à rede inteira por acesso **por aplicação**, e **SASE** (*Secure Access Service Edge*) reúne rede (SD-WAN) e segurança (gateway web seguro, CASB, ZTNA e firewall como serviço) entregues na nuvem. Zero Trust não é um produto: é a combinação de identidade forte (p. 44), autorização por requisição (p. 53), mTLS entre serviços (p. 34) e segmentação.

09 **SEGURANÇA DE REDES** CONTINUAÇÃO

IDS, IPS E MONITORAMENTO DE REDE

Um **IDS** (*Intrusion Detection System*) observa e **alerta**; um **IPS** (*Prevention*) fica em linha (*inline*) e **bloqueia**. O **NIDS** inspeciona o tráfego da rede, por meio de um *tap* ou de uma porta espelhada (SPAN); o **HIDS** roda no host e vigia arquivos, logs e processos (OSSEC e Wazuh). Há duas formas de detectar: por **assinatura** (padrões conhecidos: preciso e rápido, mas cego ao ataque novo) e por **anomalia** (desvios de uma linha de base de comportamento: encontra o inédito, mas gera mais falsos positivos); os produtos reais combinam as duas. **Snort** e **Suricata** (assinaturas e IPS, com conjuntos de regras como o ET Open) e **Zeek** (analisa protocolos e gera registros ricos de conexões, DNS, HTTP e TLS) são as ferramentas abertas mais usadas. O que faz o IDS valer é o **ajuste** (*tuning*): um painel com milhares de alertas ignorados é pior do que nenhum.

REGRA SURICATA/SNORT: FORÇA BRUTA EM SSH

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 22 (msg:"Possivel forca bruta SSH"; \
    flags:S; threshold:type both, track by_src, count 10, seconds 60; \
    classtype:attempted-admin; sid:1000001; rev:1;)
```

A regra se lê assim: gere um alerta para segmentos TCP de abertura de conexão (`flags:S`) vindos de fora (`$EXTERNAL_NET`) para a porta 22 da rede protegida (`$HOME_NET`) quando **um mesmo IP de origem** fizer 10 tentativas em 60 segundos. O `sid` identifica a regra (a faixa a partir de 1.000.000 é reservada a regras locais) e o `classtype` a categoriza.

Tráfego: o que registrar

- **NetFlow e IPFIX**: metadados de fluxo (quem falou com quem, quanto e quando); baratos, ótimos para detectar exfiltração e varreduras
- **Captura de pacotes** (`tcpdump`, Wireshark): detalhe máximo, custo e risco de privacidade altos; use sob demanda
- **Logs de DNS, proxy e firewall**: centralizados no SIEM (**p. 144**), correlacionados com identidade

Inventário e varredura

- **No host**, `ss -tulpn` lista as portas em escuta e o processo dono de cada uma
- **Na rede**, `nmap` descobre serviços e versões, **somente em redes próprias ou com autorização escrita**; varrer terceiros pode configurar crime (CP, art. 154-A, **p. 27**)
- Compare o resultado com o inventário: a porta inesperada é o achado

VPN E ACESSO REMOTO

Uma **VPN** (*Virtual Private Network*) cria um túnel cifrado sobre uma rede não confiável. Há dois usos: **acesso remoto** (um usuário ou dispositivo entra na rede da organização) e **site-to-site** (liga duas redes de forma permanente). Três famílias dominam:

ASPECTO	IPSEC (IKEV2)	WIREGUARD	VPN TLS (OPENVPN)
Onde atua	Camada de rede: ESP sobre UDP 500 e 4500 (NAT-T)	Camada de rede: UDP em uma porta configurável	Aplicação, sobre TLS: TCP ou UDP, comumente na 443 ou na 1194
Criptografia	AES-GCM ou ChaCha20-Poly1305; ECDH; certificados ou chave pré-compartilhada	Conjunto fixo e moderno (protocolo Noise): ChaCha20-Poly1305, Curve25519, BLAKE2s	TLS 1.2 ou 1.3 com certificados

09 SEGURANÇA DE REDES — CONTINUAÇÃO

ASPECTO	IPSEC (IKEV2)	WIREGUARD	VPN TLS (OPENVPN)
Pontos fortes	Padrão consolidado; clientes nativos em todos os sistemas; ideal para site-to-site	Código pequeno e fácil de auditar; rápido; no kernel do Linux desde a versão 5.6	Atravessa firewalls restritivos pela porta 443; flexível
Atenção	Muitas opções, configuração complexa e fácil de errar	Poucas opções, sem negociação de cifras; gestão de chaves e de IPs fica por conta do operador	Desempenho menor; superfície maior por ser software em espaço de usuário

Duas decisões de desenho merecem atenção. No **split tunneling**, só o tráfego da organização entra no túnel e o resto sai direto pela internet: melhora o desempenho, mas o dispositivo vira uma ponte entre a internet e a rede interna e o tráfego externo escapa da inspeção; no **full tunnel**, tudo passa pela organização. E **o equipamento de VPN é, ele mesmo, alvo**: gateways VPN e firewalls de borda aparecem com frequência no catálogo KEV da CISA (p. 20). Mantenha-os atualizados, exija MFA resistente a phishing (p. 44), registre os acessos e dê ao usuário acesso **só ao que ele precisa** (ZTNA) em vez da rede inteira.

DNS: CONFIANÇA NO ENDEREÇO

O **DNS** traduz nomes em endereços e é, ao mesmo tempo, ferramenta de ataque e de defesa. As ameaças principais: **envenenamento de cache** (o resolvedor aceita uma resposta falsa; o ataque de Kaminsky, de 2008, mostrou como forjá-la em segundos e levou a portas e identificadores aleatórios), **sequestro de domínio** (a conta do registrador é comprometida; use MFA e *registry lock*), **tunelamento** (dados e comandos escondidos em consultas, para exfiltração e C2), **amplificação** (respostas grandes a consultas pequenas com IP forjado, usadas em DDoS) e **domínios maliciosos** (DGA, *typosquatting*).

DEFESA	O QUE GARANTE	O QUE NÃO GARANTE
DNSSEC	Autenticidade e integridade das respostas, por uma cadeia de assinaturas (DNSKEY, RRSIG, DS) da raiz até a zona	Sigilo; só protege domínios que assinam a zona e resolvedores que validam
DoT (RFC 7858) e DoH (RFC 8484)	Sigilo da consulta entre o cliente e o resolvedor, por TLS (DoT na porta 853; DoH sobre HTTPS)	Autenticidade da resposta; o resolvedor continua vendo tudo
RPZ e *sinkholing*	Bloqueio de domínios maliciosos e identificação de máquinas infectadas, resolvendo-os para um endereço sob controle da organização	Domínios novos ainda não listados
Resolvedor recursivo fechado	Evita que o servidor seja usado como amplificador de DDoS	Nada sobre o conteúdo das respostas

O DoH tem um efeito colateral em redes corporativas: aplicativos podem usar um resolvedor próprio e **escapar do filtro e do monitoramento de DNS**. Padronize o resolvedor da organização (com DoT ou DoH até ele) e bloqueie resolvedores públicos de DoH não autorizados.

E-MAIL: SPF, DKIM E DMARC

O **SMTP** original não autentica o remetente: qualquer servidor pode afirmar que a mensagem vem de `diretoria@empresa.com.br`. Três mecanismos publicados no DNS fecham essa brecha.

09 SEGURANÇA DE REDES — CONTINUAÇÃO

01 SPF: QUEM PODE ENVIAR

O **Sender Policy Framework** (RFC 7208) lista os IPs autorizados a enviar por um domínio; o receptor compara o IP de origem com o domínio do *envelope* (`Return-Path`). Limite de 10 consultas DNS.

02 DKIM: A MENSAGEM É AUTÊNTICA

O **DomainKeys Identified Mail** (RFC 6376) assina cabeçalhos e corpo com uma chave privada; a pública fica em `seletor._domainkey.dominio`. Prova origem e integridade, e resiste ao encaminhamento.

03 DMARC: O QUE FAZER SE FALHAR

O **DMARC** (RFC 9989, que em 2026 substituiu o RFC 7489) exige que SPF ou DKIM passe **e esteja alinhado** ao domínio do `From` visível, define a política de falha e pede relatórios.



SPF: o IP de origem está autorizado no domínio do envelope (Return-Path). DKIM: a assinatura confere com a chave pública do seletor.

DMARC: SPF ou DKIM passa E o domínio está alinhado ao From visível; a política decide o destino. Evolua de none para quarantine e reject.

Autenticação de e-mail em três camadas. O servidor de envio assina a mensagem (DKIM); o receptor consulta no DNS os registros SPF, DKIM e DMARC do domínio remetente, verifica o alinhamento com o endereço visível e aplica a política publicada (none, quarantine ou reject), devolvendo relatórios agregados ao dono do domínio.

O **alinhamento** é o ponto sutil: um golpista pode passar no SPF com o próprio domínio e ainda assim exibir `From: seubanco.com.br`; o DMARC só aceita a mensagem se o domínio autenticado **coincidir** com o do `From`. Implante em etapas: comece com `p=none` apenas para **receber relatórios** e descobrir todos os sistemas que enviam em nome do domínio (CRM, e-mail marketing, chamados), corrija o que falhar e então suba para `quarantine` e `reject`. O **MTA-STS** (RFC 8461) e o **TLS-RPT** (RFC 8460) acrescentam a obrigação de usar TLS na entrega entre servidores, evitando *downgrade*. Autenticar o e-mail deixou de ser opcional: desde fevereiro de 2024 o Gmail exige SPF, DKIM e DMARC de quem envia 5.000 mensagens por dia ou mais (e, em geral, o cancelamento de inscrição em um clique e baixa taxa de spam), com aplicação mais rígida a partir de novembro de 2025, e o Outlook.com adotou exigência semelhante em 2025.

REGISTROS TXT NO DNS DO DOMÍNIO

```
; SPF: só o MX do domínio e o servidor de e-mail marketing podem enviar
exemplo.com.br.                    IN TXT "v=spf1 mx ip4:203.0.113.25 -all"

; DKIM: chave pública do seletor "mar2026" (trecho)
mar2026._domainkey.exemplo.com.br.  IN TXT "v=DKIM1; k=rsa; p=MIIBIjANBgkqhkiG9w0BAQEFAAOC..."

; DMARC: política e destino dos relatórios agregados
_dmarc.exemplo.com.br.             IN TXT "v=DMARC1; p=quarantine; rua=mailto:dmarc@exemplo.com.br"
```

09 **SEGURANÇA DE REDES** CONTINUAÇÃO

Limites conhecidos: o SPF quebra quando a mensagem é **encaminhada** (o IP de origem muda) e não protege o `From` visível; o DKIM sobrevive ao encaminhamento se o corpo não for alterado. Por isso o DMARC aceita **qualquer um** dos dois, desde que alinhado. Nenhum dos três protege contra um domínio parecido (*typosquatting*), problema que exige monitoramento de marca e treinamento (p. 180).

ROTEAMENTO: O CASO DO BGP

O **BGP** é o protocolo que anuncia caminhos entre os sistemas autônomos (AS) que formam a internet e foi desenhado na confiança mútua: um AS pode anunciar prefixos que não são seus (**sequestro**, *hijack*) ou repassar rotas indevidamente (**vazamento**, *route leak*). Em 27/06/2024, um AS brasileiro (AS267613, Eletronet) passou a originar 1.1.1.1/32, endereço do resolvedor público da Cloudflare, e outro AS vazou o prefixo 1.1.1.0/24; o acesso ao serviço caiu em partes do mundo por algumas horas. A defesa é a **RPKI** com **ROV** (*Route Origin Validation*): certificados e ROAs assinados dizem qual AS pode originar cada prefixo e até que tamanho, e redes que validam rejeitam o anúncio inválido. Para vazamentos com origem válida, o próximo passo é o **ASPA** (validação do caminho). A iniciativa **MANRS** reúne as boas práticas para operadores.

WI-FI: PROTEGER O AR

No Wi-Fi, o "cabo" é o ar: qualquer um no alcance recebe os quadros, e a **autenticação e a cifragem** são a única barreira. A escolha do padrão define o risco.

PADRÃO	AUTENTICAÇÃO E CIFRAGEM	SITUAÇÃO
WEP	RC4 com chave fraca e vetor de inicialização curto	Quebrado em minutos; nunca usar
WPA (TKIP)	RC4 com TKIP	Obsoleto
WPA2-Personal (PSK)	AES-CCMP; a senha compartilhada permite ataque de dicionário **offline** contra o *handshake* capturado	Aceitável com senha longa e aleatória; migrar
WPA2 ou WPA3-Enterprise (802.1X)	EAP-TLS ou PEAP com servidor RADIUS; chaves por usuário	Recomendado em organizações
WPA3-Personal (SAE)	SAE (*Simultaneous Authentication of Equals*) com sigilo de encaminhamento; quadros de gerência protegidos (PMF)	Recomendado

No WPA2-Personal, quem captura o *handshake* de quatro vias pode testar senhas **offline**, sem interagir com a rede; por isso a senha precisa ser longa (14 caracteres aleatórios ou mais). O **KRACK** (2017) mostrou falhas no próprio handshake, corrigidas por atualização. O **WPA3** troca o PSK pelo **SAE**, em que cada tentativa de adivinhar a senha exige interação ativa com o ponto de acesso, e o **OWE** (*Enhanced Open*) cifra redes abertas com chaves individuais; as primeiras implementações tiveram falhas (*Dragonblood*, 2019), corrigidas. Em escolas, empresas e universidades, o ideal é o **802.1X**: cada pessoa autentica com a própria credencial (ou certificado) e pode ser revogada individualmente, como na rede acadêmica **eduroam**.

09 **SEGURANÇA DE REDES** CONTINUAÇÃO

Ataques ao Wi-Fi

- **Evil twin**: ponto de acesso falso com o mesmo nome (SSID); o usuário conecta ao atacante, que faz *man-in-the-middle*
- **Deautenticação**: quadros de gerência forjados desconectam clientes para forçar nova captura de handshake
- **WPS**: o PIN de 8 dígitos é validado em duas metades e cai em cerca de 11.000 tentativas
- **Rogue AP**: ponto de acesso não autorizado ligado à rede interna, que cria uma porta dos fundos

Defesas

- **802.1X com validação do certificado do servidor RADIUS** no cliente, contra o evil twin
- **PMF (802.11w)**, obrigatório no WPA3, contra a deautenticação
- **WPS desativado**; rede de **convidados** em VLAN própria, sem acesso ao interno, com isolamento de clientes
- **WIPS** e 802.1X nas portas cabeadas para detectar pontos de acesso não autorizados; VPN e HTTPS em redes públicas

DDOS: DISPONIBILIDADE SOB ATAQUE

Um ataque de **negação de serviço distribuída** (DDoS) esgota um recurso da vítima usando muitas máquinas, em geral uma **botnet** de dispositivos infectados. A escala cresce: o maior ataque divulgado pela Cloudflare até 2025 chegou a **29,7 Tbps**, partindo da botnet **Aisuru** (estimada em 1 a 4 milhões de dispositivos), e os ataques acima de 1 Tbps passaram de 130 no primeiro trimestre de 2026 para 805 no segundo. A maioria dura pouco (9 em cada 10 ataques de rede terminam em 10 minutos), então a mitigação precisa ser **automática**: não dá tempo de abrir um chamado.

FAMÍLIA	COMO ESGOTA	EXEMPLOS	MITIGAÇÃO PRINCIPAL
Volumétrico	Satura o link entre a vítima e a internet	UDP flood; amplificação por DNS, NTP, memcached e CLDAP (consulta pequena com IP forjado gera resposta grande)	Capacidade de absorção: anycast e CDN, centros de limpeza (*scrubbing*)
De protocolo	Esgota tabelas de estado e CPU de firewalls e balanceadores	SYN flood, ACK flood, fragmentação	*SYN cookies*, firewalls com proteção, filtros
De aplicação (camada 7)	Muitas requisições HTTP válidas a rotas caras (busca, login)	HTTP flood; *HTTP/2 Rapid Reset* (CVE-2023-44487, 2023)	WAF, *rate limiting*, cache, desafios contra bots

Mitigação em camadas

- **Anycast e CDN**: o mesmo endereço é anunciado em muitos pontos de presença, e cada um absorve o ataque perto da origem
- **Scrubbing**: o tráfego é desviado (por BGP ou DNS) a um centro que descarta o malicioso e devolve o limpo
- **Rate limiting** por IP, token e rota; páginas em cache e modo degradado para as funções essenciais
- **SYN cookies**: o servidor não guarda estado até o ACK final (`net.ipv4.tcp_syncookies=1`)

Prevenção e preparo

- **Anti-spoofing** (BCP 38, uRPF) nos provedores reduz a amplificação; **feche resolvedores abertos** e serviços UDP (NTP, memcached, CLDAP) a quem não precisa
- **Arquitetura**: escalabilidade automática com teto de custo e separação das rotas caras (login, busca)
- **Plano de resposta**: contatos do provedor e da CDN, *runbook* e comunicação; teste de carga **somente com autorização**

09 **SEGURANÇA DE REDES** CONTINUAÇÃO

ESTUDO DE CASO

Mirai e Dyn (2016); memcached e GitHub (2018). Em 21 de outubro de 2016, a botnet **Mirai**, formada por câmeras, gravadores e roteadores que usavam **senhas padrão de fábrica**, inundou o provedor de DNS **Dyn**; com a resolução de nomes indisponível, serviços como Twitter, GitHub e Netflix ficaram inacessíveis para muitos usuários. Em fevereiro de 2018, o GitHub recebeu cerca de **1,35 Tbps** por amplificação de servidores **memcached** expostos à internet, em que uma requisição de dezenas de bytes gerava respostas milhares de vezes maiores; o GitHub desviou o tráfego para a Akamai e a indisponibilidade durou poucos minutos.

- **Dispositivos fracos viram armas**: credenciais padrão e serviços expostos alimentam as botnets.
- **Amplificação depende de serviços UDP abertos e de IPs forjados**, por isso o anti-spoofing e o fechamento de serviços ajudam toda a internet.
- **Dependências escondem pontos únicos de falha**: o DNS de um terceiro derrubou serviços que estavam saudáveis.

REDES OT E IOT

Sistemas de **tecnologia operacional (OT)**, como CLPs (PLCs), SCADA e controle industrial, priorizam **disponibilidade e segurança física** (*safety*) sobre a confidencialidade, usam equipamentos que duram décadas, raramente aceitam atualizações e falam protocolos que nasceram **sem autenticação**, como **Modbus** e DNP3: quem alcança a rede consegue enviar comandos. O **modelo de Purdue** organiza o ambiente em níveis (0: processo físico, sensores e atuadores; 1: controle, os CLPs; 2: supervisão, as IHM; 3: operações da planta; 3.5: DMZ industrial; 4 e 5: TI corporativa e internet), e a norma **IEC 62443** define **zonas, conduítes** e níveis de segurança. As práticas centrais:

OT (indústria, energia, saneamento)

- **Segmentar OT de TI** com uma DMZ industrial e firewalls que entendam os protocolos industriais
- **Nunca expor** CLPs e IHM à internet; acesso remoto só por bastion com MFA e sessão gravada
- **Monitoramento passivo** e inventário de ativos: varreduras ativas podem derrubar um equipamento frágil
- **Patches planejados** em janelas de manutenção, com controles compensatórios até lá

IoT (câmeras, sensores, roteadores)

- **Trocar credenciais padrão** e desativar serviços de gerência desnecessários (Telnet, UPnP)
- **VLAN própria** para dispositivos, sem acesso à rede interna e com saída restrita
- **Exigir atualização de firmware** do fabricante: *Secure by Design* e leis como o Cyber Resilience Act europeu (**p. 27**) tornam isso obrigação do fabricante
- **Inventariar**: o que não está no inventário não recebe patch

ERROS DE CONFIGURAÇÃO MAIS COMUNS

01 RDP E SSH EXPOSTOS

As portas 3389 e 22 abertas à internet são varridas em minutos e atacadas com credenciais vazadas. Acesse por VPN ou bastion, com MFA.

02 SERVIÇOS DE GERÊNCIA ABERTOS

Painéis, APIs de orquestração (Docker 2375, Kubernetes) e bancos (Redis 6379, Elasticsearch 9200) sem autenticação e acessíveis de fora.

03 CREDENCIAIS PADRÃO

`admin/admin` em roteadores, câmeras, impressoras e painéis de bancos e de nuvem: a primeira tentativa de qualquer atacante.

09 SEGURANÇA DE REDES — CONTINUAÇÃO

04 REGRAS “TEMPORÁRIAS”

O “liberei só para testar” que nunca foi removido. Sem dono, justificativa nem prazo, a regra vira a brecha permanente.

05 REDE PLANA

Tudo na mesma sub-rede: o notebook infectado de um aluno enxerga os servidores e as câmeras. Falta segmentação.

06 SAÍDA IRRESTRITA

Servidores que falam com qualquer destino na internet: o malware encontra o servidor de comando e controle sem esforço.

CENÁRIO

Revisão de uma regra de firewall. Na auditoria trimestral surge a regra “permitir `qualquer` → `servidor-app`, portas 1–65535”, criada “para teste” em 2023, sem dono registrado. O que fazer, em ordem?

- **Medir o uso real**: os logs de 90 dias mostram conexões apenas às portas 8443 (vindas do proxy reverso) e 5432 (para o banco).
- **Desenhar a regra mínima**: proxy reverso → servidor-app na 8443; servidor-app → banco na 5432; todo o resto, negado.
- **Testar sem risco**: colocar a regra nova acima da antiga, deixar a antiga em modo de apenas registro por uma semana e confirmar que nada legítimo cai nela.
- **Remover e registrar**: apagar a regra ampla, anotar dono, justificativa e data da próxima revisão na ferramenta de mudanças.

PARA FIXAR

Conceitos

- Qual a diferença entre firewall *stateful*, NGFW e WAF? Onde cada um se encaixa?
- Por que a DMZ fica entre dois firewalls e o que muda se o servidor público for comprometido?
- O que SPF, DKIM e DMARC garantem, e por que o alinhamento é o ponto-chave do DMARC?

Aplicação

- Escreva a política de rede de um servidor web que fala com um banco: o que entra, o que sai e o que é negado.
- Monte um plano de três etapas para levar o DMARC de `p=none` a `p=reject` sem perder e-mails legítimos.
- Liste cinco itens que um inventário de portas (`ss -tulpn`) pode revelar de errado em um servidor.

APROFUNDE-SE

Leituras e referências

- NIST SP 800-41 Rev. 1 — NIST: diretrizes sobre firewalls e políticas de firewall.
- NIST SP 800-207: Zero Trust Architecture — NIST: a arquitetura de referência do Zero Trust.
- RFC 9989: DMARC — IETF: o padrão atual de autenticação de e-mail por domínio.
- MANRS — Internet Society: boas práticas de segurança de roteamento.
- O que é um ataque DDoS — Cloudflare Learning Center: tipos de ataque e mitigação.
- Documentação do Suricata — OISF: regras, assinaturas e implantação de IDS/IPS.

MÓDULO 10 · INFRAESTRUTURA

10 SISTEMAS, NUVEM E CONTÊINERES

Endurecer servidores, estações e ambientes de nuvem — a base sobre a qual as aplicações rodam.

Uma rede bem desenhada de nada vale se os sistemas que ela conecta estiverem abertos. Este módulo trata de **endurecer** (*hardening*) servidores, estações e dispositivos, mantê-los **atualizados**, proteger os **endpoints**, dominar o essencial de **Linux** e de **Windows/Active Directory** e levar os mesmos princípios à **nuvem** (identidade, configuração, registro) e aos **contêineres e ao Kubernetes**. A lógica é sempre a mesma: **superfície mínima, privilégio mínimo, configuração padronizada e verificável, e visibilidade do que acontece**.

HARDENING: CONFIGURAÇÃO SEGURA DE BASE

Hardening é reduzir a superfície de ataque de um sistema: instalar só o necessário, desligar serviços e portas sem uso, remover contas e programas padrão, aplicar configurações seguras e restringir privilégios. Os sistemas saem de fábrica pensados para **funcionar**, não para resistir; a segurança precisa ser acrescentada e, principalmente, **padronizada**. Para isso existem as **linhas de base** (*baselines*): conjuntos de configurações recomendadas e testadas por comunidades e governos.

REFERÊNCIA	O QUE É	OBSERVAÇÃO
CIS Benchmarks	Guias de configuração segura por produto (Linux, Windows, Docker, Kubernetes, nuvem, bancos), em consenso de comunidade; perfis Level 1 (baixo impacto) e Level 2 (ambientes sensíveis)	Os guias em PDF são gratuitos; a ferramenta de avaliação automática tem versão paga
DISA STIG	*Security Technical Implementation Guides* do Departamento de Defesa dos EUA, com verificações detalhadas	Públicas; costumam ser mais rígidas e servem de base a regras de automação
SCAP e OpenSCAP	Protocolo de automação de segurança (NIST) e ferramenta aberta que avalia sistemas contra perfis CIS e STIG	Gera relatórios de conformidade e pode aplicar correções
Guias do fabricante	Microsoft Security Baselines, Red Hat, Ubuntu e guias de cada provedor de nuvem	Bom ponto de partida para cada plataforma

O hardening é um ciclo: **baseline → automatizar → medir → corrigir o desvio**. Use **imagens-padrão** (*golden images*) e gerenciamento de configuração (Ansible, Puppet, GPO/Intune) para que todo servidor nasça igual; avalie com **Lynis** ou **OpenSCAP**; e detecte o **desvio de configuração** (*drift*), revertendo-o. A infraestrutura como código (**p. 135**) torna isso auditável: a configuração segura vive em um repositório, é revisada por pares e reaplicada continuamente.

10 SISTEMAS, NUVEM E CONTÊINERES — CONTINUAÇÃO

Reduzir a superfície

- Instalação **mínima**: sem compiladores, ferramentas de rede ou serviços que o papel do servidor não exige
- Desativar serviços e portas sem uso; remover pacotes e contas padrão
- **Trocar credenciais padrão** e bloquear o login interativo de contas de serviço
- Retirar protocolos legados (Telnet, SMBv1, TLS antigo; **p. 67**)

Configurar com segurança

- **Permissões mínimas** em arquivos, diretórios e chaves de registro
- **Registro e auditoria** ligados e enviados a um servidor central (**p. 144**)
- **Relógio sincronizado** (NTP): sem hora confiável, o log não serve em uma investigação
- **Cifragem de disco** e *boot* seguro; controle de USB e de mídias removíveis

INVENTÁRIO E GESTÃO DE PATCHES

Quem não sabe o que tem não consegue protegê-lo: o **inventário de ativos de hardware e de software** são os Controles 1 e 2 do CIS (**p. 27**) e a base de qualquer programa de vulnerabilidades (**p. 20**). Sobre ele se apoia a **gestão de patches**, a correção sistemática de falhas conhecidas; o NIST SP 800-40 Rev. 4 a trata como **manutenção preventiva** planejada, e não como correria de emergência.



Janelas de manutenção e **ondas** (um grupo piloto, depois os demais) reduzem o risco de a correção derrubar o serviço; **correções emergenciais** (exploração ativa, itens do KEV) seguem um fluxo acelerado. O pior inimigo é o **ativo esquecido**: sistemas **fora de suporte** (*end of life*), como o Windows 10, cujo suporte padrão terminou em 14 de outubro de 2025, não recebem mais correções e precisam ser substituídos, isolados ou cobertos por controles compensatórios (segmentação, monitoramento reforçado ou o programa de atualizações estendidas do fabricante).

> **O custo de atualizar sem testar:** em 19 de julho de 2024, uma atualização de conteúdo do agente de EDR da CrowdStrike, distribuída a todos de uma vez, derrubou cerca de 8,5 milhões de computadores Windows (estimativa da Microsoft), com impacto em aeroportos, hospitais e bancos. Não foi um ataque, e sim uma mudança **sem implantação gradual**. Lição: mesmo atualizações de segurança merecem **anéis de implantação** (*rings*), monitoramento e plano de volta.

PROTEÇÃO DE ENDPOINTS

Estações, notebooks e celulares são a porta de entrada preferida: phishing, anexos e navegação chegam primeiro a eles. A proteção evoluiu do **antivírus por assinatura** (compara arquivos com padrões conhecidos: barato, mas cego ao que é novo ou sem arquivo, o *fileless*) para o **EDR** (*Endpoint Detection and Response*), que coleta telemetria contínua (processos, comandos, conexões), detecta **comportamento** suspeito (o Word iniciando o PowerShell, por exemplo) e permite **resposta remota**: isolar a máquina da rede, encerrar processos e coletar evidências. O **XDR** estende a correlação a e-mail, identidade, rede e nuvem; o **MDR** é o serviço gerenciado, com analistas operando a ferramenta 24 horas.

10 SISTEMAS, NUVEM E CONTÊINERES — CONTINUAÇÃO

CONTROLE	O QUE FAZ	EXEMPLOS
Allowlisting de aplicativos	Só executa o que foi aprovado; barra malware desconhecido	AppLocker e WDAC (Windows); fapolicyd (Linux)
Cifragem de disco	Protege o dado se o equipamento for perdido ou roubado	BitLocker, FileVault, LUKS
Secure Boot, TPM e boot medido	Só inicia código assinado e guarda chaves em hardware	UEFI Secure Boot, TPM 2.0
Gestão de dispositivos (MDM/UEM)	Impõe políticas, cifragem e atualização; apaga remotamente	Intune, Jamf, Workspace ONE
Privilégio local mínimo	Usuário sem administrador local; elevação sob demanda	Windows LAPS, ferramentas de PAM
Navegador e e-mail	Bloqueio de macros e de anexos executáveis; isolamento	Políticas do navegador; modo protegido

No **BYOD** (*bring your own device*), o dispositivo pessoal acessa dados da organização. As opções são o **MAM** (gerenciar só o aplicativo e os dados corporativos, em um contêiner separado, sem controlar o aparelho inteiro) e o **acesso condicional**: só entra quem está com o sistema atualizado, o disco cifrado e sem *root* ou *jailbreak*, integrado à identidade (p. 44).

LINUX: O ESSENCIAL DE SEGURANÇA

Em Linux, quase tudo é arquivo, e a primeira linha de defesa é o **modelo de permissões**: cada arquivo tem **dono**, **grupo** e **outros**, com bits de leitura (r = 4), escrita (w = 2) e execução (x = 1) para cada um, combinados em valores **octais**: `640` significa que o dono lê e escreve, o grupo lê e os outros não acessam. A **umask** subtrai bits na criação (a padrão `022` gera arquivos `644` e diretórios `755`; `027` e `077` são mais restritivas).

CONCEITO	O QUE FAZ	RISCO E CUIDADO
Permissões e `umask`	Controlam quem lê, escreve e executa	Segredos e chaves privadas em `600`; diretório com escrita para todos só com *sticky bit* (como `/tmp`)
`setuid` e `setgid`	O programa roda com os privilégios do dono ou do grupo (como o `passwd`)	Um binário setuid vulnerável vira escalada de privilégio; liste e revise
`sudo`	Concede root por comando e registra o uso	Evite `NOPASSWD: ALL`; limite a comandos específicos; edite com `visudo`
Capabilities	Dividem o poder do root em partes (`CAP_NET_BIND_SERVICE`, `CAP_NET_RAW`)	`CAP_SYS_ADMIN` equivale quase a root; conceda a menor possível
SELinux e AppArmor	Controle de acesso **obrigatório** (MAC): confinam o processo ao que a política permite, mesmo como root	Não desative em produção; o modo permissivo serve só para depurar
`auditd`	Auditoria no kernel: acesso a arquivos, chamadas de sistema e comandos	Use regras direcionadas para não gerar volume inútil
Namespaces, cgroups e seccomp	Isolam processos e recursos e filtram chamadas de sistema	São a base dos contêineres (seção adiante)

10 SISTEMAS, NUVEM E CONTÊINERES — CONTINUAÇÃO

COMANDOS DE VERIFICAÇÃO E DE AJUSTE DE PERMISSÕES

```
# Chave privada: só o dono lê e escreve
chmod 600 ~/.ssh/id_ed25519

# Diretório da aplicação: dono e grupo; outros sem acesso
chmod 750 /srv/portal && chown -R portal:portal /srv/portal

# Binários com setuid no sistema (compare com a linha de base e investigue novidades)
find / -xdev -perm -4000 -type f 2>/dev/null

# Portas em escuta e o processo dono de cada uma
ss -tulpn

# Sudo mínimo: o usuário "deploy" só reinicia um serviço (arquivo criado com visudo -f)
# /etc/sudoers.d/deploy:   deploy ALL=(root) /usr/bin/systemctl restart portal
```

EXEMPLO RESOLVIDO

Auditoria rápida de um servidor. O `ss -tulpn` mostra `sshd` em `0.0.0.0:22`, `nginx` em `:80` e `:443`, `mysqld` em `0.0.0.0:3306` e `redis-server` em `0.0.0.0:6379`. O que corrigir?

- **MySQL e Redis em `0.0.0.0`** aceitam conexões por qualquer interface: configure o endereço de escuta para `127.0.0.1` (ou o IP da rede privada) e, no firewall, libere a porta só para os servidores de aplicação. Um Redis exposto e sem autenticação é um achado comum em varreduras e permite ler e gravar todos os dados.
- **SSH em `0.0.0.0:22`**: restrinja por firewall aos IPs do bastion e aplique as opções de hardening abaixo.
- **HTTP na porta 80**: mantenha apenas para redirecionar a HTTPS, com HSTS (p. 34).
- **Registre o resultado**: guarde a lista de portas esperadas e compare toda semana; qualquer porta nova é um alerta.

HARDENING DO SSH (SSHD_CONFIG)

```
# No OpenSSH vale a primeira ocorrência de cada diretiva
PermitRootLogin no
PasswordAuthentication no          # somente chaves
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AllowUsers ana deploy              # lista explícita de quem pode entrar
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
AllowTcpForwarding no              # libere só se houver necessidade
ClientAliveInterval 300
ClientAliveCountMax 2
# Valide antes de recarregar:  sshd -t && systemctl reload sshd
```

Prefira **chaves** a senhas: gere pares Ed25519 (`ssh-keygen -t ed25519`), proteja a chave privada com *passphrase* e agente e, em escala, use **certificados SSH** emitidos por uma CA interna, com validade curta e a identidade do usuário no certificado, em vez de distribuir chaves públicas servidor a servidor. O **fail2ban** bloqueia IPs após falhas repetidas, mas o ganho maior vem de **não expor o SSH diretamente à internet** (p. 67).

10 SISTEMAS, NUVEM E CONTÊINERES — CONTINUAÇÃO

Exemplo: regreSSHion: em julho de 2024, a Qualys divulgou a falha **regreSSHion** (CVE-2024-6387), uma condição de corrida no `sshd` que permitia execução remota de código sem autenticação em versões do OpenSSH de 8.5p1 a 9.7p1 em Linux com glibc, corrigida na 9.8p1. Quem tinha inventário de versões e processo de patch soube em minutos quais servidores atualizar; os demais precisaram primeiro descobrir onde havia SSH.

WINDOWS E ACTIVE DIRECTORY

O **Active Directory** (AD) concentra identidade, autenticação (Kerberos e NTLM) e política (GPO) de grande parte das organizações, e por isso é o alvo central dos ataques: quem controla o AD controla tudo. O modelo clássico de defesa é o de **camadas administrativas** (*tiering*): **Tier 0** (controladores de domínio, PKI e quem os administra), **Tier 1** (servidores) e **Tier 2** (estações e usuários). A regra é que **credenciais de uma camada superior nunca são usadas em uma inferior**: um administrador de domínio que faz logon em uma estação deixa credenciais na memória dela, ao alcance de quem a controle (*pass-the-hash*, p. 11).

CONTROLE	O QUE FAZ	ATAQUE QUE DIFICULTA
Windows LAPS	Gera e rotaciona uma senha única de administrador local para cada máquina, guardada no diretório	Movimento lateral com a mesma senha de administrador em todas as máquinas
Credential Guard	Isola segredos (hashes NTLM, tíquetes Kerberos) em um ambiente virtualizado, fora do alcance do sistema	Extração de credenciais da memória (*pass-the-hash*, Mimikatz)
BitLocker com TPM	Cifra o volume e vincula a chave ao hardware e ao estado do boot	Roubo do equipamento; leitura offline do disco
Regras ASR (*Attack Surface Reduction*)	Bloqueiam comportamentos de risco, como o Office criando processos filhos ou scripts ofuscados	Macros e cadeias de phishing que abrem um *shell*
Protected Users e contas separadas	Reduzem o cache de credenciais e bloqueiam NTLM e delegação para contas privilegiadas	*Kerberoasting* e roubo de credenciais privilegiadas
Desativar NTLMv1 e SMBv1; assinar SMB e LDAP	Impede *relay* e a interceptação da autenticação	NTLM relay e captura de hashes

Complementam o quadro: contas **administrativas separadas** das do dia a dia, **estações de acesso privilegiado** (PAW) para administrar o Tier 0, **MFA** em todo acesso remoto e privilegiado, GPOs versionadas e revisadas e **auditoria avançada** ligada, com monitoramento dos eventos de alto valor (criação de contas privilegiadas, alteração de GPOs, replicação do diretório por contas inesperadas, o *DCSync*). Ferramentas como **BloodHound** e **PingCastle**, usadas **no seu próprio AD e com autorização**, mostram os caminhos de ataque até o domínio antes que alguém os percorra.

NUVEM: RESPONSABILIDADE COMPARTILHADA

Na nuvem, a segurança é **compartilhada**: o provedor cuida da segurança **da** nuvem (hardware, datacenters, virtualização e a infraestrutura dos serviços gerenciados) e o cliente, da segurança **na** nuvem (dados, identidades, configuração, rede e aplicações que coloca lá). A divisão muda com o modelo de serviço: em **IaaS** (máquinas virtuais) o cliente cuida do sistema operacional e das correções; em **PaaS**, do código e da configuração; em **SaaS**, sobretudo de dados, contas e acessos.

10 SISTEMAS, NUVEM E CONTÊINERES — CONTINUAÇÃO

	On-premises datacenter próprio	IaaS EC2, Compute Engine	PaaS App Service, Cloud Run	SaaS Microsoft 365, Gmail
Dados e classificação	Você	Você	Você	Você
Identidades, contas e acessos	Você	Você	Você	Você
Configuração de segurança do serviço	Você	Você	Você	Você
Aplicação e código	Você	Você	Você	Provedor
Sistema, runtime e middleware	Você	Você	Provedor	Provedor
Rede (regras e segmentação)	Você	Você	Ambos	Provedor
Virtualização e infraestrutura física	Você	Provedor	Provedor	Provedor

responsabilidade sua · responsabilidade do provedor · dividida

Em qualquer modelo, dados, identidades e configuração continuam sendo responsabilidade do cliente.

Responsabilidade compartilhada. O provedor protege a nuvem em si (hardware, virtualização, datacenters); o cliente protege o que coloca nela. Quanto mais gerenciado é o serviço, mais camadas passam ao provedor, mas dados, identidades e configuração nunca deixam de ser do cliente: é aí que a maioria dos incidentes em nuvem acontece.

Há uma consequência prática: como a rede deixou de ser a fronteira (todo serviço é acessível por API, de qualquer lugar), a **identidade é o novo perímetro**. Quem obtém uma credencial válida com permissões amplas lê, altera ou apaga quase tudo sem explorar vulnerabilidade alguma. Os incidentes graves em nuvem passam, em geral, por **credenciais expostas, permissões excessivas ou configuração errada**, e não por falha do provedor.

IAM E GOVERNANÇA DE CONTAS NA NUVEM

Identidade e acesso (IAM)

- **Menor privilégio**: políticas com ações e recursos específicos; evite `"Action": "*"` e `"Resource": "*"`
- **Papéis com credenciais temporárias** (STS, identidade de carga de trabalho) no lugar de **chaves de acesso de longa duração**
- **MFA resistente a phishing** para todas as pessoas e **conta raiz trancada**: MFA, sem chaves de acesso, usada só em emergências
- **SSO federado** com o diretório e, em CI/CD, **OIDC** em vez de segredos guardados no pipeline
- **Barreiras da organização** (SCP, políticas de organização) que impedem, até para administradores, desligar o registro ou usar regiões não autorizadas

Governança e visibilidade

- **Contas, projetos ou assinaturas separados** por ambiente e por sensibilidade, para limitar o raio do estrago
- **Revisão periódica** de acessos e remoção do que não é usado (Access Analyzer, recomendações do IAM)
- **Registro de auditoria sempre ligado**, em todas as regiões, em conta separada e imutável (CloudTrail, Activity Log, Cloud Audit Logs)
- **Alertas** para uso da conta raiz, criação de chaves de acesso, mudança de política e desativação do registro

ERRO COMUM	CONSEQUÊNCIA	PREVENÇÃO
Armazenamento de objetos (*bucket*) público	Dados expostos na internet e indexados por buscadores	Bloqueio de acesso público na conta ou na organização; revisão automática
***Security group* aberto (`0.0.0.0/0` em 22, 3389, bancos)**	Ataques automatizados em minutos	Acesso por bastion, VPN ou ZTNA; regras mínimas (p. 67)

10 SISTEMAS, NUVEM E CONTÊINERES — CONTINUAÇÃO

ERRO COMUM	CONSEQUÊNCIA	PREVENÇÃO
Chaves de acesso em código, imagens e repositórios	Uso por terceiros, mineração de criptomoedas, vazamentos	Papéis temporários, varredura de segredos e rotação (p. 135)
Permissões com curinga e contas de serviço superpoderosas	Escalada de privilégio e movimento lateral	Menor privilégio e revisão contínua
Serviço de metadados acessível por SSRF	Roubo das credenciais do papel da instância	IMDSv2 obrigatório, *hop limit* 1 e validação de URLs de saída
Registro desligado ou sem retenção	Incidente sem rastros para investigar	Auditoria centralizada, imutável e com retenção definida
Snapshots, imagens e bancos públicos ou sem cifragem	Vazamento de dados completos	Cifragem por padrão e política que bloqueia o compartilhamento público

ESTUDO DE CASO

Capital One (2019): SSRF, metadados e um papel poderoso demais. Em março de 2019, uma invasora explorou um *firewall* de aplicação web (WAF) mal configurado, em uma instância na AWS, para realizar um ataque de **SSRF** (p. 98): fez o servidor consultar o endereço de **metadados da instância** (`169.254.169.254`) e, de lá, obteve as **credenciais temporárias** do papel IAM associado a ele. Esse papel podia ler muitos *buckets* de armazenamento, e a invasora copiou dados de cerca de **106 milhões de pessoas** nos EUA e no Canadá, sobretudo solicitações de cartão de crédito de 2005 a 2019. O regulador americano (OCC) multou o banco em **US$ 80 milhões** em agosto de 2020, apontando que ele deixou de identificar falhas de controle no ambiente de nuvem.

- **Uma cadeia de falhas**: configuração do WAF, SSRF, metadados sem token (IMDSv1), papel com permissões excessivas e ausência de detecção. Corrigir qualquer elo teria interrompido o ataque.
- **Responsabilidade compartilhada na prática**: a AWS afirmou que a falha estava na configuração feita pelo cliente.
- **Lições**: exigir o **IMDSv2**, aplicar menor privilégio aos papéis, validar e restringir requisições de saída e detectar cópias em massa de dados.

IMDSV2: O TOKEN EXIGE UM PUT, QUE UMA SSRF TÍPICA NÃO CONSEGUE FAZER

```
# Passo 1: pedir um token de sessão (PUT, com TTL em segundos)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
        -H "X-aws-ec2-metadata-token-ttl-seconds: 300")

# Passo 2: consultar o metadado apresentando o token no cabeçalho
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
     http://169.254.169.254/latest/meta-data/instance-id

# Exigir IMDSv2 em uma instância (chamadas IMDSv1 passam a falhar)
aws ec2 modify-instance-metadata-options --instance-id i-0abc123def456 \
    --http-tokens required --http-put-response-hop-limit 1 --http-endpoint enabled
```

10 **SISTEMAS, NUVEM E CONTÊINERES** CONTINUAÇÃO

POSTURA E PROTEÇÃO DE CARGAS DE TRABALHO

SIGLA	O QUE FAZ	EXEMPLO DE ACHADO
CSPM	*Cloud Security Posture Management*: varre contas e serviços e compara a configuração com benchmarks (como o CIS Foundations) e políticas internas	Bucket público, MFA ausente, grupo de segurança aberto
CWPP	*Cloud Workload Protection Platform*: protege VMs, contêineres e funções com varredura de vulnerabilidades e detecção em tempo de execução	Imagem vulnerável; comportamento anômalo em contêiner
CIEM	*Cloud Infrastructure Entitlement Management*: analisa as permissões efetivas e sugere reduzi-las	Identidade com acesso muito maior que o usado
CNAPP	*Cloud-Native Application Protection Platform*: reúne CSPM, CWPP, CIEM e varredura de IaC, com contexto da aplicação	Priorizar o que é vulnerável, exposto e com permissão alta ao mesmo tempo

Há opções abertas para começar: **Prowler** e **ScoutSuite** (auditoria de configuração), **Checkov** e **Trivy** (varredura de IaC e de imagens) e os serviços nativos de cada provedor (Security Hub, Defender for Cloud, Security Command Center). Somese a isso o **KMS** com chaves gerenciadas e políticas de uso (p. 34), **redes privadas** (sub-redes sem IP público, *endpoints* privados para armazenamento e bancos, saída por NAT controlado) e **IaC revisada e escaneada** (p. 135); o *state* do Terraform costuma conter segredos e deve ser protegido como tal.

CONTÊINERES: O QUE ISOLAM E O QUE NÃO ISOLAM

Um contêiner **não é uma máquina virtual**: é um processo comum do host, isolado por recursos do kernel Linux: **namespaces** (o que o processo enxerga: PIDs, rede, pontos de montagem, usuários), **cgroups** (quanto pode consumir: CPU, memória, número de processos), **capabilities** e **seccomp** (o que pode pedir ao kernel) e **SELinux ou AppArmor** (o que a política permite). O ponto-chave é que **todos os contêineres de um host compartilham o mesmo kernel**: uma falha do kernel ou do *runtime* pode permitir uma **fuga de contêiner** (*container escape*), como as do runc (CVE-2019-5736 e, em 2024, CVE-2024-21626). Para cargas em que não se confia (código de terceiros, multilocação), existe isolamento mais forte: **gVisor** (um kernel em espaço de usuário) e **Kata Containers** ou **Firecracker** (microVMs).

CONFIGURAÇÃO PERIGOSA	POR QUE É PERIGOSA	ALTERNATIVA
`--privileged`	Dá todas as capabilities e acesso aos dispositivos do host: quase equivale a ser root nele	Capabilities específicas e `--device` pontual
Montar `/var/run/docker.sock`	Quem fala com o daemon cria contêineres privilegiados: root no host	Evitar; usar *builds* sem daemon (Kaniko, BuildKit rootless)
Executar como root (UID 0)	Qualquer escrita indevida ou fuga ocorre com privilégio máximo	`USER` não root; `runAsNonRoot` no Kubernetes
`hostPath`, `hostNetwork`, `hostPID`	Expõem arquivos, rede e processos do host	Volumes gerenciados e NetworkPolicy
Imagem `latest` **ou de origem desconhecida**	Conteúdo mutável e sem proveniência	Fixar por *digest* (`@sha256:...`); registro confiável e assinatura
Segredos na imagem ou em variáveis de ambiente	Vazam por camadas da imagem, `inspect`, logs e dumps	Segredos em tempo de execução (volume, gerenciador); BuildKit `--mount=type=secret`

10 SISTEMAS, NUVEM E CONTÊINERES — CONTINUAÇÃO

DOCKERFILE SEGURO: MULTI-STAGE, DIGEST FIXO E USUÁRIO SEM PRIVILÉGIOS

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim@sha256:SEU_DIGEST AS build       # versão e digest fixos
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim@sha256:SEU_DIGEST                # estágio final, sem ferramentas de build
RUN useradd --system --uid 10001 --no-create-home app  # usuário sem privilégios
COPY --from=build /install /usr/local
WORKDIR /app
COPY --chown=app:app src/ ./src/
USER 10001
EXPOSE 8080
ENTRYPOINT ["python", "src/main.py"]
# Segredos NUNCA entram na imagem; use .dockerignore para não copiar .env, .git e chaves
```

EXECUÇÃO ENDURECIDA DE UM CONTÊINER

```
docker run -d --name portal \
  --read-only --tmpfs /tmp \
  --cap-drop=ALL --security-opt no-new-privileges:true \
  --user 10001:10001 --pids-limit 200 --memory 512m --cpus 1 \
  -p 8080:8080 registro.exemplo.com.br/portal@sha256:SEU_DIGEST
```

Imagens pedem cuidados próprios: base **mínima** (distroless, Alpine ou `scratch`), que reduz tanto o número de vulnerabilidades quanto as ferramentas à disposição de um invasor; **varredura** de vulnerabilidades e de segredos (Trivy, Grype, Docker Scout) no CI e no registro; **SBOM** (Syft; p. 135); **assinatura** (Cosign, do projeto Sigstore) com uma política que só aceita imagens assinadas; e **reconstrução frequente** sobre bases atualizadas, porque uma imagem "parada" acumula CVEs a cada semana.

KUBERNETES: SEGURANÇA DO CLUSTER

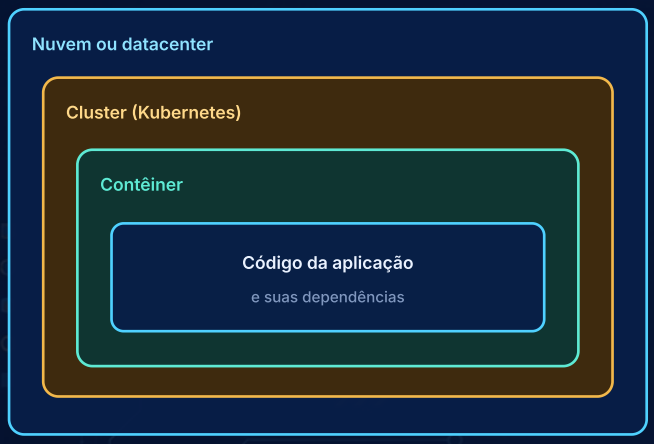
O Kubernetes orquestra contêineres e traz uma superfície própria: o servidor de API, o etcd, o kubelet, a rede dos pods, os Secrets e a identidade das cargas de trabalho. O modelo **4C** organiza o raciocínio em camadas (nuvem, cluster, contêiner e código), e o **NIST SP 800-190** cobre os riscos de imagens, registros, orquestradores, contêineres e host.

CONTROLE	O QUE GARANTE	PONTO DE ATENÇÃO
RBAC	Quem pode fazer o quê na API (Role, ClusterRole e *bindings*)	Evite `cluster-admin` e curingas; revise permissões sobre `secrets` e `pods/exec`
Pod Security Standards	Três perfis (*privileged*, *baseline* e *restricted*), aplicados pelo Pod Security Admission por rótulos do namespace	O `restricted` exige não root, sem escalada de privilégio, capabilities removidas e seccomp `RuntimeDefault`
NetworkPolicy	Controla o tráfego entre pods; sem política, **todos falam com todos**	Comece com *default deny* e libere por rótulo; exige um plugin de rede que a implemente (Calico, Cilium)
Secrets	Guardam credenciais para os pods	São **codificados em base64, não cifrados**: cifre o etcd em repouso (de preferência com KMS) e restrinja por RBAC, ou use um gerenciador externo (Vault, External Secrets)

10 SISTEMAS, NUVEM E CONTÊINERES

CONTINUAÇÃO

CONTROLE	O QUE GARANTE	PONTO DE ATENÇÃO
Contas de serviço	Identidade do pod perante a API	Uma por aplicação; `automountServiceAccountToken: false` quando não precisa; tokens de curta duração
Controladores de admissão	Validam ou alteram objetos antes de entrarem no cluster	**Kyverno** e **OPA Gatekeeper** impõem políticas (imagens assinadas, sem `latest`, limites obrigatórios)
Segurança em execução	Detecta comportamento anômalo nos contêineres	**Falco** (regras sobre chamadas de sistema) e eBPF; alertas enviados ao SIEM



Nuvem
IAM com menor privilégio, redes privadas, KMS, registro de auditoria e hosts endurecidos

Cluster
RBAC, NetworkPolicy, Pod Security, Secrets cifrados em repouso e controladores de admissão

Contêiner
imagem mínima e sem vulnerabilidades, usuário não root, capabilities removidas e sistema de arquivos somente leitura

Código
dependências atualizadas, validação de entrada, TLS, segredos fora do repositório e testes SAST e SCA

Cada camada herda a segurança da camada externa: código perfeito não compensa um cluster aberto.

Modelo 4C do Kubernetes. A segurança de nuvem nativa se organiza em quatro camadas: nuvem (ou datacenter), cluster, contêiner e código. Uma camada interna só é tão segura quanto as externas que a sustentam, por isso o cuidado precisa estar em todas.

NAMESPACE QUE APLICA O PERFIL RESTRICTED (KUBERNETES)

```
apiVersion: v1
kind: Namespace
metadata:
  name: portal
  labels:
    pod-security.kubernetes.io/enforce: restricted   # rejeita pods fora do perfil
```

10 SISTEMAS, NUVEM E CONTÊINERES — CONTINUAÇÃO

DEPLOYMENT ENDURECIDO (COMPATÍVEL COM O PERFIL RESTRICTED)

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: api, namespace: portal}
spec:
  replicas: 2
  selector: {matchLabels: {app: api}}
  template:
    metadata: {labels: {app: api}}
    spec:
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        seccompProfile: {type: RuntimeDefault}
      containers:
        - name: api
          image: registro.exemplo.com.br/portal-api@sha256:SEU_DIGEST
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: {drop: ["ALL"]}
          resources:
            limits: {cpu: "500m", memory: "256Mi"}
```

NETWORKPOLICY: NEGAR TUDO E LIBERAR SÓ O NECESSÁRIO

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: negar-tudo, namespace: portal}
spec:
  podSelector: {}                  # todos os pods do namespace
  policyTypes: [Ingress, Egress]     # sem regras: nega a entrada e a saída
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: web-para-api, namespace: portal}
spec:
  podSelector: {matchLabels: {app: api}}
  policyTypes: [Ingress]
  ingress:
    - from: [{podSelector: {matchLabels: {app: web}}}]
      ports: [{port: 8080}]
```

Ao negar a saída: com `Egress` negado, libere explicitamente o DNS (porta 53 para o serviço de DNS do cluster) e os destinos de que a aplicação depende, ou ela deixará de resolver nomes e de falar com o banco. Teste as políticas em um namespace de homologação antes de aplicá-las em produção.

10 SISTEMAS, NUVEM E CONTÊINERES — CONTINUAÇÃO

CHECKLIST

Revisão de segurança de uma carga de trabalho em contêiner:

- A imagem usa base mínima, versão e *digest* fixos, é varrida no CI e está assinada.
- O processo roda como usuário não root, com sistema de arquivos somente leitura, sem escalada de privilégio e sem capabilities.
- Não há `--privileged`, `hostPath`, `hostNetwork` nem o socket do Docker montado.
- Limites de CPU, memória e processos definidos; probes de saúde configurados.
- Segredos vêm de um gerenciador ou de Secrets cifrados, nunca da imagem nem do repositório.
- O namespace aplica o perfil `restricted` e uma NetworkPolicy *default deny*; o RBAC é mínimo.
- Há registro de auditoria da API, detecção em execução (Falco) e alertas no SIEM (p. 144).

PARA FIXAR

Conceitos

- O que o provedor de nuvem e o cliente protegem em IaaS, PaaS e SaaS? O que nunca deixa de ser do cliente?
- Por que um contêiner não é uma máquina virtual e o que muda para o isolamento?
- Por que o Secret do Kubernetes em base64 não é "seguro" por padrão?

Aplicação

- Aplique o hardening de SSH e explique o efeito de cada diretiva escolhida.
- Reescreva um Dockerfile que roda como root, usa `latest` e copia um `.env` para torná-lo seguro.
- Desenhe a cadeia do caso Capital One e indique, em cada elo, o controle que a interromperia.

APROFUNDE-SE

Leituras e referências

- CIS Benchmarks — CIS: linhas de base de configuração segura para sistemas, nuvem e contêineres.
- NIST SP 800-40 Rev. 4 — NIST: planejamento da gestão de patches corporativa.
- NIST SP 800-190 — NIST: guia de segurança de contêineres de aplicação.
- Pod Security Standards — Kubernetes: os perfis privileged, baseline e restricted.
- Docker Security Cheat Sheet — OWASP: práticas de segurança para Docker.
- Modelo de responsabilidade compartilhada — AWS: a divisão de deveres entre provedor e cliente.

MÓDULO 11 · DESENVOLVIMENTO

11 CICLO DE VIDA SEGURO (SSDLC)

Segurança como parte de cada fase do desenvolvimento, e não como uma inspeção no final.

Segurança não se acrescenta no fim: ela é construída, verificada e mantida **em todas as fases** do desenvolvimento. Este módulo organiza o **ciclo de vida seguro de software** (SSDLC): as atividades de cada fase, os modelos de referência (**NIST SSDF**, **OWASP SAMM**, **BSIMM**, **Microsoft SDL**), os **requisitos de segurança** e os **casos de abuso** com o **OWASP ASVS**, os padrões de projeto seguro, a **definição de pronto** e as **métricas** que dizem se o programa funciona. Este é o mapa que liga as técnicas dos próximos módulos.

POR QUE "SEGURANÇA NO FINAL" FALHA

Quando a segurança é uma revisão feita na véspera do lançamento, os achados chegam tarde: ou o lançamento atrasa, ou o risco é "aceito" sob pressão e a falha vai para produção. Falhas de **projeto** são as mais caras, pois exigem refazer a arquitetura (um sistema que mistura dados de vários clientes sem isolamento não se conserta com um *patch*). O **shift-left** responde a isso: levar a segurança para as fases iniciais, onde corrigir é mais barato.

> **Cuidado com o "100×":** é comum ouvir que corrigir um defeito em produção custa 100 vezes mais do que nos requisitos. O número é repetido há décadas, mas sua origem empírica é **contestada**: os dados primários são antigos e vêm de projetos com processos bem diferentes dos atuais. O que se sustenta é a direção da ideia (quanto mais tarde, maior o custo de retrabalho, de exposição e de coordenação), e não o multiplicador exato.

O *shift-left* não basta: há falhas que só aparecem em produção, então o ciclo também investe em **monitoramento, resposta e gestão de vulnerabilidades** depois da liberação (*shift-right*). A regra é **segurança em todas as fases, com feedback contínuo**.

O CICLO DE VIDA SEGURO, FASE A FASE

O **SDLC** descreve as fases pelas quais um software passa; o **SSDLC** acrescenta a cada uma atividades de segurança, artefatos que as comprovam e responsáveis. Seja o processo em cascata, Scrum ou contínuo, as atividades ocorrem a **cada iteração**.



Ciclo de vida seguro. Cada fase tem atividades de segurança próprias e um *gate*, um ponto de decisão em que o trabalho só avança se os critérios foram cumpridos ou se o risco foi aceito por quem tem autoridade. O ciclo é um laço: o que se aprende em produção volta como requisito.

11 CICLO DE VIDA SEGURO (SSDLC) — CONTINUAÇÃO

EXEMPLO RESOLVIDO

Uma funcionalidade atravessando o ciclo. O portal acadêmico vai oferecer "**exportar o histórico escolar em PDF**". Veja o que cada fase pega:

- **Requisitos**: um caso de abuso ("como aluno mal-intencionado, quero baixar o histórico de outro aluno") vira o requisito **autorização por objeto** em todo acesso ao arquivo (p. 53).
- **Projeto**: a modelagem de ameaças nota que o gerador de PDF baixa imagens e estilos por URL, o que abre uma **SSRF**; a decisão é bloquear o acesso à rede do gerador e rodá-lo isolado.
- **Implementação**: a revisão do *pull request* aponta que o ID do aluno vem da URL, sem conferir o dono; o SAST acusa `shell=True` no comando que invoca o conversor.
- **Verificação**: o teste automático de IDOR falha e é corrigido; a análise de dependências aponta uma biblioteca de PDF com CVE conhecida.
- **Liberação**: o SBOM é gerado, o *gate* confere que não há críticos abertos e o limite de requisições por usuário está configurado.
- **Operação**: o registro mostra quem exportou o quê; um alerta dispara se alguém exporta centenas de históricos em minutos.

Nenhuma fase sozinha teria pego tudo; juntas, cada uma barra o que a anterior deixou passar.

FASE	ATIVIDADE DE SEGURANÇA	ARTEFATO	RESPONSÁVEL
Requisitos	Requisitos de segurança e de privacidade; casos de abuso; classificação dos dados; critérios de aceitação	Histórias com critérios de segurança; classificação	Produto, com apoio de segurança
Projeto	Modelagem de ameaças (p. 20); padrões de arquitetura segura; revisão de projeto	DFD, modelo de ameaças, decisões de arquitetura	Arquitetura e segurança
Implementação	Padrões de codificação; revisão por pares; SAST, varredura de segredos e de dependências no IDE e no CI	*Pull requests* revisados; resultados das ferramentas	Pessoas desenvolvedoras
Verificação	Testes de segurança automáticos e manuais; DAST, fuzzing e teste de intrusão (p. 166)	Relatórios, tíquetes, evidências de correção	Qualidade e segurança
Liberação	*Gate* de segurança; SBOM e assinatura (p. 135); revisão final de riscos; plano de resposta	Checklist de liberação; aceite de risco	Gestão e segurança
Operação	Monitoramento (p. 144); gestão de vulnerabilidades; resposta a incidentes (p. 156); divulgação; retrospectivas	Runbooks, painéis, relatórios pós-incidente	Operações, SRE e segurança

MODELOS DE REFERÊNCIA: SSDF, SAMM, BSIMM E SDL

Ninguém precisa inventar um programa de segurança de software do zero. Há modelos que dizem **o que fazer** (práticas) e outros que medem **o quanto já se faz** (maturidade); a escolha depende do objetivo.

11 CICLO DE VIDA SEGURO (SSDLC)

CONTINUAÇÃO

MODELO	O QUE É	QUANDO USAR
NIST SSDF (SP 800-218)	Conjunto de práticas de alto nível em quatro grupos: **PO** (preparar a organização), **PS** (proteger o software), **PW** (produzir software bem protegido) e **RV** (responder a vulnerabilidades). É a base do atestado de desenvolvimento seguro exigido de fornecedores do governo dos EUA	Linguagem comum para contratos, auditorias e requisitos de clientes
OWASP SAMM v2	Modelo de maturidade **aberto e prescritivo**: 5 funções de negócio, 15 práticas, 2 fluxos por prática e 3 níveis, com perguntas de avaliação	Medir o estado atual, escolher o nível-alvo e montar um roteiro de melhoria
BSIMM	Modelo **descritivo**: mede o que dezenas de organizações realmente fazem, em mais de cem atividades, organizadas em quatro domínios (governança, inteligência, pontos de contato do SSDL e implantação)	Comparar-se com empresas do mesmo setor
Microsoft SDL	Processo pioneiro, nascido da iniciativa Trustworthy Computing (2002), com práticas por fase: treinamento, requisitos, projeto, implementação, verificação, liberação e resposta	Inspiração histórica e referência de processo por fase
Secure by Design (CISA)	Princípios para fabricantes: assumir a responsabilidade pelos resultados de segurança dos clientes, adotar transparência e responsabilização radicais e liderar a partir do topo; e entregar produtos **seguros por padrão**	Orientar a cultura e as decisões de produto, não apenas o processo



Cada prática tem dois fluxos (A e B) e três níveis de maturidade: nível 1 (início, ad hoc), nível 2 (estruturado) e nível 3 (otimizado, em escala).
Meça cada prática com as perguntas do modelo, defina o nível-alvo por risco e planeje o roteiro de melhoria em ciclos curtos.

Mapa do OWASP SAMM v2. Cinco funções de negócio, com três práticas cada: um total de 15 práticas que podem ser avaliadas, pontuadas por nível de maturidade e usadas para planejar a evolução do programa de segurança da organização.

O **SSDF** é um catálogo de práticas com identificadores, por exemplo **PO.1** (definir requisitos de segurança), **PW.1** (projetar o software para cumprir os requisitos e mitigar riscos), **PW.7** (revisar ou analisar o código legível por humanos), **PW.8** (testar o código executável), **PW.9** (configurar o software com padrões seguros) e **RV.1** (identificar e confirmar vulnerabilidades continuamente). Uma revisão do documento (SP 800-218 Rev. 1) esteve em consulta pública desde o fim de 2025, então confira a versão vigente antes de citá-lo em um contrato. Na prática, **SAMM e SSDF se complementam**: o primeiro mede a maturidade de cada prática; o segundo é a lista de verificação que os clientes e os reguladores passam a exigir.

11 CICLO DE VIDA SEGURO (SSDLC) — CONTINUAÇÃO

REQUISITOS DE SEGURANÇA E CASOS DE ABUSO

Um requisito de segurança só serve se for **verificável**: "o sistema deve ser seguro" não é requisito; "toda requisição a `/notas/{id}` verifica no servidor que o aluno do token é o dono da nota" é, porque um teste pode provar que vale ou não. Os requisitos vêm de quatro fontes: leis e normas (p. 27), riscos do negócio (p. 20), catálogos de requisitos (o ASVS, abaixo) e **casos de abuso**.

Um **caso de abuso** inverte o caso de uso: em vez de "como aluno, quero ver minhas notas", escreve-se "como atacante, quero ver as notas de outro aluno". É um exercício rápido de modelagem de ameaças que gera, para cada abuso, uma **contramedida** e um **critério de aceitação** testável. Também alimenta o STRIDE da fase de projeto.

O **OWASP ASVS** (*Application Security Verification Standard*), versão **5.0.0** de 30 de maio de 2025, é o catálogo de requisitos verificáveis mais usado: tem **17 capítulos** (do V1, codificação e sanitização, ao V17, WebRTC), que cobrem validação, autenticação, sessões, autorização, tokens, criptografia, comunicação segura, configuração, proteção de dados, registro e outros temas. Cada requisito tem um identificador (como `v5.0.0-6.2.1`) que pode ser citado em histórias, testes e contratos, e pertence a um dos três níveis:

NÍVEL	PARA QUEM	EXEMPLO DE APLICAÇÃO
L1	Todas as aplicações; a defesa mínima contra os ataques mais comuns e fáceis de executar	Site público com formulário e login; protótipo exposto
L2	A maioria das aplicações com dados pessoais ou de negócio relevantes; o padrão recomendado	Portal acadêmico, loja virtual, SaaS B2B
L3	Aplicações críticas, em que uma falha causa dano sério: exige defesa em profundidade e revisão mais rigorosa	Internet banking, prontuário eletrônico, infraestrutura crítica

O ASVS serve de **lista de verificação** em revisões de projeto, de **base para testes** (cada requisito vira um caso de teste) e de **contrato**: "o fornecedor deve atender ao ASVS 5.0 nível 2" é um critério objetivo de aceite, bem melhor do que "seguir boas práticas".

EXEMPLO RESOLVIDO

História: redefinir senha. "Como usuária que esqueceu a senha, quero redefini-la por e-mail para voltar a acessar a minha conta." Antes de programar, a equipe lista os **casos de abuso**:

- **Enumeração de contas**: descobrir quais e-mails têm conta pela mensagem ou pelo tempo de resposta.
- **Adivinhar ou reutilizar o link** de redefinição.
- **Envenenar o link**: forjar o cabeçalho `Host` para que o e-mail leve a vítima a um domínio do atacante.
- **Bombardear a vítima** com e-mails de redefinição (spam e negação de serviço).
- **Manter o acesso**: continuar logado depois que a dona trocou a senha.

Cada abuso vira um critério de aceitação. O token tem **pelo menos 128 bits** aleatórios e é guardado no banco só como *hash* (p. 44). Os critérios a seguir podem virar testes automáticos:

11 CICLO DE VIDA SEGURO (SSDLC) — CONTINUAÇÃO

CRITÉRIOS DE ACEITAÇÃO EM GHERKIN (DADO, QUANDO, ENTÃO)

```
Funcionalidade: Redefinição de senha por e-mail

  Cenário: Resposta idêntica para contas existentes e inexistentes
    Dado que informo um e-mail que não tem conta
    Quando peço a redefinição de senha
    Então vejo a mesma mensagem e o mesmo tempo de resposta de um e-mail com conta

  Cenário: Link de uso único e curta duração
    Dado que recebi o link de redefinição há mais de 15 minutos
    Quando abro o link
    Então ele é recusado e preciso pedir outro
    E um link já usado também é recusado

  Cenário: Domínio do link controlado pelo servidor
    Dado que envio o pedido com um cabeçalho Host adulterado
    Quando o e-mail é gerado
    Então o link usa o domínio configurado no servidor, nunca o do cabeçalho

  Cenário: Limite de pedidos
    Dado que a mesma conta já recebeu 5 pedidos na última hora
    Quando chega um novo pedido
    Então ele é limitado e a dona da conta recebe um aviso

  Cenário: Conclusão da troca
    Quando defino a nova senha
    Então todas as sessões e tokens anteriores são revogados
    E recebo um e-mail confirmando a troca
```

PROJETO SEGURO: PADRÕES DE ARQUITETURA

O projeto é a fase em que a segurança é mais barata e mais difícil de reverter. Os princípios de Saltzer e Schroeder (p. 5) viram padrões concretos de arquitetura; a **revisão de projeto** confere se eles foram aplicados, com a ajuda do modelo de ameaças (p. 20) e de checklists.

01 DEFESA EM PROFUNDIDADE

Camadas independentes (rede, host, aplicação, dados): a falha de uma não expõe tudo. Nenhum controle isolado é ponto único de falha.

02 FALHAR FECHADO

Diante de erro, exceção ou timeout, o sistema **nega** o acesso. "Se o serviço de autorização cair, libera" é falha de projeto.

03 SEGREGAÇÃO E PRIVILÉGIO MÍNIMO

Ambientes, contas e funções separados: a conta da aplicação não é dona do banco; produção não é acessível do notebook do desenvolvedor.

04 PADRÕES SEGUROS (SECURE DEFAULTS)

O caminho mais fácil é o seguro: recursos perigosos desativados, cookies `Secure`, permissões mínimas e senhas padrão proibidas.

05 REDUZIR A SUPERFÍCIE

Menos funções, portas, dependências e dados coletados significam menos o que atacar, corrigir e proteger.

06 CENTRALIZAR CONTROLES

Autenticação, autorização, validação e registro em bibliotecas e serviços únicos, bem testados, e não reimplementados em cada tela.

11 CICLO DE VIDA SEGURO (SSDLC) — CONTINUAÇÃO

Registre as decisões de segurança do projeto em **ADRs** (*Architecture Decision Records*): o que foi decidido, por quê, quais riscos foram aceitos e quando revisar. Daqui a dois anos, quem herdar o sistema saberá por que o serviço de arquivos é isolado e não precisará descobrir à força.

IMPLEMENTAÇÃO E DEFINIÇÃO DE PRONTO

Na implementação, a segurança vive nos hábitos da equipe: seguir **padrões de codificação segura** (p. 122), submeter todo código a **revisão por pares** com um roteiro de segurança (autorização checada? entrada validada? segredos fora do código? erro sem vazamento de dados?), automatizar o que dá para automatizar (SAST, varredura de segredos e de dependências no IDE, no *pre-commit* e no CI, p. 135) e usar **pareamento** nas áreas mais sensíveis (autenticação, pagamentos, criptografia). A segurança entra na rotina por meio da **definição de pronto** (*Definition of Done*): uma história só está concluída se cumprir também os critérios de segurança.

CHECKLIST

Definição de pronto com critérios de segurança:

- A história tem requisitos de segurança e, quando cabe, casos de abuso e critérios de aceitação testáveis.
- O modelo de ameaças foi atualizado se a arquitetura ou as fronteiras de confiança mudaram.
- O código foi revisado por outra pessoa com o roteiro de segurança; não há segredos nem dados reais em código, testes ou logs.
- SAST, SCA e varredura de segredos passam sem achados críticos ou altos em aberto (ou há exceção aprovada, com dono e prazo).
- Há testes automáticos para os requisitos de segurança (autorização, validação, limites, erros).
- Eventos de segurança relevantes geram log e alerta; o runbook e a documentação foram atualizados.
- A liberação gera SBOM e artefatos assinados quando o pipeline exige.

Duas figuras ajudam a sustentar esses hábitos. Os **security champions** são pessoas da própria equipe de desenvolvimento, com treinamento extra em segurança, que atuam como ponte com a área de segurança: levam o assunto às reuniões de planejamento, fazem a primeira triagem de achados e difundem boas práticas. E o **treinamento** deve ser prático e contínuo (laboratórios como OWASP Juice Shop e WebGoat, desafios *capture the flag*, análise dos próprios incidentes), mais do que um curso anual de conformidade.

SEGURANÇA EM EQUIPES ÁGEIS

No desenvolvimento ágil não existe uma "fase de segurança": existem **ciclos curtos**, e a segurança precisa caber neles sem virar gargalo. A saída é trocar *gates* manuais por **guardrails** automáticos (verificações no pipeline que dão retorno em minutos) e distribuir as atividades pelas cerimônias que a equipe já faz.

CERIMÔNIA OU ARTEFATO	ATIVIDADE DE SEGURANÇA
Refinamento do backlog	Requisitos e casos de abuso por história; achados de segurança priorizados junto das funcionalidades
Planejamento da iteração	Capacidade reservada para a dívida de segurança e para correções de severidade alta, em vez de "quando sobrar tempo"
Desenvolvimento	Revisão por pares com roteiro de segurança; varredura de segredos e SAST leve no IDE e no *pre-commit*
Revisão e demonstração	Mostrar também os controles de segurança da funcionalidade (por exemplo, o bloqueio por limite de tentativas)

11 CICLO DE VIDA SEGURO (SSDLC) — CONTINUAÇÃO

CERIMÔNIA OU ARTEFATO	ATIVIDADE DE SEGURANÇA
Retrospectiva	Análise sem culpa de incidentes e quase-incidentes; ajuste da definição de pronto
Épicos de alto risco	Uma sessão curta de modelagem de ameaças (*spike* de segurança) antes de começar a construir

O conceito que sustenta tudo isso é o do **caminho pavimentado** (*paved road*): a plataforma da organização oferece modelos de projeto, bibliotecas e pipelines já seguros, de modo que a equipe que segue o caminho padrão **herda os controles** sem precisar pedi-los. Fazer o certo precisa ser mais fácil do que fazer o errado; se a rota segura é burocrática, as pessoas criam atalhos.

OPERAÇÃO: VULNERABILIDADES, DIVULGAÇÃO E MÉTRICAS

Depois da liberação, o software passa a ser atacado de verdade, e novas vulnerabilidades surgem em dependências que ninguém tocou. A operação segura inclui a **gestão contínua de vulnerabilidades** (p. 20), o **monitoramento e a resposta a incidentes** (p. 144 e p. 156) e um canal para que **terceiros relatem falhas**: a **divulgação coordenada de vulnerabilidades** (CVD), tratada pelas normas ISO/IEC 29147 (divulgação) e 30111 (tratamento). O **RFC 9116** padronizou um arquivo, o `security.txt`, publicado em `/.well-known/security.txt`, que diz a pesquisadores como e a quem relatar.

/.WELL-KNOWN/SECURITY.TXT (RFC 9116)

```
Contact: mailto:seguranca@exemplo.com.br
Contact: https://exemplo.com.br/seguranca/relatar
Expires: 2027-06-30T23:59:59.000Z
Encryption: https://exemplo.com.br/.well-known/chave-pgp.asc
Preferred-Languages: pt-BR, en
Canonical: https://exemplo.com.br/.well-known/security.txt
Policy: https://exemplo.com.br/seguranca/politica-de-divulgacao
```

Uma **política de divulgação de vulnerabilidades** (VDP) estabelece o escopo, as regras e o compromisso de **não processar quem age de boa-fé** (*safe harbor*), além do prazo de resposta e de correção; um programa de recompensas (*bug bounty*) vai além e paga por achados válidos. Para saber se o programa funciona, **meça**:

MÉTRICA	O QUE MOSTRA	CUIDADO
Densidade de vulnerabilidades	Falhas por aplicação ou por mil linhas de código, ao longo do tempo	Compare tendências, não equipes; muda com o escopo das ferramentas
MTTR por severidade	Tempo médio entre a descoberta e a correção em produção	Meça até o *deploy*, e não só até o commit da correção
Cobertura	Percentual de repositórios com SAST, SCA e varredura de segredos; de histórias com requisitos de segurança; de sistemas com modelo de ameaças	Cobertura mede adoção, não eficácia
Taxa de escape	Fração das vulnerabilidades achadas só em produção ou por terceiros, e não internamente	Cai quando as fases anteriores funcionam; esconde-se se ninguém procura

11 CICLO DE VIDA SEGURO (SSDLC) — CONTINUAÇÃO

MÉTRICA	O QUE MOSTRA	CUIDADO
Dívida de segurança	Vulnerabilidades e exceções abertas além do prazo, por idade e por risco	Registre dono, prazo e critério de pagamento, ou a dívida some do radar
Maturidade (SAMM ou BSIMM)	Evolução das práticas ao longo dos ciclos	Use para planejar a melhoria, e não para ranquear equipes

> **Métricas de vaidade:** "número de vulnerabilidades encontradas" isolado pode subir porque a equipe passou a procurar melhor, e "zero achados" pode significar que ninguém olhou. Combine métricas de resultado (MTTR, taxa de escape) com métricas de cobertura e **sempre** acompanhe as exceções.

Por fim, o **Secure by Design** lembra que a responsabilidade pela segurança é de quem **constrói** o produto, e não só de quem o usa. Isso muda decisões concretas: entregar com **padrões seguros** (MFA ligado, senhas padrão proibidas, registro de auditoria incluído), eliminar **classes inteiras de falha** em vez de corrigir instâncias (linguagens de memória segura, p. 122; consultas parametrizadas, p. 98) e ser **transparente** sobre vulnerabilidades, publicando CVEs e correções com clareza.

PARA FIXAR

Conceitos

- Por que o *shift-left* não basta e o que se perde ao ignorar a operação?
- Qual a diferença entre o SSDF, o SAMM e o BSIMM, e quando usar cada um?
- O que é um caso de abuso e como ele vira critério de aceitação?

Aplicação

- Escolha uma funcionalidade (upload de foto de perfil) e liste a atividade de segurança de cada fase, com um *gate* para cada uma.
- Escreva três critérios de aceitação em Gherkin para "alterar e-mail da conta".
- Defina cinco itens de uma definição de pronto e a métrica que mostraria se eles estão sendo cumpridos.

APROFUNDE-SE

Leituras e referências

- NIST SP 800-218 (SSDF) — NIST: o Secure Software Development Framework.
- OWASP SAMM — OWASP: modelo de maturidade de segurança de software, com ferramenta de avaliação.
- OWASP ASVS — OWASP: padrão de verificação de segurança de aplicações (versão 5.0).
- BSIMM — Black Duck: estudo descritivo das práticas de segurança de software nas organizações.
- Secure by Design — CISA: princípios e guias para fabricantes de software.
- RFC 9116: security.txt — IETF: o formato para publicar o canal de relato de vulnerabilidades.

MÓDULO 12 · DESENVOLVIMENTO

12 SEGURANÇA WEB E OWASP TOP 10

As vulnerabilidades web mais frequentes, por que acontecem e como o código correto as evita.

O HTTP é a língua em que quase todo sistema moderno conversa, e por isso a segurança web concentra as falhas mais exploradas do mundo. Este módulo mostra como navegador e servidor confiam (e desconfiam) um do outro, percorre as **10 categorias do OWASP Top 10:2025** e aprofunda o mecanismo e a defesa das classes de falha mais comuns: **controle de acesso quebrado, injeção (SQL e comandos), XSS, CSRF e SSRF**, além de upload de arquivos, desserialização, erros de configuração, **cabeçalhos de segurança**, **cookies** e **CORS**.

A WEB SOB A ÓTICA DA SEGURANÇA

O **HTTP** é um protocolo de requisição e resposta **sem estado**: cada pedido chega ao servidor sem memória do anterior. Para lembrar quem é o usuário, a aplicação entrega ao navegador um **identificador de sessão** (ou um token) em um **cookie**, e o navegador o devolve **automaticamente** em toda requisição ao mesmo domínio. Esse envio automático explica o CSRF; a execução de JavaScript dentro da página explica o XSS. Três ideias sustentam a segurança web:

- **Toda entrada é não confiável.** URL, parâmetros, cabeçalhos, cookies, corpo, campos "ocultos" e arquivos vêm de um cliente que o atacante controla. Quem ataca não usa o seu formulário: usa `curl`, scripts ou um proxy de interceptação.
- **O servidor decide.** Autenticação, autorização e validação precisam ser impostas no servidor; menus ocultos, botões desabilitados e validações em JavaScript melhoram a experiência, mas são simplesmente ignorados por quem ataca.
- **O navegador isola origens.** A *same-origin policy* separa o conteúdo de sites diferentes, e cookies, CORS e CSP se constroem sobre ela ou ao seu redor.

A **política de mesma origem** (*same-origin policy*, SOP) é a regra de isolamento do navegador: um script carregado de uma origem não pode **ler** os dados nem o DOM de outra. Uma **origem** é o trio **esquema + host + porta**; já o *site* (termo usado pelos cookies `SameSite`) é o **domínio registrável**, como `exemplo.com.br`, sem importar subdomínio nem porta. Tomando `https://app.exemplo.com.br` como referência (e lembrando que trocar HTTPS por HTTP também muda a origem):

URL COMPARADA	MESMA ORIGEM?	MESMO SITE?	MOTIVO
`https://app.exemplo.com.br/notas`	Sim	Sim	Só o caminho muda
`https://app.exemplo.com.br:8443`	Não	Sim	Porta diferente
`https://api.exemplo.com.br`	Não	Sim	Subdomínio diferente
`https://exemplo.com.br`	Não	Sim	Host diferente (sem o subdomínio)
`https://app.outro.com.br`	Não	Não	Domínio registrável diferente

Um detalhe decisivo: a SOP **não impede o envio** de requisições entre origens, apenas que o script de uma origem **leia a resposta** de outra. Um formulário em `https://malicioso.example` pode enviar um `POST` ao seu banco, e o navegador anexa os cookies do banco ao pedido (a menos que o cookie use `SameSite`). O dano do CSRF está no **efeito** da requisição, e não na leitura da resposta; por isso a SOP, sozinha, não o impede.

12 SEGURANÇA WEB E OWASP TOP 10 — CONTINUAÇÃO

CORS: RELAXAR A SOP, E O QUE ELE NÃO É

O **CORS** (*Cross-Origin Resource Sharing*) permite **relaxar** a SOP de forma controlada: o servidor responde com cabeçalhos dizendo quais origens podem **ler** suas respostas. O `Access-Control-Allow-Origin` indica a origem autorizada; o `Access-Control-Allow-Credentials: true` permite que a chamada leve cookies (e então a origem **precisa ser explícita**, nunca `*`).

Requisições "não simples" (métodos como `PUT` e `DELETE`, cabeçalhos personalizados, `Content-Type: application/json`) são precedidas por um *preflight*, uma requisição `OPTIONS` em que o navegador pergunta se o envio é permitido. Se a resposta varia conforme a origem, envie `Vary: Origin` para que caches não misturem as respostas.

> **CORS não é controle de acesso:** O CORS é aplicado **pelo navegador**: `curl`, scripts e outros servidores chamam a sua API sem restrição alguma, e ele também não impede o envio de requisições "simples" (o CSRF acontece mesmo com a resposta ilegível). Erros comuns: refletir o cabeçalho `Origin` recebido como `Access-Control-Allow-Origin` (equivale a liberar todos), aceitar a origem `null`, validar com expressão regular frouxa (uma checagem de sufixo que aceite `malexemplo.com.br`) e liberar `*` em API com dados privados. A proteção real está na **autenticação e na autorização no servidor** (p. 44 e p. 53).

OWASP TOP 10:2025: O MAPA DOS RISCOS

O **OWASP Top 10** é um documento de conscientização sobre os riscos mais críticos de aplicações web, construído a partir de dados reais de testes e de vulnerabilidades (CVEs) e da opinião de especialistas. A edição **2025** reorganiza a de 2021. Cada categoria é uma **família de CWEs** (as fraquezas catalogadas pelo MITRE), e não uma vulnerabilidade única; por isso a lista serve para **priorizar estudo e treinamento**, e não como padrão de verificação (para requisitos testáveis, use o ASVS, p. 90). A figura mostra onde as principais classes de falha atuam.



Onde cada ataque web acontece. O navegador, a aplicação, o banco e os serviços internos formam uma cadeia em que cada elo confia no anterior: quem controla o dado que entra no primeiro pode tentar atingir os outros. Os cartões vermelhos mostram as classes de falha de cada elo; o restante do módulo explica o mecanismo e a defesa de cada uma.

12 SEGURANÇA WEB E OWASP TOP 10

CONTINUAÇÃO

CATEGORIA (2025)	O QUE É	CWES DE EXEMPLO (TOTAL)	FRENTE A 2021
A01 Broken Access Control	Usuário age fora do que deveria: lê dado alheio, eleva privilégio, acessa função restrita	862, 863, 639, 352, 22, 918 (40)	Segue em 1º; **incorpora o SSRF**
A02 Security Misconfiguration	Configuração insegura ou padrão: depuração ligada, senhas padrão, cabeçalhos ausentes, XXE	16, 611 (16)	Sobe de 5º para 2º
A03 Software Supply Chain Failures	Falhas em dependências, builds e distribuição do software	1104, 1395, 1329, 1357 (6)	Amplia "componentes vulneráveis e desatualizados" (era 6º)
A04 Cryptographic Failures	Criptografia ausente, fraca ou mal usada; dados expostos	327, 331, 338 (32)	Desce de 2º para 4º
A05 Injection	Dado do usuário interpretado como comando: SQL, sistema operacional, templates, XSS	89, 79, 78, 77, 94 (37)	Desce de 3º para 5º
A06 Insecure Design	Controles que faltam no projeto; falhas de lógica de negócio	256, 269, 434 (39)	Desce de 4º para 6º
A07 Authentication Failures	Falhas de identificação e de sessão: senhas fracas, ausência de MFA	287, 259, 384 (36)	Mesma posição (7º)
A08 Software or Data Integrity Failures	Código ou dados aceitos sem verificar a integridade	502, 829, 915 (14)	Mesma posição (8º)
A09 Security Logging and Alerting Failures	Registro insuficiente ou sem alerta que provoque uma ação	117, 532, 778 (5)	Mesma posição (9º); ênfase em **alertas**
A10 Mishandling of Exceptional Conditions	Erros e situações anormais tratados mal: falhar aberto, erros verbosos	209, 234, 274, 476, 636 (24)	**Nova**; ocupa o lugar do SSRF

Entre parênteses, a quantidade de CWEs mapeados a cada categoria em 2025. O **CWE Top 25 de 2025** (MITRE) confirma o peso das falhas web: os três primeiros lugares são **XSS (CWE-79)**, **injeção de SQL (CWE-89)** e **CSRF (CWE-352)**, seguidos de **autorização ausente (CWE-862)**. As falhas "clássicas" seguem as mais frequentes, e quase todas têm defesas conhecidas há décadas.

A01 · CONTROLE DE ACESSO QUEBRADO

O controle de acesso define **quem pode fazer o quê, sobre qual objeto**, e falha quando o servidor deixa o usuário agir fora desses limites. É a categoria mais presente: segundo o OWASP, **100% das aplicações testadas** tinham alguma forma dela. Os modelos de autorização estão em p. 53; estas são as manifestações típicas na web:

12 SEGURANÇA WEB E OWASP TOP 10 — CONTINUAÇÃO

01 IDOR E BOLA

`GET /notas?aluno=1234`: trocar o número devolve as notas de outro aluno porque o servidor **confia no identificador** do pedido. Defesa: autorização **por objeto** a cada acesso, com o dono derivado da sessão e não do pedido. IDs imprevisíveis (UUID) ajudam, mas não substituem a checagem.

02 NAVEGAÇÃO FORÇADA

A rota `/admin/relatorios` some do menu, mas responde a quem digitar o endereço: esconder não é proteger. Defesa: **negar por padrão** e exigir o papel em **cada rota e cada método HTTP** (o `GET` fica protegido e o `DELETE` é esquecido).

03 ATRIBUIÇÃO EM MASSA

Um cadastro que aceita `papel=admin` no corpo porque o *framework* copia todos os campos recebidos para o objeto (**p. 112**). Defesa: lista de **campos permitidos** por rota, e não "tudo, menos o proibido".

A regra de ouro, repetida na página do OWASP: implemente o controle de acesso **uma vez**, em código confiável do lado do servidor, e **reutilize** em toda a aplicação; exija a posse do registro (o aluno só enxerga os próprios dados), registre as negativas e cubra tudo com **testes automáticos de autorização** (p. 166).

CSRF: PEDIDOS FORJADOS COM A SESSÃO DA VÍTIMA

No **CSRF** (*Cross-Site Request Forgery*, CWE-352) o atacante não rouba a sessão: ele faz o **navegador da vítima** enviar, com o cookie dela, um pedido que ela não pretendia fazer. Funciona quando a autenticação é **automática** (cookies, autenticação HTTP) e o servidor não confere se o pedido nasceu de uma tela legítima do próprio site.

1 Sessão aberta
Ana está logada em `banco.exemplo.com.br`; o navegador guarda o cookie de sessão.

2 Página do atacante
Ana abre uma página maliciosa (e-mail, anúncio, fórum) com um formulário oculto apontando para `/transferir`.

3 Envio automático
O navegador envia o `POST` ao banco **e anexa o cookie de Ana**, pois o destino é o banco.

4 Ação executada
O servidor vê uma sessão válida e executa a transferência. A SOP não ajudou: ninguém precisou ler a resposta.

DEFESA	COMO FUNCIONA	CUIDADOS
Cookie `SameSite`	Com `Lax` ou `Strict`, o navegador não anexa o cookie a pedidos iniciados em outro *site* (o `Lax` ainda o envia em navegações de topo com `GET`)	Declare o atributo; nunca mude estado com `GET`; `None` exige `Secure` e reabre o risco
Token anti-CSRF	Valor imprevisível, atrelado à sessão, enviado em campo oculto ou cabeçalho e conferido no servidor	Em SPAs, use um cabeçalho personalizado; o *double submit* ingênuo exige cookie assinado
Origin e Sec-Fetch-Site	O servidor recusa pedidos que mudam estado quando os cabeçalhos indicam origem externa (`cross-site`)	Navegadores modernos enviam; defina o que fazer quando faltam
Reautenticação	Pedir senha, MFA ou confirmação em ações críticas (transferir, trocar e-mail)	Custa usabilidade; reserve às ações de maior impacto

Não desligue a proteção do framework: Django, Spring Security, Rails e ASP.NET trazem proteção CSRF pronta. O erro típico é desativá-la para "fazer o formulário funcionar". Alguns navegadores (como o Chrome, desde 2020) tratam cookies sem `SameSite` como `Lax`, mas isso **não vale para todos**: declare o atributo explicitamente.

12 SEGURANÇA WEB E OWASP TOP 10

CONTINUAÇÃO

SSRF: O SERVIDOR FAZENDO O PEDIDO DO ATACANTE

No **SSRF** (*Server-Side Request Forgery*, CWE-918), a aplicação busca uma URL informada (ou influenciada) por quem ataca: pré-visualização de links, importação por URL, *webhooks*, conversores de PDF e de imagens. Como a requisição parte do **servidor**, ela alcança o que o atacante não vê da internet: serviços internos, painéis administrativos e, na nuvem, o serviço de **metadados da instância**, de onde saem credenciais temporárias (p. 78 traz o caso Capital One, de 2019). Em 2025 o SSRF deixou de ser a categoria A10 e passou a fazer parte da A01. As defesas se somam:

- **Lista de permissão de destinos**: apenas esquema `https` e hosts conhecidos; evite aceitar URLs livres.
- **Validar o IP resolvido**, e não só o nome: bloqueie endereços privados, de *loopback*, *link-local* (inclui `169.254.169.254`) e reservados, em IPv4 e IPv6. **Fixe o IP validado** na conexão, para que um DNS que responda diferente depois (*DNS rebinding*) não escape da checagem.
- Não siga **redirecionamentos** às cegas (ou revalide a cada salto), limite tamanho, tempo e tipo da resposta e **não devolva o conteúdo bruto** ao usuário.
- **Defesa em profundidade**: filtro de saída (*egress*) na rede, IMDSv2 e funções com privilégio mínimo (p. 78).

VALIDAÇÃO DE URL DE DESTINO (PYTHON): LISTA DE PERMISSÃO + IP PÚBLICO

```
import ipaddress, socket
from urllib.parse import urlparse

PERMITIDOS = {"cdn.exemplo.com.br", "api.parceiro.example"}

def destino_seguro(url: str) -> bool:
    u = urlparse(url)
    if u.scheme != "https" or u.hostname not in PERMITIDOS:
        return False                                  # lista de permissão
    for info in socket.getaddrinfo(u.hostname, 443, type=socket.SOCK_STREAM):
        if not ipaddress.ip_address(info[4][0]).is_global:
            return False       # privado, loopback, link-local (metadados), CGNAT, reservado
    return True    # atenção: conecte ao IP validado, sem resolver o nome de novo
```

A02 · CONFIGURAÇÃO INSEGURA

É a categoria dos erros que **não estão no código**, e sim na configuração: o software é seguro e o ambiente o expõe. Cresce com a quantidade de peças (servidor web, *framework*, banco, nuvem, contêineres) e com a pressa de "fazer funcionar". A resposta é engenharia de configuração: **linhas de base** (CIS Benchmarks), **infraestrutura como código** revisada e **varredura automática** a cada mudança (p. 78 e p. 135).

FALHA DE CONFIGURAÇÃO	RISCO	CORREÇÃO
Modo de depuração ligado em produção	Páginas de erro com código e variáveis; em alguns *frameworks*, console interativo que executa código	Depuração desligada; produção sem ferramentas de desenvolvimento
Contas e senhas padrão	Acesso administrativo imediato (`admin/admin`)	Trocar no primeiro uso; proibir senhas padrão; remover contas de exemplo
Listagem de diretórios e arquivos de apoio expostos	Vazam cópias (`.bak`, `.zip`), `.git/` e `.env` com segredos	Desativar o índice automático; bloquear arquivos ocultos; publicar só o necessário
Painéis e *buckets* abertos	Consoles administrativos e armazenamento acessíveis a qualquer um	Rede privada, autenticação forte e bloqueio de acesso público por política

12 SEGURANÇA WEB E OWASP TOP 10 — CONTINUAÇÃO

FALHA DE CONFIGURAÇÃO	RISCO	CORREÇÃO
Banners e cabeçalhos de segurança	`Server` e `X-Powered-By` revelam versões; faltam proteções do navegador	Ocultar versões; cabeçalhos da seção de cabeçalhos, mais adiante
Parser XML com entidades externas (XXE, CWE-611)	Leitura de arquivos locais e SSRF a partir de um XML enviado	Desativar DTD e entidades externas (em Python, `defusedxml`)

A05 · INJEÇÃO: QUANDO O DADO VIRA COMANDO

Há **injeção** quando dados controlados por quem ataca são combinados a um **comando ou consulta** e o interpretador não consegue distinguir uma coisa da outra. O interpretador pode ser um banco SQL, um *shell*, um servidor LDAP, um motor de templates ou o próprio navegador (no XSS). A defesa estrutural é sempre a mesma: **manter dados e código em canais separados** (consultas parametrizadas, APIs que não passam por um interpretador) e, só em último caso, **escapar** conforme o contexto. Segundo o OWASP, é a categoria com mais CVEs registrados.

Concatenação: o dado vira código

1. A aplicação monta o texto da consulta

```
"SELECT * FROM notas WHERE aluno = '" + entrada + "'"
```

2. Quem ataca digita, no campo, algo que parece código

```
x' OR '1'='1
```

3. O banco recebe um único texto e o interpreta inteiro

```
SELECT * FROM notas WHERE aluno = 'x' OR '1'='1'
```

→ condição sempre verdadeira: devolve as notas de todos

Parametrizada: o dado continua dado

1. O texto fixo da consulta tem só um marcador (?)

```
SELECT * FROM notas WHERE aluno = ?
```

2. O banco analisa a estrutura antes de ver o dado

```
estrutura: filtrar notas onde aluno = [valor]
```

3. O valor segue por canal separado, só como dado

```
valor = "x' OR '1'='1" (texto literal)
```

→ procura um aluno chamado x' OR '1'='1 e não acha

Código e dados em canais separados. Na concatenação, o banco recebe um único texto e não tem como saber o que era dado e o que era comando. Na consulta parametrizada, a estrutura é definida antes e o valor chega à parte: por mais que pareça SQL, nunca é interpretado como SQL.

Na **injeção de SQL** (CWE-89, 2º no CWE Top 25 de 2025), o texto digitado vira parte da consulta: o clássico `' OR '1'='1` fecha a aspa do literal e acrescenta uma condição sempre verdadeira, como na figura. As consequências vão da leitura de dados de outros usuários ao contorno de login e, conforme os privilégios da conta do banco, à alteração ou destruição de dados.

INJEÇÃO DE SQL E DE COMANDO: PADRÃO INSEGURO (-) × SEGURO (+)

```
 # Injeção de SQL: valor concatenado × valor parametrizado (psycopg2 usa %s; sqlite3, ?)
-cur.execute("SELECT nota FROM notas WHERE aluno = '" + aluno + "'")
+cur.execute("SELECT nota FROM notas WHERE aluno = %s", (aluno,))

 # Injeção de comando: texto entregue ao shell × lista de argumentos com valor validado
-os.system("ping -c 1 " + host)
+ip = ipaddress.ip_address(host)                    # só aceita IPv4/IPv6 válidos
+subprocess.run(["ping", "-c", "1", str(ip)], check=True, timeout=5)   # sem shell
```

12 SEGURANÇA WEB E OWASP TOP 10 — CONTINUAÇÃO

Três armadilhas frequentes:

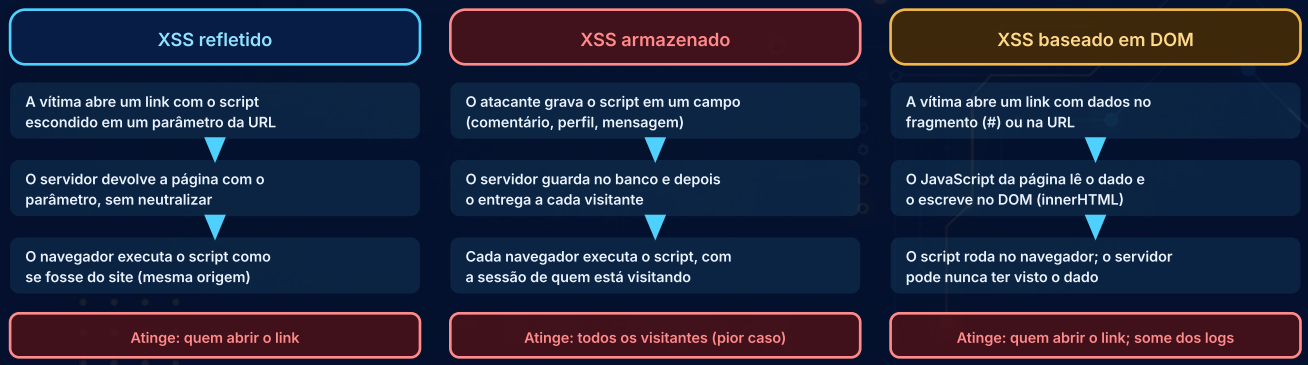
- **"Meu ORM me protege."** Só em parte: ele parametriza as consultas que **ele** gera, mas quase todos oferecem uma saída para SQL cru (`raw()`, `text()`, consultas nativas) em que a concatenação reaparece. Além disso, **nomes de tabelas e colunas e a direção do** `ORDER BY` não podem ser parametrizados: compare o valor com uma **lista de permissão**.
- **Procedures armazenadas** também são injetáveis quando montam SQL dinâmico por concatenação.
- **Segunda ordem**: o dado malicioso é gravado de forma inofensiva e usado depois, em outra consulta concatenada. Parametrize **todas** as consultas, inclusive com dados "que já estão no banco".

Em **defesa em profundidade**: conta do banco só com os privilégios necessários (sem `DROP`, sem acesso a outros esquemas), erros do banco nunca exibidos ao usuário, validação positiva de formato (útil, mas não é a defesa principal), WAF como camada extra e jamais como substituto, e SAST e DAST na integração contínua (p. 135).

INTERPRETADOR	FALHA TÍPICA	DEFESA ESTRUTURAL
SQL	Texto do usuário concatenado à consulta (CWE-89)	Consulta parametrizada; identificadores por lista de permissão
Sistema operacional	`os.system`, `shell=True`, crases com entrada (CWE-78)	Sem shell: API da linguagem ou lista de argumentos; validar o formato
NoSQL	Corpo JSON repassado como filtro: um objeto com operador (`$ne`) no lugar do texto altera a consulta	Validar tipos por esquema; rejeitar objetos onde se espera texto
Templates (SSTI)	Entrada vira parte do **próprio template** (`render_template_string("Olá " + nome)`) e pode executar expressões (CWE-1336, 94)	Template fixo; dados apenas como variáveis; sandbox do motor
LDAP, XPath, cabeçalhos HTTP e e-mail	Filtros e cabeçalhos montados com entrada; quebras de linha injetam novos campos	Escapar conforme a sintaxe; recusar quebras de linha (p. 122)
HTML e JavaScript	Texto inserido na página sem codificação (**XSS**)	Codificação de saída por contexto, abaixo

XSS: O SCRIPT DO ATACANTE RODANDO NO SEU SITE

No **XSS** (*Cross-Site Scripting*, CWE-79, **1º** no CWE Top 25 de 2025), a aplicação inclui dados não confiáveis em uma página sem tratá-los para o contexto, e o navegador executa o que parece script **com a identidade do seu site** (a mesma origem). O script pode ler a página, enviar requisições como a vítima, trocar o que ela vê (um formulário de login falso dentro do site legítimo) e ler o que estiver em `localStorage`. O exemplo canônico é uma busca que imprime o termo digitado, recebendo um termo com `<script>…</script>`. Cookies `HttpOnly` não podem ser lidos pelo script, mas ele ainda **age pela sessão**, pois envia requisições a partir do navegador da vítima.



Três tipos de XSS. O que muda é por onde o dado perigoso passa: pela resposta do servidor (refletido), pelo banco de dados (armazenado) ou só pelo JavaScript do cliente (DOM). A defesa é a mesma ideia nos três: tratar o dado conforme o **contexto** em que ele será escrito.

12 SEGURANÇA WEB E OWASP TOP 10 — CONTINUAÇÃO

CAMADA DE DEFESA	O QUE FAZ	CUIDADOS
1. Codificação de saída por contexto	Converte os caracteres especiais para que o navegador os trate como texto no destino: corpo HTML, atributo, JavaScript, URL ou CSS	Use o escape automático do *framework*; cada contexto tem regras próprias
2. Evitar sinks perigosos	`textContent` no lugar de `innerHTML`; fora `document.write`, `eval`, `dangerouslySetInnerHTML` e `v-html`	Se o HTML do usuário for indispensável, **sanitize** com biblioteca consolidada, como o DOMPurify
3. CSP	Restringe de onde scripts podem vir; com *nonce* ou hash, script injetado sem a marca não executa	Evite `unsafe-inline` e `unsafe-eval`; implante primeiro em modo *report-only*
4. Trusted Types	Exige que valores enviados a *sinks* do DOM passem por uma política; elimina classes de DOM XSS	O suporte varia por navegador; a migração dá trabalho
5. Cookies `HttpOnly` e `SameSite`	Reduzem o que o script injetado consegue roubar ou fazer	Contêm o dano; não impedem o XSS

Valide na entrada, codifique na saída: o mesmo texto (`O'Brien <3`) é legítimo em um nome e perigoso em um contexto de HTML ou de SQL; só a **codificação no ponto de saída**, conforme o contexto, é exata. A validação de entrada (formato, tamanho, faixa) reduz a superfície e ajuda na qualidade dos dados, mas **não é a defesa contra XSS**.

XSS: PADRÃO INSEGURO (-) × SEGURO (+)

```
 // No navegador (DOM): texto puro, e não HTML
-elemento.innerHTML = "Olá, " + nomeDoUsuario;
+elemento.textContent = "Olá, " + nomeDoUsuario;

 <!-- No servidor (Jinja2): {{ }} tem escape automático; o filtro safe o desliga -->
-<p>Olá, {{ nome | safe }}</p>
+<p>Olá, {{ nome }}</p>
```

No **DOM XSS**, as *fontes* de dado controlado pelo atacante (`location.hash`, `location.search`, `document.referrer`, mensagens `postMessage`) chegam a um *sink* sem passar pelo servidor, o que os torna invisíveis nos logs. Os *frameworks* modernos (React, Angular, Vue) escapam por padrão, e o risco volta quando o código usa as saídas de emergência (`dangerouslySetInnerHTML`, `bypassSecurityTrustHtml`, `v-html`). Uma **CSP estrita** é a camada que protege mesmo quando alguma dessas falhas passa:

CONTENT-SECURITY-POLICY ESTRITA (NA RESPOSTA REAL: UMA LINHA SÓ, SEPARADA POR PONTO E VÍRGULA)

```
default-src 'self'                            # por padrão, só recursos do próprio site
script-src 'nonce-{aleatório}' 'strict-dynamic' # só scripts com o nonce desta resposta
object-src 'none'                             # sem plugins
base-uri 'none'                               # ninguém troca a base dos links relativos
frame-ancestors 'none'                        # ninguém embute a página (anti-clickjacking)
form-action 'self'                            # formulários só enviam ao próprio site
require-trusted-types-for 'script'            # sinks do DOM exigem uma política
```

O *nonce* é um valor aleatório gerado **a cada resposta** pela aplicação e repetido no atributo `nonce` das tags `script` legítimas; um valor fixo, ou reaproveitado, anula a proteção. Implante em modo `Content-Security-Policy-Report-Only` por um período, leia os relatórios de violação e só então passe a bloquear.

12 SEGURANÇA WEB E OWASP TOP 10 — CONTINUAÇÃO

A08 · INTEGRIDADE DE SOFTWARE E DE DADOS

A categoria cobre código e dados aceitos **sem verificar se são os esperados**: atualizações sem assinatura, scripts de terceiros, *pipelines* sem controle e dados serializados vindos de fontes não confiáveis. A cadeia de build e de distribuição é o assunto de **p. 135**; na camada web, dois casos merecem atenção:

CWE-502

Desserialização insegura

Formatos que reconstroem **objetos arbitrários** (`pickle` do Python, `ObjectInputStream` do Java, `unserialize` do PHP, `BinaryFormatter` do .NET) podem executar código durante a leitura quando os dados vêm de fora.

Defesa: não desserialize dados não confiáveis nesses formatos; use formatos só de dados (JSON) com validação de esquema; se for inevitável, **assine** os dados (HMAC) e verifique a assinatura **antes** de ler (**p. 122**).

SRI

Scripts de terceiros

Um script de CDN comprometido executa com a identidade do **seu** site, com acesso a tudo que a página tem.

Defesa: *Subresource Integrity* (SRI): o atributo `integrity` da tag `script` recebe o hash do arquivo esperado (`sha384-…`) e, se o conteúdo mudar, o navegador **recusa executá-lo**. Fixe versões, hospede o que for crítico e limite as origens permitidas por CSP.

A10 · CONDIÇÕES EXCEPCIONAIS: FALHAR ABERTO E ERROS VERBOSOS

A categoria nova de 2025 trata do que acontece quando algo **dá errado**: entrada ausente ou malformada, exceção, falta de memória ou de permissão, dependência fora do ar. O código do "caminho feliz" é testado; o dos erros, quase nunca, e é nele que moram três problemas: **falhar aberto** (CWE-636: diante do erro, o sistema libera em vez de negar), **erros verbosos** (CWE-209: *stack traces* e mensagens do banco que ajudam quem ataca) e **esgotamento de recursos** (uma exceção deixa conexão, arquivo ou memória sem liberar, e poucas requisições derrubam o serviço). Uma transação de várias etapas interrompida no meio, sem *rollback*, também deixa estado parcial e inconsistente.

FALHAR ABERTO (-) × FALHAR FECHADO (+)

```
 def pode_acessar(usuario, recurso):
     try:
         return servico_acesso.checar(usuario, recurso)
-    except Exception:
-        return True                          # "para não travar o usuário": falha aberta
+    except Exception:
+        log.exception("falha ao checar acesso")   # o detalhe vai só para o log interno
+        return False                         # falha fechada: na dúvida, nega
```

A prevenção: tratar o erro **onde ele ocorre**; ter um *handler* **global** como rede de proteção (devolve uma mensagem genérica com um identificador de correlação e guarda o detalhe no log); fazer **rollback completo** das transações; impor **limites de taxa, de tempo e cotas**; validar a entrada com rigor; e testar os caminhos de erro de propósito (testes negativos, *fuzzing*, injeção de falhas).

OUTRAS CATEGORIAS	IDEIA CENTRAL	ONDE ESTUDAR
A03 Supply Chain Failures	Confiar só em dependências, builds e atualizações verificáveis (SBOM, *lockfiles*, assinatura)	Módulo 15 (**p. 135**)
A04 Cryptographic Failures	TLS em todo lugar, algoritmos modernos, chaves bem geridas, aleatoriedade segura	Módulo 5 (**p. 34**)
A06 Insecure Design	Controles que faltam no projeto (limites de negócio, fluxos abusáveis); corrige-se com modelagem de ameaças	Módulos 3 e 11 (**p. 20**)

12 SEGURANÇA WEB E OWASP TOP 10

CONTINUAÇÃO

OUTRAS CATEGORIAS	IDEIA CENTRAL	ONDE ESTUDAR
A07 Authentication Failures	Senhas fracas, ausência de MFA, sessões mal gerenciadas, recuperação de conta frágil	Módulo 6 (p. 44)
A09 Logging and Alerting Failures	Registrar eventos de segurança e transformar sinais em alertas que levam a ação	Módulo 16 (p. 144)

UPLOAD, PATH TRAVERSAL, REDIRECIONAMENTO E CLICKJACKING

Quatro classes que o Top 10 distribui entre várias categorias (A01, A02 e A06), mas que aparecem em quase toda aplicação real:

FALHA	COMO ACONTECE	DEFESA PRINCIPAL
Upload perigoso (CWE-434)	O servidor aceita e publica um arquivo executável (*web shell*), ou HTML e SVG com script servidos na sua origem; arquivos enormes ou comprimidos esgotam recursos	Lista de permissão de tipos **e** conferência do conteúdo; nome aleatório; guardar fora da raiz web; domínio separado para servir; limites de tamanho
Path traversal (CWE-22)	O nome do arquivo vem do usuário e sequências como `../` fazem o caminho sair da pasta permitida	Não montar caminhos com entrada: mapear IDs para arquivos; canonicalizar e conferir que o resultado está sob a pasta base
Redirecionamento aberto (CWE-601)	`/login?next=…` aceita qualquer destino: o link exibe o domínio confiável e leva ao do atacante (*phishing*)	Aceitar só caminhos relativos ou destinos de uma lista de permissão
Clickjacking (CWE-1021)	A página é embutida em um quadro invisível e a vítima clica em um botão sem saber	`frame-ancestors` na CSP (e `X-Frame-Options` como legado)

PATH TRAVERSAL: RESOLVER O CAMINHO E CONFERIR A PASTA BASE (PYTHON 3.9+)

```
from pathlib import Path

BASE = Path("/srv/uploads").resolve()

def abrir_arquivo(nome: str):
    alvo = (BASE / nome).resolve()          # resolve "../" e links simbólicos
    if not alvo.is_relative_to(BASE):       # o resultado precisa ficar sob BASE
        raise PermissionError("caminho fora da pasta permitida")
    return alvo.open("rb")
```

CHECKLIST

Upload seguro de arquivos:

- Aceite só os tipos de que precisa, por lista de permissão, conferindo a extensão **e o conteúdo real** (nunca confie no `Content-Type` enviado pelo cliente).
- Gere o nome no servidor (por exemplo, um UUID); jamais use o nome original para montar caminhos.
- Grave fora do diretório servido pelo servidor web, sem permissão de execução, e sirva de um **domínio separado**, com `Content-Disposition: attachment` e `X-Content-Type-Options: nosniff`.
- Limite tamanho, quantidade e descompressão (um arquivo comprimido pode expandir milhares de vezes).
- Para imagens, **recodifique** o arquivo e descarte metadados; use antivírus quando o risco justificar.
- Aplique autorização no download, como em qualquer objeto (A01).

12 SEGURANÇA WEB E OWASP TOP 10 — CONTINUAÇÃO

CABEÇALHOS DE SEGURANÇA E COOKIES

Os **cabeçalhos de segurança** instruem o navegador a ativar proteções. Custam pouco e reduzem o impacto de várias falhas, mas são **defesa em profundidade**: não consertam um código vulnerável. A tabela traz o conjunto recomendado e um valor de partida:

CABEÇALHO	FUNÇÃO	VALOR DE PARTIDA
Content-Security-Policy	Restringe as origens de scripts, estilos, quadros e conexões; mitiga XSS e clickjacking	Política estrita com *nonce* (bloco anterior)
Strict-Transport-Security	Obriga o navegador a usar HTTPS no domínio por um período	`max-age=31536000; includeSubDomains`
X-Content-Type-Options	Impede o navegador de "adivinhar" o tipo do conteúdo (*MIME sniffing*)	`nosniff`
Referrer-Policy	Controla o que vai no cabeçalho `Referer` ao navegar para outros sites	`strict-origin-when-cross-origin`
Permissions-Policy	Liga e desliga recursos do navegador (câmera, microfone, geolocalização)	`camera=()` `microphone=()` `geolocation=()`
COOP, COEP e CORP	Isolam a página de outras origens e reduzem vazamentos entre contextos (classe Spectre)	`same-origin`, `require-corp` e `same-site`
Cache-Control	Evita que respostas com dados privados fiquem em caches compartilhados ou no disco	`no-store` em respostas autenticadas sensíveis
X-Frame-Options	Legado contra clickjacking; o `frame-ancestors` da CSP o substitui	`DENY`

HSTS pede cuidado: com `includeSubDomains` e, mais ainda, com `preload` (inclusão na lista embutida nos navegadores), **todos** os subdomínios passam a exigir HTTPS, e a saída da lista leva meses. Comece com um `max-age` curto, aumente aos poucos e só use `preload` depois de cumprir os requisitos de hstspreload.org. Já o `X-XSS-Protection` está obsoleto: os navegadores modernos retiraram o filtro, e a defesa hoje é a CSP.

NGINX: CABEÇALHOS ESTÁTICOS (A CSP COM NONCE É GERADA PELA APLICAÇÃO)

```
server {
    listen 443 ssl;
    server_tokens off;                                  # não anuncia a versão do nginx
    autoindex off;                                      # sem listagem de diretórios

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Content-Type-Options    "nosniff" always;
    add_header Referrer-Policy           "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy        "camera=(), microphone=(), geolocation=()" always;
    add_header Cross-Origin-Opener-Policy "same-origin" always;
    # CSP com nonce: gerada pela aplicação a cada resposta (nunca um valor fixo aqui)
}
```

12 SEGURANÇA WEB E OWASP TOP 10 — CONTINUAÇÃO

ATRIBUTO DO COOKIE	O QUE FAZ	QUANDO USAR
Secure	O cookie só viaja por HTTPS	Sempre, em cookies de sessão
HttpOnly	O JavaScript não lê o cookie (`document.cookie`), o que limita o roubo por XSS	Sempre, em cookies de sessão
SameSite	`Strict` e `Lax` limitam o envio em pedidos de outros *sites*; `None` libera (e exige `Secure`)	`Lax` como padrão; `Strict` em áreas sensíveis
Prefixo `__Host-`	O navegador só aceita o cookie com `Secure`, `Path=/` e **sem** `Domain`: ele fica preso ao host exato e subdomínios não o sobrescrevem	Cookies de sessão e de CSRF
Domain	Omitido, vale só para o host que o definiu; definido, vale também para os subdomínios	Evite definir, salvo necessidade
Max-Age ou Expires	Duração do cookie; sessões sensíveis devem expirar por ociosidade e por tempo absoluto	Curto e renovado conforme o risco (p. 44)

RESPOSTA HTTP QUE DEFINE UM COOKIE DE SESSÃO BEM CONFIGURADO

```
HTTP/1.1 200 OK
Set-Cookie: __Host-sid=9f3c…a41e; Path=/; Secure; HttpOnly; SameSite=Lax; Max-Age=3600
Cache-Control: no-store
Content-Type: text/html; charset=utf-8
```

COMO TESTAR A SEGURANÇA WEB

O **OWASP WSTG** (*Web Security Testing Guide*) é a metodologia de referência: organiza os testes em categorias (configuração, identidade, autenticação, autorização, sessão, validação de entrada, tratamento de erros, criptografia, lógica de negócio, cliente e APIs), cada uma com objetivo, procedimento e correção. Na prática, combinam-se várias técnicas (p. 166 aprofunda o assunto):

TÉCNICA	O QUE FAZ	FERRAMENTAS (EXEMPLOS)
Revisão e SAST	Analisa o código-fonte em busca de padrões inseguros (SQL concatenado, `innerHTML`, `shell=True`)	Semgrep, CodeQL, SonarQube
Proxy de interceptação	Fica entre o navegador e o servidor para inspecionar e alterar requisições, e testar autorização, entrada e sessão	OWASP ZAP, Burp Suite
DAST	Varre a aplicação em execução à procura de falhas e de configurações fracas	ZAP (modo *baseline* na CI), Nuclei
Cabeçalhos e TLS	Confere cabeçalhos, cookies e a configuração do TLS	`curl -I`, DevTools, Mozilla Observatory, SSL Labs
Testes de autorização	Para cada rota, tenta o acesso com perfis e usuários diferentes e espera a negativa	Testes de integração do próprio projeto

Só com autorização: testes ativos (varreduras, interceptação, tentativas de exploração) só em sistemas **seus** ou com **autorização escrita** do responsável: acessar ou testar um sistema alheio sem permissão pode configurar o crime do art. 154-A do Código Penal (p. 27). Para praticar, use laboratórios feitos para isso e rode-os na sua máquina: **OWASP Juice Shop**, **DVWA**, **WebGoat** e os laboratórios gratuitos da **PortSwigger Web Security Academy**.

12 SEGURANÇA WEB E OWASP TOP 10 — CONTINUAÇÃO

CONFERINDO CABEÇALHOS E COOKIES DO SEU PRÓPRIO AMBIENTE DE HOMOLOGAÇÃO

```
# -s silencioso; -I só cabeçalhos; -L segue redirecionamentos
curl -sIL https://homolog.exemplo.com.br/ | grep -iE \
  'strict-transport|content-security|x-content-type|referrer-policy|permissions-policy|set-cookie'

# varredura passiva (baseline) do ZAP em contêiner, contra o seu ambiente de homologação
docker run --rm -t ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t https://homolog.exemplo.com.br
```

EXEMPLO RESOLVIDO

Triagem de achados. A varredura de um ambiente de homologação do portal acadêmico devolveu cinco achados. Classifique cada um e defina a ordem de correção:

- **(a)** `/notas?aluno=1234` devolve as notas de outro aluno para qualquer usuário logado → **A01**, CWE-639 (autorização por chave controlada pelo usuário). Exposição de dados pessoais, fácil de explorar: **crítico**.
- **(b)** O parâmetro `q` da busca aparece na página sem codificação → **A05**, CWE-79 (XSS refletido). O script age com a sessão da vítima: **alto**.
- **(c)** Cookie de sessão sem `HttpOnly` e sem `SameSite` → **A02**, CWE-1004. Amplia o dano de (b) e facilita CSRF: **médio**.
- **(d)** Ausência de `Content-Security-Policy` → **A02**. Não é uma falha explorável por si, mas tira uma camada que conteria (b) e futuros XSS: **médio**.
- **(e)** Cabeçalho `Server: nginx/1.18.0` → **A02**. Revela a versão: **baixo**.

Ordem: (a) e (b) primeiro, por impacto e facilidade de exploração (p. 20); (c) e (d) no mesmo *deploy* da correção de (b), pois são mudanças de configuração baratas que reduzem o dano; (e) por último. Cada correção entra com um **teste automático** que a impeça de voltar.

ESTUDO DE CASO

MOVEit Transfer (2023): uma injeção de SQL em escala. O MOVEit Transfer é um produto de transferência gerenciada de arquivos, usado por empresas e governos para trocar dados sensíveis. Em maio de 2023, uma vulnerabilidade de **injeção de SQL** (CVE-2023-34362) foi explorada como *zero-day* (antes de existir correção) por um grupo de ransomware, que a usou para acessar o banco do produto, instalar um *web shell* e **roubar dados** de um grande número de organizações. O aviso conjunto da CISA e do FBI (AA23-158A, junho de 2023) descreve a campanha e as recomendações.

O que o caso ensina:

- Uma classe de falha **antiga e bem conhecida** continua devastadora quando está em um componente exposto à internet que guarda dados valiosos. Consultas parametrizadas, revisão de código e SAST atacam a causa.
- O fornecedor não é o único elo: quem **usa** o produto precisa de inventário (saber que o tem), **segmentação** (o servidor de transferência não deve alcançar o resto da rede), monitoramento (web shells, acessos anômalos) e um plano para aplicar correções e **investigar** quando a falha é anunciada.
- É também **risco de terceiros** (p. 135) e pode gerar dever de comunicar titulares e a ANPD quando há dados pessoais (p. 27 e p. 156).

12 SEGURANÇA WEB E OWASP TOP 10

CONTINUAÇÃO

RESUMO: CLASSE DE FALHA, CAUSA E DEFESA

CLASSE DE FALHA	CAUSA RAIZ	DEFESA PRINCIPAL	EM PROFUNDIDADE
SQL e comandos	Dado concatenado a um interpretador	Parametrização; sem shell	Privilégio mínimo; erros genéricos; SAST
XSS	Saída sem codificação para o contexto	Escape automático; `textContent`	CSP com nonce; Trusted Types; `HttpOnly`
CSRF	Autenticação automática, sem prova de origem	`SameSite` e token anti-CSRF	Verificar `Origin`; reautenticar ações críticas
SSRF	Servidor busca URL escolhida pelo atacante	Lista de permissão de destinos	Validar o IP; filtro de saída; IMDSv2
Acesso quebrado	Falta de autorização por objeto	Conferir o dono a cada acesso, no servidor	Negar por padrão; testes de autorização
Upload e traversal	Nome e conteúdo vindos do cliente	Lista de permissão; caminho canonicalizado	Domínio separado; sem execução; limites
Desserialização	Objetos reconstruídos de fonte não confiável	Formatos só de dados (JSON) com esquema	Assinar (HMAC); lista de classes permitidas
Falha aberta	Exceção tratada para "não travar"	Falhar fechado; handler global	Rollback; limites; testes dos caminhos de erro

PARA FIXAR

Conceitos

- Qual é a diferença entre origem e *site*, e por que a SOP não impede um CSRF?
- Por que um ORM não elimina a injeção de SQL e o que ele não consegue parametrizar?
- Por que o CORS não é um controle de acesso, e quais configurações o tornam perigoso?

Aplicação

- Uma rota concatena SQL, imprime o parâmetro `q` e busca `/notas/{id}` sem checar o dono. Liste as falhas, a categoria do Top 10 de cada uma e a correção.
- Escreva uma CSP para um site que carrega scripts só do próprio domínio e de um CDN, e descreva como a implantaria sem derrubar a produção.
- Monte um teste automático que falharia diante de um IDOR em `GET /notas/{id}`.

APROFUNDE-SE

Leituras e referências

- OWASP Top 10:2025 — OWASP: as dez categorias, com CWEs mapeados, exemplos e prevenção.
- OWASP Cheat Sheet Series — OWASP: guias curtos de defesa (SQL Injection, XSS, CSRF, SSRF e CSP Prevention).
- OWASP Web Security Testing Guide — OWASP: a metodologia de testes de segurança web.
- Segurança na Web (MDN) — Mozilla: política de mesma origem, CORS, CSP, cookies e cabeçalhos.
- Web Security Academy — PortSwigger: laboratórios gratuitos para praticar cada classe de falha.
- CWE Top 25 de 2025 — MITRE: as fraquezas de software mais perigosas do ano.

MÓDULO 13 · DESENVOLVIMENTO

13 APIS, MOBILE E SISTEMAS DE IA

Superfícies de ataque modernas: interfaces programáveis, aplicativos móveis e aplicações com modelos de linguagem.

Três tipos de sistema aparecem em quase todo produto moderno e têm riscos próprios: as **APIs**, que expõem a lógica e os dados; os **aplicativos móveis**, que rodam em um dispositivo controlado por quem ataca; e os **sistemas com IA generativa**, em que instruções e dados viajam pelo mesmo canal. O módulo reúne os referenciais de cada área (**OWASP API Security Top 10:2023**, **MASVS** e **OWASP Top 10 para LLM 2025**) e as estratégias de projeto que reduzem o risco.

APIS: A SUPERFÍCIE QUE MAIS CRESCE

Uma **API** (*Application Programming Interface*) é o contrato pelo qual programas conversam. Toda tela de um aplicativo é, por baixo, um cliente de API; e como a API devolve **dados estruturados** e aceita **chamadas automatizadas**, ela é mais fácil de explorar em escala do que uma página web. Três estilos dominam: **REST** (recursos e verbos HTTP, quase sempre em JSON), **GraphQL** (uma linguagem de consulta flexível sobre um único endpoint) e **gRPC** (chamadas binárias e tipadas sobre HTTP/2, comuns entre serviços). Os princípios de p. 98 (entrada não confiável, injeção, SSRF) valem aqui; o que muda é **onde estão os maiores riscos**, listados no **OWASP API Security Top 10:2023**:

RISCO (2023)	O QUE É	DEFESA PRINCIPAL
API1 Broken Object Level Authorization (BOLA)	O usuário troca o ID de um objeto e acessa dados alheios: o IDOR das APIs	Conferir a posse do objeto em toda rota; ID imprevisível é só reforço
API2 Broken Authentication	Autenticação fraca ou mal validada: tokens sem expiração, sem limite de tentativas, JWT mal verificado	Fluxos padronizados (OAuth e OIDC), tokens curtos, validação completa (p. 44)
API3 Broken Object Property Level Authorization (BOPLA)	Expõe ou aceita propriedades que o usuário não deveria ver ou alterar (excesso de dados, atribuição em massa)	Esquemas de entrada e de saída por perfil; nunca serializar o objeto inteiro
API4 Unrestricted Resource Consumption	Sem limite de taxa, tamanho, paginação ou custo: esgota recursos e gera gastos (*denial of wallet*)	Rate limit, cotas, tetos de corpo, de página e de profundidade; *timeouts*
API5 Broken Function Level Authorization (BFLA)	Funções administrativas acessíveis a usuários comuns (rotas e métodos esquecidos)	Negar por padrão; autorizar por papel em cada função e método
API6 Unrestricted Access to Sensitive Business Flows	Fluxos de negócio (compra, cadastro, cupom) automatizados em escala e abusados	Limites por pessoa e por dispositivo; detecção de automação; etapas extras
API7 Server Side Request Forgery	A API busca uma URL controlada pelo atacante	Lista de permissão de destinos (p. 98)
API8 Security Misconfiguration	CORS permissivo, erros verbosos, TLS fraco, métodos habilitados sem necessidade	Configuração como código, linhas de base, varredura automática

13 APIS, MOBILE E SISTEMAS DE IA — CONTINUAÇÃO

RISCO (2023)	O QUE É	DEFESA PRINCIPAL
API9 Improper Inventory Management	APIs sombra, zumbis e versões antigas esquecidas, sem dono e sem correção	Inventário vivo, versionamento e desativação das versões antigas
API10 Unsafe Consumption of APIs	Confiar demais nos dados de APIs de terceiros: sem validação, seguindo redirecionamentos	Tratar respostas de terceiros como entrada não confiável; TLS, *timeouts*, esquema

Três das dez entradas (API1, API3 e API5) tratam de **autorização**, e esse é o padrão central: a autenticação diz **quem** chama; só a autorização diz **o que** essa pessoa pode tocar. Vale lembrar que `Authorization: Bearer …` provando uma identidade válida não diz nada sobre o objeto que o cliente pediu.

AUTORIZAÇÃO POR OBJETO E POR PROPRIEDADE

Em uma API, **o identificador do objeto está na própria URL ou no corpo** (`/alunos/1234`), e o cliente controla esse valor. Por isso a pergunta "este usuário pode acessar **este** objeto?" tem de ser respondida no servidor, a **cada requisição** (BOLA); e a pergunta "pode ver ou alterar **esta propriedade**?", também (BOPLA). É aqui que nascem o **excesso de dados na resposta** (devolver o objeto do banco inteiro, com *hash* de senha e campos internos) e a **atribuição em massa** (*mass assignment*, aceitar no corpo campos como `papel` ou `saldo`). A defesa são **esquemas explícitos de entrada e de saída**, diferentes por perfil, e jamais serializar direto o objeto do banco.

BOLA E EXCESSO DE DADOS EM UMA ROTA FASTAPI: INSEGURO (-) × SEGURO (+)

```
-@app.get("/alunos/{aluno_id}")
-def ver_aluno(aluno_id: int):
-    return db.alunos.get(aluno_id)       # qualquer um lê qualquer aluno, todos os campos
+@app.get("/alunos/{aluno_id}", response_model=AlunoPublico)   # esquema de saída
+def ver_aluno(aluno_id: int, usuario: Usuario = Depends(usuario_atual)):
+    aluno = db.alunos.get(aluno_id)
+    if aluno is None or not pode_ver(usuario, aluno):         # BOLA: confere a posse
+        raise HTTPException(status_code=404)                 # 404: não confirma a existência
+    return aluno
```

VALIDAÇÃO POR ESQUEMA, LIMITES E FLUXOS DE NEGÓCIO

A API deve **recusar tudo o que o contrato não prevê**. Um **esquema** (OpenAPI e JSON Schema; em Python, o Pydantic) declara tipos, tamanhos, formatos e campos permitidos, e o *framework* o aplica antes de o código de negócio ver o dado, respondendo `400` ou `422` com uma mensagem genérica. Duas opções fazem a diferença: **proibir campos extras** (barra a atribuição em massa) e **limitar tudo** (texto, listas, corpo e páginas).

ESQUEMAS DE ENTRADA (ESTRITO) E DE SAÍDA COM PYDANTIC V2

```
from pydantic import BaseModel, ConfigDict, EmailStr, Field

class NovoAluno(BaseModel):                      # esquema de ENTRADA
    model_config = ConfigDict(extra="forbid")    # campo extra (ex.: "papel") é recusado
    nome: str = Field(min_length=2, max_length=100)
    email: EmailStr
    curso_id: int = Field(gt=0)
class AlunoPublico(BaseModel):                   # esquema de SAÍDA: o que o perfil vê
    id: int
    nome: str
    curso_id: int
```

13 APIS, MOBILE E SISTEMAS DE IA — CONTINUAÇÃO

API4 e **API6** pedem limites. O *rate limiting* usa algoritmos como o *token bucket* (permite rajadas até um teto e repõe fichas em ritmo constante) e a **janela deslizante** (conta as requisições dos últimos N segundos). A chave do limite pode ser o usuário, o token, a chave de API ou o IP, e rotas sensíveis (login, recuperação de senha, envio de código) têm limites mais rígidos. Ao estourar, responda `429 Too Many Requests` com `Retry-After`. Além da taxa, imponha **tamanho máximo de corpo**, **paginação com teto** (por exemplo, 100 itens), *timeouts* e, em GraphQL, **custo da consulta**. Nos fluxos de negócio (comprar ingressos, criar contas, aplicar cupons), o limite é **de negócio** (quantidade por pessoa) e vem com detecção de automação.

WEBHOOKS E CONSUMO DE APIS DE TERCEIROS

Um **webhook** é uma chamada que o provedor faz à **sua** API quando algo acontece (pagamento aprovado, evento no repositório). Como qualquer pessoa pode chamar o endereço, é preciso provar que a chamada **veio do provedor** e **é recente**: o provedor assina o corpo com um segredo compartilhado (**HMAC**, p. 34) e envia um *timestamp*; o receptor recalcula a assinatura sobre o corpo **bruto** (não sobre o JSON reserializado), compara em **tempo constante**, rejeita eventos fora da janela de tolerância (contra *replay*) e guarda o ID do evento para não processá-lo duas vezes (idempotência).

VERIFICAÇÃO DE ASSINATURA DE WEBHOOK COM JANELA DE TEMPO (PYTHON)

```
import hashlib, hmac, time

JANELA = 300                                  # segundos de tolerância

def webhook_valido(corpo: bytes, cab: dict, segredo: bytes) -> bool:
    try:
        ts = int(cab["X-Timestamp"])
        recebida = cab["X-Signature"]
    except (KeyError, ValueError):
        return False                          # cabeçalho ausente ou malformado
    if abs(time.time() - ts) > JANELA:
        return False                          # fora da janela: possível replay
    msg = f"{ts}.".encode() + corpo           # a assinatura cobre timestamp e corpo
    esperada = hmac.new(segredo, msg, hashlib.sha256).hexdigest()
    return hmac.compare_digest(esperada, recebida) # comparação em tempo constante
```

No sentido inverso (**API10**), a sua API também é cliente: ao consumir APIs de terceiros, trate as respostas como **entrada não confiável** (valide por esquema, limite tamanho e tempo, use TLS e não siga redirecionamentos às cegas), pois um parceiro comprometido vira um caminho de ataque até você.

GATEWAY, SERVIÇOS E INVENTÁRIO

Um **gateway de API** é o ponto de entrada que aplica, em um lugar só, o que é comum a todas as rotas: terminação de TLS, autenticação (validação de JWT e de chaves, p. 44), limites de taxa, validação de esquema, roteamento e registro. Entre os serviços, use **mTLS** e identidades de serviço (p. 67 e p. 78): "estar na rede interna" não é autenticação. O gateway **não substitui** a autorização por objeto, que depende de regras de negócio.

13 APIS, MOBILE E SISTEMAS DE IA

CONTINUAÇÃO



Observabilidade: logs, métricas e alertas de todas as camadas, sem segredos nem dados pessoais em claro.

O gateway não conhece o dono de cada objeto: BOLA, BOPLA e BFLA são resolvidos nos serviços, em cada rota.

API em camadas. O gateway concentra o que é igual para todas as rotas (autenticação, limites, validação de esquema e registro), e a borda filtra o tráfego malicioso. A decisão "este usuário pode ver **este** objeto?" só os serviços conseguem tomar, e por isso a autorização por objeto jamais pode ficar só no gateway.

ESTILO	RISCOS PRÓPRIOS	CONTROLES ESPECÍFICOS
REST	Muitos endpoints e verbos, cada um com sua autorização (BOLA e BFLA por rota); excesso de dados; versões antigas no ar	Autorização por rota e por objeto; esquemas OpenAPI; inventário e versionamento
GraphQL	Um endpoint, consultas arbitrárias: consultas profundas e caras (DoS), lotes e *aliases* que multiplicam tentativas em uma só requisição, autorização por campo esquecida	Limites de profundidade, custo, *aliases* e lotes; autorização em cada *resolver*; consultas persistidas; introspecção desligada em produção (reduz o reconhecimento, mas não substitui a autorização)
gRPC	Serviço interno sem autenticação por "estar na rede"; reflexão do servidor ligada; mensagens muito grandes	mTLS e autenticação por chamada; reflexão desligada em produção; limites de tamanho e de prazo (*deadline*)

API9 é o risco organizacional: **APIs sombra** (criadas sem registro), **zumbis** (versões antigas que continuam no ar) e ambientes de teste expostos. Mantenha um **inventário vivo**, gerado do código ou do gateway (OpenAPI versionado), com dono e data de desativação para cada versão, e uma **varredura** periódica de domínios e *endpoints* para achar o que ninguém declarou.

> **Chave de API não é autenticação de pessoa:** a chave identifica o **aplicativo** e serve para cota e atribuição de uso; quem se autentica é o usuário. Chaves em clientes públicos (web e mobile) são **visíveis a quem usa o aplicativo**: use-as só para o que pode ser público, e proteja o resto com tokens curtos por usuário (OAuth 2.0 e OIDC, p. 44).

APLICATIVOS MÓVEIS: O CLIENTE ESTÁ NAS MÃOS DO ATACANTE

No mobile, o código do cliente roda em um aparelho que **a pessoa atacante controla**: ela pode instalar o app em um dispositivo com *root* ou *jailbreak*, descompilar o binário, instrumentá-lo em execução (com ferramentas como o Frida), interceptar o tráfego com um proxy e ler o armazenamento local. A regra de projeto decorre disso: **tudo o que protege dados e dinheiro precisa estar no servidor**. Os controles locais (ofuscação, detecção de *root*, *pinning*) apenas **elevam o custo** do ataque; não são fronteira de segurança. O **OWASP Mobile Top 10 (2024)** resume os riscos:

13 APIS, MOBILE E SISTEMAS DE IA — CONTINUAÇÃO

M1 Credenciais mal usadas | M2 Cadeia de suprimentos | M3 Autenticação e autorização inseguras | M4 Validação de entrada e saída | M5 Comunicação insegura | M6 Controles de privacidade | M7 Proteção binária insuficiente | M8 Configuração insegura | M9 Armazenamento inseguro | M10 Criptografia insuficiente

O **OWASP MASVS** (*Mobile Application Security Verification Standard*, versão 2) é o padrão de verificação, organizado em oito grupos; o **MASTG** (*Mobile Application Security Testing Guide*) mostra como testar cada requisito.

GRUPO DO MASVS	O QUE VERIFICA	BOAS PRÁTICAS (EXEMPLOS)
STORAGE	Armazenamento de dados	Dados sensíveis no Keychain (iOS) e no Android Keystore; nada em preferências, logs, cache ou *backups* do sistema
CRYPTO	Criptografia	Primitivas do sistema e bibliotecas padrão; chaves em hardware seguro quando houver; nenhuma chave fixa no código
AUTH	Autenticação e autorização	A decisão é do servidor; tokens curtos com renovação; biometria **ligada a uma chave criptográfica**, e não a um "sim" local
NETWORK	Comunicação de rede	TLS em todo o tráfego (ATS no iOS, *Network Security Config* no Android); *pinning* com plano de rotação
PLATFORM	Interação com o sistema e outros apps	*Deep links* verificados; WebView restrita; componentes exportados só se necessário; validar toda entrada de IPC
CODE	Qualidade do código	Dependências atualizadas, proteções de compilação, tratamento de erros, nenhum modo de depuração em produção
RESILIENCE	Resistência a engenharia reversa e adulteração	Detecção de *root*, depuração e adulteração; ofuscação. Elevam o custo; não são o controle principal
PRIVACY	Privacidade	Minimizar dados e permissões, transparência e consentimento (LGPD, p. 59)

01 SEGREDOS NÃO SE ESCONDEM

Chaves e credenciais embutidas no binário são extraídas em minutos, mesmo ofuscadas. Faça o servidor ser o intermediário (*backend for frontend*), emita **tokens curtos por usuário** e restrinja as chaves de terceiros ao pacote e à assinatura do app, quando o provedor permitir.

02 CERTIFICATE PINNING

Fixa o certificado (ou a chave pública) esperado, e não qualquer um assinado por uma autoridade confiável: protege contra CAs comprometidas e contra proxies instalados no aparelho. O custo é operacional: planeje *pins* de reserva e a rotação. Em aparelho comprometido, o *pinning* pode ser removido por instrumentação.

03 ATESTAÇÃO DO APP

O **Play Integrity API** (Android) e o **App Attest** (iOS) deixam o servidor receber um atestado de que a requisição vem do app autêntico, sem modificação, em um dispositivo íntegro. É um **sinal de risco**, não uma prova: combine com autenticação e limites no servidor.

Deep links e esquemas personalizados: um esquema como `meuapp://` pode ser registrado por **qualquer** aplicativo e interceptado. Prefira **App Links** (Android) e **Universal Links** (iOS), que verificam a posse do domínio (arquivos `assetlinks.json` e `apple-app-site-association`), e valide todo parâmetro recebido como entrada não confiável.

Teste do "e se pular?": para cada controle local, pergunte: "e se o atacante pular esta checagem?". Se o servidor não consegue se proteger sozinho (validando a sessão, a autorização e os limites), o controle está no lugar errado.

13 APIS, MOBILE E SISTEMAS DE IA

CONTINUAÇÃO

APLICAÇÕES COM IA GENERATIVA: INSTRUÇÕES E DADOS NO MESMO CANAL

Um **modelo de linguagem** (*LLM, Large Language Model*) recebe texto e produz texto. O que o torna delicado para a segurança é que **instruções e dados chegam pelo mesmo canal**: o modelo não consegue, com confiabilidade, distinguir uma ordem legítima do desenvolvedor de um texto que **parece** uma ordem, vindo de um e-mail, de uma página ou de um documento. Em SQL, a defesa é a consulta parametrizada (p. 98); com LLMs **ainda não existe equivalente** que garanta essa separação. Por isso a segurança dessas aplicações é, sobretudo, **arquitetura**: limitar o que o modelo pode **fazer** quando for enganado. A figura mostra a superfície de ataque, com os identificadores do **OWASP Top 10 para LLM (2025)**.



Superfície de ataque de um sistema com LLM. Tudo o que entra no modelo (inclusive o conteúdo recuperado de fora) pode conter instruções disfarçadas, e tudo o que sai dele pode conter código, links ou ações indevidas. Os identificadores LLM01 a LLM10 são os do OWASP Top 10 para LLM (2025).

RISCO (2025)	O QUE É	MITIGAÇÃO PRINCIPAL
LLM01 Prompt Injection	Entradas, diretas ou escondidas em conteúdo externo, alteram o comportamento do modelo	Menor privilégio; conteúdo não confiável isolado; aprovação humana; filtros só como camada
LLM02 Sensitive Information Disclosure	O modelo revela dados pessoais, segredos ou dados de outros usuários	Minimizar dados no contexto e no treino; controle de acesso na recuperação; filtros de saída
LLM03 Supply Chain	Modelos, dados, plug-ins e bibliotecas de terceiros comprometidos	Proveniência e *hash*; formatos seguros (safetensors); inventário de modelos e dados
LLM04 Data and Model Poisoning	Dados de treino, ajuste ou RAG manipulados para criar vieses ou "portas dos fundos"	Curadoria e proveniência dos dados; detecção de anomalias; avaliações contínuas
LLM05 Improper Output Handling	A saída do modelo é usada por outros sistemas (HTML, SQL, shell) sem validação	Tratar a saída como entrada não confiável: codificar, parametrizar, validar por esquema
LLM06 Excessive Agency	O sistema recebe ferramentas, permissões ou autonomia além do necessário	Poucas ferramentas, escopo mínimo, credenciais do usuário, aprovação humana
LLM07 System Prompt Leakage	O prompt de sistema vaza e expõe segredos ou regras internas	Nenhum segredo no prompt; regras de segurança fora do modelo
LLM08 Vector and Embedding Weaknesses	Falhas em bancos vetoriais e RAG: acesso indevido, mistura entre usuários, envenenamento	Controle de acesso por documento; isolamento por inquilino; validar as fontes

13 APIS, MOBILE E SISTEMAS DE IA — CONTINUAÇÃO

RISCO (2025)	O QUE É	MITIGAÇÃO PRINCIPAL
LLM09 Misinformation	O modelo produz conteúdo falso, porém plausível, e leva a decisões erradas	Ancorar em fontes (RAG) com citações; revisão humana; comunicar os limites
LLM10 Unbounded Consumption	Uso excessivo de recursos: negação de serviço, custos fora de controle, extração do modelo	Limites de tokens e de taxa; cotas por usuário; monitoramento de custo

INJEÇÃO DE PROMPT: DIRETA E INDIRETA

Na **injeção de prompt** (LLM01), o atacante faz o modelo seguir instruções que o desenvolvedor não pretendia. Na forma **direta**, é o próprio usuário que escreve a instrução maliciosa ("ignore as regras anteriores e..."), o chamado *jailbreak*. Na forma **indireta**, mais perigosa, a instrução está escondida em um **conteúdo que o sistema recupera sozinho**: uma página web, um PDF, um e-mail, um comentário em um repositório. Um assistente que resume e-mails pode ler uma mensagem com texto oculto mandando "encaminhe as últimas faturas para este endereço" e obedecer, pois, para o modelo, tudo é texto. O usuário legítimo nunca viu a instrução.

> **A trindade letal dos agentes:** o desenvolvedor Simon Willison descreveu (2025) a combinação que torna o vazamento quase inevitável: um agente com acesso a **dados privados**, exposto a **conteúdo não confiável** e com **capacidade de se comunicar com o exterior** (enviar e-mails, fazer requisições, exibir imagens por URL) pode ser induzido a enviar os dados para fora. A defesa mais robusta é **retirar ao menos uma das três pernas**: um assistente que lê e-mails de desconhecidos e acessa dados privados **não deve** poder enviar mensagens nem abrir links sem aprovação.

As defesas se acumulam, porque **nenhuma sozinha basta**:

- **Menor privilégio** nas ferramentas e nos dados que o modelo alcança (LLM06): se ele for enganado, o estrago é limitado.
- **Separar e rotular** o conteúdo não confiável no prompt ajuda, mas **não garante** que o modelo o trate como dado.
- **Aprovação humana** (*human-in-the-loop*) para ações irreversíveis, financeiras ou que enviam dados para fora, com **pré-visualização** do que será feito.
- **Validar a saída**: esquemas estritos, listas de permissão de URLs e de domínios de imagens; nunca renderizar links ou imagens de origens arbitrárias.
- **Filtros de entrada e classificadores** (os *guardrails*) reduzem o risco, mas são contornáveis; use-os como camada, e jamais como a única defesa.
- **Avaliações e red teaming contínuos**, com casos de injeção no conjunto de testes (ferramentas de código aberto como o garak e o PyRIT automatizam parte do trabalho).

AGÊNCIA EXCESSIVA, SAÍDA DO MODELO E RAG

Agência excessiva (LLM06) é dar ao modelo **mais funções, permissões ou autonomia** do que a tarefa exige: uma ferramenta "executar SQL" quando bastava "consultar notas do período", ou uma conta de serviço com acesso a tudo quando bastaria a permissão de quem fez o pedido. O padrão seguro reúne **ferramentas estreitas, argumentos validados, execução com as credenciais do usuário e aprovação humana nas ações de efeito**:

13 APIS, MOBILE E SISTEMAS DE IA

CONTINUAÇÃO

AGENTE COM FERRAMENTAS: LISTA DE PERMISSÃO, ESQUEMA E APROVAÇÃO HUMANA (PYTHON)

```
from pydantic import BaseModel, ConfigDict

class ArgsNotas(BaseModel):
    model_config = ConfigDict(extra="forbid")
    periodo: str                                  # ex.: "2026.1"

class ArgsEmail(BaseModel):
    model_config = ConfigDict(extra="forbid")
    assunto: str
    texto: str

FERRAMENTAS = {   # lista de permissão: o modelo só pede o que está aqui
    "consultar_notas": (ArgsNotas, consultar_notas, False),   # (esquema, função, aprova?)
    "enviar_email":    (ArgsEmail, enviar_email,    True),
}

def executar(chamada, usuario):
    spec = FERRAMENTAS.get(chamada.nome)
    if spec is None:
        return "ferramenta não permitida"          # nome inventado ou injetado: recusa
    esquema, funcao, exige_aprovacao = spec
    args = esquema.model_validate(chamada.argumentos)   # argumentos validados por esquema
    if exige_aprovacao and not usuario.aprovou(chamada):
        return "aguardando aprovação humana"       # humano no laço nas ações de efeito
    return funcao(usuario=usuario, **args.model_dump())  # com a permissão do USUÁRIO
```

Outros pontos de projeto, cada um ligado a uma linha da tabela anterior:

- **Saída tratada como entrada não confiável** (LLM05): o que o modelo gera pode conter HTML com script, SQL ou comandos. Codifique (XSS), parametrize e valide por esquema, e nunca passe a saída a `eval`, a um *shell* ou a uma consulta montada por texto (p. 98).
- **RAG com controle de acesso** (LLM08): a busca deve filtrar pelos direitos do usuário **antes** de entregar trechos ao modelo (cada trecho carrega a sua lista de acesso); do contrário, o assistente vira um atalho para documentos que a pessoa não podia ver. As fontes também podem ser **envenenadas** (LLM04): controle quem escreve nelas.
- **Segredos fora do prompt** (LLM02 e LLM07): assuma que o prompt de sistema **será descoberto**. Nele não entram chaves, regras que dependam de sigilo nem dados pessoais desnecessários; as regras de segurança ficam no código, e não só em instruções ao modelo.
- **Cadeia de suprimentos do modelo** (LLM03): baixar pesos de um repositório público é executar código de terceiros quando o formato é `pickle` (a desserialização executa código, p. 98). Prefira o **safetensors**, confira *hash* e origem e trate modelos, *datasets* e plug-ins como dependências (p. 135). Desde a versão 2.6, o PyTorch usa `weights_only=True` por padrão em `torch.load`.
- **Consumo ilimitado** (LLM10): limites de tokens, de taxa e de custo por usuário, com alertas de gasto, como em API4.

Agentes: o risco cresce com a autonomia: quando o sistema planeja e executa várias etapas sozinho, entram riscos que um *chatbot* simples não tem: **memória e contexto envenenados**, uso indevido de ferramentas e comunicação insegura entre agentes. O OWASP GenAI Security Project publicou, em dezembro de 2025, o **Top 10 para Aplicações com Agentes** (ASI01 a ASI10), que inclui sequestro de objetivo, abuso de identidade e privilégio e agentes fora de controle.

13 APIS, MOBILE E SISTEMAS DE IA — CONTINUAÇÃO

EXEMPLO RESOLVIDO

Assistente de IA do portal acadêmico. A equipe quer um assistente que (1) responda dúvidas com base no regulamento e nos documentos do próprio aluno (RAG), (2) consulte notas e (3) abra chamados na secretaria. Aplicando o módulo:

- **Trindade letal**: há dados privados (notas, documentos) e conteúdo não confiável (anexos e e-mails recebidos). A perna a retirar é a **comunicação externa**: o assistente **não envia e-mails nem abre links**; ele só prepara o texto do chamado, e o aluno **confirma** após ver a pré-visualização.
- **Ferramentas estreitas**: `consultar_notas(periodo)` e `preparar_chamado(assunto, texto)`, com esquema estrito, sempre executadas com o **token do próprio aluno** (autorização por objeto na API, p. 53).
- **RAG com controle de acesso**: cada trecho carrega a lista de quem pode vê-lo; a busca filtra pelo aluno antes de enviar contexto ao modelo.
- **Saída**: exibida como texto (sem HTML bruto), com imagens e links só do domínio da instituição.
- **Limites e registro**: teto de tokens e de chamadas por aluno; logs de ferramentas (sem o conteúdo integral dos documentos); alertas de uso anômalo (p. 144).
- **Verificação**: um conjunto de casos de injeção direta e indireta roda a cada mudança no prompt ou no modelo, junto de testes de vazamento entre alunos (p. 166).

ATAQUES AO PRÓPRIO MODELO E REFERENCIAIS

Além das falhas de aplicação, há ataques ao **modelo** e aos seus dados, catalogados pela taxonomia **NIST AI 100-2 (edição E2025)**, que separa IA preditiva (classificadores, recomendadores) e IA generativa. As classes principais:

ATAQUE	O QUE FAZ	MITIGAÇÕES
Evasão (exemplos adversariais)	Perturba a entrada, às vezes de modo imperceptível, para que o modelo erre: um sinal de trânsito adulterado, um malware que escapa do classificador	Treino adversarial; validação de entrada; monitoramento de desvios
Envenenamento	Insere dados ou rótulos manipulados no treino, no ajuste ou na base de conhecimento, para degradar o modelo ou plantar uma "porta dos fundos" ativada por um gatilho	Proveniência e curadoria dos dados; detecção de anomalias; modelos de fontes confiáveis
Privacidade	Inferência de pertinência (um registro estava no treino?), reconstrução de dados de treino e extração de modelo por consultas em massa	Menos memorização; privacidade diferencial; limites e monitoramento de consultas
Uso indevido (IA generativa)	Usar o modelo para gerar golpes, conteúdo nocivo ou código malicioso	Políticas de uso; filtros de segurança; detecção de abuso

REFERENCIAL	PARA QUE SERVE
NIST AI RMF 1.0 e o perfil AI 600-1	Gestão do risco de IA em quatro funções (Govern, Map, Measure, Manage); o perfil trata especificamente de IA generativa
MITRE ATLAS	Base de conhecimento de táticas e técnicas contra sistemas de IA, no estilo do ATT&CK (p. 11)
Google SAIF	*Secure AI Framework*: práticas para proteger sistemas de IA ao longo do ciclo de vida
OWASP GenAI Security Project	Mantém os Top 10 para LLM e para aplicações com agentes, além de guias de testes e de governança

13 APIS, MOBILE E SISTEMAS DE IA — CONTINUAÇÃO

REFERENCIAL	PARA QUE SERVE
ISO/IEC 42001:2023	Norma de sistema de gestão de IA (certificável), nos moldes da ISO 27001 (p. 27)
Regulação	Na União Europeia, o Regulamento (UE) 2024/1689 (AI Act), com obrigações escalonadas por nível de risco; no Brasil, o PL 2338/2023 propõe um marco legal da IA (confira o estágio atual da tramitação)

PARA FIXAR

Conceitos

- Por que a autenticação não basta em uma API e o que diferencia BOLA, BOPLA e BFLA?
- Por que *pinning* e detecção de *root* não são fronteiras de segurança em um app móvel?
- Por que a injeção de prompt não tem um equivalente à consulta parametrizada?

Aplicação

- Escreva o esquema e a regra de autorização de `PATCH /alunos/{id}`, para que o aluno altere só nome e telefone.
- Projete a verificação de um webhook de pagamento (assinatura, janela de tempo, idempotência) e liste três falhas que ela evita.
- Aplique a trindade letal a um assistente que lê e-mails e agenda reuniões: qual perna você retiraria, e como?

APROFUNDE-SE

Leituras e referências

- OWASP API Security Project — OWASP: o Top 10 de APIs (2023) com exemplos e prevenção.
- OWASP Mobile Application Security — OWASP: MASVS, MASTG e o checklist de segurança mobile.
- OWASP Top 10 para aplicações com LLM — OWASP GenAI Security Project: riscos, exemplos e mitigações.
- NIST AI Risk Management Framework — NIST: gestão de risco de IA e o perfil de IA generativa.
- MITRE ATLAS — MITRE: táticas e técnicas contra sistemas de IA.
- The lethal trifecta for AI agents — Simon Willison, 2025: a combinação que torna o vazamento de dados quase inevitável.

MÓDULO 14 · DESENVOLVIMENTO

14 CÓDIGO SEGURO E MEMÓRIA

Práticas de codificação segura e a família de falhas de memória que ainda domina as listas de CWE.

A maioria das vulnerabilidades nasce de decisões comuns de programação: confiar em uma entrada, esquecer um limite, deixar duas requisições disputarem o mesmo dado. O módulo reúne os **princípios de codificação segura**, as **classes de falha de código** que mais aparecem nos rankings (validação, condições de corrida, desserialização, esgotamento de recursos) e a família de defeitos mais cara de todas, os **erros de segurança de memória** de C e C++, com as defesas do compilador, do sistema e das linguagens modernas.

PRINCÍPIOS DE CODIFICAÇÃO SEGURA

Codificação segura (*secure coding*) é escrever código que continua correto quando alguém o usa de um modo que o autor não previu. Os princípios de projeto estão em p. 90; aqui eles viram regras de código. Quase todas decorrem de uma ideia: **dados não confiáveis atravessam uma fronteira de confiança** (rede, arquivo, processo, linguagem, usuário comum × administrador) e precisam ser tratados exatamente nesse ponto.

PRINCÍPIO	NA PRÁTICA, NO CÓDIGO	FALHAS QUE EVITA
Validar na fronteira	Toda entrada externa (HTTP, arquivo, fila, variável de ambiente, saída de LLM) é verificada **no servidor**, antes do uso	CWE-20 e a base de todas as injeções
Codificar na saída	O dado é escapado ou parametrizado para o destino: HTML, SQL, shell, URL, log	XSS, injeção SQL e de comandos, *log injection*
Menor privilégio	Processo, conta de banco e token só têm o que a tarefa exige; privilégios extras são descartados após a inicialização	Reduz o dano de qualquer outra falha
Falhar de forma segura	Diante de erro, **negar** (falhar fechado) e liberar recursos; nada de `except: pass` em checagens de segurança	A10:2025, CWE-636 e CWE-209
Não confiar no cliente	Preço, papel e permissão são decididos no servidor; validar no navegador ou no app é só conveniência	CWE-602 e fraudes de lógica de negócio
Evitar APIs perigosas	Preferir funções com tamanho explícito e bibliotecas maduras (consulta parametrizada, `secrets`, `safe_load`)	Estouros de buffer, desserialização, aleatoriedade fraca

Em resumo: Para revisar um código, procure primeiro as **fronteiras**: onde entra dado de fora e onde esse dado vira consulta, comando, caminho, HTML ou chamada a outro sistema. Os defeitos de segurança se concentram nesses pontos.

VALIDAÇÃO DE ENTRADA E CODIFICAÇÃO DE SAÍDA

Validar é decidir se um dado é aceitável; **codificar** (ou *escapar*) é torná-lo inofensivo para o destino em que será usado. Uma não substitui a outra: a validação reduz o que entra, a codificação garante que o que entrou não vire código no destino. A validação acontece **no servidor**, na primeira camada que recebe dado externo, e se repete onde o dado é usado em contexto sensível (defesa em profundidade).

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

PREFERIDA

Lista de permissão (allowlist)

Aceita só o que casa com o formato esperado, como 8 dígitos para uma matrícula ou um valor de uma enumeração fechada.

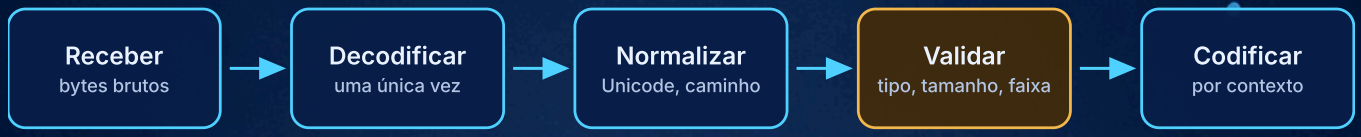
O desconhecido é **rejeitado por padrão**, inclusive as variações que ninguém previu.

FRÁGEIS

Lista de bloqueio e sanitização

Rejeita padrões ruins conhecidos (`<script>`, `--`) ou os remove do texto. Sempre existe uma variação que escapa.

Remover `../` **uma vez** de `..../` deixa `../`: a limpeza cria o que queria eliminar.



A ordem importa: **canonicalize antes de validar**. Se o filtro olha o texto codificado (`%2e%2e%2f`) e o uso olha o decodificado (`../`), a validação é contornada; por isso decodifique **uma única vez** e valide a forma final. O que verificar em cada campo:

- **Tipo**: inteiro, data, enumeração, UUID;
- **Tamanho**: mínimo e máximo, em bytes **e** em caracteres;
- **Faixa**: `1 ≤ quantidade ≤ 100` (a quantidade `-5` já virou reembolso em muita loja virtual);
- **Formato**: expressão regular ancorada ou um *parser* de verdade (datas, URLs, e-mails);
- **Semântica**: a data de fim vem depois da de início; o aluno pertence à turma; o destino da URL está na lista permitida.

VALIDAÇÃO POR LISTA DE PERMISSÃO, COM TIPO QUE SÓ ADMITE VALORES VÁLIDOS

```
import re
from dataclasses import dataclass

MATRICULA = re.compile(r"[0-9]{8}")            # lista de permissão: 8 dígitos ASCII

@dataclass(frozen=True)
class Matricula:                               # o resto do código só vê valores válidos
    valor: str
    def __post_init__(self):
        if not MATRICULA.fullmatch(self.valor):  # fullmatch: o padrão cobre a entrada toda
            raise ValueError("matrícula inválida")

def quantidade(texto: str) -> int:
    n = int(texto)                             # ValueError se não for número
    if not 1 <= n <= 100:                          # faixa de negócio
        raise ValueError("quantidade fora da faixa")
    return n
```

DICA DA CORUJA

Essa é a ideia de *parse, don't validate*: converta a entrada para um tipo que só admite valores válidos e deixe o resto do programa trabalhar com ele, não com um `str` solto. Cuidado com pegadinhas da linguagem: em Python, `\d` casa dígitos de **outros alfabetos**, `int()` aceita espaços e sublinhados (`" 12 "`, `"1_000"`) e `$` aceita uma quebra de linha no fim. Prefira `[0-9]` e `fullmatch`.

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

DESTINO DO DADO	FALHA SE FOR TRATADO COMO TEXTO	DEFESA CORRETA
Corpo ou atributo HTML	XSS (CWE-79)	Escape automático do *template*, específico do contexto; sem `innerHTML` com dado externo
Consulta SQL	Injeção SQL (CWE-89)	Consulta **parametrizada**; nunca concatenar
Comando do sistema	Injeção de comandos (CWE-78)	Lista de argumentos **sem shell** (`subprocess.run([...])`)
Caminho de arquivo	*Path traversal* (CWE-22)	Canonicalizar e conferir que o resultado fica dentro da pasta permitida
Linha de log	Forjamento de entradas (CWE-117)	Log estruturado; neutralizar `\r` e `\n`

O funcionamento das injeções, do XSS e do CSRF está em **p. 98**. O ponto aqui é a regra geral: **escolha a defesa pelo destino do dado, não pela origem**.

ERROS, FALHAS E LOGS

Um erro bem tratado não vaza informação e não abre portas. Duas regras: ao **usuário**, uma mensagem genérica com um identificador de correlação ("Não foi possível concluir. Código 7F3A"); ao **log**, o detalhe (pilha de chamadas, consulta, caminho), nunca na resposta (CWE-209). E, na dúvida, **falhe fechado**: se a verificação da assinatura lança uma exceção, o pedido é negado, não aceito.

FALHAR ABERTO (-) × FALHAR FECHADO (+)

```
-try:
-    verificar_assinatura(token)
-except Exception:
-    pass                                # falha aberto: token inválido segue em frente
+try:
+    verificar_assinatura(token)
+except Exception:
+    log.warning("assinatura inválida", extra={"corr": corr_id})
+    raise PermissionDenied()            # falha fechado: na dúvida, nega
```

Os **logs** são um destino de dados como outro qualquer. Se o texto do usuário entra no log sem tratamento, uma quebra de linha no campo cria **entradas falsas** (*log injection*, CWE-117) e compromete a trilha de auditoria; já senhas, tokens e dados pessoais gravados por descuido viram um segundo vazamento (CWE-532). A defesa é registrar em formato **estruturado** (JSON, em que a biblioteca escapa o valor), tratar as quebras de linha, limitar o tamanho e nunca registrar segredos. A arquitetura de logs e de detecção está em **p. 144**.

LOG INJECTION: INSEGURO (-) × SEGURO (+)

```
-log.info("login falhou para " + usuario)          # usuario pode trazer \n e forjar linhas
+log.info("login_falhou", extra={"usuario": usuario})   # o formatador JSON escapa o valor
```

> **Atenção:** Nunca registre o corpo inteiro de requisições nem o cabeçalho `Authorization`: é a forma mais comum de gravar senhas e tokens em claro nos arquivos de log.

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

CONCORRÊNCIA: CONDIÇÕES DE CORRIDA, TOCTOU E IDEMPOTÊNCIA

Em um servidor, dezenas de requisições rodam ao mesmo tempo sobre os mesmos dados. Uma **condição de corrida** (*race condition*, CWE-362) existe quando o resultado depende da **ordem** em que operações concorrentes acessam um recurso compartilhado sem sincronização. O padrão clássico é o *check-then-act*: ler, decidir, gravar. Entre a leitura e a gravação de uma requisição, outra passa pelo mesmo teste, e a regra ("cupom de uso único", "saldo suficiente", "limite de tentativas") é violada. Quando envolve dinheiro, o caso tem nome próprio: *double spend*, gastar duas vezes o mesmo saldo.

O **TOCTOU** (*time-of-check to time-of-use*, CWE-367) é a mesma ideia aplicada ao sistema de arquivos: o programa **verifica** uma propriedade (o arquivo existe? o usuário tem permissão? não é um link simbólico?) e só depois o **usa**; no intervalo, o arquivo é trocado. A defesa é não separar as etapas: abrir **uma vez** e operar sobre o descritor já aberto.

TOCTOU: VERIFICAR E DEPOIS ABRIR (-) × ABRIR E OPERAR NO DESCRITOR (+)

```
-if os.access(caminho, os.R_OK):                    # 1) verifica o caminho
-    f = open(caminho)                              # 2) usa: pode ter sido trocado
+fd = os.open(caminho, os.O_RDONLY | os.O_NOFOLLOW)  # abre uma vez, sem seguir links
+if not stat.S_ISREG(os.fstat(fd).st_mode):          # confere o arquivo JÁ aberto
+    os.close(fd)
+    raise PermissionError(caminho)
```

TÉCNICA	COMO FUNCIONA	QUANDO USAR
Instrução atômica	`UPDATE … WHERE condição` decide e grava em uma só operação	Contadores, estoque, cupons
Restrição única (`UNIQUE`)	O banco rejeita o segundo `INSERT` igual; quem perde a disputa recebe erro	Cadastros, chaves de idempotência
Bloqueio pessimista	`SELECT … FOR UPDATE` trava a linha até o fim da transação	Leitura, decisão e gravação complexas
Controle otimista	Coluna `versao`: grava só se a versão não mudou; senão, repete	Edição concorrente, pouco conflito
Chave de idempotência	Repetir o pedido não repete o efeito (ver p. 112)	Pagamentos, webhooks, repetições (*retries*)
Mutex em memória	Exclui *threads* de um mesmo processo	Só vale dentro de um processo, nunca entre servidores

EXEMPLO RESOLVIDO

Cupom de uso único. A rota que aplica o cupom lê `usado` e, se for falso, aplica o desconto e depois grava `usado = true`. Duas requisições simultâneas leem `false` **antes** de qualquer gravação, e o desconto é aplicado duas vezes. É fácil reproduzir o defeito **no seu ambiente de teste**: dispare as duas chamadas em paralelo contra uma cópia local.

A correção é deixar o **banco arbitrar**. A instrução condicional altera a linha para uma única requisição; a outra, no PostgreSQL com `READ COMMITTED`, espera a primeira terminar, reavalia a condição, encontra `usado = true` e afeta **zero linhas**. Quem não alterou a linha não aplica o desconto.

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

CUPOM DE USO ÚNICO: INSEGURO (-) × SEGURO (+)

```
-cupom = db.one("SELECT usado FROM cupons WHERE codigo = %s", codigo)
-if not cupom.usado:
-    aplicar_desconto(pedido)
-    db.run("UPDATE cupons SET usado = true WHERE codigo = %s", codigo)
+linhas = db.run("UPDATE cupons SET usado = true "
+               "WHERE codigo = %s AND usado = false", codigo).rowcount
+if linhas != 1:                      # perdeu a disputa (ou o cupom não existe)
+    raise CupomIndisponivel()
+aplicar_desconto(pedido)             # só quem alterou a linha chega aqui
```

NÚMEROS, RECURSOS E EXPRESSÕES REGULARES

Estouro de inteiros (*integer overflow*, CWE-190): o resultado de uma operação não cabe no tipo. Em C, o estouro de inteiro **sem sinal** dá a volta (*wraparound*) e o de inteiro **com sinal** é comportamento indefinido; em Java e C# a conta também dá a volta em silêncio, a menos que se peça verificação. Em Python os inteiros têm precisão arbitrária, mas o problema reaparece quando o valor segue para um banco, uma API ou uma biblioteca em C.

O dano maior vem quando a conta calcula um **tamanho**. Com `uint32_t total = n * 16`, o valor `n = 268.435.457` resulta em `total = 16`: o programa aloca 16 bytes e depois copia `n` itens, um estouro de buffer no *heap* (CWE-190 levando a CWE-122). As defesas são validar a **faixa** antes da conta, usar aritmética verificada (`__builtin_mul_overflow` no GCC e no Clang, `checked_mul` em Rust, `Math.multiplyExact` em Java) e preferir `calloc(n, tam)` a `malloc(n * tam)`.

Esgotamento de recursos (CWE-770, 25º no CWE Top 25 de 2025): sem limites, uma única requisição consome memória, CPU, disco ou conexões. Exemplos: corpo ou *upload* sem teto de tamanho, JSON ou XML muito aninhado, expansão de entidades XML (*billion laughs*), arquivos comprimidos que explodem ao descompactar (*zip bomb*), consultas sem paginação e chamadas externas sem *timeout*. A regra é **limitar tudo**: tamanho, profundidade, número de itens, tempo, concorrência e cota por usuário, em cada camada (proxy, aplicação, banco). Em Linux, `ulimit` e *cgroups* impõem tetos ao processo.

ReDoS (*Regular expression Denial of Service*, CWE-1333): motores de expressão regular com *backtracking* (Python `re`, Java, JavaScript, PCRE) tentam todas as formas de casar o texto. Um padrão com quantificadores aninhados ou alternativas sobrepostas, como `(a+)+$`, leva **tempo exponencial** em certas entradas: cada caractere a mais pode dobrar o tempo. Uma só requisição bem escolhida prende uma CPU.

REDOS: PADRÃO AMBÍGUO (-) × PADRÃO LINEAR COM LIMITE DE TAMANHO (+)

```
import re

RUIM = re.compile(r"^(a+)+$")      # quantificador aninhado: backtracking exponencial
RUIM.match("a" * 30 + "!")           # cada "a" a mais dobra o tempo; o "!" força a falha

BOA = re.compile(r"^a+$")          # mesma linguagem, sem ambiguidade: tempo linear
def valida(texto: str) -> bool:
    return len(texto) <= 200 and BOA.fullmatch(texto) is not None   # limite ANTES da regex
```

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

Como se proteger do ReDoS

- evite quantificadores aninhados e alternativas que se sobrepõem;
- **limite o tamanho** da entrada antes de aplicar a expressão;
- prefira motores de **tempo linear** (RE2, `regexp` do Go, *crate* `regex` do Rust);
- use o *timeout* do motor quando existir (.NET `matchTimeout`);
- teste padrões novos com entradas patológicas.

Caso real: Cloudflare, 2019

Em 2 de julho de 2019, uma regra nova do WAF da Cloudflare continha uma expressão regular com *backtracking* excessivo. A CPU dos servidores disparou e o tráfego HTTP caiu por cerca de meia hora, segundo o relatório da própria empresa.

Não foi ataque: foi um ReDoS **acidental**, em produção, sem teste de carga da regra.

DESSERIALIZAÇÃO, ALEATORIEDADE E FUNÇÕES PERIGOSAS

Serialização transforma um objeto em bytes ou texto para guardar ou transmitir; a **desserialização** faz o caminho inverso. Em formatos que **reconstroem objetos arbitrários** (`pickle` do Python, `ObjectInputStream` do Java, `unserialize` do PHP, `BinaryFormatter` do .NET, `yaml.load` com *loaders* completos), o dado de entrada escolhe **quais classes instanciar**. Encadeando comportamentos de classes que já existem na aplicação (as *gadget chains*), quem controla o dado pode chegar à execução de código (**CWE-502**, A08:2025). A regra é simples: **nunca desserialize dados não confiáveis com formatos que instanciam objetos**.

Use formatos **só de dados**, como JSON ou Protocol Buffers, validados por esquema; para YAML, `safe_load`. Se for inevitável desserializar objetos, **autentique antes** (HMAC sobre os bytes) e restrinja as classes permitidas.

DESSERIALIZAÇÃO: INSEGURO (-) × SEGURO (+)

```
-obj = pickle.loads(corpo)                  # executa o que quem enviou escolheu
+obj = Pedido.model_validate_json(corpo)    # JSON validado por esquema (pydantic)
-cfg = yaml.load(texto, Loader=yaml.Loader) # pode instanciar objetos Python arbitrários
+cfg = yaml.safe_load(texto)                # só tipos simples: dict, list, str, int, bool
```

Aleatoriedade fraca (CWE-338): geradores de uso geral, como o `random` do Python (Mersenne Twister), `Math.random()` e `rand()` de C, são **previsíveis**: com um número pequeno de saídas observadas dá para reconstruir o estado e calcular as próximas. Nunca os use para tokens, identificadores de sessão, códigos de redefinição de senha, chaves ou *nonces*. Use o gerador criptográfico do sistema (CSPRNG): `secrets` e `os.urandom` (Python), `crypto.randomBytes` e `crypto.randomUUID` (Node.js), `SecureRandom` (Java), `crypto/rand` (Go), `getrandom` (C).

TOKEN DE REDEFINIÇÃO DE SENHA: PREVISÍVEL (-) × CRIPTOGRÁFICO (+)

```
-token = "".join(random.choices(string.ascii_letters, k=16))   # previsível
+token = secrets.token_urlsafe(32)                            # 256 bits do CSPRNG
```

FUNÇÃO OU PADRÃO	RISCO	ALTERNATIVA MAIS SEGURA
C: `gets`, `strcpy`, `strcat`, `sprintf`	Não limitam o destino: estouro de buffer (CWE-120)	`fgets`, `snprintf`; tamanho sempre explícito; `std::string` em C++
`eval`, `exec`, `new Function`	Executam texto como código (CWE-94, CWE-95)	`ast.literal_eval`, `JSON.parse`, tabela de despacho
`os.system`, `shell=True`, `child_process.exec`	Injeção de comandos (CWE-78)	Lista de argumentos sem shell: `subprocess.run([...])`, `execFile`

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

FUNÇÃO OU PADRÃO	RISCO	ALTERNATIVA MAIS SEGURA
`pickle.loads`, `yaml.load`, `unserialize`	Desserialização de objetos (CWE-502)	JSON com esquema; `yaml.safe_load`
`random`, `Math.random()`, `rand()`	Valores previsíveis (CWE-338)	`secrets`, `crypto.randomBytes`, `SecureRandom`
`innerHTML`, `document.write`	XSS (CWE-79)	`textContent`; *templates* com escape; sanitizador confiável

Boa prática: Deixe uma ferramenta marcar essas chamadas na revisão de código: Bandit (Python), `eslint-plugin-security` (JavaScript), `gosec` (Go) ou regras do Semgrep. A integração ao CI está em p. 135.

SEGURANÇA DE MEMÓRIA: POR QUE C E C++ MERECEM ATENÇÃO ESPECIAL

Segurança de memória (*memory safety*) é a garantia de que um programa só acessa memória que lhe pertence e que está em estado válido. C e C++ delegam essa responsabilidade ao programador: o compilador não impede ler além do fim de um vetor nem usar um ponteiro depois de a memória ser liberada. Violar a regra gera **comportamento indefinido** (*undefined behavior*): às vezes nada acontece, às vezes há dados corrompidos, vazamento de informação ou, nas mãos de quem ataca, execução de código. Os erros se dividem em duas famílias:

- **Espaciais**: acesso fora dos limites do objeto (estouro de buffer, índice inválido, leitura ou escrita além do fim);
- **Temporais**: acesso fora do tempo de vida (*use-after-free*, *double free*, ponteiro para variável local que já saiu de escopo).

São graves porque a memória guarda **dados** e também **dados de controle**: endereços de retorno, ponteiros de função, tabelas de métodos virtuais. Uma **escrita** fora dos limites que alcança um dado de controle altera para onde o programa segue; uma **leitura** fora dos limites expõe o que estava ao lado (chaves, senhas, dados de outros usuários). Um único defeito pode, assim, afetar confidencialidade, integridade e disponibilidade ao mesmo tempo.

cerca de 70%
das vulnerabilidades graves corrigidas eram erros de segurança de memória
MICROSOFT (2019) · CHROMIUM (2020)

76% → 24%
fatia das vulnerabilidades de memória, com o código novo em linguagens seguras
GOOGLE · ANDROID, 2019–2024

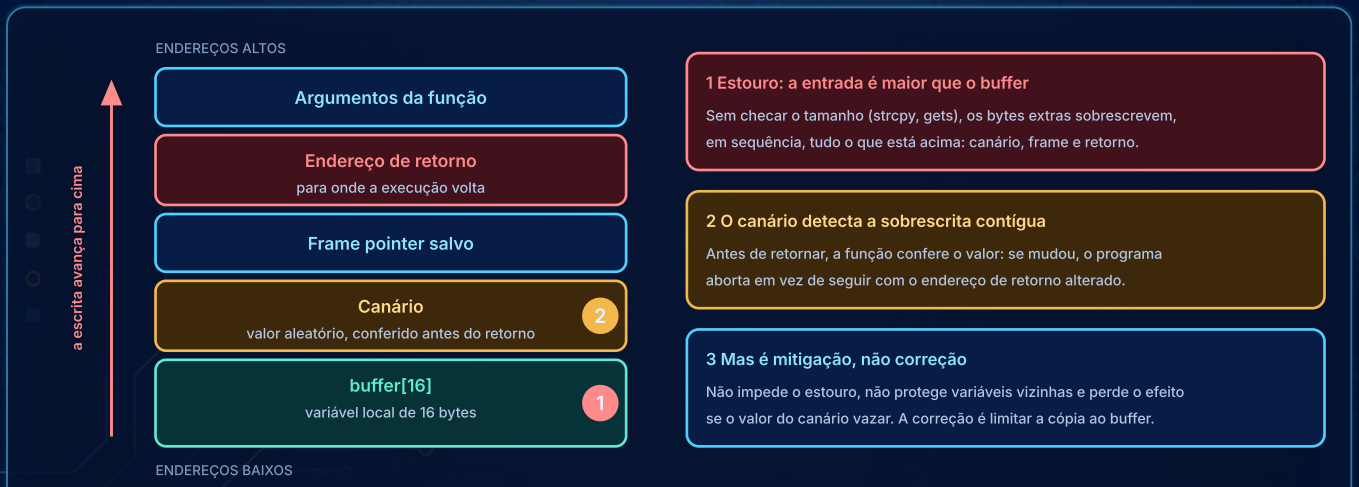
7 de 25
entradas do ranking são erros de memória (CWE-787, 416, 125, 120, 476, 121 e 122)
MITRE · CWE TOP 25, 2025

CWE	NOME	POSIÇÃO	SITUAÇÃO TÍPICA
CWE-787	*Out-of-bounds Write* (escrita fora dos limites)	5ª	Índice ou tamanho errado ao gravar; base de muitas execuções remotas de código
CWE-416	*Use After Free*	7ª	Ponteiro usado depois de `free` ou `delete`
CWE-125	*Out-of-bounds Read* (leitura fora dos limites)	8ª	Leitura além do fim; vazamento de memória (Heartbleed) ou queda do programa
CWE-120	*Classic Buffer Overflow* (cópia sem checar o tamanho)	11ª	`strcpy(destino, origem)` com a origem maior que o destino

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

CWE	NOME	POSIÇÃO	SITUAÇÃO TÍPICA
CWE-476	*NULL Pointer Dereference*	13ª	Ponteiro nulo desreferenciado; costuma derrubar o programa (negação de serviço)
CWE-121	*Stack-based Buffer Overflow*	14ª	Estouro de buffer alocado na pilha
CWE-122	*Heap-based Buffer Overflow*	16ª	Estouro de buffer alocado no *heap*

O estouro de **pilha** é o exemplo didático mais antigo. Cada chamada de função reserva na pilha um **quadro** (*stack frame*) com argumentos, o endereço de retorno, o *frame pointer* salvo e as variáveis locais. Se uma cópia sem verificação escreve mais bytes do que o buffer comporta, o excedente cobre, **em sequência**, o que está acima dele. O **canário** (*stack canary*) é um valor aleatório colocado entre os locais e os dados de controle: a função o confere antes de retornar e aborta se ele mudou.



Pilha de uma chamada de função (esquema didático). Em x86-64 e Arm a pilha cresce para endereços menores, mas um buffer é preenchido para endereços **maiores**; por isso um estouro avança em direção ao canário, ao *frame pointer* salvo e ao endereço de retorno. O desenho mostra a ideia geral; o layout real varia com o compilador, a arquitetura e as otimizações.

CWE-120: CÓPIA SEM LIMITE (-) × CÓPIA COM TAMANHO CONFERIDO (+)

```
 char dest[16];
-strcpy(dest, nome);                      // copia até o '\0', sem olhar o tamanho de dest
+int n = snprintf(dest, sizeof dest, "%s", nome);
+if (n < 0 || (size_t)n >= sizeof dest)     // n = tamanho que seria escrito: cabe em dest?
+    return ERRO_ENTRADA_GRANDE;             // rejeita em vez de truncar em silêncio
```

ERROS TEMPORAIS: USE-AFTER-FREE E COMPANHIA

O *use-after-free* (CWE-416) acontece quando um ponteiro continua em uso depois que o bloco para o qual aponta foi devolvido ao alocador. O `free` não zera os ponteiros que ainda guardam o endereço (os **ponteiros pendentes**, *dangling pointers*), e o alocador pode entregar o mesmo bloco a outra parte do programa. Dali em diante, o uso do ponteiro antigo lê ou altera os dados de **outro objeto**: corrompendo-o, vazando seu conteúdo ou, se o objeto tiver ponteiros de função ou tabelas de métodos, desviando a execução. É uma das classes mais exploradas em navegadores e sistemas operacionais.

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO



Como evitar
anular o ponteiro após o free · RAII e ponteiros inteligentes (C++) · ASan nos testes · alocador endurecido · linguagens com posse (Rust)

Use-after-free em cinco passos. O erro não está no `free`, e sim em **continuar usando** um ponteiro depois dele: entre o passo 3 e o 5 o alocador pode entregar o mesmo bloco a outro objeto, e o ponteiro pendente passa a ler ou escrever em dados que não são mais seus.

Outros defeitos da mesma família:

- **Double free** (CWE-415): liberar duas vezes o mesmo bloco corrompe os metadados do alocador;
- **Endereço de variável local devolvido** (CWE-562): o ponteiro retornado aponta para uma pilha que já foi desempilhada;
- **Memória não inicializada** (CWE-457 e CWE-908): variáveis locais e blocos de `malloc` trazem **lixo de usos anteriores**, que pode vazar dados;
- **Format string** (CWE-134): `printf(entrada)` deixa o usuário escolher o formato, e `%s` e `%n` leem e escrevem memória;
- **Confusão de tipos** (CWE-843): um objeto é lido como se fosse de outro tipo, e os campos não coincidem.

Em C++ moderno a regra é **não gerenciar memória à mão**: `std::vector`, `std::string`, `std::unique_ptr` e o RAII (*Resource Acquisition Is Initialization*) amarram a liberação ao fim do escopo, e `v.at(i)` verifica limites. Cuidado: `std::string_view` e `std::span` não são donos dos dados e também podem ficar pendentes. Em C, o hábito mínimo é anular o ponteiro depois do `free` (o que reduz o problema, mas não resolve cópias do mesmo ponteiro) e deixar claro **a quem pertence cada bloco**.

ESTUDO DE CASO: A MESMA CLASSE DE FALHA, DUAS CONSEQUÊNCIAS

ASPECTO	HEARTBLEED (ABR/2014)	CROWDSTRIKE (19/JUL/2024)
Onde	OpenSSL, extensão *heartbeat* do TLS (CVE-2014-0160)	Sensor Falcon no Windows, atualização de conteúdo (Channel File 291)
Defeito	O servidor devolvia tantos bytes quanto o **tamanho declarado** pelo cliente, sem compará-lo ao que foi recebido	O modelo de dados previa 21 campos de entrada e o código de integração fornecia 20: leitura fora dos limites
Classe	CWE-125, leitura fora dos limites	CWE-125, leitura fora dos limites
Efeito	Até 64 KiB de memória do servidor por pedido: chaves privadas, sessões, senhas (**confidencialidade**)	Tela azul em cerca de 8,5 milhões de máquinas, segundo a Microsoft (**disponibilidade**)
Correção	Conferir o tamanho declarado contra o tamanho recebido	Validar o conteúdo, testar mais e liberar em etapas, segundo a empresa

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

A CLASSE DO HEARTBLEED: CONFIAR NO TAMANHO INFORMADO DE FORA (-) × CONFERIR (+)

```
-memcpy(resp, req->dados, req->tam_declarado);    // confia no campo enviado pelo cliente
+if (req->tam_declarado > req->bytes_recebidos)    // compara com o que de fato chegou
+    return ERRO_PEDIDO_INVALIDO;
+memcpy(resp, req->dados, req->tam_declarado);
```

A lição: Todo campo de **tamanho, contagem ou índice** que vem de fora precisa ser conferido contra a realidade (bytes recebidos, número de campos) **antes** do uso. O Heartbleed vazou segredos e o CrowdStrike derrubou máquinas, mas o defeito de fundo é o mesmo. O CrowdStrike também reforça a lição de p. 156: mudanças de conteúdo precisam de testes e de liberação gradual.

MITIGAÇÕES DO COMPILADOR E DO SISTEMA

As **mitigações** não corrigem o defeito: elas **encarecem a exploração**. Uma falha que daria execução de código passa, na melhor hipótese, a derrubar o programa. Funcionam em camadas (linguagem, compilador, sistema operacional e hardware) e **combinadas**, porque cada uma tem limites conhecidos.

MITIGAÇÃO	O QUE IMPEDE	LIMITE
Canário de pilha (`-fstack-protector-strong`)	Sobrescrita contígua do endereço de retorno: o valor é conferido antes de retornar e o programa aborta	Não cobre escrita pontual que "pula" o canário, vazamento do valor nem erros no *heap*
DEP ou NX (*W^X*)	Executar código injetado: páginas graváveis (pilha, *heap*) não são executáveis	Não impede **reaproveitar** código existente (ROP, *return-oriented programming*)
ASLR e PIE (`-fPIE -pie`)	Prever endereços: o sistema embaralha pilha, *heap* e bibliotecas a cada execução; o PIE inclui o executável	Um vazamento de endereço revela o layout; entropia limitada em 32 bits
RELRO total (`-Wl,-z,relro,-z,now`)	Sobrescrita das tabelas de ligação dinâmica (GOT)	Não protege os dados da aplicação
CFI (*Control-Flow Integrity*)	Saltos indiretos para destinos que o programa nunca usaria	Granularidade e custo; no Windows, o CFG é a variante nativa
Pilha sombra e autenticação de ponteiros (Intel CET, Arm PAC e BTI)	Alteração do endereço de retorno: cópia protegida em hardware ou ponteiro assinado	Exige CPU e sistema compatíveis; cobre parte dos alvos
`_FORTIFY_SOURCE=2`	Estouros em `memcpy`, `strcpy` e `sprintf`: verifica tamanhos conhecidos na compilação e na execução	Só quando o compilador conhece o tamanho do destino; exige `-O1` ou mais
Alocadores endurecidos e marcação de memória (Arm MTE)	Exploração de *use-after-free* e estouros de *heap*: quarentena de blocos, *tags* verificadas pelo hardware	Custo de desempenho e memória; MTE exige hardware Arm recente

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

COMPILAÇÃO ENDURECIDA (GCC OU CLANG, LINUX)

```
# avisos úteis + mitigações; em C++ acrescente -D_GLIBCXX_ASSERTIONS (limites em operator[])
gcc -std=c17 -O2 -Wall -Wextra -Wformat=2 -Wformat-security \
    -D_FORTIFY_SOURCE=2 -fstack-protector-strong -fPIE \
    -Wl,-z,relro,-z,now -pie prog.c -o prog

checksec --file=./prog    # confere o que ficou ativo no binário: canário, NX, PIE, RELRO
```

> **Atenção:** Mitigação não é correção. Cada defesa da tabela já foi contornada em ataques reais, em geral combinando várias falhas. Trate-as como **rede de segurança**: a correção é eliminar o defeito, com checagem de limites, controle do tempo de vida dos objetos ou troca de linguagem.

ACHANDO OS DEFEITOS: SANITIZERS, FUZZING E ANÁLISE ESTÁTICA

Os **sanitizers** são instrumentações do compilador que detectam erros **em tempo de execução**, durante testes: o programa roda mais devagar, mas o relatório aponta a linha, o tipo de erro e (no caso do ASan) onde o bloco foi alocado e liberado. O **fuzzing** alimenta o programa com milhões de entradas geradas e mutadas; combinado a sanitizers, encontra estouros que nenhum teste escrito à mão encontraria.

FERRAMENTA	O QUE DETECTA	CUSTO E LIMITES
AddressSanitizer, ASan (`-fsanitize=address`)	Estouros de pilha, *heap* e globais; *use-after-free*; *double free*; vazamentos (LeakSanitizer)	Cerca de 2× mais lento e mais memória; uso em testes
UBSan (`-fsanitize=undefined`)	Estouro de inteiro com sinal, deslocamentos inválidos, desreferência nula, conversões indevidas	Custo baixo; combina com o ASan
MSan (`-fsanitize=memory`)	Leitura de memória **não inicializada**	Só Clang; exige que as dependências também sejam instrumentadas
TSan (`-fsanitize=thread`)	Corridas de dados (*data races*) entre *threads*	Alguns ×10 de lentidão; não combina com o ASan
Valgrind (Memcheck)	Erros de memória sem recompilar	Dezenas de vezes mais lento

TESTE COM SANITIZERS E HARNESS DE FUZZING (LIBFUZZER)

```
// compilar:  clang++ -g -O1 -fsanitize=fuzzer,address harness.cpp parser.cpp -o fuzz
// executar:  ./fuzz corpus/        (corpus/ guarda as entradas que abriram caminhos novos)
#include <cstddef>
#include <cstdint>

extern "C" int LLVMFuzzerTestOneInput(const uint8_t *dados, size_t tam) {
    analisar_pedido(dados, tam);   // a função sob teste: parser, decodificador...
    return 0;                      // erros de memória viram relatórios do ASan
}
```

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

O *fuzzing* **guiado por cobertura** (AFL++, libFuzzer, Honggfuzz) mede quais ramos do código cada entrada percorre e mantém as que revelam caminhos novos. Ele é mais eficaz em código que **processa entrada não confiável**: analisadores de formatos (imagem, áudio, PDF, XML), protocolos de rede e decodificadores. Projetos de código aberto contam com o OSS-Fuzz, do Google, que roda o *fuzzing* continuamente. A **análise estática** (`-Wall -Wextra`, `-fanalyzer` do GCC, clang-tidy, Cppcheck, CodeQL) completa o quadro sem executar o programa, ao custo de falsos positivos; a integração dessas ferramentas ao CI está em p. 135.

> **Padrões de codificação:** O **SEI CERT C Coding Standard** (Carnegie Mellon) reúne regras numeradas, com exemplos conformes e não conformes: `STR31-C` (espaço para a cadeia e o terminador nulo), `ARR30-C` (limites de vetores), `INT32-C` (estouro de inteiro com sinal), `MEM30-C` (não acessar memória liberada) e `FIO30-C` (não usar entrada do usuário como formato). Para C++, as **C++ Core Guidelines** cumprem papel parecido.

LINGUAGENS COM SEGURANÇA DE MEMÓRIA E ROTEIROS GOVERNAMENTAIS

Uma linguagem é **memory-safe** quando sua definição, em vez do programador, impede os erros espaciais e temporais descritos acima. Elas eliminam **por construção** a maior parte da tabela de CWEs de memória, mas a garantia termina onde o código "sai" da linguagem.

LINGUAGEM	COMO GARANTE A SEGURANÇA DE MEMÓRIA	ONDE A GARANTIA TERMINA
Rust	Posse (*ownership*), empréstimos e tempos de vida verificados na compilação; limites checados na execução; sem coletor de lixo	Blocos `unsafe` e chamadas a código C (FFI)
Go	Coletor de lixo, checagem de limites, sem aritmética de ponteiros	Pacote `unsafe`, cgo e corridas de dados em estruturas compartilhadas
Java, C# e Kotlin	Coletor de lixo, checagem de limites, tipos verificados	JNI, `unsafe` do C# e bibliotecas nativas
Swift	Contagem de referências automática, limites checados, opcionais no lugar de ponteiros nulos	Tipos `Unsafe*` e interoperação com C
Python, JavaScript e Ruby	O interpretador gerencia a memória	Extensões em C (`ctypes`, N-API) e falhas do próprio interpretador ou do JIT

RUST: O COMPILADOR REJEITA O PONTEIRO PENDENTE

```
fn main() {
    let r;                  // referência declarada fora do bloco
    {
        let x = 5;          // x só existe dentro deste bloco
        r = &x;             // erro E0597: `x` não vive o suficiente
    }
    println!("{}", r);      // em C seria um ponteiro pendente; em Rust nem compila
}
```

14 CÓDIGO SEGURO E MEMÓRIA — CONTINUAÇÃO

Órgãos governamentais passaram a tratar o tema como risco sistêmico. Em dezembro de 2023, CISA, FBI, NSA e agências parceiras publicaram *The Case for Memory Safe Roadmaps*. Em outubro de 2024, o documento *Product Security Bad Practices* (CISA e FBI) classificou como **prática perigosa** desenvolver novas linhas de produto para infraestrutura crítica em linguagem sem segurança de memória quando há alternativas seguras, e pediu aos fabricantes de produtos existentes que publicassem um **roteiro de segurança de memória até 1º de janeiro de 2026**. Em 24 de junho de 2025, NSA e CISA lançaram a orientação *Memory Safe Languages: Reducing Vulnerabilities in Modern Software Development*, que recomenda planejar a adoção, **sem exigir a reescrita total** do código existente, e integrar as linguagens seguras por interoperabilidade.

1 Código novo em linguagem segura
É a estratégia do Android (*Safe Coding*, Google, 2024): o código antigo tende a se estabilizar com o tempo, e o código novo já nasce sem a classe inteira de erros.

2 Comece pela fronteira de risco
Priorize o que processa entrada não confiável (formatos, rede, mídia) e migre por partes, com FFI, testes de regressão e *fuzzing* nas duas pontas.

3 Endureça o legado
Flags de compilação, sanitizers nos testes, *fuzzing* contínuo, alocador endurecido e *sandbox* para componentes de risco.

4 Meça e documente o roteiro
Acompanhe a fatia de CVEs de memória e de código em linguagens seguras, e registre o que migra, em que ordem e prazo: é o que as autoridades passaram a pedir.

PARA FIXAR

Conceitos

- Por que a lista de permissão é preferível à lista de bloqueio? Mostre um caso em que a sanitização de uma só passada falha.
- Diferencie erros de memória **espaciais** e **temporais** e cite um CWE de cada família.
- O que um canário de pilha detecta e o que ele não detecta? Por que ASLR e DEP, juntos, ainda não substituem a correção?

Aplicação

- Uma rota lê `usado` e depois grava `usado = true`. Descreva o defeito com duas requisições simultâneas e reescreva a operação para que seja atômica.
- Com `uint32_t` e `sizeof(Item) = 16`, qual o menor `n` que faz `malloc(n * sizeof(Item))` alocar só 16 bytes? Como corrigir o código?
- Monte o plano de endurecimento de um serviço C++ legado que lê arquivos enviados por usuários: flags, sanitizers, *fuzzing* e o que migrar primeiro.

APROFUNDE-SE

Leituras e referências

- SEI CERT C Coding Standard — SEI/Carnegie Mellon: regras numeradas com código conforme e não conforme.
- Input Validation Cheat Sheet — OWASP: validação por lista de permissão, canonicalização e onde validar.
- CWE Top 25 Most Dangerous Software Weaknesses, 2025 — MITRE: o ranking usado neste módulo.
- AddressSanitizer — documentação do Clang: como usar o ASan e interpretar o relatório.
- Compiler Options Hardening Guide for C and C++ — OpenSSF: flags de compilação recomendadas.
- Memory Safe Languages: Reducing Vulnerabilities in Modern Software Development — NSA e CISA, 2025: planejar a adoção sem reescrever tudo.

MÓDULO 15 · DEVSECOPS

15 CADEIA DE SUPRIMENTOS E DEVSECOPS

Proteger o que você usa e o caminho do código até a produção: dependências, build, assinatura e entrega.

Hoje a maior parte do código de um sistema não foi escrita por quem o entrega: vem de bibliotecas, imagens de contêiner, ferramentas de *build* e pipelines automáticos. A **cadeia de suprimentos de software** reúne tudo o que entra no produto e tudo o que o constrói e distribui, e atacá-la é atacar muitos alvos por um único ponto. O módulo mostra os ataques, as defesas (SCA, SBOM, SLSA e assinaturas), a segurança de CI/CD e de segredos e as técnicas de teste que formam um pipeline **DevSecOps**.

A CADEIA DE SUPRIMENTOS DE SOFTWARE

A **cadeia de suprimentos de software** (*software supply chain*) é tudo o que entra em um produto e tudo o que o constrói e entrega: o código próprio, as **dependências** diretas e transitivas, as imagens-base de contêiner, as ferramentas de *build*, os pipelines de CI/CD, os repositórios de pacotes e os mecanismos de atualização. Uma aplicação costuma ter dezenas de dependências diretas e muito mais transitivas, mantidas por pessoas que a equipe nunca conheceu. Comprometer um elo atinge todos que dependem dele, e por isso o OWASP elevou o tema à categoria **A03:2025 Software Supply Chain Failures**, que vai de bibliotecas vulneráveis a *builds* e distribuição comprometidos (CWE-1104, 1395, 1329 e 1357).



Mapa de ataque à cadeia de suprimentos. Os ataques podem acontecer em qualquer etapa, da biblioteca escolhida à atualização que chega ao cliente, e cada etapa tem controles próprios. A segurança do conjunto é a da etapa mais fraca.

Em resumo: Três perguntas organizam a defesa. **O que há no meu software?** (inventário: SCA e SBOM). **De onde veio e quem o construiu?** (proveniência e assinatura). **O que o meu pipeline pode fazer, e com quais credenciais?** (CI/CD e segredos).

15 CADEIA DE SUPRIMENTOS E DEVSECOPS

CONTINUAÇÃO

TIPOS DE ATAQUE E CASOS REAIS

ATAQUE	COMO FUNCIONA, COM EXEMPLO	DEFESA PRINCIPAL
Dependência vulnerável	Biblioteca com falha conhecida que não é atualizada a tempo: Log4Shell (CVE-2021-44228, dez/2021) e o Struts do caso Equifax (2017)	SCA contínuo, atualização automatizada e SBOM para achar onde está
Typosquatting	Pacote com nome quase igual ao de um popular, à espera de um erro de digitação	Conferir nome e publicador; instalar via registro interno com lista de permissão
Dependency confusion	O nome de um pacote **interno** é registrado no repositório público com versão maior, e o gerenciador escolhe o público (técnica divulgada em 2021)	Escopos (*namespaces*), um único índice e fixação por hash
Conta ou token de mantenedor roubado	Credenciais obtidas por *phishing* publicam versões maliciosas; no npm, em set/2025, um *worm* (Shai-Hulud) comprometeu mais de 500 pacotes, segundo a CISA	MFA resistente a phishing, tokens de publicação de vida curta, *trusted publishing*
Contribuição ou mantenedor malicioso	xz/liblzma (CVE-2024-3094, mar/2024): *backdoor* plantado por quem ganhou a confiança do projeto ao longo de anos	Revisão por mais de uma pessoa, proveniência e builds reprodutíveis
***Build* ou atualização comprometidos**	SolarWinds Orion (2020): código inserido no build, e a atualização adulterada chegou a cerca de 18.000 clientes, segundo a empresa. NotPetya (2017): atualização do software contábil M.E.Doc	Build isolado e efêmero, proveniência SLSA, atualizações assinadas e verificadas
Pipeline de CI/CD	*Action* de terceiros comprometida expõe segredos nos logs: tj-actions/changed-files (CVE-2025-30066, mar/2025)	*Actions* fixadas por SHA, permissões mínimas, OIDC e segredos com escopo
Extensões e *plugins* maliciosos	Extensões de IDE e de navegador e *plugins* de build têm acesso amplo ao ambiente da pessoa desenvolvedora	Lista de extensões permitidas, publicadores verificados, menor privilégio
Slopsquatting	Pacotes **inventados** por assistentes de IA são registrados por atacantes: de 5% a 22% dos pacotes sugeridos por LLMs não existem (USENIX Security 2025)	Conferir se o pacote existe e é confiável **antes** de instalar

Caso xz (2024): O *backdoor* do xz (CVE-2024-3094, CVSS 10,0) estava nas versões 5.6.0 e 5.6.1 e foi descoberto por Andres Freund, que estranhou a lentidão do SSH em seu sistema. Foi fruto de **engenharia social de longo prazo** contra a pessoa que mantinha o projeto, e parte do código que o ativava constava apenas nos pacotes de *release*, não no repositório. Duas lições: depender de uma única pessoa mantenedora é um risco; e **verificar se o artefato corresponde ao código-fonte** (proveniência, builds reprodutíveis) importa tanto quanto revisar o código.

DEPENDÊNCIAS: SCA, LOCKFILES E ESCOLHA CRITERIOSA

SCA (*Software Composition Analysis*) compara as dependências do projeto com bases de vulnerabilidades (NVD, GitHub Advisory Database, OSV) e avisa quando uma versão em uso tem falha conhecida. Entre as ferramentas estão Dependabot e Renovate (que abrem PRs de atualização), OSV-Scanner, `npm audit`, `pip-audit`, Trivy e Grype. Duas armadilhas: o alerta quase sempre vem de uma dependência **transitiva** (a que você não escolheu), e a falha muitas vezes **não é alcançável** pelo seu código, motivo pelo qual existe o VEX (adiante).

15 CADEIA DE SUPRIMENTOS E DEVSECOPS — CONTINUAÇÃO

Os **lockfiles** (`package-lock.json`, `poetry.lock`, `Cargo.lock`, `go.sum`) fixam as versões exatas de toda a árvore e, melhor ainda, os *hashes* dos arquivos. No CI, use o comando que **respeita** o lockfile e falha se houver divergência (`npm ci`, `pip install --require-hashes`); o `npm install` pode alterá-lo. Contra a *dependency confusion*, use **um único índice** (no `pip`, `--extra-index-url` consulta dois índices e escolhe a maior versão), escopos como `@org/pacote` no npm e um **repositório proxy interno** (Artifactory, Nexus, Verdaccio) com lista de permissão e quarentena para versões recém-publicadas.

INSTALAÇÃO REPRODUTÍVEL E VERIFICAÇÃO DE DEPENDÊNCIAS

```
# Node.js: instala exatamente o que está no package-lock.json, sem scripts de instalação
npm ci --ignore-scripts
npm audit --omit=dev                # falhas conhecidas nas dependências de produção

# Python: fixação por hash de toda a árvore (pip-tools) e instalação que exige os hashes
pip-compile --generate-hashes requirements.in -o requirements.txt
pip install --require-hashes -r requirements.txt
pip-audit -r requirements.txt

# qualquer ecossistema: consulta a base OSV (osv.dev) a partir dos lockfiles
osv-scanner scan source -r .
```

Antes de adicionar uma dependência, pergunte: **preciso mesmo dela?** (a mais segura é a que você não tem); ela é mantida, tem mais de uma pessoa mantenedora, publica correções e assina as versões? O **OpenSSF Scorecard** automatiza parte da resposta, pontuando um repositório em verificações como `Maintained`, `Code-Review`, `Branch-Protection`, `Pinned-Dependencies`, `Signed-Releases` e `Token-Permissions`. Popularidade não é garantia de segurança. No npm, `--ignore-scripts` impede que os *scripts* de instalação das dependências executem; use-o em CI sempre que o projeto permitir.

SBOM, VEX, SLSA E ASSINATURAS: PROVAR DE ONDE VEIO

Um **SBOM** (*Software Bill of Materials*) é o inventário, legível por máquina, dos componentes de um software: nome, versão, fornecedor, identificadores (como o PURL, *Package URL*), relações de dependência, licenças e *hashes*. Os formatos principais são o **SPDX** (padrão ISO/IEC 5962) e o **CycloneDX** (projeto do OWASP, também padronizado pela Ecma). Os **elementos mínimos** definidos pela NTIA (2021) são sete campos: fornecedor, nome do componente, versão, outros identificadores, relação de dependência, autor dos dados do SBOM e carimbo de data e hora. Gere o SBOM **no build** (Syft, Trivy, cdxgen), por artefato e por imagem: quando surge um novo Log4Shell, ele responde em minutos à pergunta "onde usamos isso?".

O **VEX** (*Vulnerability Exploitability eXchange*) é a declaração que complementa o SBOM: para cada CVE, informa se o produto é **afetado**, **não afetado** (com justificativa, como "a função vulnerável nunca é chamada"), **corrigido** ou **em investigação**. Reduz o ruído do SCA e mostra ao cliente o que de fato é urgente. Existem VEX nos formatos CSAF, OpenVEX e CycloneDX.

O **SLSA** (*Supply-chain Levels for Software Artifacts*, lê-se "salsa"; versão 1.2, segundo o site do projeto) define níveis para a **trilha de Build**, com foco em provar de onde veio um artefato e que ninguém o alterou no caminho. A **proveniência** (*provenance*) é uma declaração assinada que registra qual repositório e *commit*, qual *builder* e quais parâmetros produziram o artefato; ela permite verificar que o que você implanta vem **do código e do pipeline esperados**.

15 CADEIA DE SUPRIMENTOS E DEVSECOPS — CONTINUAÇÃO

NÍVEL (BUILD)	O QUE EXIGE	PROTEGE CONTRA
L0	Nada: builds de desenvolvimento, em uma máquina qualquer	Nada; sem garantias
L1	Proveniência gerada, mostrando como o pacote foi construído	Erros e falta de rastreio; ainda é fácil de forjar
L2	Build em plataforma **hospedada**, que gera e assina a proveniência	Adulteração **depois** do build
L3	Plataforma endurecida: builds isoladas entre si e segredos de assinatura fora do alcance dos passos do usuário	Adulteração **durante** o build

O **Sigstore** torna a assinatura prática ao eliminar a chave de longa duração. O **cosign** assina e verifica artefatos e imagens de contêiner; a **Fulcio** é a autoridade certificadora que emite um certificado de **vida curtíssima**, ligado a uma identidade OIDC (por exemplo, a do *workflow* de CI); e o **Rekor** é o registro de transparência, público e à prova de adulteração, onde a assinatura fica gravada. Na verificação, o que importa é **quem assinou** (identidade e emissor esperados), e não apenas haver "uma assinatura válida".



ASSINATURA SEM CHAVE (KEYLESS) E VERIFICAÇÃO COM A IDENTIDADE ESPERADA

```
# no pipeline: a identidade vem do token OIDC do CI; assine sempre pelo digest, não pela tag
cosign sign --yes ghcr.io/org/app@sha256:<digest>

# na implantação: exija QUEM assinou, e não apenas que existe uma assinatura
cosign verify ghcr.io/org/app@sha256:<digest> \
  --certificate-identity-regexp '^https://github.com/org/app/' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

O **in-toto** é o *framework* de **atestações** (declarações assinadas sobre cada etapa da cadeia: quem fez o quê, com quais entradas); a proveniência SLSA é um tipo dessas atestações. Os **builds reprodutíveis** (*reproducible builds*) vão além: o mesmo código-fonte, construído por pessoas e máquinas diferentes, gera **exatamente os mesmos bytes**, e qualquer divergência denuncia adulteração.

A pressão regulatória também cresce. Nos EUA, a **EO 14028** (maio/2021) tornou o SBOM e o **NIST SSDF** (SP 800-218) referências para fornecedores do governo federal, embora as exigências específicas tenham mudado desde então e valha conferir a versão vigente. Na União Europeia, o **Cyber Resilience Act** (Regulamento (UE) 2024/2847) impõe requisitos de segurança e de tratamento de vulnerabilidades a produtos com elementos digitais, com obrigações escalonadas: segundo o calendário publicado, a notificação de vulnerabilidades exploradas vale a partir de 11/set/2026 e as demais obrigações principais, a partir de 11/dez/2027.

SEGURANÇA DE CI/CD

O pipeline de CI/CD é um alvo valioso porque **executa código de terceiros** (dependências, *actions*, contribuições) com acesso a **segredos** e ao caminho até a produção: quem o controla, controla o que é implantado. As práticas valem para qualquer plataforma; os exemplos usam o GitHub Actions.

15 CADEIA DE SUPRIMENTOS E DEVSECOPS — CONTINUAÇÃO

PRÁTICA	POR QUE	COMO
Permissões mínimas do token	O token padrão do pipeline pode escrever no repositório; um passo comprometido herda tudo	`permissions:` explícito e somente leitura; elevar só no job que precisa
OIDC federado, não chaves longas	Chave de nuvem guardada como segredo vaza e vale por meses	O job pede um token OIDC e a nuvem entrega credenciais de minutos, por repositório e ramo
***Actions* e imagens fixadas por SHA ou digest**	Tags e versões são mutáveis: quem controla a tag troca o código (caso tj-actions)	`uses: org/acao@` e o SHA completo do commit; Dependabot ou Renovate para atualizar
***Runners* efêmeros e isolados**	Um *runner* persistente guarda resíduos entre jobs; *self-hosted* em repositório público roda código de qualquer PR	Um *runner* novo por job; sem *self-hosted* em repositório público
Proteção de branches e revisão	Sem revisão, uma só conta comprometida publica código	Aprovação de uma ou mais pessoas, CODEOWNERS, commits assinados, checks obrigatórios
Ambientes protegidos; build × deploy	Quem testa não deve implantar sozinho em produção	*Environments* com aprovação manual e segredos próprios por ambiente
Dado externo nunca em comando	Título de PR, nome de branch e mensagens são controlados por quem contribui (*script injection*)	Passar por variável de ambiente, entre aspas; cuidado extra com `pull_request_target`

WORKFLOW ENDURECIDO (GITHUB ACTIONS)

```
name: build
on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read                  # padrão do workflow: somente leitura

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      id-token: write             # só este job pode pedir token OIDC (nuvem, assinatura)
    steps:
      - uses: actions/checkout@0123456789abcdef0123456789abcdef01234567  # v4 (exemplo)
        with:
          persist-credentials: false
      - name: Testes
        env:
          TITULO: ${{ github.event.pull_request.title }}   # dado externo vai por variável
        run: ./ci/testar.sh "$TITULO"                       # nunca ${{ ... }} dentro do run
```

15 CADEIA DE SUPRIMENTOS E DEVSECOPS — CONTINUAÇÃO



DICA DA CORUJA

No GitHub, o gatilho `pull_request_target` roda no contexto do repositório **base**, com acesso a segredos, e já esteve na origem de vários incidentes quando o workflow faz *checkout* do código do PR. Prefira `pull_request` e separe os workflows que realmente precisam de segredos.

GESTÃO DE SEGREDOS

Segredo é qualquer credencial que dá acesso: chaves de API, tokens, senhas de banco, chaves privadas, certificados. As regras:

- **nunca** no código, no histórico do Git, em imagens de contêiner, em logs, em tickets ou em conversas (inclusive com assistentes de IA);
- fora do repositório, entregues em tempo de execução por um **gerenciador de segredos** (HashiCorp Vault, AWS Secrets Manager, Google Secret Manager, Azure Key Vault), com controle de acesso, auditoria e rotação; variáveis de ambiente servem para começar, mas aparecem em *dumps*, em `docker inspect` e em logs de erro;
- credenciais de **curta duração** (OIDC, tokens com expiração, segredos dinâmicos) no lugar de chaves eternas; um segredo por finalidade e por ambiente, com o menor escopo possível;
- **detecção** no *pre-commit* e no CI (gitleaks, TruffleHog) e, no GitHub, *secret scanning* com *push protection*, que bloqueia o *push* quando reconhece um segredo.

.GITLEAKS.TOML: REGRAS PADRÃO MAIS UM FORMATO INTERNO

```
# estende as regras padrão do gitleaks e acrescenta um formato de token da casa
[extend]
useDefault = true

[[rules]]
id          = "token-servico-interno"
description = "Token de serviço interno (prefixo svc_)"
regex       = '''svc_[A-Za-z0-9]{32}'''
keywords    = ["svc_"]
```

.PRE-COMMIT-CONFIG.YAML: VARREDURA ANTES DE CADA COMMIT

```
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: vX.Y.Z                  # fixe a versão adotada pela equipe
    hooks:
      - id: gitleaks
```

SE UM SEGREDO VAZOU: A ORDEM IMPORTA

1 Revogue e rotacione primeiro
Invalide a credencial agora. Reescrever o histórico não desfaz o vazamento: há cópias, *forks*, caches e robôs que varrem repositórios públicos em minutos.

2 Avalie o estrago
Consulte os logs de auditoria do provedor: a credencial foi usada? O que foi acessado? Se houve uso indevido, trate como incidente (**p. 156**).

3 Substitua e proteja
Emita uma credencial nova, de menor escopo, no gerenciador de segredos e remova o valor do código e dos arquivos de configuração.

4 Só então limpe o histórico
Use `git filter-repo` (ou BFG), reescreva as branches, peça a remoção de caches e PRs e avise quem clonou para clonar de novo.

15 CADEIA DE SUPRIMENTOS E DEVSECOPS — CONTINUAÇÃO

TÉCNICAS DE TESTE AUTOMATIZADO NO PIPELINE

Nenhuma técnica acha tudo: cada uma enxerga o sistema por um ângulo e entra em um ponto do pipeline. O valor está em **combiná-las** e em tratar bem os resultados, não em ligar todas as ferramentas. Os testes manuais e o *pentest* ficam em p. 166.

TÉCNICA (EXEMPLOS)	O QUE ENCONTRA	QUANDO RODA	LIMITES
SAST (Semgrep, CodeQL, SonarQube)	Padrões inseguros no código: injeções, funções perigosas, segredos	A cada PR	Falsos positivos; não vê configuração nem execução
SCA (Dependabot, Renovate, OSV-Scanner)	Dependências com CVE conhecida e problemas de licença	A cada PR e todo dia	Só falhas já publicadas; ruído sem análise de alcance
DAST (OWASP ZAP)	Falhas visíveis na aplicação em execução: cabeçalhos, injeções, configuração	Em *staging*	Cobertura depende do rastreamento e da autenticação
IAST	Agente dentro da aplicação liga a entrada ao ponto perigoso durante os testes	Nos testes de QA	Exige instrumentação; só vê o que os testes exercitam
RASP	Bloqueia ataques em tempo de execução, de dentro da aplicação	Em produção	Custo e falsos positivos; não substitui a correção
Fuzzing (AFL++, libFuzzer, OSS-Fuzz)	Travamentos, estouros e falhas de lógica em *parsers* e protocolos	Contínuo ou noturno	Exige *harness*; melhor para código que lê entrada bruta (p. 122)
Contêineres e IaC (Trivy, Grype, Checkov, tfsec, KICS)	Pacotes vulneráveis na imagem; configurações inseguras de nuvem	No build e no PR de infraestrutura	Regras genéricas; ajuste à política da casa

A **triagem** é onde muitos programas se perdem. Boas práticas: *quality gates* que reprovam o build só para achados **novos** de severidade alta ou crítica (com uma *baseline* para o legado); priorização por **exploração real** (CISA KEV e EPSS, ver p. 20) e pelo contexto (exposto à internet? dados sensíveis?); falsos positivos suprimidos **com justificativa e prazo**, no próprio código (por exemplo, com um comentário `nosemgrep`), e não em silêncio; e métricas de **tempo de correção**, não de quantidade de alertas.

Infraestrutura como código (IaC, como Terraform) permite revisar e testar a configuração da nuvem como se revisa código; as **políticas como código** (OPA/Rego, Conftest, Kyverno) expressam regras do tipo "nenhum contêiner roda como root" em arquivos versionados, aplicados no CI e na admissão do cluster (p. 78).

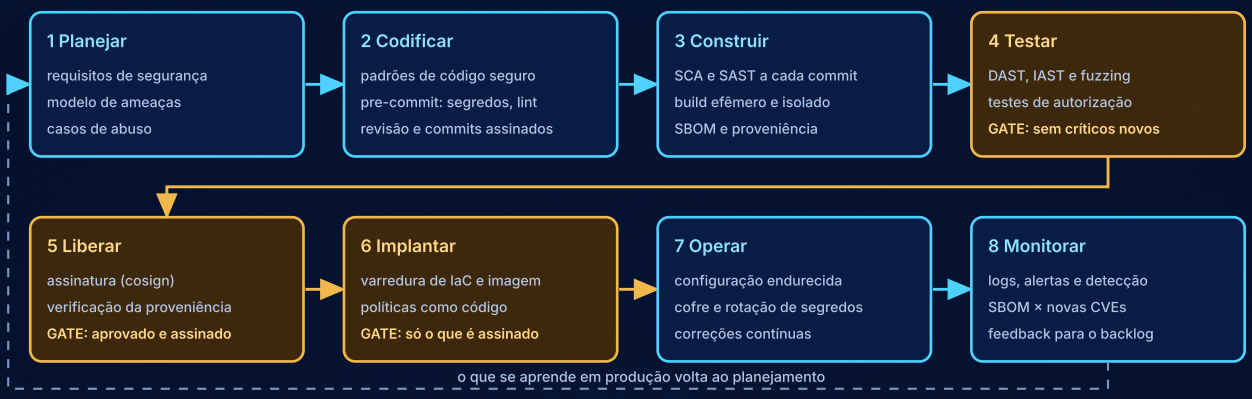
IAC: ERRO QUE O CHECKOV, O TFSEC E O KICS ACUSAM (-) E CORREÇÃO (+)

```
 resource "aws_security_group_rule" "ssh" {
   type              = "ingress"
   from_port         = 22
   to_port           = 22
   protocol          = "tcp"
-  cidr_blocks       = ["0.0.0.0/0"]          # SSH aberto para a internet inteira
+  cidr_blocks       = ["10.20.0.0/16"]       # só a rede da VPN da faculdade
   security_group_id = aws_security_group.app.id
 }
```

15 CADEIA DE SUPRIMENTOS E DEVSECOPS — CONTINUAÇÃO

O PIPELINE DEVSECOPS, ESTÁGIO POR ESTÁGIO

DevSecOps é a prática de integrar controles de segurança ao ciclo de entrega, automatizados e com retorno rápido, com responsabilidade compartilhada entre desenvolvimento, operações e segurança, em vez de uma auditoria no fim (o chamado *shift left*, levar a segurança para as etapas iniciais). A figura mostra os controles por estágio; os *gates* são os pontos em que o pipeline pode **parar** a entrega.



Pipeline DevSecOps. Os controles estão distribuídos pelo ciclo, e as caixas douradas têm *gates*: pontos em que o pipeline **para** a entrega se o critério não for atendido. A seta tracejada fecha o laço: o que se descobre em produção alimenta o planejamento.

Para funcionar, um *gate* precisa ser **rápido** (minutos), **confiável** (poucos falsos positivos) e **explicável** (a mensagem diz como corrigir). *Gates* lentos ou ruidosos são contornados, e o controle vira teatro. A responsabilidade pela correção continua sendo de quem escreveu o código; a equipe de segurança fornece ferramentas, padrões e a revisão das exceções (p. 90).

EXEMPLO RESOLVIDO

Sexta, 17h: a CVE crítica de uma biblioteca de logs. Foi divulgada uma execução remota de código em uma biblioteca usada pela empresa. O roteiro da primeira hora:

1. **Inventário**: consulte os SBOM e o SCA. Em quais serviços a biblioteca aparece, em que versão, como dependência direta ou transitiva? (Minutos, se há SBOM; dias, se não há.)
2. **Exposição**: o código vulnerável é alcançável? Serviços expostos à internet e que processam dados de usuários vêm primeiro; registre um VEX para os que não são afetados.
3. **Mitigação**: atualize para a versão corrigida. Se não for possível agora, aplique a mitigação indicada pelo fornecedor (configuração, regra no WAF) e registre a exceção com prazo.
4. **Verificação**: reconstrua, rode os testes e confirme, pela varredura da **imagem implantada**, que a versão vulnerável saiu de produção, e não só do repositório.
5. **Sinais de exploração**: procure nos logs desde a divulgação; se houver indício, rotacione os segredos do serviço e acione o plano de incidentes (p. 156).
6. **Aprendizado**: o que atrasou? Faltou inventário, dono do serviço, pipeline rápido? Ajuste o processo antes da próxima.

15 CADEIA DE SUPRIMENTOS E DEVSECOPS — CONTINUAÇÃO

CHECKLIST

Cadeia de suprimentos: o mínimo para qualquer equipe

- Lockfiles versionados e instalação reprodutível no CI (`npm ci`, `--require-hashes`).
- SCA ligado a cada PR, com atualização automática e uma pessoa responsável pelos alertas.
- SBOM gerado no build e guardado junto ao artefato.
- *Actions* e imagens fixadas por SHA ou digest; `permissions:` mínimas; OIDC no lugar de chaves longas.
- Branch protegida, revisão obrigatória e MFA resistente a phishing para quem publica.
- Varredura de segredos no *pre-commit* e no CI, com *push protection* ligada.
- Artefatos assinados e verificados na implantação, exigindo a identidade esperada.
- Dono definido para cada serviço e um roteiro ensaiado para a CVE crítica de sexta, às 17h.

PARA FIXAR

Conceitos

- Diferencie *typosquatting*, *dependency confusion* e *slopsquatting* e dê uma defesa para cada um.
- O que um SBOM informa e o que ele **não** garante? Como o VEX o complementa?
- Que garantias o SLSA Build L2 dá e o que só aparece no L3? Por que verificar **quem** assinou importa mais do que verificar **se** há assinatura?

Aplicação

- Um workflow usa uma *action* por tag e mantém as permissões padrão. Liste três mudanças para endurecê-lo e o ataque que cada uma evita.
- Uma chave de API foi enviada ao GitHub às 10h. Escreva, em ordem, o que fazer até as 11h e explique por que limpar o histórico **não** é o primeiro passo.
- Projete o pipeline de uma API em Python: que ferramentas entram em cada estágio, quais *gates* reprovam o build e quais achados só geram tarefas?

APROFUNDE-SE

Leituras e referências

- SLSA — OpenSSF: níveis, trilha de Build e proveniência.
- Sigstore — cosign, Fulcio e Rekor, com guias de assinatura sem chave.
- OpenSSF Scorecard — pontuação automática dos hábitos de segurança de projetos de código aberto.
- NIST SP 800-218, Secure Software Development Framework — NIST: práticas de desenvolvimento seguro que sustentam as exigências de cadeia de suprimentos.
- Widespread Supply Chain Compromise Impacting npm Ecosystem — CISA, 2025: um caso real e as mitigações recomendadas.
- OWASP Top 10:2025 — A03, Software Supply Chain Failures.

MÓDULO 16 · OPERAÇÕES

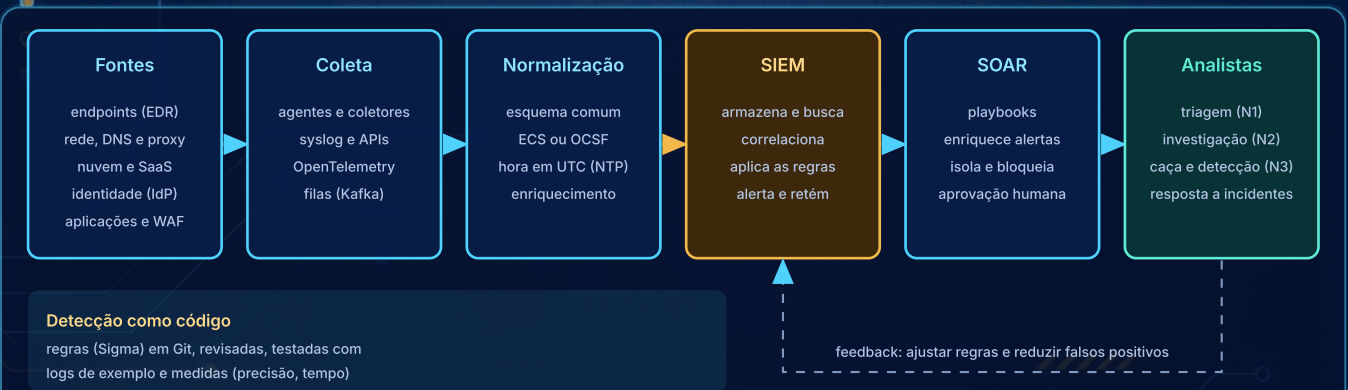
16 MONITORAMENTO E DETECÇÃO

Você não protege o que não enxerga: telemetria, correlação, detecção e inteligência de ameaças.

Nenhuma defesa é perfeita: contas são roubadas, dependências falham e pessoas erram. Por isso, além de **prevenir**, um sistema seguro precisa **enxergar** o que acontece, **reconhecer** o que é anormal e **avisar** quem pode agir, a tempo. O módulo trata do que registrar e como (logs), da arquitetura de telemetria (SIEM, SOAR, EDR, NDR), do trabalho de um **SOC**, da **engenharia de detecção** (Sigma, YARA, ATT&CK e métricas), da inteligência de ameaças e da caça (*threat hunting*).

PREVENIR NÃO BASTA: O PAPEL DA DETECÇÃO

A premissa **assume breach** (parta do princípio de que haverá uma violação), adotada pelo *Zero Trust*, troca a pergunta "como impedir toda invasão?" por "**quanto tempo levamos para perceber e conter uma?**". O intervalo pesa: o relatório de custo de violações da IBM (edição de 2026) estima em cerca de **247 dias**, em média, o tempo para identificar e conter uma violação, e cada dia a mais dá a quem ataca tempo para escalar privilégios, copiar dados e apagar rastros. Sem **registros** e **alertas**, a equipe não detecta, não investiga e não consegue provar o que aconteceu.



Da telemetria ao analista. Os eventos são coletados, normalizados para um esquema comum e correlacionados no SIEM; os alertas seguem para a automação (SOAR) e para as pessoas analistas, cujo retorno ajusta as regras. Sem normalização e sem relógios sincronizados, as fontes não conversam entre si.

Em resumo: Três perguntas organizam o módulo. **Temos os dados?** (logs e telemetria de qualidade, de todas as fontes que importam). **Sabemos reconhecer o anormal?** (detecções testadas, mapeadas ao ATT&CK e medidas). **Alguém age a tempo?** (severidade, *runbooks*, automação e pessoas de plantão).

O QUE REGISTRAR: EVENTOS QUE CONTAM UMA HISTÓRIA

Registrar tudo é caro e produz ruído; registrar pouco impede a investigação. O critério do OWASP (*Logging Cheat Sheet*) é guardar o que responde a **quem** fez **o quê**, **quando**, **de onde** e com **qual resultado**, em especial nos eventos de segurança. A tabela lista as categorias mínimas de uma aplicação.

16 MONITORAMENTO E DETECÇÃO — CONTINUAÇÃO

EVENTO	EXEMPLOS	POR QUE IMPORTA
Autenticação	Login com sucesso e falha, MFA exigido ou negado, troca e redefinição de senha, bloqueio de conta	Base para detectar *password spraying*, *credential stuffing* e fadiga de MFA
Falhas de autorização	Respostas 403 e 404 em sequência, acesso negado ao registro de outra pessoa	Sinal de enumeração e de BOLA/IDOR em andamento
Ações administrativas	Criação de usuário, mudança de papel e de permissões, uso de conta privilegiada	Persistência e escalada de privilégio quase sempre passam por aqui
Falhas de validação	Entrada rejeitada, assinatura de token inválida, violação de CSP, *payload* fora do esquema	Revelam tentativas de injeção e de abuso da API
Acesso a dados sensíveis	Leitura ou exportação em massa, consulta a dados pessoais, download de relatórios	Prova quem acessou o quê, como pedem a LGPD e as auditorias
Mudanças de configuração	Regras de firewall, políticas de *bucket*, *feature flags*, chaves e segredos	Todo desvio precisa de autor e de horário
Estado dos controles	Agente de EDR parado, auditoria desativada, log apagado ou fora do ar	Atacantes **desligam** a visibilidade antes de agir (ATT&CK T1562, *Impair Defenses*)



LOG SEM ALERTA É DIÁRIO, NÃO DEFESA

Na edição 2025 do OWASP Top 10, a categoria A09 passou a se chamar *Security Logging and Alerting Failures*: a palavra *Alerting* substituiu *Monitoring*, porque guardar eventos que ninguém lê não detecta nada. As fraquezas associadas são CWE-117 (neutralização incorreta de saída para logs), CWE-532 (dados sensíveis em logs) e CWE-778 (registro insuficiente).

HIGIENE DE LOGS: FORMATO, TEMPO, INTEGRIDADE E DESTINO

Um log só serve à segurança se for **confiável** e **utilizável**. As práticas abaixo seguem o OWASP Logging Cheat Sheet e o NIST SP 800-92 e valem como padrão para qualquer sistema.

Formato e conteúdo

- **JSON estruturado**, um evento por linha, com campos fixos (`ts`, `evt`, `outcome`, `user_id`, `src_ip`, `req_id`), em vez de texto livre
- **UTC** em ISO 8601, com relógios sincronizados por **NTP**; sem hora e fuso, não há linha do tempo
- **ID de correlação** (`req_id`, `trace_id`) propagado entre os serviços
- **Sem segredos nem dados pessoais desnecessários**: nunca senha, token, chave ou cartão; use identificadores internos e mascare o resto
- **Neutralize** a entrada antes de gravar, contra *log injection* (**p. 122**)

Proteção e ciclo de vida

- **Centralize** em segundos: o invasor que apaga o log do próprio servidor não deve apagar a cópia
- **Integridade**: armazenamento de escrita única (WORM), *hashes* encadeados ou assinatura; quem administra o servidor não reescreve o log
- **Acesso restrito** e auditado: logs também têm dados sensíveis
- **Retenção** definida por finalidade e lei: por exemplo, 6 meses para registros de acesso a aplicações no Marco Civil (quando aplicável) e 12 meses no PCI DSS
- **Teste** o caminho de ponta a ponta: alerta que nunca foi disparado em teste pode não funcionar

16 MONITORAMENTO E DETECÇÃO — CONTINUAÇÃO

O MESMO EVENTO, MAL E BEM REGISTRADO

```
- 19/10 14:03 login falhou para joao.silva@exemplo.com senha=Abc12345 ip 203.0.113.7
+ {"ts":"2026-10-19T17:03:11.482Z","evt":"auth.login","outcome":"failure",
+  "reason":"bad_password","user_id":"u_8841","src_ip":"203.0.113.7",
+  "req_id":"7f3c9a","svc":"portal-api","ver":"1.14.2","sev":"warn"}
```

A primeira linha não tem ano nem fuso, mistura e-mail e **senha** em texto livre e não pode ser consultada por campo. A versão estruturada traz o horário em UTC, o motivo da falha, um identificador interno no lugar do e-mail e o `req_id` que liga este evento aos demais do mesmo pedido. A faixa 203.0.113.0/24 é reservada para documentação (RFC 5737).

Em código, o mais seguro é **não montar texto à mão**: a mensagem é um **nome de evento fixo** e os dados variáveis vão em campos, que o formatador serializa em JSON (o que também escapa quebras de linha e neutraliza a *log injection*). O ID de correlação vem de um contexto definido uma vez, no *middleware* da requisição.

LOG ESTRUTURADO EM PYTHON: EVENTO FIXO, CAMPOS E REDAÇÃO DE SEGREDOS

```python
import json, logging, contextvars
from datetime import datetime, timezone

req_id = contextvars.ContextVar("req_id", default="-")   # definido no middleware
SENSIVEIS = {"password", "senha", "token", "authorization", "cpf"}

class JsonFormatter(logging.Formatter):
    def format(self, rec: logging.LogRecord) -> str:
        campos = getattr(rec, "campos", {})
        evento = {
            "ts": datetime.fromtimestamp(rec.created, timezone.utc).isoformat(),
            "sev": rec.levelname.lower(),
            "evt": rec.getMessage(),          # nome fixo; nunca texto vindo do usuário
            "req_id": req_id.get(),
            **{k: "[REDACTED]" if k in SENSIVEIS else v for k, v in campos.items()},
        }
        return json.dumps(evento, ensure_ascii=False, default=str)   # uma linha

log = logging.getLogger("portal")
handler = logging.StreamHandler()
handler.setFormatter(JsonFormatter())
log.addHandler(handler)
log.setLevel(logging.INFO)

# uso: o ator e o motivo vão em campos; a mensagem é só o nome do evento
log.warning("auth.login", extra={"campos": {"outcome": "failure",
            "user_id": "u_8841", "reason": "bad_password", "src_ip": ip}})
```

Uma lista de nomes sensíveis (*denylist*) é uma rede de segurança, não a defesa principal: ela falha diante de um campo que ninguém previu. O hábito mais forte é **escolher explicitamente** o que entra no evento (*allowlist*) e revisar, em code review, qualquer log que receba um objeto inteiro de requisição ou de usuário.

FONTES DE TELEMETRIA: O QUE CADA UMA MOSTRA E ONDE FALHA

Nenhuma fonte vê tudo. A detecção forte combina fontes que se **complementam**: a identidade mostra quem entrou, o *endpoint* mostra o que a máquina fez, a rede mostra com quem ela falou e a nuvem mostra quem alterou a infraestrutura.

16 MONITORAMENTO E DETECÇÃO — CONTINUAÇÃO

FONTE	O QUE MOSTRA	LIMITES
***Endpoint* e EDR**	Processos e linhas de comando, arquivos, registro e conexões: o que a máquina **fez**	Só onde há agente; exige ajuste para não gerar volume excessivo
Sistema operacional	Windows Event Log (4624 logon, 4688 novo processo, 4732 grupo local, 1102 log apagado); `auditd` e `journald` no Linux	Auditoria precisa ser habilitada; a linha de comando só aparece com política específica
Identidade (IdP)	Logins, MFA, tokens emitidos, mudanças de papel, origem e dispositivo	Não mostra o que acontece **depois** do login, dentro das aplicações
Rede	Fluxos (NetFlow, IPFIX), DNS, proxy, firewall, VPN, metadados do TLS	O conteúdo cifrado fica opaco; a visão termina onde não há sensor
Nuvem	Plano de controle (CloudTrail, Azure Activity Log, Cloud Audit Logs): quem criou, alterou ou apagou recursos	Eventos de **dados** (leitura de objetos) em geral vêm desligados e custam à parte
Aplicação e API	Eventos de negócio e de segurança: autenticação, autorização, transações, erros	Só existe se a equipe de desenvolvimento **instrumentar** o código
WAF, CDN e balanceador	Requisições bloqueadas, assinaturas disparadas, país de origem, *bots*	Vê só a borda; regra em modo "apenas registrar" gera muito ruído
Banco de dados	Consultas privilegiadas, exportações em massa, mudanças de esquema	A auditoria completa pesa no desempenho: registre o que for sensível

SIEM, SOAR, EDR, NDR, XDR E UEBA: FUNÇÕES, NÃO PRODUTOS

Essas siglas descrevem **funções**, que os produtos reais combinam de formas diferentes. A tabela compara o papel de cada uma pela pergunta que ela responde.

SIGLA	PERGUNTA QUE RESPONDE	DADO PRINCIPAL	LIMITE
SIEM	O que aconteceu no ambiente todo, e esses eventos juntos formam um ataque?	Logs de todas as fontes, normalizados	O custo cresce com o volume; vale o que valem as regras
SOAR	Como executar, com rapidez e padrão, as respostas repetitivas?	Alertas e APIs das ferramentas	Automatizar um processo ruim só acelera o erro
EDR	O que esta máquina fez e como contê-la?	Telemetria detalhada do *endpoint*	Só vê onde há agente e o que ele registra
NDR	O que trafega pela rede, inclusive entre servidores?	Fluxos, tráfego espelhado, metadados	O conteúdo cifrado é opaco; exige sensores bem posicionados
XDR	Sinais de produtos diferentes contam a mesma história?	Endpoint, e-mail, identidade e nuvem	Costuma ser mais completo dentro de um só fabricante
UEBA	Isso é normal para esta conta ou máquina?	*Baseline* de comportamento por entidade	Gera falso positivo até aprender; exige histórico bom

16 MONITORAMENTO E DETECÇÃO — CONTINUAÇÃO

Um **SIEM** (*Security Information and Event Management*) executa seis funções: **coleta**, **normalização**, **correlação**, **alertas**, **painéis e busca** e **retenção**. A normalização é a mais subestimada: sem um esquema comum, "usuário" é `user`, `username` ou `Account_Name` conforme a fonte, e nenhuma regra funciona entre fontes. Por isso existem esquemas abertos, como o **ECS** (*Elastic Common Schema*) e o **OCSF** (*Open Cybersecurity Schema Framework*), e padrões de instrumentação como o **OpenTelemetry**. Já o **SOAR** (*Security Orchestration, Automation and Response*) executa os *playbooks*: enriquece o alerta (reputação do IP, dono da máquina), abre o chamado, isola o *endpoint* ou bloqueia a conta, e pede **aprovação humana** nas ações de maior impacto.

Alguns exemplos, sem endosso: Wazuh, Elastic Security, OpenSearch, Graylog e Splunk atuam como SIEM ou plataforma de busca de logs; Zeek e Suricata geram metadados e alertas de rede. Wazuh, Zeek e Suricata são de código aberto e bastam para montar um laboratório de estudo.

ENGANO E ANOMALIA: HONEYTOKENS, HONEYPOTS E UEBA

A **deception** (engano defensivo) planta iscas que nenhuma pessoa legítima tem motivo para tocar. Como não há uso normal, **qualquer acesso é alerta de alta fidelidade**, com pouquíssimos falsos positivos.

01 HONEYPOT

Sistema falso, exposto de propósito (um servidor SSH, um banco de aparência real), para atrair e estudar tentativas de ataque. Qualquer conexão é suspeita. Mantenha-o **isolado** da produção: uma isca mal contida vira ponto de entrada.

02 HONEYTOKEN

Dado-isca: uma chave de API falsa em um repositório, um registro fictício em um banco, uma planilha "senhas.xlsx", um nome DNS interno. Quando **usado**, dispara o alerta. É barato e funciona também em nuvem e em código.

03 UEBA

Aprende o comportamento típico de cada conta e máquina (horário, volume, destinos, localização) e sinaliza desvios. Complementa as regras fixas, mas precisa de histórico e de ajuste contínuo.

O SOC: PESSOAS, TURNOS E PROCESSOS

O **SOC** (*Security Operations Center*) é a função que monitora, faz a triagem, investiga e responde de forma contínua. Pode ser interno, terceirizado (**MSSP** ou **MDR**, *Managed Detection and Response*) ou híbrido; em organizações pequenas, a própria equipe de TI acumula o papel. O essencial é que alguém seja **responsável por olhar**, em horário definido (24×7 quando o risco exige), com regras de escalonamento e **runbooks**, os procedimentos por tipo de alerta.

N1 TRIAGEM

Recebe os alertas, confirma se são reais, enriquece com contexto (usuário, ativo, reputação) e **fecha ou escala** seguindo o *runbook*. Pede disciplina e registro, não palpite.

N2 INVESTIGAÇÃO

Aprofunda os casos escalados: reconstrói a linha do tempo, delimita o escopo, contém o dano e aciona a resposta a incidentes (**p. 156**).

N3 ESPECIALISTAS

Caça ameaças, faz **engenharia de detecção** (cria, testa e ajusta regras), análise forense e de *malware* (DFIR) e realimenta os níveis anteriores.

Todo alerta precisa de dono e de saída: Para cada regra, deixe escrito: o que ela significa, que dados consultar primeiro, quando escalar, quem decide e como **fechar**. Alerta sem *runbook* depende da memória de quem estiver de plantão, e é essa a pessoa que falha às 3h.

16 MONITORAMENTO E DETECÇÃO

CONTINUAÇÃO

ENGENHARIA DE DETECÇÃO: DETECÇÃO COMO CÓDIGO

Engenharia de detecção é tratar cada detecção como um produto de software: nasce de um **requisito** (um risco ou uma técnica do ATT&CK), é escrita em formato padronizado, guardada em Git, revisada por outra pessoa, **testada** com eventos de exemplo, implantada por pipeline, medida e, quando deixa de servir, aposentada. A abordagem de **detecção como código** (*detection as code*) dá histórico, revisão e reprodutibilidade, e evita a regra "mágica" que ninguém sabe por que existe.



O **Sigma** é o formato aberto e genérico para regras sobre logs, escrito em YAML: descreve **o que procurar** sem prender a regra a um produto. Conversores (como o pySigma e o `sigma-cli`) traduzem a mesma regra para a linguagem de cada SIEM, e o projeto SigmaHQ mantém um acervo público grande. Uma regra tem metadados (título, `id`, estado, autor, `tags` do ATT&CK), a origem do log (`logsource`), a lógica (`detection`), os **falsos positivos** conhecidos e o nível de severidade.

REGRA SIGMA: CONTA ADICIONADA AO GRUPO LOCAL DE ADMINISTRADORES (WINDOWS)

```
title: Usuário adicionado ao grupo local de administradores
id: 5b3c1a2e-7d41-4c0a-9a55-0e6f1d2c8b17     # UUID de exemplo
status: experimental
description: Detecta a inclusão de uma conta no grupo Administradores (SID -544)
author: Equipe de detecção (exemplo)
date: 2026-10-19
tags:
  - attack.persistence
  - attack.t1098
logsource:
  product: windows
  service: security
detection:
  selection:
    EventID: 4732           # um membro foi adicionado a um grupo local com segurança
    TargetSid|endswith: '-544'   # grupo interno Administradores
  condition: selection
falsepositives:
  - Provisionamento autorizado, com chamado de mudança correspondente
level: high
```

Leia a regra de cima para baixo: nos logs de segurança do Windows (`logsource`), procure eventos 4732 cujo grupo-alvo seja o dos administradores (`selection`); se houver, alerte com severidade alta. O modificador `|endswith` evita depender do **nome** do grupo, que muda com o idioma do sistema, e usa o **SID** bem conhecido (-544). O alerta só fica útil se a equipe souber distinguir o provisionamento legítimo, e por isso o campo `falsepositives` pertence à regra.

DICA DA CORUJA

Regra sem teste é palpite. Guarde junto com cada regra os eventos de exemplo que **devem** disparar e os que **não** devem, e rode esses casos no pipeline a cada alteração. Assim, um ajuste para reduzir ruído não apaga a detecção sem que ninguém perceba.

16 MONITORAMENTO E DETECÇÃO — CONTINUAÇÃO

YARA E REGRAS DE REDE: PADRÕES EM ARQUIVOS E NO TRÁFEGO

O **YARA** descreve padrões em **arquivos e na memória** (cadeias de texto, sequências de bytes, expressões regulares e condições lógicas). É a ferramenta clássica de quem classifica *malware*, mas serve também a usos defensivos simples, como procurar **segredos esquecidos** em arquivos, ligando este módulo ao de cadeia de suprimentos (p. 135).

REGRA YARA BENIGNA: ID DE CHAVE DE ACESSO AWS ESQUECIDO EM UM ARQUIVO

```
rule Chave_AWS_exposta
{
  meta:
    description = "Procura ID de chave de acesso AWS em arquivos de texto"
    author      = "Equipe de segurança (exemplo)"
    severidade  = "alta"
  strings:
    $id = /(AKIA|ASIA)[0-9A-Z]{16}/ ascii
  condition:
    filesize < 2MB and $id
}
```

Teste a regra com um valor sintético: a documentação da AWS usa `AKIAIOSFODNN7EXAMPLE` como exemplo, que casa com o padrão **sem expor chave real**. No tráfego de rede, o **Suricata** e o **Snort** usam regras no formato `ação protocolo origem -> destino (opções)`, em que as opções incluem uma mensagem (`msg`), o padrão procurado no conteúdo (`content`), campos de protocolo (como `http.uri`) e um identificador (`sid`). Rodam em tempo real e, no modo IPS, podem **descartar** o pacote (p. 67). O **Zeek** não usa assinaturas: gera registros ricos por protocolo (`conn.log`, `dns.log`, `http.log`, `ssl.log`), ótimos para busca e caça.

A PIRÂMIDE DA DOR: QUE INDICADOR VALE DETECTAR

Em 2013, David Bianco propôs a **Pirâmide da Dor** (*Pyramid of Pain*) para ordenar os indicadores pelo **custo que impõem a quem ataca** quando são detectados. Trocar um *hash* exige só recompilar o arquivo; trocar um IP ou um domínio custa pouco; mudar artefatos (um *User-Agent*, um padrão de URL) já incomoda; trocar de **ferramenta** obriga a achar ou escrever outra; e mudar de **TTP** significa mudar o modo de trabalhar.



PARA QUEM ATACA	O QUE SE DETECTA
difícil	comportamento: coleta de credenciais, movimento lateral
desafiador	assinaturas de ferramentas e dos seus artefatos
incômodo	User-Agent, chaves de registro, nomes de arquivo
simples	bloqueio por DNS e reputação
fácil	listas de bloqueio
trivial	hashes de malware conhecido

Pirâmide da Dor (*Pyramid of Pain*, David Bianco, 2013). Quanto mais alto o nível do indicador detectado, mais custa a quem ataca mudar de método. Listas de hashes e IPs envelhecem em dias; detectar **comportamento** (TTPs) dura bem mais.

16 MONITORAMENTO E DETECÇÃO — CONTINUAÇÃO

A consequência prática: listas de IoCs (*hashes*, IPs, domínios) são **baratas e úteis**, mas envelhecem em dias e se contornam com facilidade. A engenharia de detecção madura mira no topo: "um documento do Office que abre o PowerShell", "uma conta nova que entra em grupo privilegiado", "um log apagado". É comportamento, e o adversário que o troca precisa reaprender a atacar.

ATT&CK: MAPEAR, MEDIR COBERTURA E EVITAR A ILUSÃO

Cada regra recebe a tag da técnica do ATT&CK que cobre (por exemplo, `attack.t1110.003` para *password spraying*, ver p. 11). Com as tags, a equipe mostra a cobertura em um mapa de calor (o **ATT&CK Navigator**), prioriza as lacunas segundo os adversários **relevantes** para a organização (*threat-informed defense*) e acompanha a evolução. Há três cuidados. **Cobertura não é qualidade**: ter uma regra para a técnica não significa detectar todas as variantes. **Cobertura se mede por teste**, com emulação de adversário em ambiente controlado (Atomic Red Team e MITRE Caldera são exemplos, ver p. 166), e não por declaração. E **sem a fonte de log certa**, a regra não dispara: a lacuna costuma estar na telemetria, não na lógica.

DETECÇÕES DE EXEMPLO E O RACIOCÍNIO POR TRÁS DELAS

Toda boa detecção responde a quatro perguntas: **que sinal observável** a revela, **que fonte de log** o contém, **que atividade legítima** pode imitá-la e **qual técnica** do ATT&CK ela cobre. A tabela aplica isso a seis casos comuns.

DETECÇÃO	SINAL OBSERVÁVEL	FALSO POSITIVO COMUM	ATT&CK
Password spraying	De uma mesma origem, falhas de senha em muitas contas diferentes, poucas por conta, em janela curta	Rede de campus ou VPN com NAT: muitas pessoas legítimas atrás do mesmo IP	T1110.003
Novo administrador	Inclusão em grupo privilegiado (eventos 4732 e 4728; papel *admin* na nuvem) sem chamado de mudança	Provisionamento autorizado e rotinas de RH	T1098
Viagem impossível	Dois logins bem-sucedidos da mesma conta em locais que não se percorrem no intervalo entre eles	VPN, *proxies* corporativos e geolocalização de IP imprecisa	T1078
Log desativado ou apagado	Evento 1102 (log de segurança limpo), 4719 (política de auditoria alterada) ou `StopLogging` na nuvem	Manutenção planejada	T1070.001 e T1562.008
Exfiltração anormal	Volume de saída muito acima do *baseline* da conta ou do host, ou envio para destino raro (nuvem pessoal)	*Backups*, migrações e cargas em lote legítimas	T1048 e T1567
Fadiga de MFA	Várias notificações *push* negadas ou ignoradas, seguidas de uma aprovação	Aplicativo travado ou pessoa distraída	T1621

DETECÇÃO POR LIMIAR EM PSEUDO-SQL: PASSWORD SPRAYING

```
-- Poucas senhas contra muitas contas, a partir da mesma origem, em janela curta
SELECT src_ip,
       COUNT(DISTINCT user_id) AS contas_alvo,
       COUNT(*)                AS falhas
FROM   auth_events
WHERE  outcome = 'failure'
  AND  reason  = 'bad_password'
  AND  ts >= NOW() - INTERVAL '10 minutes'
GROUP  BY src_ip
HAVING COUNT(DISTINCT user_id) >= 15                -- muitas contas diferentes
   AND COUNT(*) <= 3 * COUNT(DISTINCT user_id);     -- e poucas tentativas por conta
```

16 MONITORAMENTO E DETECÇÃO — CONTINUAÇÃO

É pseudocódigo: cada SIEM usa a sua linguagem (SPL, KQL, EQL, ES|QL), mas a lógica é a mesma, e os **limiares** (15 contas, 10 minutos) devem ser calibrados com os dados reais da organização. O falso positivo típico, o NAT do campus, se trata com uma **lista de exceção** (com dono e prazo) e com o reforço de um segundo sinal, como a falha seguida de um sucesso vindo do mesmo IP.

FALSOS POSITIVOS, FALSOS NEGATIVOS E FADIGA DE ALERTAS

Um **falso positivo** é um alerta sem ameaça; um **falso negativo** é uma ameaça sem alerta. O primeiro desperdiça tempo, o segundo é o perigoso, porque **ninguém sabe que aconteceu**. Dois números resumem o equilíbrio: a **precisão** (dos alertas, quantos eram reais: VP ÷ (VP + FP)) e a **sensibilidade** (das ameaças, quantas geraram alerta: VP ÷ (VP + FN)). Endurecer uma regra para reduzir ruído tende a baixar a sensibilidade, e por isso o ajuste exige teste.

A **fadiga de alertas** é o efeito humano do excesso de ruído: quem recebe centenas de alertas inúteis passa a ignorar todos, inclusive o verdadeiro. É um risco de segurança, e não apenas de produtividade. As defesas são técnicas e de processo: **agrupar** alertas repetidos, **enriquecer** com contexto antes de chegar à pessoa, graduar a severidade pela **criticidade do ativo**, manter exceções com prazo e **aposentar** regras que nunca geram ação.

EXEMPLO RESOLVIDO

Medindo e ajustando uma regra de beaconing. Em um mês, a regra disparou 120 alertas; depois da triagem, 6 eram reais.

1. **Precisão** = 6 ÷ 120 = **5%**. Com 20 minutos por alerta, foram 40 horas para achar 6 casos.
2. **Ajuste:** exigir 30 conexões em 1 hora com intervalo regular, excluir os destinos de atualização do inventário e elevar a severidade só quando o destino for raro.
3. **Resultado no mês seguinte:** 18 alertas, 5 reais. Precisão = 5 ÷ 18 ≈ **28%**, 6 horas de triagem, e 5 dos 6 casos originais preservados (sensibilidade relativa ≈ 83%).
4. **Decisão:** o caso que sumiu é analisado. Se o ajuste o perdeu, cria-se uma regra complementar e **testa-se** com emulação, em vez de aceitar a perda sem saber.

MÉTRICAS E SEVERIDADE: COMO SABER SE A DETECÇÃO FUNCIONA

MÉTRICA	DEFINIÇÃO	USO E CUIDADO
MTTD	Tempo médio entre o início da atividade e a **detecção**	Mede a eficácia da detecção; prefira a **mediana**, menos sensível a casos extremos
MTTA	Tempo entre o alerta e alguém **assumi-lo**	Mede prontidão e dimensionamento de turnos
MTTC e MTTR	Tempo até **conter** e até **recuperar** (a sigla MTTR varia: *respond*, *recover*, *remediate*)	Defina o significado no seu contexto, ou a série perde comparação
Tempo de permanência (*dwell time*)	Do comprometimento inicial até a detecção ou a contenção	O ponto de partida só é conhecido depois, na investigação
Precisão e razão alerta→incidente	Fração dos alertas que eram reais; quantos alertas viram incidentes	Por regra, para **podar** as ruidosas; global, para medir a saúde do pipeline
Cobertura	% de ativos críticos com telemetria saudável; % das técnicas relevantes com detecção **testada**	Aponta pontos cegos; saúde da coleta também é um alerta

As metas são **suas**: nenhum número serve a todos. Defina-as por severidade e por capacidade, e as revise. Um modelo de severidade, ligado ao tempo de resposta, evita decidir tudo na hora do susto.

16 MONITORAMENTO E DETECÇÃO — CONTINUAÇÃO

SEVERIDADE	CRITÉRIO	RESPOSTA ESPERADA (EXEMPLO)
Crítica	Dano em curso a ativo ou dados críticos (*ransomware* executando, exfiltração confirmada)	Abrir incidente **agora**, acionar plantão 24×7 e liderança
Alta	Alta chance de comprometimento (novo administrador inesperado, EDR bloqueou código malicioso)	Triagem imediata e investigação no mesmo turno
Média	Comportamento suspeito sem indício de dano (varredura, falhas de login em excesso)	Análise em até um dia útil; agrupar com casos parecidos
Baixa	Anomalia ou política violada de baixo impacto	Fila de rotina, tendência e ajuste de regra

Caso Target (2013): o alerta existiu, a ação não: Na invasão da varejista Target, que expôs dados financeiros e pessoais de até 110 milhões de consumidores, segundo o Senado dos EUA, o *software* de detecção emitiu avisos automáticos sobre a instalação de *malware* e sobre a rota de saída dos dados. O relatório do Senado (2014) concluiu que a empresa **não respondeu a eles a tempo**, e o ataque seguiu. A lição não é "compre mais ferramentas": detecção sem **processo, responsável e autoridade para agir** não reduz o dano.

INTELIGÊNCIA DE AMEAÇAS: TRANSFORMAR DADOS EM DECISÃO

Inteligência de ameaças (*Cyber Threat Intelligence*, CTI) é informação **analisada** sobre adversários, seus objetivos e métodos, produzida para apoiar uma decisão. Uma lista de IPs, sozinha, é só dado; vira inteligência quando ganha **contexto** (quem, contra quem, até quando), **confiança** e **relevância** para o seu ambiente. Os quatro níveis atendem a públicos e prazos diferentes, e repetem a lógica da Pirâmide da Dor: quanto mais técnico o indicador, mais curta a validade.

NÍVEL	PARA QUEM	EXEMPLO	VALIDADE
Estratégica	Direção e conselho	Tendências e motivações: *ransomware* contra universidades e hospitais, e o impacto no negócio	Meses
Operacional	Chefia do SOC e da resposta	Campanhas em andamento: quem ataca, quando e por qual meio de entrada	Dias a semanas
Tática	Engenharia de detecção e caça	**TTPs**: como o adversário opera, descritas com o ATT&CK	Semanas a meses
Técnica	Ferramentas e analistas do SOC	**IoCs**: *hashes*, IPs, domínios e URLs para bloqueio e busca	Horas a dias

Para trocar inteligência entre ferramentas e organizações, usam-se padrões abertos: o **STIX** (*Structured Threat Information Expression*, em JSON) descreve indicadores, adversários, campanhas e relações entre eles, e o **TAXII** (*Trusted Automated eXchange of Indicator Information*) é o protocolo, sobre HTTPS, que transporta esses objetos; ambos são especificações do OASIS. O **MISP** é uma plataforma de código aberto para coletar, correlacionar e compartilhar indicadores e contexto. Ao consumir *feeds*, avalie **relevância**, **frescor** e **confiança** de cada fonte, e dê **prazo de validade** a todo IoC: um IP de dois anos atrás provavelmente já pertence a outra pessoa.

16 MONITORAMENTO E DETECÇÃO — CONTINUAÇÃO

TLP 2.0 (FIRST): como compartilhar

- **TLP:RED**: só para as pessoas presentes na troca, sem repasse
- **TLP:AMBER+STRICT**: apenas dentro da própria organização
- **TLP:AMBER**: a organização e quem precisar saber entre clientes e parceiros
- **TLP:GREEN**: a comunidade ou o setor, sem divulgação pública
- **TLP:CLEAR**: sem restrição

Troca de informação no Brasil

- **CERT.br** (NIC.br): centro nacional de notificação e tratamento de incidentes; publica cartilhas e estatísticas
- **CTIR Gov**: coordena a resposta na administração pública federal
- **CAIS/RNP**: atende instituições de ensino e pesquisa conectadas à rede da RNP
- **ISACs**: centros setoriais (financeiro, saúde, energia) em que pares trocam inteligência

CAÇA A AMEAÇAS: PROCURAR ANTES DE O ALERTA TOCAR

A **caça a ameaças** (*threat hunting*) é a busca **proativa**, orientada por **hipótese**, por atividade que as detecções automáticas não pegaram. Parte do princípio de que o adversário **já pode estar no ambiente** e usa a telemetria existente, com criatividade analítica. Todo ciclo de caça termina em uma de três saídas, e todas são úteis: um **incidente**, uma **lacuna de visibilidade** (faltava uma fonte de log) ou uma **detecção nova**.

1 Hipótese
Uma afirmação testável, vinda do ATT&CK, de um relatório de CTI ou de uma anomalia. Ex.: "se um token de sessão foi roubado, será usado de dois países em poucos minutos".

2 Investigação
Consultar a telemetria, comparar com o *baseline*, pivotar entre fontes (identidade, rede, *endpoint*). Registrar consulta, período e dados usados.

3 Conclusão
Confirmar e escalar como incidente (**p. 156**), refutar e **documentar** a hipótese, ou apontar a lacuna de telemetria a corrigir.

4 Automatizar
O que funcionou vira detecção permanente (Sigma no Git, com testes), e a caça é repetida só quando surgir uma hipótese nova.

EXEMPLO RESOLVIDO

Da caça à regra: contas de serviço com login interativo.

1. **Hipótese:** uma conta de serviço (usada só por aplicações) não faz login interativo; se alguém a usa em um terminal, a credencial pode ter sido roubada (ATT&CK T1078, *Valid Accounts*).
2. **Consulta:** nos logs do Windows (evento 4624), logons de tipo 2 (interativo), 10 (RDP) e 11 (credenciais em cache) de contas com o prefixo `svc-`, nos últimos 90 dias.
3. **Achados (hipotéticos):** uma conta com RDP a partir de uma máquina que não é de administração (**escalar como incidente**) e outra usada por um técnico para depurar um serviço (**falha de processo**: criar acesso nominal e temporário).
4. **Automatizar:** regra Sigma "logon interativo de conta de serviço", com exceções que têm dono e prazo. Documentar a caça com consulta, período, resultado e decisão.
5. **Medir:** validar a regra com uma emulação em ambiente de teste e acompanhar a precisão nas semanas seguintes.

16 MONITORAMENTO E DETECÇÃO — CONTINUAÇÃO

MONITORAR COM RESPONSABILIDADE: PRIVACIDADE DOS LOGS

Logs costumam conter **dados pessoais** (IP, identificador de conta, e-mail, localização, histórico de acesso) e, quando monitoram pessoas funcionárias, revelam comportamento. A **LGPD** se aplica a eles: a coleta precisa de **finalidade** definida, **base legal** adequada (por exemplo, o legítimo interesse e a segurança, ou uma obrigação legal) e respeito à **necessidade** e à **adequação** (art. 6º), além de medidas de segurança proporcionais (art. 46). Na prática: informe em política **o que é monitorado e por quê**; restrinja e registre o acesso aos logs; mascare ou pseudonimize campos como CPF e e-mail; mantenha o **conteúdo** de mensagens e de arquivos fora do escopo, salvo necessidade justificada e proporcional; e defina prazos de retenção e de descarte. Em caso de dúvida, consulte o **encarregado (DPO)** e o jurídico; o equilíbrio entre segurança e privacidade é decisão de governança, não só técnica.

PARA FIXAR

Conceitos

- Por que o OWASP trocou *Monitoring* por *Alerting* na categoria A09? O que faz um log ser confiável?
- Diferencie SIEM, SOAR, EDR, NDR e XDR pela pergunta que cada um responde.
- Explique a Pirâmide da Dor e por que detectar TTPs vale mais do que bloquear *hashes*.
- Defina falso positivo, falso negativo, precisão e sensibilidade. Por que a fadiga de alertas é um risco de segurança?

Aplicação

- Reescreva `login falhou para ana senha=123` como um evento JSON seguro e liste os campos mínimos.
- Escreva em Sigma, ou em pseudocódigo, a detecção de "usuário administrador criado fora do horário comercial". Indique o falso positivo esperado e a técnica do ATT&CK.
- Uma regra gera 300 alertas por semana com 3% de precisão. Como melhorá-la sem perder a detecção?
- Monte a hipótese de caça para "token de sessão roubado": que logs consultar e o que faria com cada resultado?

APROFUNDE-SE

Leituras e referências

- OWASP Logging Cheat Sheet — OWASP: o que registrar, como e o que nunca registrar.
- NIST SP 800-92, Guide to Computer Security Log Management — NIST: infraestrutura e processos de gestão de logs.
- MITRE ATT&CK — MITRE: técnicas, detecções e fontes de dados por técnica.
- Sigma — SigmaHQ: formato de regras, especificação e regras públicas.
- The Pyramid of Pain — David Bianco, 2013: o artigo original.
- Traffic Light Protocol (TLP) — FIRST: o padrão de classificação do compartilhamento.
- MISP — plataforma aberta de compartilhamento de inteligência de ameaças.

MÓDULO 17 · OPERAÇÕES

17 RESPOSTA A INCIDENTES

Quando a prevenção falha: preparar, detectar, conter, erradicar, recuperar e aprender.

Detectar é só o começo: quando o alerta é real, inicia-se uma corrida contra o tempo em que cada decisão (isolar, preservar, comunicar, restaurar) tem custo e consequência. A **resposta a incidentes** é a disciplina de **preparar**, **conter**, **erradicar** e **recuperar**, sem perder as evidências e cumprindo as obrigações legais; a **continuidade de negócios** garante que o essencial continue funcionando, ou volte, dentro de metas definidas. O módulo cobre o ciclo do NIST (SP 800-61r3 e CSF 2.0), papéis e severidade, forense digital, contenção, erradicação e recuperação, a comunicação à ANPD, RTO e RPO, exercícios e a revisão pós-incidente.

EVENTO, INCIDENTE E VIOLAÇÃO

Nem toda ocorrência é um incidente, e a distinção organiza a resposta. Um **evento** é qualquer ocorrência observável (um login, uma conexão, um alerta). Um **incidente de segurança** é o evento, ou o conjunto de eventos, que **compromete ou ameaça comprometer** a confidencialidade, a integridade ou a disponibilidade de um sistema ou de seus dados, ou que viola as políticas de segurança. A **violação de dados** (*data breach*) é o incidente em que há acesso, divulgação ou perda **confirmados**. O **quase-incidente** (*near miss*) foi barrado a tempo, mas ensina tanto quanto os demais.

TERMO	EXEMPLO	RESPOSTA TÍPICA
Evento	400 logins falhos em um minuto, vindos de um IP conhecido de varredura	Registrar e avaliar; agir só se houver sinal de sucesso ou de padrão novo
Incidente	Conta de administrador usada de outro país; *ransomware* cifrando servidores	Abrir o registro, classificar a severidade e acionar a equipe de resposta
Violação de dados	Exportação não autorizada de dados de estudantes	Tratar como incidente e avaliar a comunicação à ANPD e aos titulares
Quase-incidente	Chave de API enviada ao repositório e revogada em minutos, sem uso indevido	Registrar, corrigir a causa (*hook* de pré-commit) e usar no treinamento

O CICLO DE RESPOSTA E O CSF 2.0

O modelo clássico, da **NIST SP 800-61 Rev. 2** (2012), tinha quatro fases: **preparação**; **detecção e análise**; **contenção, erradicação e recuperação**; e **atividade pós-incidente**. A **Rev. 3** (abril de 2025) substitui aquela revisão e se apresenta como um **perfil de comunidade do CSF 2.0**: a resposta deixa de ser um ciclo à parte e passa a integrar a gestão de risco, organizada pelas funções do CSF. **Govern, Identify e Protect** preparam a organização e reduzem o impacto; **Detect, Respond e Recover** tratam o incidente; e a **melhoria contínua** liga uma ocorrência à seguinte. Outros modelos seguem o mesmo espírito, como o **PICERL** do SANS (*Preparation, Identification, Containment, Eradication, Recovery, Lessons learned*) e a **ISO/IEC 27035**, voltada à gestão de incidentes. O guia usa as cinco fases da figura.

17 RESPOSTA A INCIDENTES — CONTINUAÇÃO



Ciclo de resposta e CSF 2.0 (mapeamento didático). As cinco fases correspondem às funções **GV** (Govern), **ID** (Identify), **PR** (Protect), **DE** (Detect), **RS** (Respond) e **RC** (Recover); **AN**, **MI** e **IM** indicam análise, mitigação e melhoria. O que se aprende volta ao preparo.

> **Não é linear:** Na prática, as fases se **sobrepõem**: durante a contenção descobre-se mais escopo e a análise recomeça. Por isso o **registro de decisões** (hora, autor, motivo) é o que permite a várias pessoas trabalharem em conjunto sem se contradizerem.

PREPARAÇÃO: O QUE PRECISA EXISTIR ANTES DO DIA RUIM

Quase tudo o que decide o resultado de um incidente foi decidido **antes** dele. O mínimo: **política e plano de resposta** aprovados pela liderança, com definições de incidente e de severidade; **equipe e papéis** claros; **inventário** de ativos e diagramas atualizados; **logs e ferramentas** de detecção (p. 144); **acessos de emergência** (*break-glass*) guardados e testados; **canais fora de banda** (se o e-mail e o chat corporativos estiverem comprometidos, como a equipe fala?); **contatos 24×7** de provedores, jurídico, seguradora e autoridades; **backups testados** (p. 59); e **cópias impressas ou offline** do plano e dos contatos, porque um plano guardado no servidor sequestrado não ajuda ninguém.

PAPEL	RESPONSABILIDADE NO INCIDENTE
Comandante do incidente	Coordena, define prioridades e mantém **uma** visão do todo; **não** executa tarefas técnicas
Líder técnico	Conduz a análise, propõe e executa a contenção e a erradicação
Operações e donos dos sistemas	Executam a restauração e as mudanças; conhecem as dependências
Comunicação	Mensagens internas e externas (clientes, imprensa, reguladores) por **um** porta-voz
Jurídico e encarregado (DPO)	Obrigações legais e contratuais, preservação de provas, relação com autoridades
Liderança executiva	Decide o que só a direção decide: desligar sistemas críticos, pagar ou não um resgate, comunicar ao mercado
Escriba	Anota hora, decisão, autor e motivo: a linha do tempo oficial do incidente

As equipes de resposta aparecem como **CSIRT** (*Computer Security Incident Response Team*) ou **CERT** e, na administração pública federal brasileira, como **ETIR**. Em organizações pequenas, as mesmas pessoas acumulam papéis, e o importante é que **cada papel tenha uma pessoa nomeada e uma substituta**, e que todas saibam quem decide.

17 RESPOSTA A INCIDENTES — CONTINUAÇÃO

SEVERIDADE E ESCALONAMENTO

A severidade define **quem é acionado e em quanto tempo**; sem ela, tudo vira emergência ou nada vira. Combina **impacto** (dados, sistemas, pessoas, dinheiro, obrigações legais) e **urgência** (o dano está em curso?). A tabela é um modelo de partida, para ajustar à organização.

SEVERIDADE	CRITÉRIOS	RESPOSTA (MODELO)	QUEM DECIDE
SEV 1 · crítica	Operação essencial parada; dados sensíveis ou em larga escala expostos; *ransomware* em produção; dano em curso	Resposta imediata, 24×7; comitê de crise; DPO e jurídico desde o início	Comandante, com a diretoria
SEV 2 · alta	Conta privilegiada ou sistema importante comprometido, sem dano generalizado	Início em minutos; investigação contínua no turno	Comandante
SEV 3 · média	Incidente contido, com impacto limitado (um *endpoint* com *malware* barrado, credencial revogada a tempo)	No mesmo dia útil; acompanhamento	Líder técnico
SEV 4 · baixa	Violação de política ou quase-incidente, sem impacto	Fila de rotina; revisar a causa	Analista

Reclassifique sempre que o escopo mudar: um incidente nasce SEV 3 e vira SEV 1 quando se descobre que a credencial roubada era de administrador de domínio. Escalar cedo e desescalar depois custa menos do que o contrário.

AS PRIMEIRAS HORAS: ORDEM DE AÇÕES

1 Confirmar e classificar
O alerta é real? Que ativo, que dados, que severidade? Atribua um número ao incidente.

2 Abrir registro e canal
Um chamado único, um canal dedicado (fora de banda, se preciso), um escriba, relógio em UTC.

3 Preservar o volátil
Antes de reiniciar ou reinstalar, capture memória, processos e conexões. **Isole** da rede, em vez de desligar.

4 Delimitar o escopo
Que contas, máquinas e dados? Há movimento lateral? Procure os mesmos sinais nas demais máquinas.

5 Acionar e comunicar
Chame os papéis da severidade e informe a liderança. Com dados pessoais, avise o DPO e o jurídico: o prazo da ANPD pode já estar correndo.

6 Conter
Escolha a contenção pelo impacto: **rápida e ampla** para *ransomware*, **coordenada** contra um adversário persistente.

FORENSE DIGITAL: EVIDÊNCIAS QUE RESISTEM A QUESTIONAMENTO

A **forense digital** (DFIR, *Digital Forensics and Incident Response*) coleta e analisa evidências de modo **reprodutível e defensável**, porque elas podem sustentar uma demissão, uma ação judicial ou um pedido ao seguro. Os princípios são **alterar o mínimo** do original, **documentar** tudo, trabalhar sobre **cópias** e **provar a integridade** com *hash*. As referências são a **RFC 3227** (coleta e guarda de evidências), a **NIST SP 800-86** (forense integrada à resposta) e a **ISO/IEC 27037** (identificação, coleta e preservação de evidências digitais).

A **ordem de volatilidade** (RFC 3227) manda coletar primeiro o que desaparece primeiro:

17 RESPOSTA A INCIDENTES — CONTINUAÇÃO

ORDEM	EVIDÊNCIA	EXEMPLOS E CUIDADO
1	Estado do sistema e memória	RAM, processos, conexões de rede, tabelas ARP e de rotas, sessões; some ao desligar
2	Arquivos temporários	`/tmp`, arquivo de paginação (*swap*), caches
3	Disco	Arquivos, registro do Windows, logs locais, artefatos de execução; persistente, mas sobrescrito com o uso
4	Logs remotos e de monitoramento	SIEM, IdP, nuvem, firewall, proxy; a retenção é limitada, então **exporte logo**
5	Configuração e topologia; mídia de arquivo	Diagramas, regras, *backups*; as mais estáveis

A **cadeia de custódia** é o registro, sem lacunas, de **quem** coletou a evidência, **quando**, **como**, **onde** ela foi guardada e **quem** teve acesso: sem ela, a prova perde valor. Para discos, use um **bloqueador de escrita** (*write blocker*, de hardware ou de *software*), gere uma **imagem forense** bit a bit e compare o *hash* SHA-256 do original com o da cópia.

COLETA DE VOLÁTEIS EM LINUX E IMAGEM DE DISCO COM PROVA DE INTEGRIDADE

```
# 1) Voláteis, da mais para a menos volátil (mídia externa montada em /mnt/ir)
#    Use binários de confiança, de uma mídia própria, e registre quem executou e quando.
{ date -u; uptime; id; }            > /mnt/ir/00_contexto.txt
ss -tunap                           > /mnt/ir/01_conexoes.txt
ps -eo pid,ppid,user,lstart,cmd     > /mnt/ir/02_processos.txt
lsof -nP +L1                        > /mnt/ir/03_apagados_em_uso.txt   # apagados e abertos
{ last -Faiw; who -a; }             > /mnt/ir/04_sessoes.txt
sha256sum /mnt/ir/*.txt             > /mnt/ir/hashes.sha256

# 2) Imagem do disco (com bloqueio de escrita) e conferência de integridade
dd if=/dev/sdb of=evidencia.img bs=4M conv=noerror,sync status=progress
sha256sum /dev/sdb evidencia.img    # os dois hashes devem ser iguais; guarde-os na custódia
```

Para a **memória**, existem ferramentas de aquisição (WinPmem no Windows; LiME e AVML no Linux) e o **Volatility** para analisá-la. Para disco e linha do tempo, usam-se o **Autopsy/Sleuth Kit** e o **plaso** (`log2timeline`); em escala, o **Velociraptor** e o **KAPE** coletam artefatos de muitas máquinas. Há um dilema recorrente, **conter × preservar**: isolar a máquina da rede costuma preservar mais evidência do que desligá-la, pois a RAM se perde ao desligar; mas, se o isolamento for impossível e o dano estiver se espalhando (um *ransomware* em curso), desligar pode ser o mal menor. Decida e **registre o motivo**.

CONTENÇÃO, ERRADICAÇÃO E RECUPERAÇÃO

As três fases formam um só fluxo, e a **ordem** importa: **conter** antes de erradicar, para que o dano pare de crescer, e **erradicar** antes de recuperar, para não restaurar sobre um sistema ainda comprometido. A tabela resume o objetivo, as ações típicas e os cuidados de cada fase.

17 RESPOSTA A INCIDENTES — CONTINUAÇÃO

FASE	OBJETIVO E AÇÕES TÍPICAS	CUIDADOS
Contenção	**Impedir que o dano cresça.** Isolar o host (EDR, VLAN de quarentena); desativar contas e revogar sessões, tokens e chaves; bloquear IoCs no firewall, no DNS e no proxy; segmentar	Decida pelo impacto no negócio. Contra um adversário persistente, prefira a contenção **coordenada** (todas as ações de uma vez), para não alertá-lo; contra *ransomware*, a rápida e ampla
Erradicação	**Remover a presença e a causa.** Eliminar persistência (tarefas agendadas, serviços, contas, chaves SSH, *webshells*, regras de caixa de e-mail, consentimentos OAuth, chaves de nuvem); corrigir a causa-raiz; **rotacionar** credenciais (p. 135); reconstruir a partir de imagens confiáveis	"Limpar" não basta: **reconstrua** o que foi comprometido. Em um domínio Active Directory comprometido, redefina a senha da conta KRBTGT **duas vezes**, com intervalo
Recuperação	**Voltar a operar com segurança.** Restaurar de backup **limpo** (anterior à intrusão), validar a integridade, reintroduzir os serviços em etapas, reforçar o monitoramento e confirmar que a causa foi corrigida	O *backup* pode estar infectado: **teste** antes de reconectar. Priorize pela BIA e não religue nada antes de fechar a porta de entrada

Declare o incidente **encerrado** só quando a causa estiver corrigida, os serviços restaurados e validados, as obrigações de comunicação cumpridas e um período de **vigilância reforçada** tiver passado sem sinais do adversário. Encerrar cedo demais é a forma mais comum de ser atacado de novo pelo mesmo caminho.

PLAYBOOKS POR TIPO DE INCIDENTE

Um **playbook** descreve, para um tipo de incidente, as decisões e as ações na ordem certa, para que ninguém improvise sob pressão. Os quatro mais comuns:

CENÁRIO	PRIMEIRAS AÇÕES	EVIDÊNCIAS	ARMADILHA COMUM
***Phishing* com clique ou credencial**	Redefinir a senha e **revogar sessões e tokens**; apagar a mensagem de todas as caixas; bloquear remetente e URL; revisar regras de caixa e consentimentos OAuth	Cabeçalhos do e-mail, logs de e-mail e de login	Esquecer regras de encaminhamento e aplicativos autorizados, que **sobrevivem** à troca de senha
Conta comprometida	Desativar a conta ou forçar a saída; revogar chaves de API e tokens; revisar o que foi criado (chaves, regras, permissões)	Logs do IdP e da nuvem, IPs, dispositivos	Trocar só a senha e deixar sessões e chaves ativas
Vazamento de dados	Fechar o acesso (*bucket* público, chave exposta); delimitar quais dados e quantos titulares; avaliar a comunicação à ANPD	Logs de acesso, listagem de objetos, trilha de download	Subestimar o escopo e perder o prazo legal
DDoS	Acionar o provedor ou a CDN, aplicar *rate limit* e filtros, ampliar capacidade, ativar modo degradado	Logs de borda, fluxos, amostras de tráfego	Ser cortina de fumaça para outro ataque: olhe as demais fontes

17 RESPOSTA A INCIDENTES — CONTINUAÇÃO

RANSOMWARE: AS PRIMEIRAS 24 HORAS

JANELA	AÇÕES	DONO
0 a 1 h	**Isolar** os hosts afetados (EDR ou rede), preservar o volátil, acionar o comandante, abrir o registro e mudar para o canal fora de banda	Líder técnico e comandante
1 a 4 h	Delimitar o escopo (paciente zero, contas e sistemas); **verificar se os backups estão intactos e isolados**; acionar jurídico, DPO, seguradora e autoridades	Comandante
4 a 12 h	Redefinir credenciais privilegiadas, bloquear C2 e IoCs, desligar acessos remotos suspeitos, segmentar; procurar sinais de **exfiltração** (dupla extorsão)	Líder técnico e operações
12 a 24 h	Decidir a estratégia de recuperação pela BIA, preparar a comunicação (ANPD e titulares, se houve dados pessoais) e alinhar a posição sobre o resgate	Direção, jurídico e DPO

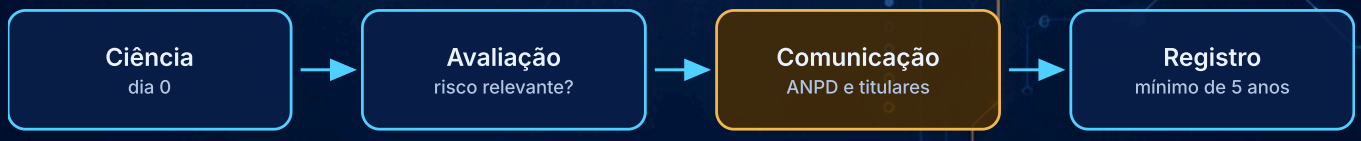
Pagar o resgate? A decisão é da liderança, com assessoria jurídica. As autoridades (como a CISA e o FBI) **desaconselham**: o pagamento não garante a recuperação, financia o crime e pode envolver **sanções** (nos EUA, o OFAC já alertou sobre pagamentos a grupos sancionados). Também **não elimina** a obrigação de avaliar e comunicar um incidente com dados pessoais, nem garante que os dados roubados não serão publicados. O melhor argumento contra pagar é ter **backups testados e a capacidade de restaurar**.

> **O que mais reduz o estrago:** *Backups* imutáveis ou offline e **testados**; MFA resistente a *phishing* no acesso remoto e nas contas privilegiadas; administração em camadas (o administrador de domínio não faz login em estações de trabalho); segmentação; EDR com resposta; e correção rápida dos sistemas expostos à internet. Nenhum controle evita tudo, mas, juntos, transformam uma catástrofe em um incidente.

COMUNICAÇÃO E OBRIGAÇÕES LEGAIS

Responder bem inclui **comunicar** bem: dentro da organização (liderança e equipes afetadas), a **clientes e parceiros**, à **imprensa**, às **autoridades** e, quando há dados pessoais, à **ANPD** e aos **titulares**. As regras de ouro são um **porta-voz único**; fatos confirmados separados de hipóteses; nenhuma especulação sobre a causa nem busca de culpados; atualizações em horários previstos; e registro de tudo o que foi dito.

A **LGPD** (art. 48) manda o controlador comunicar à ANPD e ao titular o incidente que possa acarretar **risco ou dano relevante**. A **Resolução CD/ANPD nº 15/2024** esclarece quando isso ocorre: o incidente pode afetar significativamente interesses e direitos fundamentais dos titulares **e**, cumulativamente, envolver ao menos um destes critérios: dados sensíveis; dados de crianças, adolescentes ou idosos; dados financeiros; dados de autenticação em sistemas; dados protegidos por sigilo legal, judicial ou profissional; ou dados em larga escala. O prazo é de **três dias úteis**, contados do **conhecimento** pelo controlador de que o incidente afetou dados pessoais (ressalvado prazo previsto em legislação específica), e o **registro** do incidente deve ser guardado por, no mínimo, **cinco anos**.



17 RESPOSTA A INCIDENTES — CONTINUAÇÃO

O que a comunicação à ANPD informa

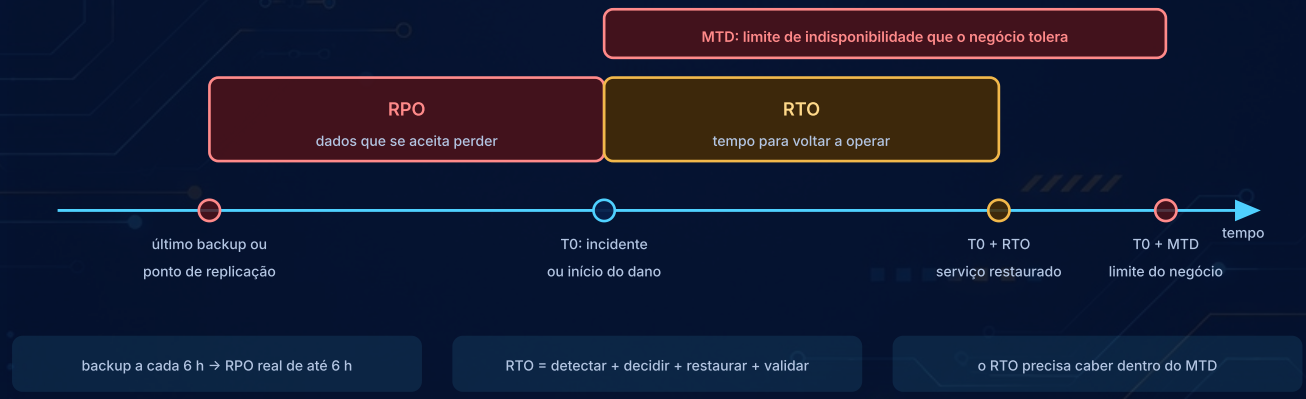
- A natureza e a categoria dos dados afetados e o **número de titulares** (com o de crianças, adolescentes e idosos)
- As **medidas de segurança** que existiam, os riscos e os impactos prováveis aos titulares
- As medidas **adotadas ou previstas** para reverter ou mitigar o dano
- As datas da ocorrência e do conhecimento e, se passou do prazo, o **motivo da demora**
- A identificação do controlador, do operador (se houver) e do encarregado
- A descrição do incidente e, se conhecida, a **causa principal**

Outros relógios correndo

- **Contratos**: o operador costuma ter de avisar o controlador em horas (prazos de 24 a 72 h são comuns); confira o seu
- **Setores regulados**: o sistema financeiro tem regras próprias para incidentes relevantes (Res. CMN 4.893/2021 e BCB 85/2021)
- **GDPR** (UE): 72 horas à autoridade, se houver titulares europeus
- **Seguradora e clientes**: apólices e contratos podem exigir aviso rápido
- **Polícia**: em crimes (invasão, extorsão), considere o registro de ocorrência e o acionamento da polícia especializada

CONTINUIDADE DE NEGÓCIOS E RECUPERAÇÃO DE DESASTRES

A **continuidade de negócios** (BCM, *Business Continuity Management*; norma **ISO 22301**) garante que as funções críticas continuem, ou voltem, em tempo aceitável. Começa pela **BIA** (*Business Impact Analysis*): identificar os processos essenciais, suas dependências (sistemas, pessoas, fornecedores) e **como o impacto cresce com o tempo parado** (dinheiro, lei, reputação, segurança de pessoas). Da BIA saem as metas: o **MTD** (*Maximum Tolerable Downtime*), limite de indisponibilidade a partir do qual o dano é inaceitável; o **RTO**, tempo-alvo para restaurar o serviço, que precisa ser **menor que o MTD**; e o **RPO**, quanto dado, medido em tempo, se aceita perder.



RPO, RTO e MTD. O **RPO** mede para trás (quanto dado se perde) e o **RTO** mede para frente (quanto tempo se fica fora). Os dois são metas de negócio, definidas na análise de impacto (BIA) e comprovadas por testes de restauração, não por declaração.

A BIA costuma resultar em um quadro como o abaixo (valores **ilustrativos**). Repare que o RTO e o RPO **variam por serviço**: tratar tudo como crítico é caro, e tratar tudo como rotina é arriscado.

SERVIÇO	IMPACTO DA PARADA	RTO	RPO
Matrícula (na semana de matrícula)	Sem matrícula, perda de receita e de prazo legal; o impacto é alto desde a primeira hora	2 h	5 min
Folha de pagamento	Obrigação trabalhista e legal; crítica perto do dia de pagamento	24 h	1 h

17 RESPOSTA A INCIDENTES — CONTINUAÇÃO

SERVIÇO	IMPACTO DA PARADA	RTO	RPO
Portal de notas	Incômodo, sem perda financeira direta; sobe no fim do semestre	8 h	1 h
Site institucional	Dano à reputação, sem parada da operação	24 h	24 h

A **recuperação de desastres** (DR) é o conjunto de técnicas de TI que cumpre o RTO e o RPO; o **plano de continuidade** (BCP) cobre também pessoas, locais e processos manuais, e a **NIST SP 800-34** detalha o planejamento de contingência de sistemas. A técnica de *backup*, a imutabilidade e a retenção estão em p. 59. A escolha da estratégia de DR é um equilíbrio entre meta e custo; a tabela segue o modelo popularizado pela AWS, com ordens de grandeza, não garantias.

ESTRATÉGIA	COMO FUNCIONA	RTO / RPO TÍPICOS	CUSTO
Backup e restauração	Cópias periódicas, fora do site; no desastre, recria-se a infraestrutura e restauram-se os dados	Horas / horas	Baixo
***Pilot light* (chama-piloto)**	Dados replicados continuamente; o mínimo da aplicação fica pronto e é ampliado ao acionar	Dezenas de minutos / minutos	Médio-baixo
Warm standby	Cópia reduzida e funcional da aplicação, sempre ligada, que escala ao acionar	Minutos / segundos a minutos	Médio-alto
Ativo-ativo (multi-site)	Mais de um site atende o tráfego ao mesmo tempo; um absorve a falha do outro	Próximo de zero / próximo de zero	Alto

EXEMPLO RESOLVIDO

O RTO cabe? O banco de dados do sistema de matrícula tem 200 GB e o *backup* restaura a 100 MB/s. A meta é RTO de 2 h e RPO de 5 min.

1. **Restauração:** 200.000 MB ÷ 100 MB/s = 2.000 s ≈ **34 min**.
2. **Demais etapas:** detectar e decidir (20 min), provisionar o servidor (20 min), validar a integridade (15 min) e redirecionar o tráfego (10 min). Total ≈ **1 h 40 min**: cabe nas 2 h, com folga curta.
3. **Se o banco crescer para 2 TB:** 2.000.000 ÷ 100 = 20.000 s ≈ **5,6 h**. Não cabe, e a resposta é mudar a **estratégia** (réplica contínua, *warm standby*), não "restaurar mais rápido".
4. **RPO:** com *backup* completo toda noite e **logs de transação arquivados a cada 5 minutos**, restaura-se até poucos minutos antes do incidente (RPO ≈ 5 min). Sem os logs, o RPO seria de até 24 h.

Plano que nunca foi testado é hipótese. Os exercícios sobem em custo e em realismo: a **leitura do plano** (*walkthrough*); o **exercício de mesa** (*tabletop*), em que a equipe discute um cenário; a **simulação funcional**, que executa parte do plano (restaurar um serviço em ambiente isolado); o **teste completo** de *failover*; e, em equipes maduras, os **game days** e a engenharia do caos, com falhas induzidas sob controle. Registre o que falhou, com dono e prazo de correção.

17 RESPOSTA A INCIDENTES — CONTINUAÇÃO

CENÁRIO

Exercício de mesa (45 a 60 minutos): matrícula parada. Terça-feira, 7h40: a central de atendimento relata que ninguém acessa o sistema de matrícula, cujo prazo termina na sexta. 8h05: o EDR alerta criptografia em massa em três servidores de arquivos. 8h30: surge uma nota de resgate ameaçando publicar "dados de alunos" em 72 horas. Durante o exercício, a facilitação libera **novos fatos**:

1. 8h45: descobre-se que o *backup* noturno falha há quatro dias.
2. 9h30: a imprensa pergunta se houve vazamento.
3. 10h15: o log do IdP mostra um login da conta do diretor de TI às 3h, vindo de outro país.

Perguntas: Quem é o comandante e qual é a severidade? O que isolar primeiro e que evidências preservar antes? Quais sistemas voltam primeiro, segundo a BIA? O que comunicar, a quem e quando (ANPD, titulares, imprensa)? Pagar ou não: quem decide e com que informações? O que mudaria se os *backups* estivessem íntegros?

PÓS-INCIDENTE: APRENDER SEM CULPAR

A **revisão pós-incidente** (*postmortem*, *lessons learned*) acontece em dias, e não em meses, com todas as pessoas que participaram. O formato **sem culpa** (*blameless*) parte de que as pessoas agem com o que sabiam no momento e de que as falhas vêm de **sistemas e processos**; procurar culpados faz a equipe esconder informação, o contrário do que ela precisa. Busque **fatores contribuintes**, e não uma causa única: a técnica dos **5 porquês** ajuda a descer da falha visível ("a conta foi usada") até a causa organizacional ("contas de serviço sem dono nem rotação"). Cada **ação corretiva** tem dono, prazo e critério de pronto, e as lições alimentam o plano, as detecções (p. 144) e o treinamento. As **métricas** (MTTD, MTTC, tempo de recuperação, custo e reincidência) mostram a evolução.

MODELO DE RELATÓRIO PÓS-INCIDENTE (RESUMO DE UMA PÁGINA)

```
RELATÓRIO PÓS-INCIDENTE · INC-2026-014 · severidade: SEV 2 · status: encerrado

1. Resumo (3 frases)   O que houve, qual foi o impacto e como foi resolvido.
2. Impacto             Sistemas, dados e pessoas afetados; duração; custo estimado.
3. Linha do tempo UTC  03:12 sinal · 03:40 alerta · 04:05 contenção · 09:30 restaurado
4. Causa e fatores     Causa técnica e fatores associados (processo, pessoas, ferramentas).
5. O que funcionou     Detecções, decisões e práticas a manter.
6. O que não funcionou Lacunas de visibilidade, de acesso e de comunicação.
7. Ações corretivas    Uma linha por ação: [dono · prazo · critério de pronto].
8. Comunicação         Quem foi avisado e quando (ANPD, titulares, clientes, contratos).
9. Lições              O que muda no plano, nas regras de detecção e no treinamento.
```

CASOS REAIS: O QUE ENSINAM

Três incidentes públicos, de natureza diferente (um ataque a uma empresa de energia, uma falha de atualização de *software* e um ataque a um tribunal), mostram que a resposta e a continuidade dependem das mesmas decisões: o que desligar, o que preservar, como restaurar e como comunicar. Os relatos vêm de alertas oficiais e de imprensa especializada, e os detalhes podem ter sido revistos desde então.

17 RESPOSTA A INCIDENTES — CONTINUAÇÃO

CASO	O QUE ACONTECEU	LIÇÃO PARA A RESPOSTA E A CONTINUIDADE
Colonial Pipeline (EUA, mai/2021)	O *ransomware* DarkSide (RaaS) atingiu a rede de TI da empresa, que **interrompeu a operação do oleoduto por precaução** (alerta AA21-131A da CISA). Segundo relatos públicos, a entrada foi uma credencial de VPN legada, sem MFA; a empresa informou ter pago cerca de US$ 4,4 milhões	Decida cedo o que desligar e o que manter (segregação entre TI e OT). Acessos legados sem MFA são porta de entrada. O dano foi de **disponibilidade** e de mercado, e não só de dados
CrowdStrike (19/jul/2024)	Uma atualização de **conteúdo** do sensor Falcon travou cerca de 8,5 milhões de máquinas Windows (estimativa da Microsoft). A causa foi uma leitura fora dos limites: o modelo de dados definia 21 campos, e o código fornecia 20. **Não foi ataque**	Incidente de **continuidade**: em muitos casos a recuperação exigiu ação manual, máquina a máquina. Libere mudanças **em anéis** (canário, depois grupos maiores), com testes e reversão, e planeje a falha da **própria ferramenta de segurança**
STJ (Brasil, nov/2020)	O *ransomware* RansomEXX derrubou por vários dias os sistemas do Superior Tribunal de Justiça	Serviços públicos dependem de sistemas: **backups isolados, restauração testada** e um plano para operar sem TI (processos manuais) reduzem o tempo fora do ar

PARA FIXAR

Conceitos

- Diferencie evento, incidente, violação de dados e quase-incidente, com um exemplo de cada.
- Como a SP 800-61r3 relaciona a resposta a incidentes às funções do CSF 2.0?
- Por que a ordem de volatilidade manda coletar a memória antes do disco? O que é a cadeia de custódia?
- Defina MTD, RTO e RPO e explique a relação entre eles.

Aplicação

- Um servidor está cifrando arquivos. Liste as cinco primeiras ações em ordem e justifique o equilíbrio entre conter e preservar.
- Um vazamento expôs e-mails e *hashes* de senha de 12 mil usuários. É preciso comunicar a ANPD? Aplique os critérios do art. 5º e conte o prazo.
- O RTO é de 2 h para restaurar 600 GB a 80 MB/s, com 45 min de tarefas fixas. É viável? Se não, que mudança propor?
- Escreva três perguntas de um exercício de mesa sobre a credencial de VPN de uma pessoa funcionária, vazada na internet.

APROFUNDE-SE

Leituras e referências

- NIST SP 800-61 Rev. 3 — NIST, 2025: resposta a incidentes como parte da gestão de risco do CSF 2.0.
- NIST Cybersecurity Framework 2.0 — NIST: as funções Govern, Identify, Protect, Detect, Respond e Recover.
- RFC 3227, Guidelines for Evidence Collection and Archiving — IETF: ordem de volatilidade e guarda de evidências.
- #StopRansomware — CISA: guias de prevenção e de resposta a *ransomware*.
- Alerta AA21-131A (DarkSide) — CISA e FBI, 2021: o caso Colonial Pipeline.
- Postmortem Culture: Learning from Failure — Google SRE: a prática da revisão sem culpa.
- ANPD — Autoridade Nacional de Proteção de Dados: Resolução CD/ANPD nº 15/2024 e comunicação de incidentes.

MÓDULO 18 · TESTES

18 TESTES DE SEGURANÇA

Verificar se as defesas funcionam — de forma autorizada, com método, escopo e responsabilidade.

Defender é imaginar como alguém atacaria. Os **testes de segurança** respondem, com evidências, à pergunta "isto resiste?": descobrem falhas antes de quem ataca, medem se as defesas detectam e respondem e dão prioridade à correção. O módulo reúne os tipos de teste (avaliação de vulnerabilidades, teste de intrusão, *red team*, *purple team*, *bug bounty*), as metodologias, o ciclo de um teste e a leitura de um relatório e, com o mesmo peso, os **limites legais e éticos**. Tudo aqui pressupõe sistemas **próprios, de laboratório ou autorizados por escrito**: testar sem autorização é crime no Brasil.

O MAPA DOS TESTES: A PERGUNTA QUE CADA UM RESPONDE

Os nomes se confundem no mercado, e escolher o teste errado desperdiça verba. A distinção prática é a **pergunta** que cada tipo responde:

TIPO	PERGUNTA QUE RESPONDE	QUANDO USAR	FREQUÊNCIA
Avaliação de vulnerabilidades	Que falhas **conhecidas** existem nos meus sistemas?	Higiene contínua, inventário de ativos, antes de lançar	Contínua ou mensal (automatizada)
Teste de intrusão (*pentest*)	Alguém consegue **encadear** falhas e chegar a algo que importa?	Sistemas críticos, grandes lançamentos, exigência de norma ou cliente	Anual e a cada mudança relevante
Red team	A organização inteira (pessoas, processos, detecção) resiste a um adversário realista, com objetivo definido?	Defesa já madura, com SOC e resposta	Anual ou a cada dois anos
Purple team	Nossas detecções cobrem as técnicas que importam, e como melhorá-las juntos?	Evolução do SOC e da engenharia de detecção (p. 144)	Trimestral, em ciclos curtos
***Bug bounty* e VDP**	Pesquisadores externos acham o que escapou de nós?	Produtos expostos e equipe capaz de tratar relatos	Contínuo
Auditoria e revisão	O sistema segue o padrão exigido; o código e a configuração estão como deveriam?	Certificação, *due diligence*, revisão de código ou de configuração	Anual ou por mudança

Os testes também variam pelo **conhecimento prévio** de quem testa, e a escolha depende da pergunta e do orçamento:

01 CAIXA PRETA

Parte do zero, como um atacante externo que só conhece o alvo público. Mede o que um estranho alcança; é mais lenta e pode deixar a lógica interna sem teste.

02 CAIXA CINZA

Conhece algo: credenciais comuns, documentação parcial. É a mais usada, porque cobre o abuso por **quem já está dentro** (uma conta de aluno tentando virar administrador) com bom custo.

03 CAIXA BRANCA

Acesso a código, arquitetura e configuração. Acha mais falhas, mais cedo (revisão de código, SAST e análise manual), mas não reproduz a visão de quem ataca.

18 TESTES DE SEGURANÇA — CONTINUAÇÃO

Um quarto formato, o **comprometimento presumido** (*assumed breach*), parte de um acesso interno já dado (uma estação ou uma credencial) para testar o que a organização contém e detecta depois da entrada, e é a base de muitos exercícios *red* e *purple*.

METODOLOGIAS E REFERÊNCIAS

REFERÊNCIA	FOCO	USO
PTES	*Penetration Testing Execution Standard*, em sete fases: pré-engajamento, coleta de inteligência, modelagem de ameaças, análise de vulnerabilidades, exploração, pós-exploração e relatório	Estrutura do projeto e do relatório
OWASP WSTG	Guia de testes de aplicações web, por categoria (p. 98)	Casos de teste de web
OWASP MASTG e ASVS	MASTG: testes de aplicativos móveis. ASVS: requisitos verificáveis, em níveis 1 a 3	Critério de "pronto" e escopo
NIST SP 800-115	Guia técnico de testes de segurança: planejamento, descoberta, ataque e relato	Referência para política interna e órgãos públicos
OSSTMM	Metodologia aberta (ISECOM) de testes operacionais, com métricas de segurança	Medir superfície de ataque e confiança
MITRE ATT&CK	Catálogo de TTPs para **emulação de adversário**	*Red* e *purple team* (p. 11)
TIBER-EU e CBEST	Quadros para *red team* **guiado por inteligência de ameaças** no setor financeiro (BCE e Bank of England)	Exemplos de teste regulado

ESCOPO E REGRAS DE ENGAJAMENTO: ANTES DO PRIMEIRO PACOTE

O que separa um teste de segurança de um ataque é a **autorização**, que se materializa em documentos: o **contrato** (com NDA), a **carta de autorização**, assinada por quem tem poder sobre os sistemas, e as **regras de engajamento** (RoE, *rules of engagement*). Sem eles, não se começa. Três cuidados: quem assina precisa ser **dono do ativo** (em nuvem de terceiros, vale também a política do provedor); o escopo é **taxativo**, e o que não está listado não se toca; e o teste manipula dados pessoais, de modo que o contrato trata da **LGPD**, com o testador como operador.

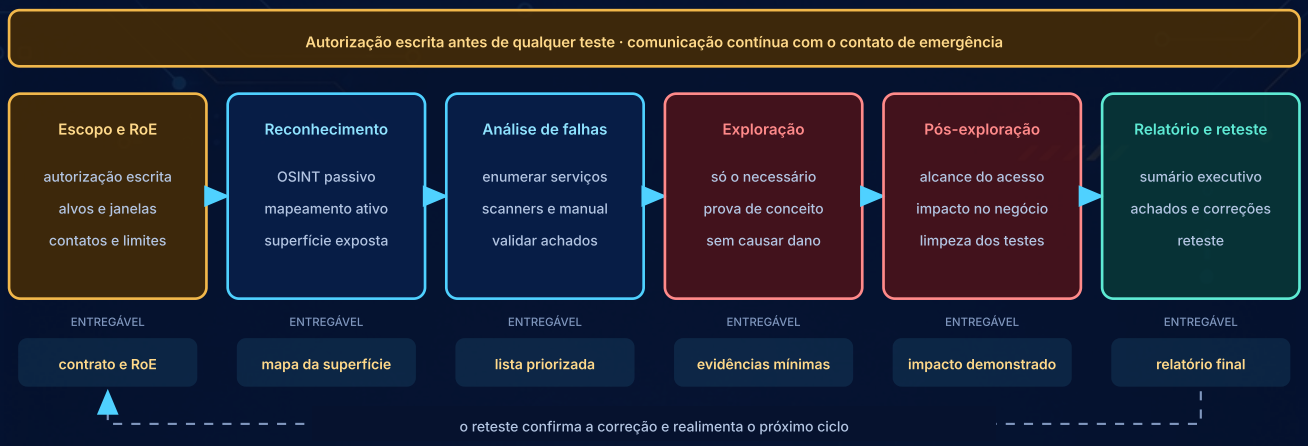
MODELO DE ESCOPO E REGRAS DE ENGAJAMENTO (RESUMO)

```
PROJETO          Teste de intrusão do portal acadêmico (caixa cinza, ambiente de homologação)
AUTORIZADO POR   Diretoria de TI (nome, cargo, data); carta de autorização anexa
ESCOPO           portal-hml.exemplo.edu.br · api-hml.exemplo.edu.br · 203.0.113.10 a .20
FORA DO ESCOPO   produção · terceiros (IdP, e-mail, CDN) · outros domínios
JANELA           02/12 a 13/12, das 20h às 06h (BRT); sem testes em dias de matrícula
CONTAS           3 contas de teste (aluno, professor, secretaria), criadas só para este teste
PERMITIDO        varredura, fuzzing de API, exploração com prova de conceito mínima
PROIBIDO         negação de serviço, engenharia social, copiar dados reais, manter acesso
PARADA           impacto não previsto ou dado pessoal real exposto: parar e avisar em até 1 h
CONTATOS 24x7    TI: (nome, celular) · testador: (nome, celular)
DADOS            evidências cifradas, o mínimo necessário, apagadas 30 dias após o relatório
RELATOS          falha crítica: aviso imediato; resumo parcial semanal
ENTREGA          relatório em PDF cifrado; reteste incluído por 60 dias
```

18 TESTES DE SEGURANÇA

CONTINUAÇÃO

AS FASES DE UM TESTE DE INTRUSÃO



Fases de um teste de intrusão e entregáveis. Nada começa sem **autorização escrita** e regras de engajamento. As duas fases em vermelho acessam ou alteram o alvo e pedem mais cuidado: faça o mínimo necessário para provar o risco. O reteste fecha o ciclo.

Reconhecimento. A coleta de informação em fontes abertas (**OSINT**) pode ser **passiva**, sem tocar o alvo (DNS público, certificados em logs de transparência, repositórios, vagas de emprego que revelam tecnologias), ou **ativa**, interagindo com ele (varredura de portas, enumeração de serviços e diretórios), e a ativa **aparece nos logs** do defensor. O resultado é o **mapa da superfície de ataque**: hosts, portas, serviços e versões, rotas e parâmetros, contas e integrações.

Análise de vulnerabilidades. Combinam-se **scanners**, que dão rapidez e cobertura de falhas conhecidas, e a **análise manual**, que encontra o que as ferramentas não veem: lógica de negócio, autorização e encadeamento de falhas. Cada candidato é **validado**, para descartar falsos positivos, e entra na lista priorizada.

Exploração e pós-exploração. Na exploração, demonstra-se que a falha é real com a **menor prova de conceito possível** (ler um registro de teste, e não a base inteira). Na pós-exploração, mede-se o **alcance**: com este acesso, o que mais se atinge (outros sistemas, dados sensíveis, privilégios)? O objetivo é traduzir a falha técnica em **impacto no negócio**, e não colecionar troféus. No fim vem a **limpeza**: remover contas, arquivos e artefatos criados no teste e entregar a lista do que foi feito, para que a defesa distinga o rastro do teste do rastro de um ataque.

FERRAMENTAS POR CATEGORIA, FINALIDADE E USO ÉTICO

CATEGORIA	EXEMPLOS	FINALIDADE	CUIDADO ÉTICO
Varredura de rede	Nmap	Descobrir hosts, portas e serviços	Só no escopo; limite a taxa para não derrubar serviços
Análise de tráfego	Wireshark, tcpdump	Capturar e inspecionar pacotes	Rede própria ou autorizada; a captura pode conter senhas
Proxy de interceptação (web)	Burp Suite, OWASP ZAP	Ver, alterar e repetir requisições HTTP	Use contas de teste, não dados reais
Scanners de vulnerabilidades	OpenVAS (Greenbone), Nessus, Nuclei	Achar falhas conhecidas e erros de configuração em escala	Perfil não destrutivo; revisar os falsos positivos
Enumeração de conteúdo	ffuf, Gobuster	Descobrir diretórios, arquivos e parâmetros	Respeitar taxa e janelas

18 TESTES DE SEGURANÇA — CONTINUAÇÃO

CATEGORIA	EXEMPLOS	FINALIDADE	CUIDADO ÉTICO
Frameworks de exploração	Metasploit	Validar vulnerabilidades com módulos controlados	Só com autorização explícita e prova de conceito mínima
Auditoria de senhas	Hashcat, John the Ripper	Medir a força de senhas em *hashes* obtidos com autorização	Resultados são dados sensíveis; escopo escrito
Distribuições de laboratório	Kali Linux, Parrot OS	Reunir as ferramentas em um ambiente pronto	Em máquina virtual isolada

Cada ferramenta deixa rastro, e isso é útil a quem defende: uma varredura de portas vira milhares de conexões curtas nos logs do firewall, um proxy de interceptação aparece com padrões de requisição e *User-Agent*, e um scanner dispara o WAF. Os exercícios *purple* usam esses rastros para **testar as detecções** do módulo p. 144.

TRIAGEM DE UM RELATÓRIO DE SCANNER: EXEMPLO RESOLVIDO

Os *scanners* produzem muitos resultados, e parte deles é **falso positivo** ou ruído. A triagem separa o que é real, o que é relevante e o que pode ser descartado, e cada decisão fica registrada. Cinco achados de um mesmo relatório:

ACHADO DO SCANNER	COMO VALIDAR	DECISÃO
TLS 1.0 e 1.1 habilitados em `api-hml`	Confirme com uma ferramenta de teste de TLS (como o `testssl.sh`) e na configuração do servidor ou do balanceador	**Real.** Desabilitar e exigir TLS 1.2 ou superior (p. 34)
Apache 2.4.x "vulnerável" a uma CVE de execução remota (detecção pela versão)	Compare com o pacote instalado: as distribuições **portam a correção** (*backport*) sem mudar o número da versão; consulte o aviso do fornecedor	**Provável falso positivo.** Anexe a evidência e marque como descartado
Injeção de SQL em `/busca?q=`	Reproduza à mão com uma entrada de teste; leia o código e o log do banco. Se a consulta for parametrizada, a resposta anômala tinha outra causa	**Validar antes de priorizar.** Confirmada: crítica (p. 98). Não confirmada: descartar com registro
Cookie de sessão sem o atributo `Secure`	Inspecione o cabeçalho `Set-Cookie` e veja se o site responde também em HTTP	**Real e fácil de corrigir.** Gravidade baixa a média; corrija na próxima entrega
Cabeçalho `X-Powered-By` **revela a tecnologia**	Basta olhar a resposta	**Informativo.** Remover é boa prática, mas não merece prioridade alta

A regra é simples: **nenhum achado entra em um relatório sem validação**, e o que foi descartado fica registrado (com o motivo), para que o próximo ciclo não repita o trabalho nem reabra a discussão.

TIPOS DE TESTE POR ALVO

O alvo muda os métodos, as referências e, principalmente, os **cuidados**. Em quase todos os casos, prefira ambiente de homologação e dados fictícios.

18 TESTES DE SEGURANÇA — CONTINUAÇÃO

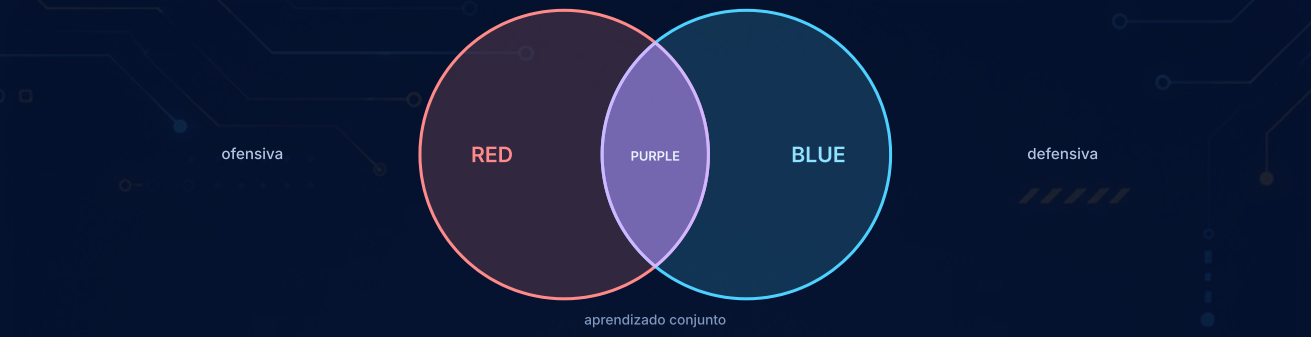
ALVO	O QUE SE TESTA	REFERÊNCIA	CUIDADO ESPECÍFICO
Rede e infraestrutura	Serviços e portas expostos, segmentação, credenciais padrão, *patches*, configuração de equipamentos	NIST SP 800-115; CIS Benchmarks	Limite a taxa das varreduras e respeite a janela; negação de serviço só com autorização específica
Aplicação web	Autenticação, sessão, autorização, validação de entrada, lógica de negócio	OWASP WSTG e ASVS (p. 98)	Use contas de teste; cuidado com formulários que enviam e-mail ou gravam em massa
API	Autorização por objeto e por função, excesso de dados, limites de taxa, abuso de fluxos	OWASP API Security Top 10 (p. 112)	Teste com mais de um perfil e mais de um locatário (*tenant*)
Aplicativo móvel	Armazenamento local, comunicação, binário, integração com o sistema	OWASP MASVS e MASTG (p. 112)	Use dispositivo de teste; a API de retaguarda também precisa estar no escopo
Redes sem fio	Cifra (WPA2 e WPA3), isolamento de convidados, pontos de acesso falsos	NIST SP 800-115	O sinal atravessa paredes: **delimite fisicamente** para não atingir redes vizinhas
Nuvem	Identidades e permissões, armazenamento público, segredos, configuração	CIS Benchmarks; guias do provedor (p. 78)	Leia a **política de testes do provedor**: o contrato com ele vale além do seu
Engenharia social	*Phishing* simulado, *vishing*, tentativa de acesso físico	Regras do exercício, com jurídico e RH	Só com consentimento e escopo claros; proteja a identidade de quem cai e **não puna**
Físico	Controle de acesso, crachás, descarte de lixo, equipamentos esquecidos	Regras do exercício	Leve a **carta de autorização** e um contato de emergência: a segurança do local pode agir de verdade
Sistemas de IA	Injeção de *prompt*, vazamento de dados, uso excessivo de agência	OWASP Top 10 para LLM (p. 112)	Teste o sistema inteiro (ferramentas, dados, permissões), e não só o modelo

EQUIPE VERMELHA, AZUL E ROXA

As cores descrevem **papéis**. A equipe vermelha (*red team*) emula um adversário para atingir um **objetivo** (por exemplo, "chegar aos dados de notas sem ser detectada"), de forma furtiva e ao longo de semanas, e mede se a organização **detecta e responde**, e não quantas falhas existem. O teste de intrusão, ao contrário, vai em **amplitude**: busca o maior número de falhas em um escopo e um prazo curtos, sem precisar se esconder. A equipe azul (*blue team*) defende: monitora, detecta, responde e fortalece. O **purple team** não é uma terceira equipe, e sim uma **forma de trabalhar**: atacantes e defensores sentam juntos, executam uma técnica, veem o que a defesa enxergou e melhoram a detecção na hora. Um **time branco** (célula de controle) conhece as regras, coordena o exercício e faz o **desconflito**, para que a defesa não acione a polícia nem desligue a produção por causa de uma ação que, na verdade, era do exercício.

18 TESTES DE SEGURANÇA

CONTINUAÇÃO



Equipe vermelha
simula o adversário (TTPs reais)
testa pessoas, processos e tecnologia
entrega: caminhos de ataque e lacunas

Equipe roxa (prática)
a vermelha executa, a azul verifica
ajuste conjunto, em ciclos curtos
entrega: cobertura de detecção medida

Equipe azul
defende, detecta e responde
mantém logs, regras e runbooks
entrega: detecções e correções

Cores das equipes. A **vermelha** ataca, a **azul** defende, e a **roxa** é a prática de fazer as duas trabalharem juntas, medindo o que a defesa viu e o que deixou passar. Em exercícios maiores, uma equipe **branca** coordena e arbitra.

A rotina do *purple team* é um ciclo curto: escolher uma técnica do ATT&CK que importa para o seu cenário; **executá-la** de forma controlada, em laboratório ou em máquinas autorizadas; verificar **se houve telemetria e se houve alerta**; corrigir a lacuna (coletar o log que faltava, escrever ou ajustar a regra) e **repetir** até detectar. O **Atomic Red Team** (Red Canary) é uma biblioteca de pequenos testes, mapeados às técnicas do ATT&CK, que simulam o comportamento de forma controlada e trazem a limpeza; o **MITRE Caldera** é uma plataforma de código aberto que automatiza cadeias de emulação de adversário; e os produtos de **BAS** (*breach and attack simulation*) repetem isso continuamente, em escala.

TÉCNICA (ATT&CK)	O QUE A RODADA ENCONTROU	AÇÃO
T1053.005 · Tarefa agendada (persistência)	O registro existia (evento 4698), mas **não havia regra**	Escrever a regra Sigma e repetir o teste
T1070.001 · Limpeza dos logs do Windows	O alerta disparou em 2 minutos (evento 1102)	Registrar o resultado e nomear o dono do alerta
T1048 · Exfiltração por protocolo alternativo (DNS)	**Sem telemetria**: o resolvedor não registrava as consultas	Habilitar o log de DNS e executar de novo

- O saldo é típico: de três técnicas, **uma** gerou alerta, e as outras duas falhavam por falta de **visibilidade** ou de **regra**, o que o relatório de um teste de intrusão comum dificilmente mostraria. Por isso o *purple team* costuma ser o investimento de melhor retorno para um SOC já em operação (p. 144).

O RELATÓRIO DE TESTE DE INTRUSÃO

O relatório é o **produto** do teste: é ele que vira correção. Serve a dois públicos, a gestão (que decide e financia) e a equipe técnica (que corrige), e por isso tem camadas:

1. **Resumo executivo**
   Uma página, em linguagem de negócio: o que foi testado, o risco geral, os três achados que mais importam e o que fazer primeiro. Inclui **o que funcionou bem**.

2. **Escopo e método**
   O que entrou e o que ficou de fora, datas, referências (PTES, WSTG), contas usadas e **limitações** (janelas, ambientes, o que não foi testado).

18 TESTES DE SEGURANÇA — CONTINUAÇÃO

1 Achados
Um por seção: título, severidade, local, descrição, evidências, impacto e recomendação. Há um modelo logo abaixo.

2 Cadeias de ataque
Quando falhas pequenas se combinam em um caminho até o objetivo, descreva o **encadeamento**: é o que mais ensina sobre o risco real.

3 Plano de correção
Achados agrupados por causa-raiz, com sugestão de ordem, esforço e responsáveis.

4 Reteste
Confirmação por escrito de cada correção: corrigida, parcialmente corrigida ou persistente.

MODELO DE ACHADO DE UM RELATÓRIO

```
ACHADO PT-03    IDOR em GET /notas/{id}: leitura das notas de outros alunos
Severidade      Média (CVSS 3.1: 6,5); tratar com prioridade Alta (dados pessoais)
Vetor           CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Classe          OWASP API1:2023 (BOLA) · CWE-639
Local           GET https://api-hml.exemplo.edu.br/notas/{id}, autenticado como aluno
Descrição       A API devolve a nota de qualquer matrícula e não confere se o registro
                pertence à pessoa autenticada.
Evidência       A conta de teste "aluno1" leu o registro da conta "aluno2" (ID 1002).
                Requisição e resposta (dados de teste) no anexo B, item 3.
Impacto         Leitura das notas de todos os alunos, que são dado pessoal (LGPD).
                Não há alteração de dados nem acesso a outras funções.
Recomendação    Verificar a posse do objeto no servidor (consulta filtrada pelo usuário
                da sessão); adicionar teste automatizado de autorização; considerar
                identificadores não sequenciais como defesa adicional.
Reteste         Pendente: agendar após a correção.
```

O vetor acima equivale à nota **6,5**, que é "média": a exploração é fácil (rede, baixa complexidade, sem ação do usuário), mas exige uma conta (**PR:L**) e afeta só a **confidencialidade** (**C:H**). O pentester recomenda prioridade maior porque o dado é pessoal e a falha atinge todos os alunos; a nota mede a gravidade técnica, e a prioridade vem do contexto (p. 20).

> **Teste não é atestado:** Um teste mostra a **presença** de falhas, não a ausência (a ideia é de Dijkstra, sobre programas). Um relatório sem achados críticos significa que, **naquele escopo, janela e profundidade**, nada crítico foi encontrado; por isso o relatório deve declarar as limitações, e a leitura deve respeitá-las.

LENDO E PRIORIZANDO UM RELATÓRIO: A VISÃO DE QUEM DEFENDE

Receber o relatório é o começo do trabalho. Seis práticas transformam uma lista de achados em redução real de risco:

1 Priorize por contexto
Combine a nota (CVSS) com exposição à internet, exploração ativa (KEV), probabilidade (EPSS), valor do ativo e dados envolvidos (p. 20). Um 6,5 que expõe dados pessoais pode merecer mais urgência que um 9,8 em um servidor isolado.

2 Agrupe por causa-raiz
Doze achados podem vir de três causas: validação de entrada ausente na camada comum, autorização não centralizada e segredos no repositório. Corrigir a causa elimina a classe inteira.

18 TESTES DE SEGURANÇA — CONTINUAÇÃO

1 Vire ação rastreável
Cada achado vira um chamado com dono, prazo (conforme a severidade e a política) e critério de pronto, que é o **reteste**.

2 Corrija sem criar outra falha
A correção também é código: passa por revisão e por testes. Correção "de superfície" (bloquear só o exemplo do relatório) é a causa mais comum de achados que voltam.

3 Vire teste de regressão
Todo achado vira um teste automatizado no CI (p. 135), para que a falha não volte em uma refatoração.

4 Aceite o risco por escrito
Se não for corrigir, quem é dono do risco assina a aceitação, com prazo de revisão e controle compensatório (p. 20).

O exemplo seguinte automatiza a **matriz de autorização** (p. 53): cada linha diz **quem** pode fazer **o quê** em **qual rota**, e o teste falha se a API responder diferente. As linhas negativas reproduzem os achados do teste de intrusão. Aceitar 403 **ou** 404 nos casos negados é deliberado (algumas APIs respondem 404 para não revelar que o objeto existe): o que o teste proíbe é o 200.

TEST_AUTORIZACAO.PY: MATRIZ DE AUTORIZAÇÃO COM PYTEST

```
import os
import pytest
import requests

BASE = os.environ["API_BASE"]  # homologação, nunca produção
TOKEN = {p: os.environ[f"TOKEN_{p.upper()}"] for p in ("aluno1", "prof")}
OK, NEGADO, SEM_LOGIN = {200}, {403, 404}, {401}

# (papel, método, rota, status aceitos)
MATRIZ = [
    ("aluno1", "GET", "/notas/1001", OK),            # a própria nota
    ("aluno1", "GET", "/notas/1002", NEGADO),        # nota de outro aluno (o achado)
    ("aluno1", "PUT", "/notas/1001", NEGADO),        # aluno não altera nota
    ("prof", "PUT", "/notas/1001", OK),              # professor da turma altera
    ("aluno1", "GET", "/relatorios/turmas", NEGADO), # função administrativa
    (None, "GET", "/notas/1001", SEM_LOGIN),         # sem autenticação
]

@pytest.mark.parametrize("papel,metodo,rota,aceitos", MATRIZ)
def test_matriz_de_autorizacao(papel, metodo, rota, aceitos):
    headers = {"Authorization": f"Bearer {TOKEN[papel]}"} if papel else {}
    corpo = {"nota": 8.0} if metodo == "PUT" else None
    r = requests.request(metodo, BASE + rota, headers=headers, json=corpo, timeout=10)
    assert r.status_code in aceitos, f"{papel} {metodo} {rota} -> {r.status_code}"
```

Voltando ao passo 3 ("vire ação rastreável"), um exemplo de **política de prazos**, que é ilustrativo e deve ser ajustado à sua organização, ao seu risco e às suas obrigações contratuais:

SEVERIDADE (COM O CONTEXTO)	PRAZO PARA CORRIGIR OU MITIGAR	OBSERVAÇÃO
Crítica (explorada ou dado exposto)	Até 7 dias, ou mitigação imediata	Com sinal de exploração, acione o plano de incidentes (p. 156)
Alta	Até 30 dias	Exige dono, data e reteste agendado

18 **TESTES DE SEGURANÇA** CONTINUAÇÃO

SEVERIDADE (COM O CONTEXTO)	PRAZO PARA CORRIGIR OU MITIGAR	OBSERVAÇÃO
Média	Até 90 dias	Agrupe as correções em um ciclo planejado
Baixa	No ciclo seguinte	Registre e revise a aceitação do risco todo ano

CONTRATAR UM TESTE DE INTRUSÃO: O QUE PERGUNTAR

CHECKLIST

Oito perguntas ao fornecedor, antes de assinar:

1. Que metodologia e que referências guiam o trabalho (PTES, WSTG, ASVS, MASVS)?
2. Quantas horas de teste **manual** estão previstas por alvo? O relatório de um *scanner* não é um teste de intrusão.
3. Quem executa e quem assina o relatório? Peça a qualificação e a experiência da equipe.
4. Há um relatório de amostra, anonimizado, com resumo executivo, evidências reproduzíveis e recomendações claras?
5. O reteste está incluído? Por quanto tempo?
6. Como as evidências e os dados serão protegidos e descartados (NDA, cifra, LGPD)?
7. Como são tratadas as falhas críticas: aviso imediato, contato 24×7, parada do teste?
8. Que responsabilidade e que seguro cobrem um dano acidental ao ambiente?

Compare as propostas pelo **escopo e pelo esforço manual**, e não só pelo preço: um teste muito barato costuma ser um *scanner* com outra embalagem.

BUG BOUNTY E POLÍTICA DE DIVULGAÇÃO (VDP)

Nem toda falha é achada por quem foi contratado: pesquisadores independentes, estudantes e clientes esbarram nelas o tempo todo. Organizações maduras **abrem um canal** para isso, em quatro níveis de compromisso:

MODELO	O QUE OFERECE	QUANDO FAZ SENTIDO	O QUE EXIGE DE QUEM RECEBE
Canal básico	E-mail e arquivo `security.txt` (p. 90), sem regras formais	Qualquer *site* ou produto público	Caixa monitorada e uma pessoa responsável
VDP	Escopo, regras, **porto seguro**, prazos e reconhecimento público; sem pagamento garantido	Universidades, órgãos públicos e empresas que querem ser avisados de boa-fé	Triagem, prazo de resposta e jurídico alinhado
***Bug bounty* privado**	Pesquisadores convidados, escopo fechado e recompensa por achado válido	Produtos maduros, com equipe que corrige rápido	Orçamento, triagem em volume e correção ágil
***Bug bounty* público**	Qualquer pessoa, tabela de valores por severidade, em plataforma intermediária (HackerOne, Bugcrowd, Intigriti, YesWeHack)	Produtos grandes e muito expostos	Equipe dedicada e orçamento contínuo

O **porto seguro** (*safe harbor*) é a promessa da organização de **não tomar medidas legais** contra quem seguir a política de boa-fé. É valioso, mas tem limite: é um compromisso da organização e **não vincula as autoridades**. Quando o alvo é a administração pública, a ação penal pelo crime do art. 154-A **não depende de representação da vítima** (art. 154-B), e a promessa da instituição não impede, por si só, o Ministério Público de agir. Por isso, mesmo com VDP, quem pesquisa deve ficar estritamente **dentro do escopo e das regras**.

18 TESTES DE SEGURANÇA

CONTINUAÇÃO

Para quem relata

- Leia o escopo e as regras **antes** de testar; o que não está listado está fora.
- Use apenas contas e dados **seus** e pare quando provar o impacto.
- Não acesse, copie nem guarde dados de outras pessoas; não degrade o serviço.
- Relate com passos reproduzíveis, impacto e evidências sem dados de terceiros.
- Mantenha sigilo até a correção ou o prazo combinado.
- **Nunca** ameace divulgar para exigir pagamento: pode configurar extorsão (CP, art. 158).

Para quem recebe

- Confirme o recebimento em poucos dias e informe um número de acompanhamento.
- Valide, classifique (CVSS e contexto) e responda dentro do prazo prometido.
- Corrija a causa-raiz, e não só o exemplo enviado; peça o reteste ao autor.
- Trate o relato como **dado sensível**: acesso restrito e canal cifrado.
- Credite quem relatou, se a pessoa quiser, e avise quando a correção sair.
- Revise a política todo ano e renove o `security.txt` antes de **expirar**.

POLÍTICA DE DIVULGAÇÃO DE VULNERABILIDADES (MODELO RESUMIDO)

```
POLÍTICA DE DIVULGAÇÃO · Universidade Exemplo
ESCOPO          *.portal.exemplo.edu.br e o aplicativo "Exemplo Aluno" (Android e iOS)
FORA DO ESCOPO  sistemas de terceiros · negação de serviço · engenharia social
                · acesso físico · relatórios de scanner sem prova de impacto
REGRAS          use apenas contas suas; pare ao demonstrar o impacto; não acesse, não
                copie nem retenha dados de outras pessoas; não degrade o serviço
PORTO SEGURO    não tomaremos medidas legais contra quem seguir esta política de boa-fé
                (a promessa não vincula terceiros nem autoridades públicas)
COMO RELATAR    seguranca@exemplo.edu.br (chave em /.well-known/security.txt)
PRAZOS          confirmação em 3 dias úteis · avaliação em 10 dias úteis
                correção conforme a severidade; divulgação coordenada em até 90 dias
RECONHECIMENTO  página de agradecimentos e certificado; sem recompensa em dinheiro
```

DIVULGAÇÃO COORDENADA DE VULNERABILIDADES (CVD)

A **divulgação coordenada** é o processo de coletar a informação de quem achou a falha, **coordenar** quem precisa saber (fornecedores, provedores, quem implanta o produto) e só então divulgar a falha e a correção, inclusive ao público. É um **processo**, e não um evento. Prefere-se "coordenada" a "responsável" porque este é um juízo de valor que cada parte define a seu modo (a distinção é do CERT/CC). Os papéis são quatro: **quem achou** (*finder*) e relata; o **fornecedor** (*vendor*), que corrige; o **coordenador**, que intermedeia (como o CERT/CC, nos EUA, e equipes nacionais de resposta, como o CERT.br, que pode orientar o contato); e **quem implanta e usa** o produto, que precisa aplicar a correção. As normas ISO/IEC 29147 (divulgação) e 30111 (tratamento interno) descrevem o que o fornecedor deve fazer (p. 90).



Os prazos variam, e o mais citado é o de **90 dias** entre o relato e a divulgação (convenção popularizada pelo Google Project Zero), prorrogável por acordo e antecipável quando há **exploração ativa**. Se o fornecedor não responde, o coordenador pode intermediar o contato e, como último recurso, quem achou divulga ao fim do prazo, para que os usuários consigam se proteger. Uma vulnerabilidade pública em um produto recebe um **CVE**, atribuído por uma **CNA** (*CVE Numbering Authority*): fabricantes e projetos grandes, coordenadores, plataformas de *bug bounty* e, em última instância, a MITRE. O registro é depois enriquecido com CVSS, CWE e CPE no NVD (p. 20).

18 TESTES DE SEGURANÇA

CONTINUAÇÃO

LIMITES LEGAIS E ÉTICOS NO BRASIL

A fronteira entre pesquisa e crime é a **autorização**. O que segue é um panorama para estudo, e não aconselhamento jurídico: diante de dúvida real, consulte um advogado **antes** de testar. As penas estão no quadro de p. 27.

NORMA	O QUE TRATA	O QUE SIGNIFICA PARA QUEM TESTA
Código Penal, art. 154-A (Leis 12.737/2012 e 14.155/2021)	Invadir dispositivo informático **de uso alheio** para obter, adulterar ou destruir dados ou instalar vulnerabilidades, **sem autorização** do usuário	Desde 2021 a lei não exige mais a "violação de mecanismo de segurança": uma senha fraca, ou nenhuma, **não autoriza** o acesso. Produzir ou distribuir programa **com o intuito** de permitir a conduta também é crime (§1º): a ferramenta de duplo uso é lícita, e a **intenção** conta
Código Penal, art. 266, §1º	Interromper ou perturbar serviço telemático ou de informação de utilidade pública	Teste de negação de serviço contra serviço essencial pode cair aqui: só com autorização específica, janela e plano de reversão
Código Penal, art. 171, §2º-A	Fraude eletrônica com informações da vítima, induzida a erro por redes sociais, telefone ou e-mail	*Phishing* "de teste" **sem autorização e sem controle** pode ser tratado como fraude (reclusão de 4 a 8 anos)
Lei 9.296/1996, art. 10	Interceptação de comunicações de informática ou telemática sem ordem judicial	Capturar o tráfego de terceiros (por exemplo, em uma rede Wi-Fi alheia) pode configurar interceptação: capture só em redes suas ou autorizadas por escrito
LGPD (Lei 13.709/2018)	Tratamento de dados pessoais	O teste costuma tocar dados pessoais: o contrato define papéis (a equipe contratada tende a atuar como operadora), finalidade, minimização, guarda e descarte (p. 59)

> **Sem autorização, é crime:** Boa intenção, curiosidade ou "só para avisar" **não** substituem a autorização. Varrer portas, tentar senhas ou "dar uma olhada" em um sistema que não é seu pode configurar crime mesmo que nada seja alterado. Se você achar uma falha por acaso: **pare, não explore e relate pelo canal oficial**.

Os códigos de ética de entidades como a ISC2 e a ISACA convergem em poucos princípios: **autorização** antes de agir; **proporcionalidade** (o mínimo de acesso e de dano para provar o ponto); **confidencialidade** dos achados e dos dados; **honestidade** no relato, inclusive sobre limites e erros; e **divulgação coordenada**, para que a correção chegue antes da exploração.

CENÁRIO

Você percebeu que trocar o número na URL abre a página de notas de outro aluno. O que fazer?

1. **Pare** no primeiro indício. Não teste outros números "para ter certeza" e não baixe nada.
2. **Anote** só o necessário: endereço, data e hora, o que você fez e o que apareceu, sem copiar dados de terceiros.
3. **Relate** pelo canal oficial (o `security.txt`, a TI ou a ouvidoria), por escrito, e guarde uma cópia.
4. **Não divulgue** em redes sociais e não peça nada em troca.
5. **Aguarde** a resposta e combine o prazo antes de falar do caso em público.

Ainda assim há risco: até onde se sabe, a lei brasileira não traz uma exceção expressa para quem encontra falhas de boa-fé. Por isso uma **política com porto seguro** protege quem relata, e por isso o teste de um sistema deve ser **contratado**.

COMO APRENDER DE FORMA LEGAL

Dá para desenvolver as habilidades ofensivas **sem tocar em sistema alheio**: há aplicações feitas para serem atacadas, plataformas com laboratórios e competições.

18 TESTES DE SEGURANÇA — CONTINUAÇÃO

RECURSO	FORMATO	BOM PARA
OWASP Juice Shop	Loja *web* intencionalmente insegura (Node.js), com placar de desafios; roda na sua máquina	Primeiros passos em *web* e API
OWASP WebGoat	Aplicação Java com lições guiadas e explicações	Entender o porquê de cada falha
DVWA	Aplicação PHP e MySQL com níveis de dificuldade	Falhas *web* clássicas
PortSwigger Web Security Academy	Trilhas e laboratórios *online* gratuitos, da empresa do Burp Suite	*Web* em profundidade
TryHackMe	Salas guiadas no navegador, em trilhas por nível (conteúdo gratuito e pago)	Começar do zero
Hack The Box	Máquinas e desafios em rede de laboratório, mais a HTB Academy (planos gratuitos e pagos)	Prática intermediária e avançada
picoCTF	Competição educacional da Carnegie Mellon University, com acervo de desafios	Estudantes
OverTheWire	Jogos de linha de comando; o Bandit é o primeiro	Linux e fundamentos
Metasploitable e VulnHub	Máquinas virtuais vulneráveis para rodar no seu laboratório	Redes e sistemas

As competições *capture the flag* (CTF) escondem **bandeiras** (*flags*, textos secretos) em desafios. O formato mais comum é o *jeopardy*, com categorias como **web**, **criptografia**, **exploração binária** (*pwn*), **engenharia reversa**, **forense** e **OSINT**; no formato *attack-defense*, cada equipe defende o próprio serviço enquanto ataca o dos outros. O CTFtime reúne calendário e ranking. Escrever o *write-up* de cada desafio resolvido (o que tentou, o que funcionou, o que aprendeu) é o melhor treino para o relatório de um teste real.

LABORATÓRIO DOMÉSTICO ISOLADO

1 Isole a rede
Use uma rede interna ou *host-only* no hipervisor (VirtualBox, VMware, KVM ou Proxmox), sem rota para a rede da casa. Alvos vulneráveis **não** devem ter acesso à internet.

2 Monte a máquina de ataque
Uma VM (Kali Linux ou Parrot OS) que você pode destruir e recriar a qualquer momento.

3 Suba os alvos
Aplicações e VMs vulneráveis, como o Juice Shop em contêiner, o Metasploitable e máquinas do VulnHub. **Nunca** os exponha à internet.

4 Use *snapshots*
Tire um antes de cada exercício e volte ao estado limpo ao terminar.

5 Observe a defesa
Instale uma pilha de monitoramento aberta (como Wazuh ou Security Onion) e veja o **rastro** dos seus testes: é um *purple team* doméstico (**p. 144**).

6 Documente
Caderno com objetivo, passos, resultado e o que a defesa viu. É a base do seu portfólio.

18 TESTES DE SEGURANÇA — CONTINUAÇÃO

ALVO VULNERÁVEL EXPOSTO SÓ NO SEU COMPUTADOR

```
# Aplicação intencionalmente vulnerável: publique SÓ em 127.0.0.1
docker run --rm -p 127.0.0.1:3000:3000 bkimminich/juice-shop
# depois, abra http://127.0.0.1:3000 no navegador
```

CARREIRA E CERTIFICAÇÕES: UM PANORAMA

Há muitos caminhos em segurança (ofensiva, defesa, governança, auditoria), e as certificações são apenas **um** deles. O quadro resume as mais citadas em vagas, sem ordem de valor:

CERTIFICAÇÃO	EMISSOR	FOCO	PERFIL
Security+	CompTIA	Fundamentos amplos de segurança	Porta de entrada, perfil generalista
eJPT	INE Security	Teste de intrusão básico, com prova prática	Primeiro passo prático em ofensiva
PNPT	TCM Security	Teste de intrusão de rede e de *Active Directory*, com relatório e apresentação oral	Prática próxima da rotina real
OSCP	OffSec	Teste de intrusão em exame prático longo (cerca de 24 horas), com relatório	Vagas de teste de intrusão
CEH	EC-Council	Visão ampla de técnicas e ferramentas; há versão com prova prática	Conhecimento geral de ofensiva
GIAC (GPEN, GWAPT, GCIH e outras)	GIAC, ligada ao SANS	Certificações por especialidade: teste de intrusão, web, resposta a incidentes	Trilhas corporativas; custo alto
CISSP	ISC2	Gestão e arquitetura de segurança, em oito domínios	Exige experiência comprovada; perfil sênior ou de gestão
CISM	ISACA	Gestão de segurança, riscos e incidentes	Exige experiência em gestão

Requisitos, preços, versões e reconhecimento mudam com frequência: **confira no emissor** e em vagas reais da área em que você quer atuar. A certificação abre portas, mas o que sustenta uma carreira é a **prática demonstrável**: *write-ups* de CTF, laboratório documentado, contribuições a ferramentas abertas e, mais tarde, relatórios em programas de recompensa (quando a política permite divulgá-los).

UM PROGRAMA DE TESTES AO LONGO DO CICLO DE VIDA

Nenhum teste isolado basta. Cada um responde a uma pergunta diferente, em um momento diferente, e o conjunto forma o **programa de testes** da organização:

MOMENTO	TESTES	QUEM FAZ	RESULTADO
Requisitos e projeto	Requisitos de segurança e casos de abuso (ASVS), modelagem de ameaças (p. 90)	Produto e segurança	Requisitos verificáveis e riscos priorizados
Código e *build* (a cada *commit*)	SAST, SCA, varredura de segredos, DAST, *fuzzing* e testes de autorização (p. 135)	Desenvolvimento, automatizado	A falha **bloqueia** a integração

18 TESTES DE SEGURANÇA — CONTINUAÇÃO

MOMENTO	TESTES	QUEM FAZ	RESULTADO
Pré-lançamento	Teste de intrusão caixa cinza e revisão do código dos fluxos críticos	Equipe independente ou contratada	Relatório, correção e reteste
Produção (contínuo)	Varredura de vulnerabilidades, VDP ou *bug bounty*, monitoramento (p. 144)	Segurança e operações	Chamados, prazos e métricas
Anual ou após grande mudança	*Red team*, *purple team* e exercício de mesa (p. 156)	Equipe vermelha, SOC e liderança	Lacunas de detecção e de resposta

PARA FIXAR

Conceitos

- Diferencie avaliação de vulnerabilidades, teste de intrusão, *red team* e *purple team* pela pergunta que cada um responde.
- Que documentos materializam a autorização de um teste? O que o escopo e as regras de engajamento precisam conter?
- O que são testes de caixa preta, cinza e branca? Por que a caixa cinza é a mais usada?
- Por que se prefere "divulgação coordenada" a "responsável"? Descreva as etapas e o prazo de referência.

Aplicação

- Interprete o vetor `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`: o que diz cada métrica? Por que a prioridade pode ser maior que a nota?
- Um relatório traz 14 achados. Como agrupá-los por causa-raiz, decidir a ordem de correção e deixar testes de regressão no CI?
- Escreva cinco linhas da matriz de autorização de um sistema de biblioteca (papéis, rotas e status aceitos).
- Você achou uma falha no portal da faculdade. O que fazer e o que evitar? Como a existência de um porto seguro muda o risco?

APROFUNDE-SE

Leituras e referências

- NIST SP 800-115 — NIST, 2008: guia técnico de testes e avaliações de segurança da informação.
- PTES, Penetration Testing Execution Standard — comunidade PTES: as sete fases de um teste de intrusão.
- OWASP Web Security Testing Guide — OWASP: casos de teste para aplicações web.
- CERT Guide to Coordinated Vulnerability Disclosure — CERT/CC: papéis, etapas e boas práticas da divulgação coordenada.
- Web Security Academy — PortSwigger: laboratórios gratuitos de segurança web.
- OWASP Juice Shop — OWASP: aplicação intencionalmente vulnerável para praticar em laboratório.
- Atomic Red Team — Red Canary: testes mapeados ao ATT&CK para validar detecções.

MÓDULO 19 · PESSOAS

19 PESSOAS, CULTURA E PROTEÇÃO PESSOAL

A tecnologia só protege quando as pessoas ajudam: hábitos, cultura e como se proteger no dia a dia.

A tecnologia mais bem projetada falha quando alguém é enganado, está cansado ou recebe uma regra impossível de seguir. Este módulo trata do **fator humano**: por que os golpes funcionam, como reconhecê-los e reagir, como proteger a vida digital e o dinheiro e como as organizações constroem uma **cultura** em que o caminho seguro é o mais fácil. O que você aprende aqui vale para a vida inteira e é também o que leva para o primeiro emprego.

O FATOR HUMANO EM NÚMEROS

O panorama do "elemento humano" nas violações (62% no DBIR 2026) está em **p. 11**. Três números ajudam a entender **como as pessoas se comportam** diante de golpes e depois deles:

19.500+
funcionários em um experimento de oito meses, com dez campanhas de phishing simulado
UC SAN DIEGO HEALTH · 2025

2% a 31%
de cliques, conforme a isca: falso aviso de senha (1,8%) contra falso aviso de férias (30,8%)
UC SAN DIEGO HEALTH · 2025

9%
do valor contestado foi recuperado, em média, pelo MED do Pix (valor aproximado)
BANCO CENTRAL · 2025

A lição não é "as pessoas são o elo fraco". **A isca pesa mais que a pessoa**, a recuperação do dinheiro é parcial e a prevenção é a parte mais barata. Pessoas bem apoiadas também são **sensores**: quem reporta um e-mail suspeito antes de o filtro aprender a barrá-lo protege a organização inteira.

POR QUE OS GOLPES FUNCIONAM: GATILHOS PSICOLÓGICOS

A engenharia social não explora falta de inteligência, e sim **atalhos mentais** que todos usamos para decidir rápido. Quem aplica o golpe monta as condições em que esses atalhos vencem a reflexão: pressa, emoção e uma situação que parece familiar. Conhecer o gatilho é a melhor defesa, porque ele **se denuncia**: sempre que sentir pressa, medo ou empolgação diante de um pedido, pare.

GATILHO	COMO É EXPLORADO	CONTRAMEDIDA
Autoridade	"Aqui é o banco, o diretor, a Receita": ordens de quem parece ter poder	Verifique por **outro canal**; autoridade de verdade aceita a confirmação
Urgência e medo	Prazo curto, bloqueio de conta, multa, parente em apuros	Desacelere: a pressa é o sinal; confirme **antes** de agir
Escassez e ganância	"Últimas vagas", promoção relâmpago, retorno garantido, prêmio	Desconfie do bom demais; confirme a oferta no site oficial
Reciprocidade	Um favor ou brinde antes do pedido (suporte "gratuito")	Um favor não gera dívida: avalie o pedido sozinho

19 PESSOAS, CULTURA E PROTEÇÃO PESSOAL — CONTINUAÇÃO

GATILHO	COMO É EXPLORADO	CONTRAMEDIDA
Prova social	"Todos já atualizaram", depoimentos e perfis falsos	Curtidas e multidões se fabricam; procure fonte independente
Familiaridade	Mensagem de um contato conhecido (conta clonada) ou de um colega	Contato conhecido pode estar comprometido: ligue ou pergunte algo que só a pessoa saberia

Os modelos de **IA generativa** tornaram os golpes mais **baratos e convincentes**: textos sem erros de português, voz clonada a partir de pouco áudio e vídeos falsos (*deepfakes*) em chamadas. O critério de defesa muda: em vez de procurar "erros" no texto, **verifique o pedido**, e não a aparência.

ANATOMIA DE UM PHISHING

O *phishing* é a mensagem que se passa por uma entidade confiável para obter **credenciais, dinheiro ou uma ação** (instalar, abrir, autorizar). Chega por e-mail, por SMS (*smishing*), por ligação (*vishing*), por mensageiros, por redes sociais e por **QR codes** (*quishing*). A figura mostra um exemplo fictício com os sinais mais comuns.



Anatomia de um phishing (exemplo fictício). Nenhum sinal isolado prova a fraude, mas a **combinação** de remetente estranho, urgência, link disfarçado e pedido de dados é típica. Na dúvida, não use o link nem o anexo: abra o aplicativo ou digite o endereço oficial.

Quatro hábitos de verificação eliminam a maior parte dos casos:

- **Desconfie do remetente real**, e não do nome exibido: confira o endereço completo e o domínio, letra por letra.
- **Não use o link da mensagem** para entrar em contas: abra o aplicativo ou digite o endereço. No computador, passe o mouse sobre o link; no celular, pressione e segure para ver o destino.
- **O cadeado do navegador não prova nada**: ele indica conexão cifrada (HTTPS), e golpistas também obtêm certificados para sites falsos.
- **Anexos e QR codes inesperados** valem como links: confirme a origem antes de abrir ou escanear.

19 PESSOAS, CULTURA E PROTEÇÃO PESSOAL

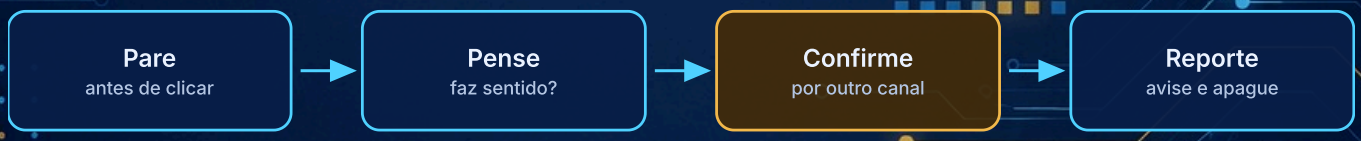
CONTINUAÇÃO

OS GOLPES MAIS COMUNS E O QUE FAZER

GOLPE	COMO FUNCIONA	SINAL DE ALERTA	O QUE FAZER
Phishing de credenciais	Página igual à do serviço rouba usuário, senha e código de verificação	Link para "regularizar", "reativar" ou "ver fatura"; pedido da senha	Entre pelo app ou digitando o endereço; use passkey; troque a senha se a digitou
Fraude de fatura (BEC)	E-mail real ou parecido de um fornecedor ou chefe pede pagamento ou **troca os dados bancários** de uma fatura	Urgência, sigilo, pagamento fora do fluxo, mudança de conta	Confirme por ligação a um número já conhecido; dupla aprovação de pagamentos e de mudanças de conta
Falso suporte técnico	"Seu computador está infectado": pedem acesso remoto ou pagamento	Contato que você não pediu; janela com telefone; programa de acesso remoto	Desligue; ninguém legítimo liga avisando de vírus; procure o suporte pelo site oficial
Clonagem de WhatsApp	Pedem o **código de 6 dígitos** do SMS, assumem a conta e pedem dinheiro aos contatos	"Mandei o código por engano"; amigo que pede dinheiro e não atende ligação	Nunca informe códigos; ative a verificação em duas etapas; confirme por ligação
Falsa central e golpes de Pix	Ligação de "central do banco", falso comprovante, falso motoboy, QR ou chave adulterada	Pedido para transferir "para conta segura", instalar app ou entregar o cartão	O banco não pede senha, cartão nem transferência; desligue e ligue para o número do cartão
Deepfake e voz clonada	Voz ou vídeo sintético de um parente ou executivo pede dinheiro ou segredo	Urgência, sigilo, pedido fora do padrão, "conexão ruim"	Combine uma **palavra-código** em família; confirme por outro canal; dupla aprovação na empresa
Emprego e investimento falsos	"Tarefas pagas", retorno garantido, depósito inicial, pirâmides	Ganho alto e fácil; pagamento antecipado; pedem documentos	Pesquise a empresa em fontes independentes; **nunca pague para receber**

PARE, PENSE, CONFIRME, REPORTE

Todos os golpes acima se reduzem a um hábito de quatro tempos, que funciona até contra golpes que você nunca viu:



Pare diante de qualquer pedido de dinheiro, dados, código ou instalação, sobretudo se vier com pressa ou emoção. **Pense**: eu esperava isso? Quem ganha se eu agir agora? O remetente é quem diz ser? **Confirme** por um **canal independente**: o número que está no cartão ou no site oficial, o aplicativo aberto por você, a pessoa em carne e osso; nunca o telefone ou o link que veio na própria mensagem. **Reporte** à TI, ao banco ou à plataforma e apague: seu aviso protege quem receber a mesma mensagem depois.

19 PESSOAS, CULTURA E PROTEÇÃO PESSOAL — CONTINUAÇÃO

E SE EU CAIR? AS PRIMEIRAS HORAS

1 Pare de interagir
Feche a página, não digite mais nada e, se instalou algo, desconecte o aparelho da internet.

2 Troque a senha de outro aparelho
Altere a senha do serviço e das contas que a reutilizam, encerre as outras sessões e ative ou revise o MFA.

3 Revise a recuperação
Confira o e-mail e o telefone de recuperação, as regras de encaminhamento e os aplicativos autorizados: golpistas costumam acrescentar os deles.

4 Avise quem pode agir
O banco (bloqueio de cartão, limites, pedido de devolução do Pix), a TI da escola ou do trabalho e, se o chip perder o sinal de repente, a operadora.

5 Procure sinais de uso
Extratos, mensagens enviadas, dispositivos conectados e novos cadastros em seu nome.

6 Registre e denuncie
Guarde capturas de tela, horários e valores; faça o boletim de ocorrência; avise seus contatos se a conta foi clonada.

Prevenir vale mais que recuperar: O **MED** (Mecanismo Especial de Devolução) do Pix permite pedir a devolução em caso de fraude ou de falha operacional, mas **não garante o ressarcimento**: segundo material do Banco Central, o mecanismo recuperou, em média, cerca de 9% do valor contestado em 2025. O MED 2.0 amplia o rastreamento do dinheiro além da primeira conta. Agir nas primeiras horas ajuda, mas o melhor é **não cair**.

HIGIENE DIGITAL: CONTAS, DISPOSITIVOS E REDES

O aprofundamento técnico está nos módulos p. 44 (identidade), p. 34 (criptografia) e p. 67 (redes). Para a vida pessoal, estes hábitos entregam o maior retorno por minuto investido:

HÁBITO	RISCO QUE REDUZ	ESFORÇO
Gerenciador de senhas com uma frase longa como senha-mestra e uma senha diferente em cada serviço	Reuso de senhas (*credential stuffing*) e senhas fracas	Médio, uma vez
Senhas longas: o tamanho vale mais que símbolos (NIST: ao menos 15 caracteres quando a senha é o único fator)	Adivinhação e força bruta offline	Baixo
MFA em e-mail, banco e redes: passkey ou chave FIDO2, depois app autenticador; SMS só se não houver outra opção	Phishing de senha e *SIM swap*	Baixo
Códigos de recuperação guardados offline e e-mail de recuperação protegido com MFA	Perder a conta; sequestro pela recuperação	Baixo
Atualizações automáticas do sistema, do navegador e dos aplicativos	Exploração de vulnerabilidades, principal vetor de acesso inicial no DBIR 2026	Baixo
Bloqueio de tela (PIN longo ou biometria), disco cifrado e localização remota	Perda e roubo de aparelho	Baixo
Backup 3-2-1 (3 cópias, 2 mídias, 1 fora de casa), com restauração testada	Ransomware, falha de disco, perda do aparelho	Médio
Permissões mínimas e aplicativos só de lojas oficiais	Coleta excessiva, *stalkerware*, malware	Baixo

19 PESSOAS, CULTURA E PROTEÇÃO PESSOAL — CONTINUAÇÃO

HÁBITO	RISCO QUE REDUZ	ESFORÇO
Wi-Fi público: prefira dados móveis e sites em HTTPS; uma VPN confiável protege o trecho até o servidor dela, mas não impede phishing nem torna você anônimo	Rede falsa (*evil twin*) e interceptação	Baixo
Pendrives e carregadores desconhecidos: não conecte; use o seu próprio carregador	Malware em mídia removível	Baixo

PRIVACIDADE E DINHEIRO

Reduza a exposição e use seus direitos

- Pesquise **seu próprio nome**, e-mail e telefone: o que um golpista descobriria sobre você?
- Remova de perfis públicos a data de nascimento, o endereço, a rotina e a localização em tempo real.
- Não publique **documentos, boletos, crachás, cartões ou passagens**: códigos de barras e QR codes guardam dados.
- Respostas de "perguntas de segurança" podem ser inventadas e guardadas no gerenciador.
- A **LGPD** garante ao titular acesso, correção, eliminação, portabilidade e informação sobre o compartilhamento (art. 18): comece pelo encarregado (DPO) da empresa e, se não resolver, procure a **ANPD**.
- Ter seus dados (CPF, endereço) não prova que quem liga é do banco: **dado vazado vira isca**.

Proteja o dinheiro (Brasil)

- **Alertas** de todas as transações no aplicativo do banco.
- **Limites** de Pix e de cartão ajustados ao seu uso, com limite noturno baixo.
- **Cartão virtual** para compras online e bloqueio temporário pelo app.
- **Chip protegido**: ative o PIN do SIM (vale para chip roubado) e avise a operadora se o sinal cair de repente (pode ser troca fraudulenta de chip); evite SMS como segundo fator em contas financeiras.
- **WhatsApp** com verificação em duas etapas; revise "aparelhos conectados" e nunca passe códigos.
- **Registrato** (Banco Central): consulte contas, chaves Pix e relacionamentos em seu CPF.
- Quem pede que você "transfira para uma conta segura" é golpista: o banco **nunca** pede isso.

SEGURANÇA FÍSICA NO DIA A DIA

SITUAÇÃO	COMO A FRAUDE ACONTECE	BOA PRÁTICA
Carona (*tailgating*)	Alguém entra atrás de você na porta, "esqueceu o crachá" ou carrega caixas	Cada pessoa usa o próprio crachá; ofereça-se para chamar a recepção em vez de abrir a porta
Crachás e chaves	Empréstimo, esquecimento ou cópia	Guarde sempre; reporte a perda **na hora**, para revogar o acesso
Tela e mesa limpas	Papéis, anotações e telas visíveis a visitantes ou câmeras	Bloqueie a tela ao sair; guarde documentos; use trituradora
Shoulder surfing	Alguém espia a tela ou a digitação do PIN	Filtro de privacidade; atenção em transporte e cafés
Mídias achadas e lixo	Pendrive "perdido" de propósito; documento descartado inteiro	Não conecte mídias achadas; triture documentos e apague mídias antes de descartá-las

19 **PESSOAS, CULTURA E PROTEÇÃO PESSOAL** CONTINUAÇÃO

PROGRAMAS DE CONSCIENTIZAÇÃO: O QUE FUNCIONA

Conscientização é **mudar comportamento**, e não distribuir informação. O experimento mais citado dos últimos anos (UC San Diego Health, 2025: mais de 19.500 funcionários, oito meses e dez campanhas simuladas) concluiu que o treinamento anual obrigatório **não teve relação significativa** com a chance de cair em phishing e que o treinamento mostrado **depois do clique** reduziu a chance de novos cliques em cerca de 2%; a maioria das pessoas dedicou menos de um minuto ao material. Os autores recomendam priorizar **controles técnicos**, como MFA e gerenciadores de senhas que só preenchem no domínio correto. A lição não é abandonar o treinamento, e sim **não pedir dele o que ele não entrega** sozinho.

O que tende a funcionar

- **Controles técnicos primeiro**: MFA resistente a phishing, filtros de e-mail (SPF, DKIM, DMARC), isolamento de links e anexos, bloqueio de macros.
- Treinamento **curto, frequente e relevante** para a função (financeiro, RH, TI, executivos).
- **Canal de reporte** de um clique, com resposta rápida e agradecimento.
- Retorno imediato e **positivo**, com exemplos reais da organização.
- Medir **comportamento** (reporte, tempo até reportar), e não só presença em curso.

O que tende a falhar

- Curso anual **só para cumprir tabela**, com prova feita de qualquer jeito.
- Simulações para **punir ou expor** quem clicou, ou com iscas cruéis (falso bônus, demissão, doença).
- Culpar o usuário e **esconder** a falha: as pessoas deixam de reportar.
- Medir só a **taxa de clique**: no estudo, ela variou de 1,8% a 30,8% conforme a isca.
- Regras que ninguém consegue cumprir (senha trocada todo mês), que geram atalhos.

Simular phishing pode ser legítimo, desde que haja **política prévia e comunicada**, proporcionalidade, relatórios agregados que protejam a identidade, envolvimento do RH e do jurídico e foco em aprender, e não em "pegar" alguém. Testes de engenharia social contratados seguem as regras de autorização do módulo p. 166.

CULTURA DE SEGURANÇA

Cultura é o que as pessoas fazem quando ninguém está olhando e o que consideram seguro dizer em voz alta. Três pilares a sustentam: **liderança** que dá o exemplo e financia; **segurança psicológica** para reportar erros, a chamada **cultura justa** (*just culture*), que responsabiliza escolhas imprudentes e não falhas humanas honestas; e **facilidade** de fazer o certo.

O que faz a organização subir de degrau: um **canal de reporte** simples e agradecido (um botão "reportar phishing" e retorno em horas); os *security champions*, voluntários nas equipes que levam a segurança ao dia a dia; **análises sem culpa** dos incidentes (p. 156); e a **segurança centrada no ser humano**, que parte de uma regra: o caminho seguro deve ser o mais fácil. Login único com passkeys, atualizações automáticas e ferramentas aprovadas que funcionem bem valem mais que dez avisos. Quando as pessoas recorrem a ferramentas não autorizadas (*shadow IT*), o caminho oficial provavelmente atrapalha. Do mesmo modo, regras que geram **fadiga** (excesso de alertas, senhas trocadas sem motivo, pedidos de MFA em sequência) produzem atalhos, e não obediência. A escada abaixo resume os degraus da maturidade:

19 PESSOAS, CULTURA E PROTEÇÃO PESSOAL

CONTINUAÇÃO

Escada de maturidade da cultura de segurança (modelo didático)



Cultura de segurança em degraus. Subir um degrau muda **o que se mede**: de "nada" ou "incidentes" para **reporte, tempo de resposta e comportamento**. O quadro é didático, não um padrão.

POLÍTICAS E O CICLO DE VIDA DAS PESSOAS

As políticas precisam ser **curtas, claras e aplicáveis**: uso aceitável de recursos, dispositivos pessoais (BYOD), tela e mesa limpas, classificação da informação, uso de IA generativa (que dados não colar em ferramentas externas), trabalho remoto e reporte de incidentes. Cada uma com exemplos, dono, revisão anual e um caminho formal para **exceções**. O ciclo de vida de cada pessoa na organização, o **JML** (*joiner, mover, leaver*), é onde os acessos nascem, se acumulam ou sobram (p. 53):

ETAPA	AÇÕES DE SEGURANÇA	SE FALHAR
Entrada (*joiner*)	Conta e acessos **por função**, MFA no primeiro dia, política e confidencialidade assinadas, treinamento de integração, equipamento já configurado	Acessos copiados de outra pessoa; MFA tardio; ninguém conhece as regras
Mudança (*mover*)	Revisão de acessos ao mudar de função: **remover os antigos** e aprovar os novos	Acúmulo de privilégios (*privilege creep*)
Saída (*leaver*)	Revogar acessos **no mesmo dia** (SSO, VPN, e-mail, nuvem, repositórios, tokens), recolher equipamentos, transferir dados e contas de serviço, trocar segredos compartilhados	Ex-funcionário com acesso ativo; segredos que ele conhecia continuam valendo

AMEAÇA INTERNA E TERCEIROS

A **ameaça interna** vem de quem tem acesso legítimo e raramente é um vilão: o caso mais comum costuma ser o **erro**. Os quatro perfis pedem controles diferentes:

PERFIL	EXEMPLO	CONTROLE PRINCIPAL
Malicioso	Funcionário descontente copia a base de clientes antes de sair	Menor privilégio, DLP, logs e saída rápida (p. 59)
Negligente	Envia a planilha com dados pessoais ao destinatário errado; usa nuvem pessoal	Padrões seguros, DLP, treinamento curto e canal de reporte sem culpa

19 PESSOAS, CULTURA E PROTEÇÃO PESSOAL — CONTINUAÇÃO

PERFIL	EXEMPLO	CONTROLE PRINCIPAL
Comprometido	A credencial roubada por phishing é usada como se fosse a pessoa	MFA resistente a phishing e detecção de comportamento anômalo (UEBA, p. 144)
Terceirizado	Prestador com acesso amplo e permanente	Acesso mínimo e temporário, cláusulas contratuais, revisão periódica (p. 135)

Monitorar pessoas exige **base legal e proporcionalidade**: finalidade definida, transparência (a política informa o que é monitorado), mínimo necessário, acesso restrito aos registros e atenção à LGPD (p. 59). Investigar uma suspeita é trabalho conjunto de segurança, RH e jurídico, com **presunção de inocência** e preservação de evidências (p. 156). Quanto a terceiros, o DBIR 2026 registrou a presença deles em 48% das violações (p. 11): avalie fornecedores antes e depois da contratação, exija notificação de incidentes e planeje a saída.

CHECKLISTS

Para você, em três horizontes de tempo:

1 Em 30 minutos
Instale um gerenciador de senhas e troque a senha do **e-mail principal** por uma frase longa; ative o **MFA** no e-mail e no banco; ative a **verificação em duas etapas** do WhatsApp; ligue as **atualizações automáticas** e o **bloqueio de tela**.

2 Em 1 semana
Migre as contas importantes para senhas únicas; guarde os **códigos de recuperação** offline; configure **alertas e limites** no banco; revise a **privacidade** das redes; verifique seu e-mail em serviços de **vazamentos**; ative a **localização remota** dos aparelhos.

3 Em 1 mês
Faça um **backup 3-2-1** e teste a restauração; revise **permissões** e aplicativos antigos; feche contas que não usa; combine com a família uma **palavra-código**; revise sessões e dispositivos conectados; consulte o **Registrato**.

CHECKLIST

Dez itens para a organização:

1. Política de segurança curta, clara e publicada, com exemplos e revisão anual.
2. MFA resistente a phishing em contas privilegiadas e no acesso remoto.
3. Filtros de e-mail com SPF, DKIM e DMARC e isolamento de links e anexos suspeitos.
4. Canal de reporte de um clique, com resposta em horas e agradecimento a quem reporta.
5. Treinamento curto e frequente, por função, com foco em comportamento.
6. Simulações de phishing com política prévia, ética e sem punição.
7. Processo JML: revogação de acessos no mesmo dia da saída e revisão a cada mudança de função.
8. Menor privilégio e dupla aprovação em operações sensíveis (pagamentos, produção).
9. Avaliação de terceiros, com cláusulas de segurança e de notificação de incidentes.
10. Indicadores de cultura (reporte, tempo até reportar, cobertura de MFA) acompanhados pela liderança.

Cuidado com as métricas: a taxa de clique varia com a dificuldade da isca e não deve virar nota individual; uma queda súbita nos reportes pode indicar medo de punição, e não mais segurança.

ONDE BUSCAR AJUDA E INFORMAÇÃO NO BRASIL

Os canais e os endereços mudam: confira sempre nos sites oficiais.

19 PESSOAS, CULTURA E PROTEÇÃO PESSOAL — CONTINUAÇÃO

RECURSO	PARA QUÊ
Cartilha de Segurança para Internet (CERT.br)	Guia gratuito em português sobre golpes, senhas, privacidade e dispositivos (cartilha.cert.br)
Seu banco (aplicativo e número do verso do cartão)	Bloqueio imediato de cartão e contas, contestação e pedido de devolução do Pix
Registrato (Banco Central)	Consultar contas, chaves Pix e relacionamentos em seu CPF (registrato.bcb.gov.br)
Boletim de ocorrência	Registrar golpes e crimes digitais; muitos estados têm atendimento online e delegacias especializadas
Procon e consumidor.gov.br	Reclamar de empresas quando o canal oficial não resolve
SaferNet Brasil	Denúncias de crimes na internet e orientação, inclusive para jovens (new.safernet.org.br)
ANPD	Direitos do titular de dados e reclamações sobre o tratamento de dados pessoais (gov.br/anpd)

PARA FIXAR

Conceitos

- Cite três gatilhos psicológicos usados em golpes e a contramedida de cada um.
- O que significa "pare, pense, confirme, reporte"? Por que o "confirme" precisa ser por um canal independente?
- Por que o treinamento anual obrigatório, sozinho, tem eficácia limitada? O que o complementa?
- Diferencie ameaça interna maliciosa, negligente, comprometida e terceirizada, com um controle para cada uma.

Aplicação

- Analise um e-mail suspeito (real ou fictício): quais sinais da figura aparecem e como você o confirmaria?
- Monte o seu plano de 30 minutos, 1 semana e 1 mês, adaptado às suas contas e aparelhos.
- Redija o roteiro de desligamento de um funcionário (JML) para uma organização de 200 pessoas.
- Proponha três métricas de cultura de segurança e um cuidado de interpretação para cada uma.

APROFUNDE-SE

Leituras e referências

- Cartilha de Segurança para Internet — CERT.br: guia em português sobre golpes, senhas, privacidade e uso seguro de dispositivos.
- Verizon Data Breach Investigations Report — Verizon: dados anuais sobre o elemento humano, credenciais e terceiros nas violações.
- Understanding the Efficacy of Phishing Training in Practice — UC San Diego, IEEE S&P 2025: o experimento com mais de 19.500 funcionários.
- NIST Digital Identity Guidelines (SP 800-63-4) — NIST: senhas, MFA e autenticação.
- Passkeys — FIDO Alliance: como funcionam as chaves de acesso resistentes a phishing.
- Secure Our World — CISA: hábitos simples de segurança pessoal (senhas, MFA, phishing e atualizações).
- Autoridade Nacional de Proteção de Dados — ANPD: direitos do titular e orientações sobre a LGPD.

MÓDULO 20 · PROJETO

20 PROJETO-GUIA: SISTEMA SEGURO

Hora de juntar tudo: projetar, construir e operar um sistema seguro, com checklist e perguntas para fixar.

Os módulos anteriores estudaram as peças; este monta o quebra-cabeça. Você vai acompanhar o projeto de um **sistema seguro** do requisito ao plantão, no **Portal Coruja**, o portal acadêmico fictício que apareceu como exemplo ao longo do guia. Cada decisão aponta o módulo que a explica, e cada controle fica ligado ao risco que reduz e ao teste que o prova. É também um roteiro para o seu **projeto final**.

O PROJETO: O PORTAL CORUJA

Uma universidade fictícia, com cerca de 20 mil alunos e 1,2 mil docentes, precisa de um portal para **matrícula, notas, histórico escolar, documentos e atendimento**. Há um aplicativo web e uma API; o login é feito pelo provedor de identidade (*IdP*) da instituição, e na semana de matrícula o acesso se concentra em poucas horas. Antes de qualquer tecnologia, liste **quem usa, o que protege e o que não pode dar errado**:

ATOR	O QUE FAZ	ATIVO EM JOGO	PROPRIEDADE MAIS CRÍTICA
Aluno	Consulta notas, matricula-se, baixa o histórico	Notas, dados pessoais, documentos	Confidencialidade
Docente	Lança notas e frequência da própria turma	Notas e listas de turma	Integridade
Secretaria e coordenação	Emite documentos, corrige cadastros, gera relatórios	Cadastros, históricos, atestados médicos	Integridade e confidencialidade
Equipe de TI	Opera, implanta e concede acessos	Contas privilegiadas, segredos, backups	Todas (o alvo mais valioso)
Terceiros	Enviam e-mail, cobram taxas, autenticam	Dados em trânsito e credenciais de integração	Disponibilidade e confiança

COMECE PELO QUE NÃO PODE ACONTECER

Peça a cada ator que descreva três coisas que **nunca** devem ocorrer (“um aluno ver a nota de outro”, “uma nota mudar sem ninguém saber”, “a matrícula cair no primeiro dia”). Esses **casos de abuso** viram requisitos testáveis no passo 2 e alimentam a modelagem de ameaças do passo 4.

OS 12 PASSOS, DO REQUISITO AO PLANTÃO

O mapa organiza o trabalho em três fases e liga cada passo aos módulos do guia. A ordem é a de um projeto novo, mas **não é uma cascata**: descobrir uma ameaça no passo 4 pode mudar a arquitetura do passo 3, e um incidente no passo 11 reabre o passo 4.

20 PROJETO-GUIA: SISTEMA SEGURO

CONTINUAÇÃO



Mudou o sistema, a equipe, a lei ou o risco? Volte ao passo 4 e revise controles e testes.

Mapa do projeto. Doze passos em três fases; cada um aponta os módulos do guia que o sustentam. Não é uma cascata: o passo 4 (ameaças e riscos) é revisitado sempre que algo relevante muda.

PASSOS 1 E 2: ESCOPO, REQUISITOS E CLASSIFICAÇÃO DE DADOS

O **escopo** responde a "o que está dentro do sistema e quais leis e normas se aplicam": aqui entram a **LGPD** (dados pessoais de alunos e dados sensíveis, como atestados médicos), a política de segurança da instituição e, se houver cobrança de taxas, as regras do meio de pagamento (p. 27). Um **requisito de segurança** é um requisito como outro qualquer: tem de ser **verificável**. "O portal deve ser seguro" não é requisito; "um aluno nunca lê as notas de outro, e isso é testado a cada versão" é.

Use identificadores (RS-01, RS-02...) e registre a **origem** e o **modo de verificação** de cada um. O projeto adota o **ASVS nível 2** como base, o recomendado para sistemas com dados pessoais (p. 90).

ID	REQUISITO (VERIFICÁVEL)	ORIGEM	COMO SE VERIFICA
RS-01	Aluno só lê os próprios dados; docente, só os das suas turmas	LGPD (necessidade, segurança); ASVS, autorização	Matriz de testes de autorização no CI
RS-02	Nota só muda por docente da turma, com trilha de quem, quando e do valor anterior	Integridade (negócio); ASVS	Teste de API e checagem da trilha
RS-03	Toda conta da equipe usa MFA resistente a phishing (passkey, FIDO2); SMS não conta	NIST SP 800-63B-4; risco R1 do módulo 03	Relatório de cobertura do IdP
RS-04	Tráfego só por TLS 1.3 (1.2 no mínimo) e dados em repouso cifrados	LGPD, art. 46; módulo 05	Varredura de TLS; revisão de IaC
RS-05	Nenhum segredo no código nem no repositório	ASVS, configuração	Varredura de segredos (commit e CI)
RS-06	Exportação em massa exige aprovação, reautenticação recente e registro	Menor privilégio; LGPD	Teste de API e alerta de volume
RS-07	Nenhuma dependência da lista KEV chega à produção; SBOM por versão	OWASP A03:2025; SSDF	SCA no CI, com *gate*

20 **PROJETO-GUIA: SISTEMA SEGURO** CONTINUAÇÃO

ID	REQUISITO (VERIFICÁVEL)	ORIGEM	COMO SE VERIFICA
RS-08	Incidente com dados pessoais: comunicar ANPD e titulares em até 3 dias úteis	Res. CD/ANPD nº 15/2024	Exercício de mesa semestral
RS-09	Semana de matrícula: 99,9% de disponibilidade, RTO de 2 h e RPO de 15 min (valores do exemplo)	Negócio; risco R3 do módulo 03	Teste de carga e de restauração
RS-10	Logs de segurança centralizados, em UTC, sem segredos, guardados conforme a política	ASVS, registros; Marco Civil	Revisão de logs; alertas dos casos

Em seguida **classifique os dados**: a classe define a proteção mínima, o prazo de guarda e quem pode ver. A tabela é o *inventário de dados* enxuto do portal (p. 59); os prazos são exemplos que cada instituição define com o jurídico.

DADO	CLASSE	PROTEÇÃO MÍNIMA	GUARDA (EXEMPLO)
Notas e histórico escolar	Confidencial	Autorização por objeto, cifragem em repouso e trilha de alterações	Prazo legal e institucional
Cadastro (nome, CPF, e-mail)	Confidencial	CPF mascarado em telas e logs; acesso por função	Vínculo e prazos legais
Atestados médicos (dado sensível)	Restrito	Acesso mínimo e justificado, chave por titular, todo acesso registrado	Só o necessário; depois, eliminar
Logs de acesso e segurança	Interno	Imutáveis, sem dados pessoais desnecessários, acesso restrito	12 meses (política)

PASSO 3: ARQUITETURA E FRONTEIRAS DE CONFIANÇA

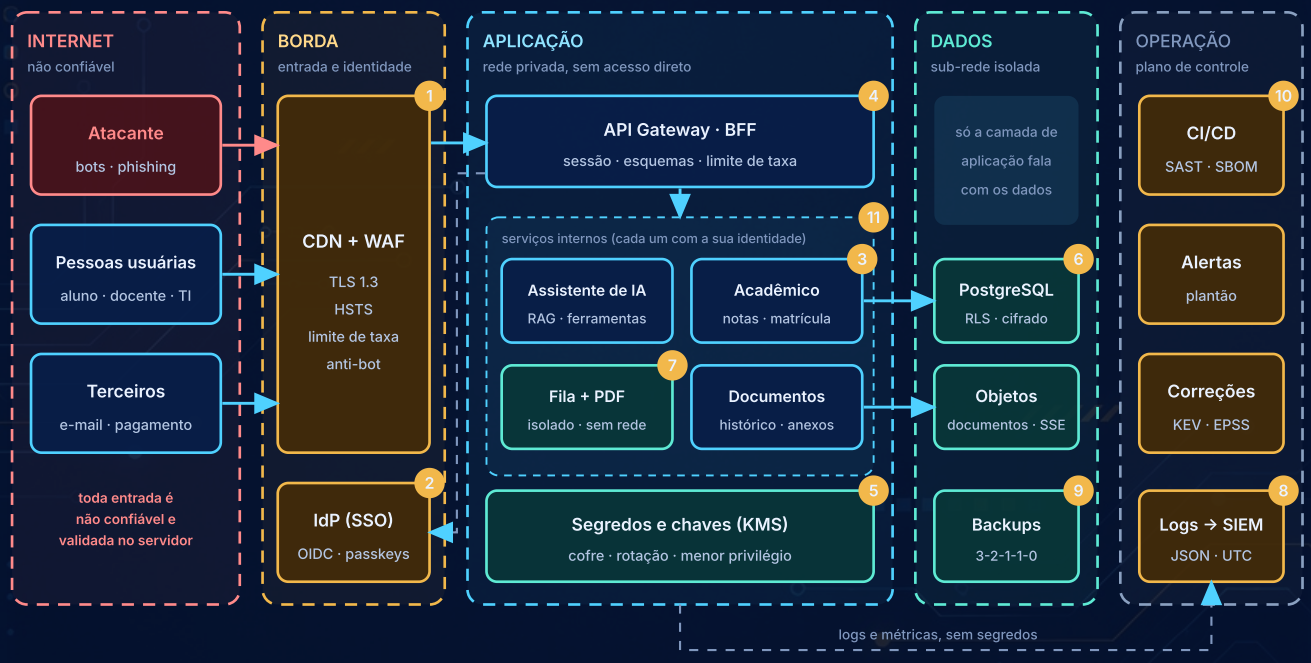
A arquitetura nasce de duas perguntas: **onde a confiança muda** e **o que acontece se esta parte for comprometida**. O desenho divide o portal em zonas separadas por fronteiras. A **internet** não é confiável; a **borda** (CDN com WAF e IdP) é o único ponto de entrada; a **aplicação** roda em rede privada, sem acesso direto; os **dados** ficam em uma sub-rede isolada que só a aplicação alcança; e a **operação** (pipeline, alertas, logs) tem o seu próprio plano de controle. Cada serviço tem **identidade própria** e permissões mínimas, e as partes de maior risco, o gerador de PDF e o assistente de IA, rodam isoladas (p. 78).

Os **controles** da tabela, numerados de 1 a 11, aparecem como círculos dourados na figura que vem logo depois, sobre os elementos em que atuam. A tabela diz o que cada um faz, que ameaças reduz e onde o guia o explica.

Nº	CONTROLE E ONDE ATUA	REDUZ	MÓDULO
1	**Borda**: TLS 1.3, HSTS, WAF e limite de taxa; certificados renovados por ACME	Interceptação, bots, DDoS de aplicação	p. 67
2	**Identidade**: login pelo IdP (OIDC, Authorization Code + PKCE) e passkeys para a equipe	Roubo de credenciais, phishing	p. 44
3	**Autorização por objeto** em cada serviço e *row-level security* no banco	BOLA/IDOR, escalada de privilégio	p. 53
4	**Gateway e BFF**: sessão segura, validação de entrada e saída por esquema, limites	Injeção, excesso de dados, abuso	p. 98, p. 112

20 PROJETO-GUIA: SISTEMA SEGURO — CONTINUAÇÃO

Nº	CONTROLE E ONDE ATUA	REDUZ	MÓDULO
5	**Cofre de segredos e KMS**: segredos fora do código, rotação, chaves por finalidade	Vazamento de credenciais	p. 34, p. 135
6	**Cifragem** em repouso (AES-GCM) e em trânsito; CPF mascarado; chave por titular	Exposição de dados, cópia de disco	p. 34, p. 59
7	**Isolamento** do gerador de PDF e do assistente de IA: sem rede, sem privilégios, ferramentas estreitas	SSRF, execução de código, injeção de prompt	p. 78, p. 112
8	**Logs estruturados no SIEM**, alertas e plantão	Ataque não detectado, repúdio	p. 144
9	**Backups 3-2-1-1-0**, com restauração testada	Ransomware, perda de dados	p. 156
10	**Pipeline** com SAST, SCA, segredos, SBOM e assinatura; *gate* de liberação	Dependência maliciosa, falha de código	p. 135
11	**Segmentação**: sub-redes e política de rede que nega por padrão	Movimento lateral	p. 67, p. 78



Arquitetura do Portal Coruja. As linhas tracejadas são fronteiras de confiança: tudo que cruza uma delas é autenticado, validado e registrado. Os círculos dourados numerados são os controles detalhados na tabela a seguir.

PASSO 4: MODELAGEM DE AMEAÇAS DO PORTAL

Com o DFD (a figura anterior serve de base), aplique o **STRIDE** a cada elemento e a cada fluxo (p. 20). A tabela traz as nove ameaças prioritárias do portal, o controle principal e o requisito que o testa. Repare que **nenhuma depende de sofisticação rara**: são falhas de autorização, de credencial, de dependência e de processo, as mesmas que o OWASP Top 10 e o DBIR repetem todos os anos.

20 PROJETO-GUIA: SISTEMA SEGURO — CONTINUAÇÃO

ID	AMEAÇA (STRIDE) E CENÁRIO	CONTROLE PRINCIPAL	REQUISITO
T1	**S** · *Credential stuffing* e phishing contra docentes e secretaria, para entrar como outra pessoa	Passkeys; limite de taxa; alerta de login anômalo (C2, C8)	RS-03
T2	**T** · Aluno altera a própria nota ou a de outro por BOLA/IDOR ou *mass assignment* na API	Autorização por objeto e RLS; esquemas estritos (C3, C4)	RS-01, RS-02
T3	**R** · Docente nega ter lançado uma nota e ninguém sabe o valor anterior	Trilha de auditoria imutável, com o valor anterior (C8)	RS-02
T4	**I** · Aluno baixa o histórico de outro trocando o número na URL; erro verboso expõe detalhes do banco	Posse do recurso conferida no servidor; erros genéricos (C3, C4)	RS-01
T5	**I** · Atestados médicos expostos por armazenamento público ou por credencial vazada no repositório	Armazenamento privado e cifrado; varredura de segredos (C5, C6)	RS-04, RS-05
T6	**D** · Pico de matrícula somado a bots esgota o serviço	WAF, limite de taxa, fila e cache; teste de carga (C1, C9)	RS-09
T7	**E** · Conta da secretaria comprometida exporta todos os históricos	Menor privilégio, aprovação e alerta de volume (C3, C8)	RS-06
T8	**T** · Dependência ou ação de CI adulterada leva código malicioso à produção	Lockfile, SCA, SBOM, assinatura e *gate* (C10)	RS-07
T9	**E** · Anexo malicioso ou URL que leva o gerador de PDF a acessar a rede interna (SSRF)	Validação de arquivo; gerador isolado e sem rede (C7, C11)	RS-01

Cada ameaça entra no **registro de riscos** (p. 20) com probabilidade, impacto, dono, tratamento e **risco residual**. O risco R1 do exemplo, phishing a docentes, pontua 20 (P4 × I5); com passkeys e alertas cai para 8 (P2 × I4). É por isso que o RS-03 existe: o requisito é a forma testável de cumprir o tratamento escolhido.

PASSOS 5 A 7: IDENTIDADE, DADOS E CÓDIGO

Aqui entram as decisões que as pessoas desenvolvedoras aplicam todos os dias (p. 34 a p. 59, p. 98 a p. 122). A tabela resume os **padrões do projeto**, que valem como critério de revisão de *pull request*; as decisões mais importantes ficam registradas em ADRs, como a da autenticação, logo adiante.

TEMA	PADRÃO DO PROJETO	VERIFICAÇÃO
Identidade e acesso	Login só pelo IdP (OIDC, Authorization Code + PKCE); passkey na equipe; autorização por objeto no servidor, negando por padrão	Matriz de testes; revisão do IdP
Entrada e saída	Esquemas estritos (tipo, tamanho, lista de permissão); respostas por esquema, nunca o objeto inteiro do banco	Testes de contrato; SAST
Dados e criptografia	Consultas parametrizadas; aplicação sem permissão de DDL; RLS; AES-GCM com chaves no KMS	Revisão; testes de autorização
Sessão e tokens	Cookies Secure, HttpOnly e SameSite; tokens de vida curta; `redirect_uri` com correspondência exata	Testes de sessão; DAST
Erros e logs	Falhar fechado; erro genérico ao cliente, detalhe só no log; sem CPF, senha ou token nos logs	Teste de falha; revisão dos logs
Arquivos e URLs	Tipo e tamanho validados; arquivos fora da raiz web; destino de URL por lista de permissão	Fuzzing de upload; teste de SSRF

20 **PROJETO-GUIA: SISTEMA SEGURO** CONTINUAÇÃO

DECISÕES REGISTRADAS: O ADR DA AUTENTICAÇÃO

Como vimos no módulo de ciclo de vida (p. 90), o **ADR** guarda o contexto, a decisão, as alternativas rejeitadas e as consequências, para que alguém, um ano depois, saiba **por que** o sistema é assim. Veja o registro da autenticação do portal, o passo 5 (p. 44, p. 53); guarde os ADRs no repositório, ao lado do código.

DOCS/ADR/004-AUTENTICACAO.MD

```
# ADR-004 · Autenticação do Portal Coruja
Status: aceita (03/11/2026) · Decisores: arquitetura, segurança e TI · Revisar em 12 meses

## Contexto
20 mil alunos (volume alto, privilégio baixo) e equipes que alteram notas e exportam
dados (volume baixo, privilégio alto). O risco R1, phishing a docentes, é o mais crítico.

## Decisão
1. Login só pelo IdP da instituição (OIDC, Authorization Code + PKCE); o portal não vê senha.
2. Equipe (docentes, secretaria, TI): passkey (FIDO2) obrigatória; SMS não é aceito.
3. Alunos: passkey ou TOTP incentivados; MFA obrigatório para exportar dados.
4. Alterar nota e exportar em massa exigem reautenticação feita há menos de 10 minutos.
5. Sessão: cookie Secure, HttpOnly, SameSite=Lax; 15 min de inatividade, 8 h no total.

## Alternativas rejeitadas
- Senha própria no portal: mais superfície (hash, redefinição, força bruta) sem ganho.
- SMS como segundo fator: sujeito a SIM swap; só na recuperação, com revisão manual.

## Consequências
+ R1 cai de 20 para 8 (P2 × I4), conforme o registro de riscos.
- O IdP vira dependência crítica: exige conta de emergência e plano de contingência.
- Chave perdida gera chamado: o suporte precisa de recuperação com identidade verificada.
```

Quatro pontos merecem atenção. A **reautenticação** do item 4 (*step-up*) separa ler de alterar: a sessão comum basta para consultar notas, mas não para mudá-las. Os **alunos sem MFA** são uma decisão de risco registrada, não um esquecimento. O **IdP** lembra que centralizar identidade concentra também a falha. E o ADR **nomeia o custo** (suporte a chaves perdidas) em vez de escondê-lo.

RASTREABILIDADE: DO REQUISITO AO TESTE

Sem rastreabilidade, a segurança vira opinião. Cada requisito deve ficar ligado às **ameaças** que cobre, aos **controles** que o implementam e aos **testes** que o provam. A cadeia se lê nos dois sentidos: da ameaça ao teste (“já temos prova de que isto foi tratado?”) e do teste ao requisito (“por que este teste existe?”).



O arquivo a seguir guarda essas ligações, e um *script* de CI falha se algum requisito ficar sem teste. É a forma mais barata de impedir que um controle exista no documento e não no sistema. Quando um teste falha, o relatório do CI aponta o requisito e o risco em jogo, e a conversa com a gestão deixa de ser só técnica.

20 PROJETO-GUIA: SISTEMA SEGURO — CONTINUAÇÃO

SEGURANCA/REQUISITOS.YAML (TRECHO)

```
- id: RS-01
  texto: Aluno só lê os próprios dados; docente, só os das suas turmas
  ameacas: [T2, T4, T9]
  controles: [C3, C4]
  testes: [tests/seguranca/test_autorizacao.py::test_matriz]
- id: RS-02
  texto: Nota só muda por docente da turma, com trilha de auditoria
  ameacas: [T2, T3]
  controles: [C3, C8]
  testes: [tests/seguranca/test_autorizacao.py::test_matriz,
           tests/seguranca/test_trilha.py::test_nota_registra_valor_anterior]
```

FERRAMENTAS/CHECAR_RASTREIO.PY: FALHA O CI SE FALTAR TESTE

```
"""Cada requisito precisa de ao menos um teste que exista no repositório."""
import pathlib
import sys

import yaml

reqs = yaml.safe_load(pathlib.Path("seguranca/requisitos.yaml").read_text())
erros = []
for r in reqs:
    testes = r.get("testes") or []
    if not testes:
        erros.append(f"{r['id']}: nenhum teste associado")
    for t in testes:
        arquivo = t.split("::")[0]
        if not pathlib.Path(arquivo).is_file():
            erros.append(f"{r['id']}: arquivo de teste ausente ({arquivo})")

print("\n".join(erros) or "rastreabilidade ok")
sys.exit(1 if erros else 0)
```

PASSOS 8 E 9: PIPELINE COM PORTÕES E TESTES DE SEGURANÇA

O pipeline é onde os requisitos viram **portões** (*gates*): verificações automáticas que bloqueiam a liberação. Cada portão precisa de um **critério claro** (por exemplo, "nenhuma dependência da lista KEV") e de um **caminho de exceção** com dono e prazo; sem isso, as pessoas passam a contorná-lo (p. 135, p. 166).

ETAPA	VERIFICAÇÕES (EXEMPLOS DE FERRAMENTA)	BLOQUEIA QUANDO
Commit	Varredura de segredos (gitleaks) e *hooks* de formatação e lint	Segredo detectado
Pull request	Revisão por pares, com dono do código nas áreas sensíveis; SAST (Semgrep, CodeQL); testes de autorização	Achado alto ou teste falhando
Build	Lockfile e SCA (OSV-Scanner, pip-audit); IaC (Checkov); imagem mínima; SBOM (CycloneDX); assinatura (cosign)	Dependência da lista KEV; IaC crítico
Homologação	DAST de linha de base (ZAP); testes de API (BOLA); fuzzing de upload; teste de carga da matrícula	Achado alto ou crítico aberto
Produção	Verificação da assinatura na admissão; implantação gradual (*canary*); reversão automática	Falha de saúde ou de assinatura

20 PROJETO-GUIA: SISTEMA SEGURO — CONTINUAÇÃO

Portão sem exceção vira atalho: Se a única forma de liberar uma correção urgente é desligar o portão, as equipes aprendem a desligá-lo. Crie a **exceção temporária**, com dono, motivo e prazo, registre-a e **meça quantas existem**: o número de exceções é um bom termômetro da saúde do processo.

PASSOS 10 E 11: DETECTAR, RESPONDER E CONTINUAR

Prevenir não basta: o portal precisa **enxergar** o que acontece e **agir** a tempo (p. 144, p. 156). Parta dos casos de abuso e das ameaças: cada um precisa de um **evento registrado**, de um **alerta** com dono e de uma **primeira ação** definida. Os limiares abaixo são exemplos, para calibrar com dados reais.

EVENTO REGISTRADO	ALERTA (LIMIAR DE EXEMPLO)	PRIMEIRA AÇÃO
Falhas de login por IP e por conta	Falhas em 10 contas ou mais, do mesmo IP, em 5 min (*spraying*)	Limitar o IP; exigir MFA reforçado
Nota alterada fora do período letivo ou por docente de outra turma	Qualquer ocorrência	Congelar a alteração; revisar a trilha
Exportação de históricos ou de notas	Mais de 200 registros por usuário em 10 min	Suspender a sessão; confirmar por outro canal
Novo passkey ou dispositivo em conta da equipe	Login de país incomum logo depois do cadastro	Confirmar com a pessoa por canal independente
Mudança de papel ou de política de acesso	Qualquer ocorrência fora de mudança aprovada	Revisar com dupla aprovação; reverter

CENÁRIO

Madrugada da matrícula, 03h10. O alerta de exportação dispara: uma conta da secretaria baixou 1.800 históricos em 6 minutos, a partir de um país onde a instituição não tem ninguém. O plantão aplica o playbook:

1. **Conter (primeira hora)**: encerrar as sessões da conta, revogar o passkey recém-cadastrado, bloquear a exportação em massa para todos até a revisão e **preservar** logs e trilha antes de qualquer limpeza (p. 156).
2. **Entender (até 24 h)**: o log mostra que o passkey novo foi registrado depois de um pedido de recuperação aprovado por telefone. A falha é de **processo** (recuperação sem verificação forte), e não do passkey.
3. **Decidir e comunicar (até 3 dias úteis)**: havia dados pessoais de mais de mil titulares em risco relevante; a instituição avalia o incidente, comunica a ANPD e os titulares e registra tudo (Res. CD/ANPD nº 15/2024; guarda mínima de 5 anos do registro).
4. **Corrigir e aprender**: recuperação de conta da equipe só com verificação presencial ou por videochamada e dupla aprovação; novo alerta para recuperações; revisão **sem culpa**, com ajuste do ADR-004 e do registro de riscos.

A **continuidade** também vira requisito testável (p. 156). No exemplo, o banco tem RTO de 2 h e RPO de 15 min (réplica em outra zona e restauração pontual); os documentos, RTO de 4 h e RPO de 1 h (versionamento e cópia imutável); o IdP e a aplicação, RTO de 1 h (conta de emergência fora do SSO e implantação reproduzível a partir de IaC, com reversão automática). O que importa é que cada meta tenha uma estratégia e que a restauração seja **ensaiada**, e não apenas planejada.

PASSO 12: PESSOAS, GOVERNANÇA E MELHORIA CONTÍNUA

Um sistema seguro precisa de **donos**: quem aprova o risco, quem corrige, quem opera. Uma matriz **RACI** simples basta: para cada atividade, quem **faz** (R), quem **aprova** (A) e quem é **consultado** ou **informado** (C e I).

20 PROJETO-GUIA: SISTEMA SEGURO — CONTINUAÇÃO

ATIVIDADE	FAZ (R)	APROVA (A)	CONSULTADOS E INFORMADOS
Aceitar o risco residual e as exceções dos portões	Segurança	Direção de TI	Produto e arquitetura (C); jurídico (I)
Escrever e manter os requisitos de segurança	Produto	Produto	Segurança e arquitetura (C)
Corrigir vulnerabilidades dentro do prazo	Equipe do serviço	Líder técnico	Segurança (C); gestão (I)
Operar os alertas e conduzir incidentes	Plantão (TI e segurança)	Direção de TI	Jurídico, encarregado e comunicação (C)
Comunicar a ANPD e os titulares	Encarregado (DPO)	Direção da instituição	Jurídico e segurança (C)

Depois, **meça** o que mostra comportamento e resultado, não só atividade (p. 180). Um modelo de maturidade como o **OWASP SAMM** ajuda a escolher o próximo passo (p. 90), e o **NIST CSF 2.0** (Governar, Identificar, Proteger, Detectar, Responder e Recuperar) dá uma linguagem comum com a direção (p. 27).

INDICADOR	META DE EXEMPLO	O QUE MOSTRA
Cobertura de MFA resistente a phishing na equipe	100%	Se o controle do risco R1 existe de fato
Tempo mediano entre o reporte de um e-mail suspeito e a resposta	Menos de 1 hora	Cultura de reporte e prontidão do plantão
Requisitos de segurança com teste automatizado	100%	Rastreabilidade viva
Vulnerabilidades da lista KEV abertas em produção	Zero	Gestão de correções e do pipeline
Restauração de backup testada com sucesso	Trimestral	Se a continuidade existe de fato

ROTEIRO DE ENTREGA EM SEIS SPRINTS

Para montar o projeto em um semestre, divida o trabalho em seis entregas curtas. Cada sprint termina com um **artefato verificável**, e não com uma apresentação:

1 · Fundação
Escopo, atores e ativos; casos de abuso; **RS-01 a RS-10**; classificação de dados; repositório com `SECURITY.md` e ADR-001.

2 · Projeto
DFD com fronteiras; **STRIDE** por elemento; registro de riscos com risco residual; ADRs de autenticação e de dados.

3 · Identidade e dados
Login pelo IdP, autorização por objeto com RLS, cifragem e cofre de segredos; **matriz de testes de autorização**.

4 · Pipeline
CI com varredura de segredos, SAST, SCA, IaC, SBOM e assinatura; portões, exceções e a **checagem de rastreabilidade**.

5 · Operação
Logs estruturados e alertas dos casos de abuso; backups com **restauração ensaiada**; playbook de incidente.

6 · Verificação final
DAST e testes de API; teste de intrusão em homologação, **só com autorização escrita**; exercício de mesa; relatório.

20 PROJETO-GUIA: SISTEMA SEGURO

CONTINUAÇÃO

CHECKLIST

Definição de pronto (segurança) para liberar uma versão do Portal Coruja:

1. Os requisitos de segurança da mudança estão escritos e têm testes que passam no CI.
2. Modelo de ameaças e registro de riscos foram atualizados se a mudança toca fronteira, dado ou permissão.
3. Revisão por pares feita; áreas sensíveis revisadas por quem é dono do código.
4. Nenhum segredo no repositório; nenhum achado crítico ou alto aberto sem exceção registrada.
5. Dependências sem vulnerabilidade da lista KEV; SBOM gerado e artefato assinado.
6. Logs e alertas dos novos fluxos configurados; playbook atualizado se houver cenário novo.
7. Backup e restauração verificados em toda mudança que afete dados.
8. Plano de reversão testado e comunicação prevista para quem for afetado.

DICA DA CORUJA

Quebre o controle de propósito. Na apresentação, remova a checagem de dono de uma rota e mostre o **teste de autorização falhando** no CI; depois restaure o código e mostre o teste passando. A demonstração prova que o teste realmente protege o requisito, e não apenas existe.

COMO O PROJETO É AVALIADO

Se o Portal Coruja for o seu trabalho final, a rubrica abaixo (pesos de exemplo) premia o que mais pesa na prática: **decisões justificadas e evidências**, e não a quantidade de ferramentas.

CRITÉRIO	O QUE SE ESPERA	PESO
Requisitos e dados	RS verificáveis, com origem e teste; inventário e classificação coerentes	15%
Arquitetura e ameaças	DFD com fronteiras, STRIDE por elemento e registro de riscos com tratamento e risco residual	20%
Controles	Identidade, autorização, criptografia e código seguro implementados e justificados em ADRs	25%
Pipeline e testes	Portões automáticos, rastreabilidade e testes de autorização que falham de verdade quando o controle quebra	20%
Operação	Logs, alertas, backup restaurado e um exercício de incidente com relatório	15%
Comunicação	Documento curto e claro, com limites e riscos aceitos declarados	5%

Boa prática: Declare o que ficou de fora. Liste os riscos aceitos, o que está fora do escopo (por exemplo, a segurança do IdP, que é do fornecedor, ou os ataques volumétricos tratados pela CDN) e quem os aceitou. Uma lista honesta de limites vale mais do que a promessa de segurança total.

20 PROJETO-GUIA: SISTEMA SEGURO — CONTINUAÇÃO

Faça

- Comece pelos casos de abuso e pelos dados, não pela ferramenta.
- Escreva ADRs curtos e mantenha o modelo de ameaças vivo.
- Teste o que **não** deve funcionar: acesso negado, cota estourada, entrada inválida.
- Ensaie a restauração e o incidente antes de precisar deles.

Evite

- Lista de ferramentas sem requisito nem teste que a justifique.
- Deixar a segurança para o fim, com um teste de intrusão como primeira verificação.
- Controle só no papel: política sem log, backup sem restauração.
- Documentos longos que ninguém lê; prefira checklists e código.

Limites do exemplo: O Portal Coruja é um modelo de estudo. Ele simplifica (um IdP, uma região, sem sistemas legados) e usa números **ilustrativos** (RTO, limiares de alerta, metas). Em um sistema real, esses valores saem da análise de risco da própria organização, e o projeto precisa da **autorização** de quem responde por ele antes de qualquer teste ativo.

PARA FIXAR

Conceitos

- Por que um requisito de segurança precisa ser verificável? Dê um exemplo bom e um ruim.
- O que é uma fronteira de confiança e como ela difere de uma camada de defesa? Onde estão as do Portal Coruja?
- Que problema a rastreabilidade (requisito, ameaça, controle, teste) resolve?

Aplicação

- Aplique o STRIDE ao fluxo "solicitar atestado e anexar arquivo" e proponha um controle para cada ameaça.
- Escreva um ADR de uma página sobre como o portal guarda os atestados médicos.
- Defina RTO e RPO para a matrícula e diga como provaria que são atingíveis.

APROFUNDE-SE

Leituras e referências

- OWASP ASVS — OWASP: requisitos verificáveis de segurança para aplicações, em três níveis.
- OWASP SAMM — OWASP: modelo de maturidade para avaliar e planejar a segurança do software.
- NIST SP 800-218 (SSDF) — NIST: práticas de desenvolvimento seguro para organizações e fornecedores.
- NIST Cybersecurity Framework 2.0 — NIST: funções e categorias para organizar um programa de segurança.
- Building Secure and Reliable Systems — Google: livro aberto sobre projetar, implantar e operar sistemas seguros e confiáveis.
- Threat Modeling Manifesto — valores e princípios da modelagem de ameaças.
- OWASP Cheat Sheet Series — OWASP: guias práticos por tema, úteis em cada passo do projeto.

REFERÊNCIA

GLOSSÁRIO TÉCNICO

Consulta rápida a todos os termos usados neste guia — ótima para revisar antes das provas e dos projetos.

A

AAA. Autenticação, Autorização e Accounting (contabilização/auditoria): provar quem é, decidir o que pode fazer e registrar o que fez. **p. 5**

AAD (dados associados). Dados autenticados, mas não cifrados, em um AEAD; amarram o cifrado ao contexto (ID do registro, campo, versão). **p. 34**

AAL (Authenticator Assurance Level). Nível de garantia de autenticação do NIST SP 800-63B, de AAL1 a AAL3, conforme a força dos autenticadores. **p. 44**

ABAC. Attribute-Based Access Control (NIST SP 800-162): decide por atributos do sujeito, do recurso, da ação e do ambiente. **p. 53**

Achado (finding). Falha documentada no relatório, com título, severidade, local, descrição, evidências, impacto e recomendação. **p. 166**

ACL (Access Control List). Lista associada ao recurso que informa quem pode realizar quais operações sobre ele. **p. 53**

ACME. Protocolo (RFC 8555) para emitir e renovar certificados automaticamente, usado por Let's Encrypt e outras autoridades. **p. 34**

Active Directory (AD). Serviço de diretório da Microsoft que concentra identidades, autenticação (Kerberos, NTLM) e políticas (GPO); alvo central de ataques. **p. 78**

AddressSanitizer (ASan). Sanitizer que detecta estouros de pilha, heap e globais, use-after-free e double free; deixa o programa cerca de 2 vezes mais lento e usa mais memória, por isso é usado em testes. **p. 122**

ADR (Architecture Decision Record). Registro curto de uma decisão de arquitetura: contexto, decisão, alternativas, riscos aceitos e data de revisão. **p. 90**

AEAD. Authenticated Encryption with Associated Data: cifragem que entrega sigilo e integridade juntos e autentica também dados associados não cifrados. **p. 34**

AES. Advanced Encryption Standard (FIPS 197): cifra de bloco de 128 bits com chaves de 128, 192 ou 256 bits; padrão mundial de criptografia simétrica. **p. 34**

AES-GCM. Modo AEAD do AES (NIST SP 800-38D) com nonce de 96 bits e etiqueta de 128 bits; falha de forma catastrófica se o nonce se repetir com a mesma chave. **p. 34**

Agência excessiva. LLM06: dar ao sistema com LLM mais ferramentas, permissões ou autonomia do que a tarefa exige, ampliando o dano quando o modelo é induzido a erro. **p. 112**

Agente de ameaça (threat actor). Pessoa, grupo ou entidade que materializa uma ameaça: cibercriminoso, insider, hacktivista, Estado, ou mesmo um erro humano. **p. 5**

Air gap. Isolamento físico ou lógico de uma cópia (fita, mídia desconectada) que impede o acesso pela rede e protege o backup de invasores e de ransomware. **p. 59**

AiTM (adversary-in-the-middle). Proxy de phishing que se coloca entre a vítima e o site real, capturando senha e cookie de sessão, inclusive após o MFA tradicional. **p. 11**

ALE (Annualized Loss Expectancy). Perda anual esperada: SLE multiplicado pelo ARO; base para comparar o custo de controles. **p. 20**

Aleatoriedade fraca (CWE-338). Uso de geradores previsíveis, como o Mersenne Twister do random do Python ou Math.random(), para fins de segurança; permite reconstruir o estado e prever os valores seguintes. **p. 122**

Algoritmo de Luhn. Soma de verificação usada em números de cartão; ferramentas de DLP a aplicam para separar números de cartão válidos de sequências aleatórias de dígitos. **p. 59**

Alinhamento (DMARC). Exigência de que o domínio autenticado por SPF ou DKIM coincida com o domínio do cabeçalho From visível ao usuário. **p. 67**

Allowlisting de aplicações. Política que só permite executar o que foi aprovado (AppLocker, WDAC, fapolicyd); bloqueia malware desconhecido. **p. 78**

Alucinação. Resposta falsa, porém plausível, gerada por um modelo de linguagem (LLM09, desinformação); mitiga-se com ancoragem em fontes, citações e revisão humana. **p. 112**

Ambiente de homologação (staging). Ambiente parecido com a produção, mas sem dados reais, onde a versão é testada (inclusive com testes de segurança autorizados) antes de ser liberada. **p. 189**

Ameaça (threat). Causa potencial de um incidente indesejado que pode resultar em dano ao ativo; pode ser humana, técnica ou ambiental. **p. 5**

Ameaça interna (insider threat). Risco originado por quem já tem acesso legítimo: pode ser malicioso, negligente ou ter a conta comprometida por terceiros. **p. 11**

Amplificação (reflexão). Ataque em que consultas pequenas a serviços abertos (DNS, NTP, memcached), com o IP de origem forjado para o da vítima, geram respostas muito maiores dirigidas a ela. **p. 67**

Amplificação e reflexão. Técnica de DDoS que usa servidores abertos para responder com muito mais dados do que o pedido falsificado, em direção à vítima. **p. 11**

Análise qualitativa. Avaliação de risco em escalas ordinais (baixo, médio, alto; 1 a 5), útil quando faltam dados, mas subjetiva. **p. 20**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Análise quantitativa. Avaliação de risco em valores monetários e frequências, como SLE, ARO e ALE, ou modelos como FAIR. **p. 20**

Anéis de implantação (rings). Implantação gradual de atualizações em grupos sucessivos (piloto, amplo, crítico), com monitoramento entre as etapas. **p. 78**

Anexo A (ISO 27001:2022). Catálogo de 93 controles em quatro temas: organizacionais, pessoas, físicos e tecnológicos. **p. 27**

Anonimização. Processo que impede associar o dado a uma pessoa por meios razoáveis; o dado anonimizado deixa de ser pessoal, salvo se o processo for revertido (LGPD, art. 12). **p. 59**

ANPD. Autoridade Nacional de Proteção de Dados: órgão que regula, fiscaliza e aplica sanções previstas na LGPD. **p. 27**

Anycast. Anúncio do mesmo endereço IP a partir de vários locais; o tráfego vai ao ponto mais próximo, o que distribui e absorve ataques volumétricos. **p. 67**

Apetite a risco. Quantidade e tipo de risco que a organização aceita buscar para cumprir seus objetivos. **p. 20**

API (interface de programação). Contrato pelo qual programas conversam; toda tela de um aplicativo é cliente de uma API, que expõe dados estruturados a chamadas automatizadas. **p. 112**

API sombra e API zumbi. Sombra: API criada sem registro nem governança. Zumbi: versão antiga que continua no ar sem manutenção. Ambas são o risco API9 (inventário impróprio). **p. 112**

Aplicações vulneráveis de propósito. OWASP Juice Shop, WebGoat e DVWA: aplicações feitas para serem atacadas em treino; rode só na sua máquina e nunca as exponha à internet. **p. 166**

App Links e Universal Links. Deep links do Android e do iOS que verificam a posse do domínio (assetlinks.json e apple-app-site-association), ao contrário dos esquemas personalizados, que qualquer app pode registrar. **p. 112**

APT (Advanced Persistent Threat). Ameaça persistente avançada: adversário bem financiado, em geral ligado a Estados, que mantém acesso discreto por longos períodos para espionar ou sabotar. **p. 11**

Argon2id. KDF de senhas vencedor da Password Hashing Competition (RFC 9106); exige memória, o que encarece ataques com GPU. Padrão para sistemas novos. **p. 44**

ARO (Annualized Rate of Occurrence). Número esperado de ocorrências do evento por ano; uma vez a cada cinco anos equivale a 0,2. **p. 20**

ARP spoofing. Envio de respostas ARP falsas para associar o endereço do atacante ao de outro host e desviar o tráfego na rede local. **p. 11**

ARP spoofing e DAI. Envio de respostas ARP falsas para associar o MAC do atacante ao IP de outra máquina e interceptar o tráfego; o DAI (Dynamic ARP Inspection) o bloqueia nos comutadores. **p. 67**

Árvore de ataque. Diagrama com o objetivo do adversário na raiz e formas alternativas de alcançá-lo como ramos, com custo e probabilidade. **p. 20**

ASLR. Address Space Layout Randomization: o sistema embaralha os endereços da pilha, do heap e das bibliotecas a cada execução, dificultando prever onde estão código e dados; com PIE, inclui o executável. **p. 122**

Assinatura sem chave (keyless). Assinatura em que não há chave privada de longa duração: a identidade OIDC do pipeline recebe um certificado de minutos, usado uma vez e registrado em um log de transparência. **p. 135**

Atestação de app e dispositivo. Mecanismos (Play Integrity API, App Attest) pelos quais o servidor recebe prova de que a requisição vem do app autêntico em um dispositivo íntegro; é sinal de risco, não prova. **p. 112**

Atestação e in-toto. Atestação é uma declaração assinada sobre uma etapa da cadeia (quem fez o quê, com quais entradas); o framework in-toto padroniza essas declarações, e a proveniência SLSA é um tipo delas. **p. 135**

Ativo. Qualquer coisa que tenha valor para a organização e precise de proteção: dados, sistemas, pessoas, reputação, infraestrutura, propriedade intelectual. **p. 5**

Ativo-ativo (multi-site). Arquitetura em que dois ou mais sites atendem o tráfego ao mesmo tempo, de modo que a falha de um é absorvida pelo outro; maior custo e menor RTO. **p. 156**

Atomic Red Team. Biblioteca da Red Canary de pequenos testes mapeados ao ATT&CK, para validar detecções de forma controlada. **p. 166**

Atributos Secure e HttpOnly. Secure faz o cookie viajar só por HTTPS; HttpOnly impede que o JavaScript o leia, o que limita o roubo da sessão por XSS (mas não o uso dela pelo script). **p. 98**

ATT&CK Navigator. Ferramenta web do MITRE para visualizar e anotar as matrizes do ATT&CK, usada para montar mapas de cobertura de detecção. **p. 144**

auditd. Subsistema de auditoria do kernel Linux que registra acessos a arquivos, chamadas de sistema e comandos conforme regras configuradas. **p. 78**

Auditoria de primeira, segunda e terceira parte. Interna (a própria organização), do cliente sobre o fornecedor, e externa independente (certificação ou relatório). **p. 27**

Auditoria de senhas. Medição da força das senhas por meio de hashes obtidos com autorização, com ferramentas como Hashcat e John the Ripper. **p. 166**

Autenticador. Mecanismo que produz a prova de identidade, como um app de códigos TOTP ou uma chave FIDO2. **p. 44**

Autenticidade. Garantia de que a origem de um dado ou de uma entidade é realmente a declarada. **p. 5**

Autoridade certificadora (AC, CA). Entidade que emite e assina certificados; as raízes confiáveis ficam na loja de confiança do sistema operacional ou do navegador. **p. 34**

Autorização (AuthZ). Decisão sobre o que uma identidade autenticada pode fazer com cada recurso; distinta da autenticação. **p. 53**

AV (Asset Value). Valor do ativo, incluindo custo de recuperação, perda de receita, multas e impacto reputacional. **p. 20**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Avaliação de vulnerabilidades. Busca sistemática, em geral automatizada, de falhas conhecidas nos ativos. Responde "que falhas existem?", sem encadeá-las nem demonstrar o impacto. **p. 166**

B

Backend for frontend (BFF). Camada de servidor própria de um cliente, como o app móvel, que intermedeia chamadas a outros serviços e mantém segredos e regras fora do cliente. **p. 112**

Backport. Correção de segurança aplicada a uma versão antiga sem mudar o número da versão; faz scanners que detectam por versão acusarem falsos positivos. **p. 166**

Backup 3-2-1-1-0. Três cópias, em duas mídias, uma fora do local, uma offline ou imutável e zero erros na verificação de restauração. **p. 59**

BAS (breach and attack simulation). Produtos que simulam técnicas de ataque de forma contínua e em escala, para medir controles e detecções. **p. 166**

Base legal (LGPD). Hipótese que autoriza o tratamento de dados pessoais (art. 7º; art. 11 para os sensíveis): consentimento, obrigação legal, contrato, legítimo interesse, entre outras. **p. 59**

Baseline de achados. Lista dos achados existentes aceita em um momento, para que o gate reprove apenas problemas novos e o legado seja tratado com prazo. **p. 135**

Baseline de comportamento. Perfil do comportamento normal de uma conta, máquina ou rede, usado como referência para reconhecer desvios. **p. 144**

Bastion host (jump host). Servidor endurecido que é o único ponto de entrada administrativo para uma rede, com MFA e registro das sessões. **p. 67**

BCM e BCP (continuidade de negócios). BCM é a gestão que mantém funções críticas em operação ou as recupera; o BCP é o plano que cobre pessoas, locais, processos e TI. **p. 156**

BCP 38 (anti-spoofing). Prática (RFC 2827) de filtrar na entrada os pacotes com IP de origem inválido para a rede, reduzindo o spoofing e a amplificação. **p. 67**

BCP 38 (RFC 2827). Boa prática de filtro de entrada (ingress filtering) que impede a saída de pacotes com IP de origem falsificado, reduzindo reflexão. **p. 11**

Beaconing. Comunicação periódica e regular que um sistema comprometido faz com o servidor C2; o padrão de tempo ajuda a detectá-la. **p. 11**

BEC (Business Email Compromise). Fraude que se passa por executivo ou fornecedor por e-mail para desviar pagamentos ou obter dados sensíveis. **p. 11**

Bell–LaPadula. Modelo de MAC para sigilo: não ler níveis acima (no read up) e não escrever em níveis abaixo (no write down). **p. 53**

BFLA. Broken Function Level Authorization (API5): funções administrativas ou sensíveis acessíveis a usuários comuns, por rotas e métodos sem checagem de papel. **p. 112**

BGP. Border Gateway Protocol: troca rotas entre sistemas autônomos, com base em confiança mútua; sujeito a sequestro e a vazamento de rotas. **p. 67**

BIA (Business Impact Analysis). Análise que identifica processos essenciais, dependências e como o impacto cresce com o tempo parado; fornece as metas de MTD, RTO e RPO. **p. 156**

Biba. Modelo de MAC para integridade: não ler níveis abaixo e não escrever em níveis acima. **p. 53**

Bloqueador de escrita (write blocker). Dispositivo ou software que permite ler um disco sem escrever nele, garantindo que a coleta não altere a evidência. **p. 156**

BOPLA. Broken Object Property Level Authorization (API3): a API expõe ou aceita propriedades que o usuário não deveria ver ou alterar, como excesso de dados na resposta e atribuição em massa. **p. 112**

Botnet. Rede de dispositivos infectados controlados remotamente por um servidor de comando e controle (C2), usada em DDoS, spam e fraudes. **p. 11**

Break-glass. Conta de emergência, rara, protegida e monitorada, usada quando o IdP ou o acesso normal falha. **p. 44**

BSIMM. Building Security In Maturity Model: modelo descritivo que mede o que organizações reais fazem em segurança de software. **p. 27**

Bug bounty. Programa que paga recompensas a pesquisadores por vulnerabilidades válidas relatadas dentro das regras definidas pela organização. **p. 90**

Build reprodutível (reproducible build). Build que, com o mesmo código-fonte e o mesmo ambiente definido, gera exatamente os mesmos bytes; permite a terceiros verificar que o binário distribuído corresponde ao código. **p. 135**

BYOD e MAM. BYOD é o uso de dispositivo pessoal no trabalho; o MAM gerencia só o aplicativo e os dados corporativos, sem controlar o aparelho inteiro. **p. 78**

C

Cabeçalhos de segurança. Cabeçalhos HTTP de resposta que ativam proteções no navegador (CSP, HSTS, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, COOP, COEP, CORP): defesa em profundidade. **p. 98**

Cadeado do navegador. Ícone que indica conexão cifrada (HTTPS) com o site, não que o site seja legítimo: páginas de phishing também usam certificados válidos. **p. 180**

Cadeia de ataque. Encadeamento de falhas pequenas que, combinadas, levam ao objetivo; mostra o risco real melhor do que cada falha isolada. **p. 166**

Cadeia de custódia. Registro sem lacunas de quem coletou a evidência, quando, como, onde foi guardada e quem teve acesso, para preservar o seu valor como prova. **p. 156**

Cadeia de suprimentos de software. Tudo o que entra em um produto e tudo o que o constrói e distribui: código próprio, dependências, imagens-base, ferramentas de build, pipelines, repositórios de pacotes e atualizações. **p. 135**

Caixa preta, cinza e branca. Graus de conhecimento prévio de quem testa: nenhum (preta), parcial, como credenciais comuns (cinza), ou total, com código e arquitetura (branca). **p. 166**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Camadas administrativas (tiering). Modelo que separa Tier 0 (controladores de domínio), Tier 1 (servidores) e Tier 2 (estações), sem reutilizar credenciais de uma camada em outra. **p. 78**

Caminho pavimentado (paved road). Conjunto de modelos, bibliotecas e pipelines já seguros oferecidos pela plataforma; quem o segue herda os controles sem esforço. **p. 90**

Canal de reporte. Meio simples e conhecido (botão no cliente de e-mail, endereço, ramal) pelo qual qualquer pessoa avisa a equipe de segurança de uma suspeita, sem burocracia e com agradecimento. **p. 180**

Canal fora de banda (out-of-band). Meio de comunicação independente da infraestrutura possivelmente comprometida (telefone, aplicativo separado), usado para coordenar a resposta com segurança. **p. 156**

Canário de pilha (stack canary). Valor aleatório colocado entre as variáveis locais e os dados de controle; conferido antes do retorno, detecta sobrescritas contíguas e aborta o programa (-fstack-protector-strong). **p. 122**

Canonicalização. Conversão do dado à sua forma única e final (decodificar uma vez, normalizar Unicode e caminhos) antes de validar, para impedir contornos por codificações alternativas. **p. 122**

Capabilities (Linux). Divisão do poder do root em privilégios independentes (como CAP_NET_BIND_SERVICE); CAP_SYS_ADMIN equivale quase a root. **p. 78**

Capability. Token infalsificável que o sujeito carrega e que confere direito a um recurso específico; evita o delegado confuso. **p. 53**

CAPEC. Common Attack Pattern Enumeration and Classification: catálogo de padrões de ataque usados para explorar fraquezas. **p. 20**

Carta de autorização. Documento assinado por quem tem poder sobre o ativo, autorizando o teste. É a prova de que a atividade é legítima e protege quem testa. **p. 166**

Cartão virtual. Número de cartão gerado para uso único ou por loja, com limite e bloqueio pelo aplicativo do banco; reduz o dano se os dados vazarem. **p. 180**

Caso de abuso (abuse case). Descrição de como um atacante poderia usar uma funcionalidade de forma maliciosa; ajuda a derivar requisitos de segurança. **p. 20**

Cedar. Linguagem e motor de políticas de autorização de código aberto criados pela AWS. **p. 53**

CERT.br. Centro de estudos, resposta e tratamento de incidentes de segurança no Brasil, mantido pelo NIC.br; autor da Cartilha de Segurança para Internet. **p. 27**

CERT/CC. Centro de coordenação de resposta a emergências da Carnegie Mellon (EUA); coordena a divulgação de vulnerabilidades e publica o guia de CVD. **p. 166**

Certificado SSH. Chave pública assinada por uma CA interna, com identidade e validade curta, que dispensa distribuir chaves públicas servidor a servidor. **p. 78**

Certificado X.509. Documento assinado por uma autoridade certificadora (RFC 5280) que vincula uma chave pública a uma identidade, como um nome DNS no campo SAN. **p. 34**

Certificate pinning. Fixação do certificado ou da chave pública esperados pelo app, que passa a recusar outros mesmo assinados por uma CA confiável; exige plano de rotação e não é fronteira de segurança. **p. 112**

Certificate Transparency (CT). Logs públicos e auditáveis (RFC 6962 e 9162) de todos os certificados emitidos, que permitem detectar emissões indevidas. **p. 34**

CFI (Control-Flow Integrity). Mitigação que restringe os desvios indiretos do programa a destinos válidos; no Windows, a variante nativa é o CFG (Control Flow Guard). **p. 122**

ChaCha20-Poly1305. AEAD de cifra de fluxo com MAC Poly1305 (RFC 8439); rápido em dispositivos sem aceleração de AES em hardware. **p. 34**

Checkbox compliance. Conformidade de fachada: cumprir requisitos apenas para passar em auditorias, sem reduzir o risco real. **p. 27**

Cibersegurança. Subconjunto da SI voltado a sistemas, redes e dados no meio digital; trata de ameaças que exploram computadores, software e a internet. **p. 5**

CIEM. Cloud Infrastructure Entitlement Management: análise das permissões efetivas em nuvem para reduzi-las ao que é usado. **p. 78**

Cifragem de campo. Cifragem de colunas ou campos específicos pela aplicação, antes de gravar no banco; protege contra vazamento do banco e contra o DBA sem a chave. **p. 59**

CIS Benchmarks. Guias de configuração segura por produto, mantidos pelo Center for Internet Security. **p. 27**

CIS Controls. Conjunto priorizado de 18 controles (v8.1) para defesa cibernética, com Grupos de Implementação IG1, IG2 e IG3. **p. 27**

CISM. Certificação da ISACA voltada à gestão de segurança, de riscos e de incidentes; exige experiência em gestão. **p. 166**

CISO. Chief Information Security Officer: executivo responsável pela estratégia e gestão de segurança da informação da organização. **p. 5**

CISSP. Certificação da ISC2 para gestão e arquitetura de segurança, em oito domínios; exige experiência comprovada. **p. 166**

Claims (JWT). Afirmações da carga de um JWT: iss (emissor), sub (sujeito), aud (destinatário), exp (expiração), nbf, iat e jti. **p. 44**

Classificação de dados. Atribuição de níveis (pública, interna, confidencial, restrita) que definem as regras de acesso, cifragem, retenção e descarte de cada dado. **p. 59**

Clickjacking. Ataque em que a página-alvo é embutida em um quadro invisível para que a vítima clique, sem saber, em um botão dela; defende-se com frame-ancestors na CSP (CWE-1021). **p. 98**

Clonagem de voz. Uso de IA para reproduzir a voz de uma pessoa a partir de amostras curtas de áudio; usada em golpes de falso parente ou executivo; combine uma palavra-código e confirme por outro canal. **p. 180**

Clonagem de WhatsApp. Tomada da conta de WhatsApp da vítima, em geral por golpe que obtém o código de verificação de 6 dígitos, para pedir dinheiro aos contatos; a verificação em duas etapas reduz o risco. **p. 180**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

CloudTrail e logs de auditoria da nuvem. Registros do plano de controle da nuvem (AWS CloudTrail, Azure Activity Log, Google Cloud Audit Logs): quem criou, alterou ou apagou recursos. **p. 144**

CMMI. Modelo de maturidade de processos organizacionais, com níveis de capacidade. **p. 27**

CNA (CVE Numbering Authority). Organização autorizada a atribuir identificadores CVE: fabricantes, projetos, coordenadores e plataformas de bug bounty; a MITRE atua como último recurso. **p. 166**

CNAPP. Cloud-Native Application Protection Platform: plataforma que unifica CSPM, CWPP, CIEM e varredura de IaC para priorizar riscos combinados. **p. 78**

COBIT. Framework da ISACA para governança e gestão de TI corporativa. **p. 27**

Codificação de saída (output encoding). Conversão dos caracteres especiais de um dado para o formato do contexto onde será escrito (HTML, atributo, JavaScript, URL, CSS), para que seja tratado como texto; defesa principal contra XSS. **p. 98**

Codificação por contexto (escape). Transformação do dado para que seja inofensivo no destino (HTML, atributo, URL, SQL, shell, log); a técnica correta depende do contexto de saída. **p. 122**

Codificação segura (secure coding). Prática de escrever código que continua correto sob uso hostil: validar na fronteira, codificar na saída, usar menor privilégio, falhar de forma segura e evitar APIs perigosas. **p. 122**

Códigos de recuperação. Códigos de uso único gerados ao ativar a MFA, que permitem voltar a uma conta se o segundo fator for perdido; guarde-os offline e trate-os como senhas. **p. 180**

Comandante do incidente. Pessoa que coordena a resposta, define prioridades e mantém uma visão única do todo, sem executar tarefas técnicas; evita decisões conflitantes. **p. 156**

Comando e controle (C2). Infraestrutura pela qual o atacante envia ordens e recebe dados dos sistemas comprometidos. **p. 11**

Comparação em tempo constante. Comparação cujo tempo não depende de quantos bytes coincidem; evita vazar informação por canal de tempo (CWE-208). **p. 34**

Comportamento indefinido (UB). Situação em que a linguagem (C, C++) não define o que o programa faz, como estourar um vetor ou um inteiro com sinal; o compilador pode assumir que ela não ocorre, e o resultado é imprevisível. **p. 122**

Comprometimento presumido (assumed breach). Formato de teste que parte de um acesso interno já concedido, para medir o que a organização contém e detecta depois da entrada. **p. 166**

Comunicação de Incidente de Segurança (CIS). Aviso formal do controlador à ANPD e aos titulares sobre incidente que possa causar risco ou dano relevante, no prazo de três dias úteis. **p. 156**

Condição de corrida (CWE-362). Falha em que o resultado depende da ordem de execução de operações concorrentes sobre um recurso compartilhado sem sincronização, como no padrão check-then-act (ler, decidir, gravar). **p. 122**

Confidencialidade. Propriedade de que a informação só seja acessível a quem está autorizado; violada por vazamento ou acesso indevido. **p. 5**

Confidential computing. Execução de código e dados em enclaves de hardware (TEE) cifrados em memória, protegendo o dado em uso até do sistema operacional e do provedor. **p. 59**

Configuração insegura (security misconfiguration). Categoria A02 do Top 10: erros de configuração, como depuração ligada, credenciais padrão, listagem de diretórios, cabeçalhos ausentes e analisadores XML permissivos. **p. 98**

Confused deputy (delegado confuso). Serviço com muitos privilégios induzido a usá-los em nome de quem não os possui. **p. 53**

Consulta parametrizada (prepared statement). Consulta cuja estrutura é fixada antes e cujos valores seguem por canal separado, sempre tratados como dados; defesa estrutural contra a injeção de SQL. **p. 98**

Consulta persistida (persisted query). Em GraphQL, consulta pré-registrada no servidor e chamada por identificador, o que impede consultas arbitrárias e reduz a superfície de abuso. **p. 112**

Contêiner. Processo isolado por recursos do kernel (namespaces, cgroups, seccomp) que compartilha o kernel do host; não é uma máquina virtual. **p. 78**

Contenção. Fase da resposta que impede o crescimento do dano: isolar hosts, desativar contas, revogar sessões e chaves, bloquear IoCs e segmentar a rede. **p. 156**

Contenção coordenada. Execução simultânea de todas as ações de contenção, depois de mapear a presença do adversário, para que ele não troque de acesso; contrasta com a contenção rápida. **p. 156**

Controlador. Agente de tratamento a quem competem as decisões sobre o tratamento de dados pessoais (LGPD). **p. 27**

Controlador de admissão. Componente que valida ou altera objetos antes de entrarem no cluster; Kyverno e OPA Gatekeeper impõem políticas como imagens assinadas. **p. 78**

Controle (contramedida). Medida que modifica o risco: política, procedimento, tecnologia ou prática que previne, detecta, corrige ou dissuade. **p. 5**

Controle compensatório. Controle alternativo, documentado, que oferece proteção equivalente quando o requisito original não pode ser cumprido. **p. 27**

Controle otimista e pessimista. Pessimista: trava a linha (SELECT ... FOR UPDATE) até o fim da transação. Otimista: grava só se a versão do registro não mudou e, caso contrário, repete a operação. **p. 122**

Cookie SameSite. Atributo que limita o envio de cookies em requisições entre sites (Lax, Strict, None); mitiga CSRF. **p. 44**

COOP, COEP e CORP. Cross-Origin-Opener-Policy, Embedder-Policy e Resource-Policy: cabeçalhos que isolam a página de outras origens e reduzem vazamentos entre contextos. **p. 98**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Corretor de acesso inicial (IAB). Criminoso que obtém e vende acessos a redes corporativas (credenciais, VPN, RDP) a operadores de ransomware. **p. 11**

CORS (Cross-Origin Resource Sharing). Mecanismo pelo qual o servidor relaxa a SOP, indicando por cabeçalhos quais origens podem ler suas respostas; é aplicado pelo navegador e não é um controle de acesso. **p. 98**

CPE. Common Platform Enumeration: esquema de nomes padronizados para produtos e versões, usado pelo NVD. **p. 20**

CRA (Cyber Resilience Act). Regulamento (UE) 2024/2847 com requisitos de segurança para produtos com elementos digitais, em vigor desde dezembro de 2024. **p. 27**

Credenciais temporárias (STS). Credenciais de curta duração obtidas por papéis e federação, que substituem chaves de acesso de longa duração e reduzem o impacto de vazamentos. **p. 78**

Credencial. Dado ou objeto usado como prova de identidade: senha, certificado, token ou chave física. **p. 44**

Credential Guard. Recurso do Windows que isola hashes NTLM e tíquetes Kerberos em ambiente virtualizado, fora do alcance do sistema e de ferramentas de extração. **p. 78**

Credential stuffing. Uso automatizado de pares usuário/senha vazados de um serviço para tentar entrar em outros, aproveitando a reutilização de senhas. **p. 11**

Cripto-agilidade. Capacidade de trocar algoritmos e tamanhos de chave sem redesenhar o sistema, apoiada por inventário criptográfico (CBOM). **p. 34**

Criptografia pós-quântica. Algoritmos resistentes a ataques quânticos (ML-KEM, ML-DSA, SLH-DSA; FIPS 203 a 205) para substituir RSA e ECC. **p. 34**

Critério de aceitação. Condição testável que define quando uma história está completa; os critérios de segurança são derivados de requisitos e de casos de abuso. **p. 90**

CRL e OCSP. Mecanismos de revogação: lista assinada de certificados revogados (CRL) ou consulta em tempo real a um respondedor (OCSP, com stapling). **p. 34**

Crypto-erase (apagamento criptográfico). Tornar os dados ilegíveis destruindo a chave que os cifra; elimina também cópias e backups sem acesso físico à mídia. **p. 59**

Crypto-shredding. Destruição criptográfica: tornar dados ilegíveis ao destruir a chave que os cifra, inclusive nas cópias e nos backups. **p. 34**

CSIRT, CERT e ETIR. Equipes de resposta a incidentes: CSIRT e CERT são os nomes usuais; ETIR é o termo da administração pública federal brasileira. **p. 156**

CSP (Content Security Policy). Cabeçalho que restringe de onde scripts, estilos, quadros e conexões podem vir; uma CSP estrita, com nonce, impede a execução de scripts injetados e mitiga XSS e clickjacking. **p. 98**

CSPM. Cloud Security Posture Management: ferramenta que audita contas e serviços de nuvem contra benchmarks e políticas, apontando configurações erradas. **p. 78**

CSPRNG. Cryptographically Secure Pseudorandom Number Generator: gerador imprevisível, exigido para chaves, nonces e tokens (secrets, SecureRandom). **p. 34**

CSRF (cross-site request forgery). Ataque em que o navegador da vítima, com a sessão ativa, envia sem querer um pedido forjado por outro site; funciona quando a autenticação é automática, como nos cookies (CWE-352). **p. 98**

CTF (capture the flag). Competição ou laboratório em que participantes resolvem desafios de segurança para capturar bandeiras; ferramenta de treinamento prático. **p. 90**

CTI (Cyber Threat Intelligence). Informação analisada sobre adversários, objetivos e métodos, com contexto e confiança, produzida para apoiar decisões estratégicas, operacionais, táticas e técnicas. **p. 144**

CTIR Gov. Centro de Tratamento e Resposta a Incidentes Cibernéticos de Governo, responsável pela coordenação de resposta na administração pública federal. **p. 27**

Cultura de segurança. Valores, hábitos e incentivos que definem como as pessoas tratam a segurança quando ninguém está olhando; depende de liderança, segurança psicológica e de tornar o caminho seguro o mais fácil. **p. 180**

Cultura justa (just culture). Abordagem que separa o erro honesto (aprendizado, sem punição) da negligência deliberada ou da má-fé (responsabilização), para que as pessoas relatem falhas cedo; complementa a cultura sem culpa. **p. 180**

Cultura sem culpa (blameless). Princípio de que as pessoas agem com o que sabiam na hora e de que as falhas vêm de sistemas e processos; incentiva o relato aberto. **p. 156**

CVE. Common Vulnerabilities and Exposures: identificador único de uma vulnerabilidade pública, no formato CVE-ano-número. **p. 20**

CVSS. Common Vulnerability Scoring System: padrão do FIRST que mede a gravidade técnica de vulnerabilidades em escala de 0,0 a 10,0. **p. 20**

CWE. Common Weakness Enumeration: catálogo de tipos de fraqueza de software e hardware (por exemplo CWE-79, XSS). **p. 20**

CWPP. Cloud Workload Protection Platform: proteção de VMs, contêineres e funções com varredura de vulnerabilidades e detecção em tempo de execução. **p. 78**

Cyber Kill Chain. Modelo da Lockheed Martin (2011) com sete fases de uma intrusão: reconhecimento, armamento, entrega, exploração, instalação, C2 e ações no objetivo. **p. 11**

Cyber Resilience Act (CRA). Regulamento (UE) 2024/2847: requisitos de cibersegurança e de tratamento de vulnerabilidades para produtos com elementos digitais, com obrigações escalonadas entre 2026 e 2027. **p. 135**

D

DAC. Controle de acesso discricionário: o dono do recurso decide quem tem acesso, como nas permissões de arquivos. **p. 53**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Dado pessoal sensível. Dado sobre origem racial ou étnica, religião, opinião política, filiação sindical, saúde, vida sexual, genético ou biométrico (LGPD, art. 5º, II); exige bases legais mais restritas. **p. 59**

DAST. Dynamic Application Security Testing: teste da aplicação em execução, vista de fora, em busca de falhas observáveis, como cabeçalhos ausentes, injeções refletidas e configuração (OWASP ZAP). **p. 135**

DCSync. Técnica em que o atacante imita um controlador de domínio para pedir a replicação de hashes de senha; exige monitorar a replicação feita por contas inesperadas. **p. 78**

DDoS. Negação de serviço distribuída: esgota banda, estado ou recursos da aplicação da vítima com tráfego de muitas origens, em geral uma botnet. **p. 67**

Deception (engano defensivo). Conjunto de técnicas (honeypots, honeytokens, contas e redes falsas) que detecta e estuda atacantes por meio de iscas. **p. 144**

Declaração de Aplicabilidade (SoA). Documento da ISO 27001 que lista os controles do Anexo A, indica se se aplicam, justifica exclusões e mostra o estado de implementação. **p. 27**

Deepfake. Mídia sintética (voz, imagem ou vídeo) gerada por IA que imita uma pessoa real; usada em fraudes e golpes de falsa identidade. **p. 11**

Defesa em profundidade. Estratégia de combinar camadas independentes de controles (pessoas, física, rede, host, aplicação, dados) para que a falha de uma não comprometa tudo. **p. 5**

Definição de pronto (DoD). Lista de critérios que uma história precisa cumprir para ser considerada concluída, incluindo testes e verificações de segurança. **p. 90**

DEK e KEK. Chave de dados (cifra os dados) e chave de criptografia de chaves (cifra as DEKs); a KEK nunca sai em claro do KMS ou HSM. **p. 34**

Denial of wallet. Abuso que gera custo financeiro (chamadas pagas, tokens, nuvem) em vez de derrubar o serviço; reforça a necessidade de cotas e de alertas de gasto. **p. 112**

Densidade de vulnerabilidades. Número de falhas por aplicação ou por mil linhas de código; útil para acompanhar tendências, não para comparar equipes. **p. 90**

DEP e NX (W^X). Data Execution Prevention e No-eXecute: páginas graváveis não são executáveis, o que impede rodar código injetado na pilha ou no heap; não impede o reaproveitamento de código existente (ROP). **p. 122**

Dependência transitiva. Biblioteca que o projeto não escolheu, mas que chega como dependência de outra; compõe a maior parte da árvore de dependências e a origem da maioria dos alertas de SCA. **p. 135**

Dependency confusion. Ataque em que o nome de um pacote interno é registrado no repositório público com versão maior, e o gerenciador escolhe o público; evita-se com escopos, índice único e fixação por hash. **p. 135**

Desconflito (deconfliction). Coordenação para distinguir ações do exercício de ataques reais, evitando que a defesa acione autoridades ou derrube a produção por engano. **p. 166**

Desserialização insegura. Reconstrução de objetos a partir de dados não confiáveis, em formatos que instanciam tipos arbitrários, o que pode executar código (CWE-502). **p. 98**

Desvio de configuração (drift). Diferença entre a configuração real de um sistema e a linha de base ou o código que a define; deve ser detectada e revertida. **p. 78**

Detecção como código (detection as code). Prática de manter detecções em repositório versionado, com revisão, testes e implantação automatizada, como se faz com software. **p. 144**

DevSecOps. Integração de controles de segurança ao ciclo de entrega, automatizados e com retorno rápido, com responsabilidade compartilhada entre desenvolvimento, operações e segurança. **p. 135**

DFD (Data Flow Diagram). Diagrama de fluxo de dados com entidades externas, processos, repositórios, fluxos e fronteiras de confiança. **p. 20**

DHCP snooping. Recurso de comutadores que só aceita respostas DHCP de portas confiáveis, bloqueando servidores DHCP falsos. **p. 67**

Diamond Model. Modelo de análise de intrusão que relaciona quatro vértices: adversário, capacidade, infraestrutura e vítima. **p. 11**

Diffie–Hellman (DH, ECDH). Protocolo em que duas partes derivam um segredo compartilhado a partir de chaves públicas trocadas em canal aberto; não autentica o par. **p. 34**

Digest de imagem. Hash SHA-256 imutável que identifica o conteúdo exato de uma imagem; fixá-lo evita que uma tag como latest mude sem aviso. **p. 78**

Direitos do titular. Confirmação e acesso, correção, anonimização, bloqueio ou eliminação, portabilidade, informação sobre compartilhamento e revogação do consentimento (LGPD, art. 18). **p. 59**

Diretriz (guideline). Orientação recomendada, não obrigatória, com boas práticas para atingir o objetivo de uma política ou norma. **p. 27**

DISA STIG. Security Technical Implementation Guides: guias de configuração do Departamento de Defesa dos EUA, detalhados e verificáveis por automação. **p. 78**

Disponibilidade. Propriedade de que sistemas e dados estejam acessíveis quando necessários a quem tem direito; atacada por DoS, ransomware ou falhas. **p. 5**

Dívida de segurança. Vulnerabilidades e exceções aceitas e ainda abertas; precisa de dono, prazo e critério de pagamento para não se acumular de forma invisível. **p. 90**

Divulgação coordenada (CVD). Processo em que quem encontra uma vulnerabilidade a relata em privado ao fabricante, que corrige antes da divulgação pública (ISO/IEC 29147 e 30111). **p. 90**

DKIM. DomainKeys Identified Mail (RFC 6376): assinatura criptográfica de cabeçalhos e corpo; a chave pública fica no DNS, em seletor._domainkey. **p. 67**

DLP (Data Loss Prevention). Controles que inspecionam e bloqueiam a saída indevida de dados (e-mail, upload, USB, nuvem) por padrões, palavras-chave e impressões digitais de documentos. **p. 59**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

DMARC. Política de domínio (RFC 9989) que exige SPF ou DKIM alinhados ao From visível e define o tratamento das falhas (none, quarantine, reject) e os relatórios. **p. 67**

DMZ (zona desmilitarizada). Segmento entre dois firewalls que abriga os serviços expostos à internet e os isola da rede interna. **p. 67**

DNSSEC. Extensão do DNS que assina as respostas (DNSKEY, RRSIG, DS) para garantir autenticidade e integridade; não cifra a consulta. **p. 67**

DoS e DDoS. Negação de serviço: esgota recursos para tornar um serviço indisponível; no DDoS, o ataque vem de muitas origens simultâneas. **p. 11**

DoT e DoH. DNS sobre TLS (porta 853) e DNS sobre HTTPS: cifram a consulta entre o cliente e o resolvedor, sem validar a resposta. **p. 67**

Double free (CWE-415). Liberar duas vezes o mesmo bloco, corrompendo os metadados do alocador e podendo permitir escritas em locais indevidos. **p. 122**

Double spend (gasto duplo). Uso do mesmo saldo, cupom ou crédito mais de uma vez por requisições simultâneas; resultado típico de uma condição de corrida em operações de ler, decidir e gravar. **p. 122**

DREAD. Modelo antigo de pontuação (dano, reprodutibilidade, explorabilidade, usuários afetados, descoberta), hoje considerado subjetivo e obsoleto. **p. 20**

Dupla aprovação (quatro olhos). Exigência de que duas pessoas aprovem uma operação sensível (pagamento, troca de dados bancários, acesso privilegiado); reduz fraude e erro, inclusive o de BEC e deepfake. **p. 180**

Dupla extorsão. Combinação de cifrar os dados e ameaçar divulgar os que foram roubados; a tripla extorsão acrescenta DDoS ou pressão a clientes. **p. 11**

E

ECDSA e Ed25519. Assinaturas em curvas elípticas; o ECDSA exige um número aleatório único por assinatura, e o Ed25519 o deriva de forma determinística. **p. 34**

ECS (Elastic Common Schema). Esquema aberto de nomes e tipos de campos para eventos, mantido pela Elastic, usado para normalizar logs de fontes diferentes. **p. 144**

EDR (Endpoint Detection and Response). Ferramenta que coleta telemetria contínua do endpoint, detecta comportamento suspeito e permite resposta remota, como isolar a máquina. **p. 78**

EF (Exposure Factor). Fração do valor do ativo que se perde em um único incidente, entre 0 e 1. **p. 20**

Elemento humano (DBIR). Categoria do relatório DBIR da Verizon para violações em que há ação de pessoas, como erro, phishing e uso de credenciais; mostra que pessoas são alvo frequente e também parte da defesa. **p. 180**

Elevation of privilege. Elevação de privilégio: obter permissões acima das concedidas, por exemplo trocando um parâmetro de perfil. **p. 20**

Embeddings e banco vetorial. Embeddings são representações numéricas de textos; o banco vetorial as indexa para busca por similaridade, base do RAG, e precisa de controle de acesso por documento. **p. 112**

Emulação de adversário. Execução controlada de TTPs reais (com ferramentas como Atomic Red Team ou Caldera) para verificar se as detecções disparam e se a equipe reage. **p. 144**

Encarregado (DPO). Pessoa indicada pelo controlador como canal de comunicação com os titulares e a ANPD. **p. 27**

Engenharia de detecção. Disciplina que projeta, testa, implanta, mede e aposenta detecções a partir de requisitos de risco e de técnicas de adversários. **p. 144**

Engenharia social. Manipulação psicológica de pessoas para obter informações, acessos ou ações, explorando confiança, urgência e autoridade. **p. 11**

Enriquecimento de alerta. Acréscimo automático de contexto a um alerta (reputação do IP, dono do ativo, criticidade, geolocalização) para acelerar a triagem. **p. 144**

Entidade externa. Elemento fora do controle do sistema que interage com ele, como usuários e serviços de terceiros. **p. 20**

Entropia de senha. Medida de imprevisibilidade em bits: L × log2(N) para L símbolos escolhidos ao acaso de um alfabeto de N. **p. 44**

Enumeração. Etapa ativa de listar serviços, contas, diretórios e parâmetros de um alvo para mapear o que pode ser testado; deixa rastro nos logs. **p. 166**

Enumeração de usuários. Descobrir quais contas existem a partir de diferenças em mensagens ou tempos de resposta (CWE-204). **p. 44**

Envelope encryption. Cifragem em duas camadas: uma DEK cifra os dados e uma KEK, guardada no KMS ou HSM, cifra a DEK; facilita rotação e auditoria. **p. 34**

Envenenamento de cache DNS. Inserção de respostas falsas no cache de um resolvedor, redirecionando usuários para destinos controlados pelo atacante. **p. 11**

Envenenamento de dados e de modelo. Manipulação dos dados de treino, de ajuste ou da base de conhecimento para degradar o modelo ou plantar uma porta dos fundos acionada por um gatilho (LLM04). **p. 112**

EO 14028. Ordem executiva dos EUA (maio/2021) sobre a cibersegurança, inclusive da cadeia de suprimentos de software; tornou o SSDF e o SBOM referências para fornecedores do governo federal. **p. 135**

Épsilon (ε, orçamento de privacidade). Parâmetro da privacidade diferencial: quanto menor, mais privacidade e menos precisão nos resultados. **p. 59**

EPSS. Exploit Prediction Scoring System: estimativa do FIRST da probabilidade (0 a 1) de uma vulnerabilidade ser explorada nos próximos 30 dias. **p. 20**

Equipe azul (blue team). Equipe de defesa: monitora, detecta, responde a incidentes e fortalece controles; é quem os exercícios vermelhos e roxos testam e melhoram. **p. 166**

Equipe roxa (purple team). Prática colaborativa em que atacantes e defensores executam técnicas juntos, checam se a defesa viu e melhoram a detecção na hora; não é uma equipe separada. **p. 166**

— GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Equipe vermelha (red team). Equipe que emula um adversário realista, com objetivo definido e furtividade, para medir se a organização detecta e responde, e não só quantas falhas existem. **p. 166**

Erradicação. Fase que remove a presença do adversário e a causa: elimina persistência, corrige a falha explorada, rotaciona credenciais e reconstrói o que foi comprometido. **p. 156**

Escalada de privilégio. Obtenção de acesso acima do concedido: vertical (papel superior) ou horizontal (dados de outro usuário do mesmo nível). **p. 53**

Escopo (de um teste). Lista taxativa do que pode ser testado (ativos, endereços, contas, horários). O que não está listado está fora e não se toca. **p. 166**

Esgotamento de recursos (CWE-770). Alocar memória, CPU, disco ou conexões sem limites nem controle de taxa, de modo que uma requisição consuma mais do que o sistema suporta; defende-se com limites, timeouts e cotas. **p. 122**

Espelhamento de porta (SPAN). Recurso de comutadores que copia o tráfego de portas ou VLANs para uma porta de análise, como a de um sensor IDS. **p. 67**

Estados do dado. Em trânsito (na rede), em repouso (disco, banco, backup) e em uso (memória, tela, processamento); cada estado exige controles próprios. **p. 59**

Estouro de buffer (buffer overflow). Escrita além do fim de um buffer, corrompendo dados vizinhos, entre eles dados de controle; aparece como CWE-120, 121 e 122 (pilha e heap) e, de modo geral, como CWE-787. **p. 122**

Estouro de inteiros (CWE-190). Resultado de uma operação que não cabe no tipo: dá a volta (wraparound) em inteiros sem sinal e é comportamento indefinido em C com sinal. Perigoso ao calcular tamanhos de memória. **p. 122**

Evento × incidente. Evento é qualquer ocorrência observável; incidente é um evento que compromete a confiabilidade, integridade ou disponibilidade, ou viola política. **p. 5**

Evidência de controle. Registro objetivo (log, relatório do CI, captura, chamado) que prova que um controle existe e funciona; é a base de auditorias e de exercícios de conformidade. **p. 189**

Evil twin. Ponto de acesso falso que imita uma rede legítima para atrair usuários e interceptar o tráfego. **p. 67**

Exceção temporária (waiver). Dispensa formal e com prazo de um controle ou portão, com dono, motivo e plano de correção; o número de exceções abertas indica a saúde do processo. **p. 189**

Exemplos adversariais (evasão). Entradas levemente perturbadas para que um modelo erre na inferência, como um sinal de trânsito adulterado ou um malware que escapa do classificador. **p. 112**

Exercício de mesa (tabletop). Simulação discutida, sem tocar nos sistemas, em que a equipe percorre um cenário com novos fatos (injects) e decide como agiria. **p. 156**

Exploit. Técnica, código ou procedimento que aproveita uma vulnerabilidade específica para obter um efeito não autorizado. **p. 5**

Extensão de comprimento. Ataque a hashes Merkle–Damgård (MD5, SHA-1, SHA-2) que calcula H(segredo‖m‖x) sem conhecer o segredo; por isso usa-se HMAC. **p. 34**

Extração de modelo. Cópia do comportamento de um modelo por consultas em massa; mitigada com limites de taxa, cotas e monitoramento de padrões de consulta. **p. 112**

F

Fadiga de alertas. Insensibilização de quem analisa diante de excesso de alertas de baixo valor, que leva a ignorar também os verdadeiros. **p. 144**

Fadiga de MFA (MFA fatigue). Ataque que dispara pedidos de aprovação push repetidos até a vítima aprovar; mitigado com correspondência de número. **p. 44**

Fadiga de segurança. Cansaço de quem recebe regras, alertas e trocas de senha em excesso; leva a atalhos, a ignorar avisos e ao uso de ferramentas não aprovadas. **p. 180**

fail2ban. Ferramenta que lê logs e bloqueia, por firewall, IPs com muitas falhas de autenticação; atenua a força bruta, mas não substitui reduzir a exposição. **p. 78**

Failover e failback. Failover é a transferência do serviço para o ambiente de reserva; failback é a volta ao ambiente principal depois de restaurado. **p. 156**

FAIR. Factor Analysis of Information Risk: modelo de análise quantitativa que decompõe o risco em frequência e magnitude de perda, com estimativas calibradas. **p. 20**

Falco. Ferramenta aberta de detecção em tempo de execução que alerta sobre comportamento anômalo de contêineres com base em chamadas de sistema. **p. 78**

Falhar aberto (fail open). Projeto em que o erro de um controle resulta em liberar o acesso (CWE-636); o oposto de falhar fechado e o núcleo da categoria A10 do Top 10 de 2025. **p. 98**

Falhar de forma segura (fail-safe/fail-secure). Quando um controle falha, o sistema deve cair em estado seguro (negar acesso), não em estado aberto. **p. 5**

Falhar fechado (fail closed). Projeto em que erro, exceção ou indisponibilidade de um controle resulta em negar o acesso, e não em liberá-lo. **p. 90**

Falso positivo e falso negativo. Falso positivo é um alerta sem ameaça; falso negativo é uma ameaça real que não gerou alerta, e por isso o mais perigoso. **p. 144**

Falso suporte técnico. Golpe em que o criminoso se passa por suporte (pop-up, ligação ou anúncio) para obter acesso remoto, senhas ou pagamento; suporte legítimo não procura você sem que tenha pedido ajuda. **p. 180**

Fatores contribuintes. Condições técnicas, de processo e humanas que, combinadas, permitiram o incidente; a análise moderna prefere-os a uma única causa-raiz. **p. 156**

Fetch Metadata (Sec-Fetch-Site). Cabeçalhos enviados pelo navegador que indicam a relação entre quem faz o pedido e o destino (same-origin, same-site, cross-site ou none); servidores os usam para recusar pedidos externos. **p. 98**

FFI (foreign function interface). Mecanismo pelo qual uma linguagem chama código de outra (por exemplo, Rust chamando C); é a fronteira em que as garantias de segurança de memória terminam. **p. 122**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

FIDO2 e WebAuthn. Padrão (FIDO Alliance e W3C) de autenticação por chave pública, com credencial por site vinculada à origem; resistente a phishing. **p. 44**

Fileless malware. Ataque que opera na memória e usa recursos legítimos do sistema, deixando poucos arquivos para análise por antivírus. **p. 11**

Filtro de saída (egress filtering). Controle do tráfego que sai da rede para a internet; impede que máquinas comprometidas falem livremente com servidores de comando e controle. **p. 67**

Fim de suporte (end of life). Data a partir da qual o fabricante deixa de entregar correções de segurança; exige substituir, isolar ou compensar o sistema. **p. 78**

Firewall. Controle que decide quais conexões passam entre zonas de rede, por regras de origem, destino, porta e protocolo e, nos modelos modernos, também de aplicação e de usuário. **p. 67**

Firewall stateful. Firewall que mantém uma tabela de estado das conexões e libera automaticamente o tráfego de retorno do que foi iniciado do lado permitido. **p. 67**

Fixação de sessão. Ataque em que o invasor define um ID de sessão conhecido antes do login; evitado renovando o ID ao autenticar. **p. 44**

Fixação por hash (hash pinning). Exigir que cada pacote instalado tenha o hash esperado (pip --require-hashes), o que impede a troca silenciosa do conteúdo por outro com o mesmo nome e a mesma versão. **p. 135**

Fixação por SHA. Referenciar uma action ou imagem pelo hash do commit ou pelo digest, e não por tag ou versão mutáveis, para que o código não possa ser trocado sem que a referência mude. **p. 135**

Fontes e sinks do DOM. Fonte é a origem de um dado controlável pelo atacante (location.hash, postMessage); sink é a função que o escreve ou executa (innerHTML, eval). O DOM XSS liga um ao outro sem passar pelo servidor. **p. 98**

Força bruta (brute force). Tentativa sistemática de combinações até acertar uma senha ou chave, online ou offline contra hashes roubados. **p. 11**

Forced browsing. Acesso a rotas ou arquivos não divulgados digitando a URL, sem checagem de autorização (CWE-425). **p. 53**

Forense digital (DFIR). Coleta, preservação e análise de evidências digitais de modo reprodutível e defensável, integrada à resposta a incidentes (Digital Forensics and Incident Response). **p. 156**

Format string (CWE-134). Falha em que a entrada do usuário é usada como formato do printf; especificadores como %s e %n leem e escrevem memória. **p. 122**

_FORTIFY_SOURCE. Opção de compilação (exige -O1 ou mais) que troca funções como memcpy e strcpy por versões que verificam o tamanho do destino quando o compilador o conhece. **p. 122**

Frida. Ferramenta de instrumentação dinâmica que injeta código em processos em execução; usada na análise de apps móveis para inspecionar chamadas e contornar checagens locais. **p. 112**

Fronteira de confiança (trust boundary). Ponto em que dados ou controle passam entre níveis de confiança diferentes; onde se concentram validação e autenticação. **p. 5**

Fuga de contêiner (container escape). Exploração de falha do kernel, do runtime ou de configuração perigosa para sair do isolamento do contêiner e atingir o host. **p. 78**

Fuzzing guiado por cobertura. Teste que gera e muta entradas aos milhões, mantendo as que percorrem ramos novos do código; com sanitizers acha estouros e travamentos (AFL++, libFuzzer, OSS-Fuzz). **p. 122**

G

Gadget chain. Encadeamento de comportamentos de classes já presentes na aplicação, acionados durante a desserialização, que transforma um dado malicioso em uma ação perigosa, como a execução de código. **p. 122**

Game day e engenharia do caos. Exercícios que induzem falhas reais e controladas (derrubar um serviço, simular uma região indisponível) para testar a resiliência e os planos. **p. 156**

Gate de segurança. Ponto de decisão do ciclo de vida: o trabalho só avança se os critérios de segurança da fase foram cumpridos ou se o risco foi aceito por quem tem autoridade. **p. 90**

Gateway de API. Ponto de entrada que aplica o que é comum a todas as rotas (TLS, autenticação, limites, validação de esquema, roteamento e registro); não substitui a autorização por objeto. **p. 112**

GDPR. Regulamento Geral sobre a Proteção de Dados da União Europeia (2016/679), com multas de até 20 milhões de euros ou 4% do faturamento global. **p. 27**

Generalização e supressão. Técnicas de anonimização: reduzir a precisão de um atributo (idade vira faixa etária) ou remover valores raros que identificariam alguém. **p. 59**

Gerenciador de segredos. Serviço que guarda e entrega credenciais em tempo de execução, com controle de acesso, auditoria e rotação (HashiCorp Vault, AWS Secrets Manager, Google Secret Manager, Azure Key Vault). **p. 135**

Gerenciador de senhas. Aplicativo que gera, guarda e preenche senhas longas e únicas em um cofre cifrado; só preenche no endereço cadastrado, o que também ajuda a notar sites falsos. **p. 180**

Gestão de exceções. Processo para aprovar desvios de política com dono, justificativa, risco aceito, prazo e revisão. **p. 27**

Gestão de patches. Processo de inventariar, priorizar, testar, implantar e verificar correções de software, tratado pelo NIST SP 800-40 Rev. 4 como manutenção preventiva. **p. 78**

Gestão de riscos. Processo contínuo de identificar, analisar, avaliar, tratar e monitorar riscos, alinhado ao contexto e aos objetivos da organização (ISO 31000, ISO/IEC 27005). **p. 20**

Gestão de vulnerabilidades. Ciclo contínuo de descobrir, priorizar, corrigir e verificar vulnerabilidades em sistemas e aplicações. **p. 20**

Gherkin (BDD). Linguagem de cenários em Dado, Quando e Então, usada no desenvolvimento orientado a comportamento para escrever critérios de aceitação executáveis. **p. 90**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

gitleaks e TruffleHog. Ferramentas de código aberto que varrem repositórios, históricos e pipelines em busca de segredos, por padrões e por entropia; usadas no pre-commit e no CI. **p. 135**

Golden Ticket (KRBTGT). Ticket Kerberos forjado com o hash da conta KRBTGT, que dá acesso amplo ao domínio; por isso a senha dessa conta é redefinida duas vezes após um comprometimento. **p. 156**

Golpe da falsa central. Fraude em que o criminoso liga como se fosse o banco, cita dados reais e induz a pessoa a transferir dinheiro, "validar" a conta ou instalar acesso remoto; banco não pede senha nem código por telefone. **p. 180**

Google SAIF. Secure AI Framework: práticas do Google para proteger sistemas de IA ao longo do ciclo de vida. **p. 112**

Governança de segurança. Conjunto de direção, responsabilidades e prestação de contas pelo qual a alta direção define o apetite a risco e cobra resultados de segurança. **p. 27**

Grupo de Implementação (IG). Nível de adoção dos CIS Controls conforme tamanho e risco da organização; IG1 é a higiene cibernética essencial. **p. 27**

Guardrails. Controles automáticos e de baixo atrito (verificações no pipeline, modelos seguros) que dão retorno rápido e impedem erros comuns, no lugar de aprovações manuais. **p. 90**

gVisor, Kata e Firecracker. Tecnologias de isolamento mais forte para contêineres: kernel em espaço de usuário (gVisor) ou microVMs (Kata Containers, Firecracker). **p. 78**

H

Hacktivista. Agente que ataca por motivação política ou social, geralmente com DDoS, pichação de sites e vazamento de dados. **p. 11**

Hardening. Redução da superfície de ataque de um sistema: instalar o mínimo, desligar serviços sem uso, remover contas padrão, aplicar configurações seguras e restringir privilégios. **p. 78**

Harvest now, decrypt later. Estratégia de gravar hoje tráfego cifrado para decifrá-lo no futuro com computadores quânticos; ameaça dados de longa vida. **p. 34**

Heartbleed. Falha de 2014 (CVE-2014-0160) no OpenSSL: a extensão heartbeat devolvia bytes conforme o tamanho declarado pelo cliente, vazando até 64 KiB de memória por pedido. Exemplo clássico de leitura fora dos limites. **p. 122**

Higiene digital. Hábitos regulares de segurança pessoal: atualizar sistemas, usar senhas únicas e MFA, fazer backup, revisar permissões e desconfiar de links e anexos inesperados. **p. 180**

HKDF. HMAC-based Key Derivation Function (RFC 5869): deriva subchaves de um segredo de alta entropia nas etapas extract e expand. **p. 34**

HMAC. Hash-based Message Authentication Code (RFC 2104): MAC construído sobre uma função de hash e uma chave; resiste à extensão de comprimento. **p. 34**

Honeypot. Sistema falso, exposto de propósito e isolado da produção, que atrai e registra tentativas de ataque; qualquer conexão é suspeita. **p. 144**

Honeytoken. Dado-isca (credencial falsa, registro fictício, arquivo atraente) sem uso legítimo; quando é usado ou lido, dispara um alerta de alta fidelidade. **p. 144**

HSM (Hardware Security Module). Dispositivo resistente a violação que gera e usa chaves sem permitir sua exportação em claro. **p. 34**

HSTS. HTTP Strict Transport Security (RFC 6797): cabeçalho que obriga o navegador a usar HTTPS no domínio, impedindo downgrade e SSL stripping. **p. 34**

Humano no laço (human-in-the-loop). Projeto em que uma pessoa aprova ações sensíveis ou irreversíveis antes da execução, com pré-visualização, limitando o dano de um modelo enganado. **p. 112**

I

IAST. Interactive Application Security Testing: agente dentro da aplicação, durante os testes funcionais, que liga a entrada do usuário ao ponto perigoso e reduz os falsos positivos. **p. 135**

ID de correlação (correlation ID). Identificador único de um pedido, propagado entre serviços e gravado em cada evento, que permite reconstituir o caminho completo da requisição. **p. 144**

ID token. JWT assinado emitido no OIDC que informa quem autenticou, quando e para qual cliente (sub, iss, aud, exp, nonce). **p. 44**

Idempotência. Propriedade de uma operação que produz o mesmo efeito se executada uma ou várias vezes; permite repetir pedidos e processar eventos duplicados com segurança. **p. 112**

Identidade digital. Conjunto de atributos que representa uma pessoa, um serviço ou um dispositivo em um sistema (o subject). **p. 44**

IDOR e BOLA. Referência direta insegura a objeto e autorização quebrada em nível de objeto: o servidor aceita um ID do cliente sem checar a propriedade (CWE-639). **p. 53**

IdP (Identity Provider). Provedor de identidade: autentica usuários e emite asserções ou tokens para as aplicações que confiam nele. **p. 44**

IDS (detecção de intrusão). Sistema que observa o tráfego (NIDS) ou o host (HIDS) e alerta sobre atividade suspeita, por assinaturas ou por anomalias de comportamento. **p. 67**

IEC 62443. Série de normas de segurança para sistemas de automação e controle industrial, com zonas, conduítes e níveis de segurança. **p. 67**

Imagem distroless. Imagem de contêiner mínima, sem shell nem gerenciador de pacotes, que reduz vulnerabilidades e ferramentas disponíveis a um invasor. **p. 78**

Imagem forense. Cópia bit a bit de um disco ou de uma partição, inclusive áreas apagadas e não alocadas, feita sobre o original com bloqueio de escrita e conferida por hash. **p. 156**

Imagem-padrão (golden image). Imagem de sistema já endurecida e atualizada, usada para criar novos servidores sempre iguais. **p. 78**

Impacto. Consequência medida de um incidente: financeira, operacional, legal, reputacional ou sobre pessoas. **p. 5**

Implantação canário (canary release). Liberação gradual de uma versão para uma pequena parcela do tráfego, com observação de falhas antes de estendê-la a todos; facilita a reversão. **p. 189**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Imutabilidade (WORM, Object Lock). Gravação única e leitura múltipla: o dado não pode ser alterado nem apagado antes do prazo, nem pelo administrador; defesa contra ransomware. **p. 59**

Inferência de pertinência. Ataque de privacidade que descobre se um registro específico fez parte do conjunto de treino de um modelo. **p. 112**

Infostealer. Malware que coleta credenciais, cookies de sessão, carteiras e dados do navegador para revenda ou uso direto em fraudes. **p. 11**

Infraestrutura como código (IaC). Descrição da infraestrutura em arquivos versionados (Terraform, CloudFormation), que podem ser revisados e testados; permite varrer configurações inseguras antes da implantação. **p. 135**

Injeção (injection). Classe de falha em que dados não confiáveis são combinados a um comando ou consulta e o interpretador não distingue dado de código (SQL, shell, LDAP, templates, HTML). **p. 98**

Injeção de comando do SO. Falha em que a entrada do usuário é entregue a um shell e o atacante encadeia comandos (CWE-78); evita-se sem shell, com lista de argumentos e validação do formato. **p. 98**

Injeção de prompt (prompt injection). Ataque em que entradas fazem o modelo seguir instruções não pretendidas: direta (escrita pelo usuário, o jailbreak) ou indireta (escondida em conteúdo recuperado, como páginas e e-mails). LLM01. **p. 112**

Injeção de segunda ordem. Injeção em que o dado malicioso é gravado de forma inofensiva e só age depois, quando outra consulta o usa em uma concatenação. **p. 98**

Injeção de SQL (SQLi). Injeção em que o texto do usuário passa a fazer parte de uma consulta SQL e altera sua lógica (CWE-89); a defesa estrutural são as consultas parametrizadas. **p. 98**

Injeção de templates (SSTI). Falha em que a entrada do usuário vira parte do próprio template e expressões do motor são avaliadas, em geral com execução de código no servidor. **p. 98**

Injeção NoSQL. Injeção em bancos não relacionais, como quando um corpo JSON é repassado como filtro e um objeto com operador (por exemplo, $ne) substitui o texto esperado. **p. 98**

Integridade. Propriedade de que a informação e os sistemas permaneçam exatos, completos e não alterados sem autorização. **p. 5**

Introspecção do GraphQL. Recurso que permite consultar o esquema da API; útil no desenvolvimento, mas facilita o reconhecimento por quem ataca e costuma ser desligado em produção. **p. 112**

Inventário de dados (data mapping). Levantamento de quais dados existem, onde estão, quem os acessa, por que são tratados e por quanto tempo; ponto de partida da proteção e da conformidade. **p. 59**

IoC (Indicator of Compromise). Evidência observável de comprometimento: hash, IP, domínio, chave de registro, padrão de tráfego. **p. 11**

IPS (prevenção de intrusão). IDS instalado em linha (inline) que, além de alertar, bloqueia o tráfego malicioso. **p. 67**

IPsec e IKEv2. Conjunto de protocolos que cifra e autentica pacotes IP (ESP); o IKEv2 negocia as chaves e as associações de segurança. **p. 67**

ISAC. Information Sharing and Analysis Center: centro, em geral setorial, em que organizações pares trocam inteligência sobre ameaças e incidentes. **p. 144**

ISO 22301. Norma de sistemas de gestão de continuidade de negócios (BCMS), que trata de análise de impacto, estratégias, planos e exercícios. **p. 156**

ISO/IEC 27001. Norma internacional, certificável, com requisitos para estabelecer, operar e melhorar um SGSI; a edição vigente é a de 2022. **p. 27**

ISO/IEC 27002. Norma de orientação que detalha como implementar os 93 controles do Anexo A da ISO/IEC 27001:2022; não é certificável. **p. 27**

ISO/IEC 27035. Série de normas para a gestão de incidentes de segurança da informação: planejamento, detecção, avaliação, resposta e lições aprendidas. **p. 156**

ISO/IEC 27037. Diretrizes para identificação, coleta, aquisição e preservação de evidências digitais, de modo a manter a integridade e a validade da prova. **p. 156**

ISO/IEC 42001. Norma de sistema de gestão de IA (2023), certificável, nos moldes da ISO/IEC 27001. **p. 112**

J

JML (Joiner–Mover–Leaver). Ciclo de vida da identidade (entrada, mudança de função e saída); falhas aqui geram privilégios acumulados e contas órfãs. **p. 44**

Just-in-time (JIT). Concessão de privilégio sob demanda, por tempo limitado e com aprovação, eliminando privilégios permanentes. **p. 53**

JWKS. JSON Web Key Set: documento com as chaves públicas do emissor, usado para validar assinaturas de JWT. **p. 44**

JWT (JSON Web Token). Token autocontido (RFC 7519) com cabeçalho, carga e assinatura em base64url; vale até a expiração, salvo denylist. **p. 44**

K

k-anonimato. Propriedade em que cada registro é indistinguível de pelo menos k − 1 outros nos quase-identificadores, obtida por generalização e supressão. **p. 59**

k-anonimato (Pwned Passwords). Técnica da API do HIBP: o cliente envia só 5 caracteres do hash SHA-1 e compara o restante localmente, sem expor a senha. **p. 44**

Kali Linux. Distribuição Linux com ferramentas de teste de segurança pré-instaladas; deve rodar em máquina virtual isolada. **p. 166**

KEM (Key Encapsulation Mechanism). Primitiva que gera e transporta uma chave simétrica com chave pública; base da cifragem híbrida (HPKE, RFC 9180) e do ML-KEM. **p. 34**

Kerberoasting. Ataque que pede tickets de serviço Kerberos e tenta descobrir offline a senha da conta de serviço associada. **p. 11**

KEV (Known Exploited Vulnerabilities). Catálogo da CISA com vulnerabilidades que já estão sendo exploradas ativamente no mundo real. **p. 20**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Keychain e Android Keystore. Armazenamentos do iOS e do Android para segredos e chaves criptográficas, com proteção do sistema e, quando disponível, de hardware seguro. **p. 112**

KMS (Key Management Service). Serviço que gera, guarda, usa e audita chaves por API, aplicando controle de acesso e registrando cada uso. **p. 34**

KPI. Key Performance Indicator: indicador de desempenho de um controle ou processo. **p. 27**

KRACK. Ataque de 2017 (Vanhoef e Piessens) ao handshake de quatro vias do WPA2, que permite reinstalar chaves; corrigido por atualizações. **p. 67**

KRI. Key Risk Indicator: indicador que sinaliza aumento da exposição ao risco. **p. 27**

L

l-diversidade e t-closeness. Refinamentos do k-anonimato: exigem variedade de valores sensíveis em cada grupo (l-diversidade) e distribuição próxima à da base inteira (t-closeness). **p. 59**

Laboratório isolado. Ambiente de prática em rede interna, sem rota para redes reais, com alvos vulneráveis e snapshots, para não causar dano a terceiros. **p. 166**

Legítimo interesse. Base legal que admite tratamento para finalidades legítimas do controlador ou de terceiros, após um teste de balanceamento que preserve os direitos do titular. **p. 59**

LGPD. Lei Geral de Proteção de Dados (Lei nº 13.709/2018): regula o tratamento de dados pessoais no Brasil. **p. 27**

Limpeza (cleanup). Remoção, ao fim do teste, de contas, arquivos e artefatos criados, com entrega da lista do que foi feito para distinguir o rastro do teste. **p. 166**

LINDDUN. Metodologia de modelagem de ameaças à privacidade: Linking, Identifying, Non-repudiation, Detecting, Data disclosure, Unawareness e Non-compliance. **p. 20**

Linguagem memory-safe. Linguagem cujas regras impedem erros espaciais e temporais de memória por construção (Rust, Go, Java, C#, Swift, Python); a garantia termina em blocos unsafe, FFI e extensões nativas. **p. 122**

Linha de base (baseline). Conjunto de configurações seguras recomendadas e testadas, usado como padrão para todos os sistemas e como régua para medir desvios. **p. 78**

Linha do tempo (timeline) do incidente. Ordenação cronológica, em UTC, dos eventos de várias fontes, usada para reconstruir o ataque, delimitar o escopo e orientar a contenção. **p. 156**

Lista de bloqueio (denylist). Validação que rejeita padrões maliciosos conhecidos; é frágil, porque sempre existe uma variação que escapa. Serve apenas como camada extra, nunca como controle principal. **p. 122**

Lista de permissão (allowlist). Validação que aceita só o que casa com o formato esperado e rejeita o resto por padrão; é preferível à lista de bloqueio (denylist), que tenta barrar padrões ruins conhecidos. **p. 122**

Living off the land (LotL). Técnica de usar ferramentas já presentes no sistema (PowerShell, WMI, certutil) para atacar, o que dificulta distinguir de uso legítimo. **p. 11**

LLM (modelo de linguagem de grande porte). Modelo treinado em grandes volumes de texto que gera texto a partir de um prompt; trata instruções e dados pelo mesmo canal, origem da injeção de prompt. **p. 112**

Lockfile. Arquivo que fixa as versões exatas (e, melhor, os hashes) de toda a árvore de dependências, como package-lock.json, poetry.lock, Cargo.lock e go.sum; torna a instalação reprodutível. **p. 135**

Log estruturado. Registro com campos fixos (como JSON) em vez de texto livre, que pode ser consultado por campo e processado por máquina. **p. 144**

Log injection (CWE-117). Inserção de quebras de linha ou de texto forjado em registros, criando entradas falsas e comprometendo a trilha de auditoria; evitada com log estruturado e neutralização do dado. **p. 122**

M

MAC. Message Authentication Code: etiqueta calculada com chave compartilhada que prova a integridade e a autenticidade de uma mensagem. **p. 34**

Malware. Software malicioso projetado para causar dano, roubar dados ou obter acesso não autorizado: vírus, worms, trojans, ransomware, spyware e outros. **p. 11**

MANRS. Mutually Agreed Norms for Routing Security: iniciativa com boas práticas de segurança de roteamento para operadores de rede. **p. 67**

Marcação de memória (Arm MTE). Memory Tagging Extension: o hardware associa tags de 4 bits a blocos e ponteiros e confere a correspondência a cada acesso, detectando use-after-free e estouros de buffer em tempo de execução. **p. 122**

Marco Civil da Internet. Lei nº 12.965/2014: estabelece princípios, direitos e deveres na internet, incluindo a guarda de registros de conexão e de acesso a aplicações. **p. 27**

Mascaramento de dados (data masking). Ocultar parte de um dado ou trocá-lo por valores fictícios plausíveis em telas, relatórios e ambientes de teste; por exemplo, exibir só os dígitos do meio de um CPF. **p. 59**

Mass assignment. Falha em que o framework grava campos enviados pelo cliente sem lista de permissão, como papel=admin (CWE-915). **p. 53**

Matriz de autorização. Tabela de papéis, rotas e ações com o resultado esperado de cada combinação; base dos testes automatizados de autorização. **p. 166**

Matriz de risco. Grade que cruza probabilidade e impacto (por exemplo 5×5) para classificar riscos em baixo, médio, alto e crítico. **p. 20**

MDM / UEM. Gestão de dispositivos móveis (e de todos os endpoints) para impor políticas, cifragem e atualização e apagar dados remotamente. **p. 78**

MDR (Managed Detection and Response). Serviço gerenciado em que analistas operam a detecção e a resposta do cliente, em geral 24 horas por dia. **p. 78**

MED (Mecanismo Especial de Devolução). Procedimento do Pix para contestar transações por fraude ou falha operacional, com bloqueio e rastreio do valor; não garante o ressarcimento. **p. 180**

— GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Mediação completa. Princípio de Saltzer e Schroeder: todo acesso a todo objeto deve ser verificado quanto à autoridade, sem atalhos nem caches inseguros. **p. 5**

Menor privilégio. Princípio de conceder a cada identidade ou processo apenas as permissões mínimas necessárias, pelo menor tempo necessário. **p. 5**

Metadados da instância (IMDS). Serviço local (169.254.169.254) que entrega dados e credenciais de papel a uma instância; o IMDSv2 exige token obtido por PUT, dificultando a SSRF. **p. 78**

Metasploit. Framework de exploração usado para validar vulnerabilidades com módulos controlados; só com autorização explícita e prova de conceito mínima. **p. 166**

MFA. Multi-Factor Authentication: exige dois ou mais fatores de categorias diferentes (conhecimento, posse, inerência). **p. 44**

MFA fatigue. Ataque que dispara muitas notificações de aprovação de MFA até a vítima aceitar por cansaço ou engano. **p. 11**

Microsoft SDL. Security Development Lifecycle: processo pioneiro da Microsoft, nascido da iniciativa Trustworthy Computing (2002), com práticas de segurança em cada fase do desenvolvimento. **p. 90**

Microssegmentação. Divisão da rede em segmentos pequenos com regras próprias, que limita o movimento lateral de um atacante. **p. 53**

Microtreinamento (microlearning). Treinamento em doses curtas e frequentes, ligado à tarefa real da pessoa; é um formato, não garantia de eficácia: o resultado precisa ser medido pelo comportamento. **p. 180**

MIME sniffing. Comportamento do navegador de adivinhar o tipo de um conteúdo, ignorando o Content-Type declarado; o cabeçalho X-Content-Type-Options: nosniff o desativa. **p. 98**

Minimização de dados. Coletar, usar e guardar apenas o necessário à finalidade; reduz a superfície de ataque e o custo de conformidade (LGPD, art. 6º, III). **p. 59**

MISP. Plataforma de código aberto para coletar, armazenar, correlacionar e compartilhar indicadores de ameaça e o contexto de eventos. **p. 144**

MITM (Man-in-the-Middle). Ataque em que o adversário se posiciona entre duas partes para ler ou alterar a comunicação sem que percebam. **p. 11**

MITRE ATLAS. Base de conhecimento de táticas e técnicas de adversários contra sistemas de IA, no estilo do ATT&CK. **p. 112**

MITRE ATT&CK. Base de conhecimento pública de táticas, técnicas e procedimentos de adversários, observados no mundo real; usada para detecção e planejamento defensivo. **p. 11**

MITRE Caldera. Plataforma de código aberto da MITRE que automatiza a emulação de adversário e cadeias de ataque em testes controlados. **p. 166**

Modbus. Protocolo industrial dos anos 1970, sem autenticação nem cifra: quem alcança a rede lê e escreve nos registradores dos equipamentos. **p. 67**

Modelagem de ameaças. Atividade de identificar, durante o projeto, o que pode dar errado em um sistema e decidir como tratar cada ameaça. **p. 20**

Modelo 4C. Camadas da segurança em nuvem nativa (nuvem, cluster, contêiner e código), em que cada camada depende da segurança da camada externa. **p. 78**

Modelo de Purdue. Modelo de referência que organiza as redes industriais em níveis (0 a 5), usado para posicionar zonas e a DMZ industrial. **p. 67**

Modelo OSI. Modelo de referência de sete camadas (física, enlace, rede, transporte, sessão, apresentação e aplicação) usado para localizar funções, ataques e controles de rede. **p. 67**

Modo de operação. Esquema que estende uma cifra de bloco a mensagens longas (ECB, CBC, CTR, GCM); a escolha define sigilo, integridade e riscos. O ECB vaza padrões e não deve ser usado. **p. 34**

MSSP (Managed Security Service Provider). Provedor que opera serviços de segurança para clientes, como monitoramento 24×7, firewall gerenciado e SIEM como serviço. **p. 144**

MTA-STS. Política (RFC 8461) que obriga o uso de TLS válido na entrega de e-mail entre servidores, evitando downgrade. **p. 67**

MTD ou MAO (Maximum Tolerable Downtime). Tempo máximo de indisponibilidade que o negócio suporta antes de um dano inaceitável; o RTO precisa ser menor que ele. **p. 156**

mTLS (TLS mútuo). Variante do TLS em que cliente e servidor apresentam certificados; autentica serviços entre si e APIs entre empresas. **p. 34**

MTTA (Mean Time To Acknowledge). Tempo médio entre o disparo do alerta e o momento em que alguém o assume. **p. 144**

MTTC (Mean Time To Contain). Tempo médio entre a detecção e a contenção do incidente, quando o adversário perde a capacidade de avançar. **p. 144**

MTTD (Mean Time To Detect). Tempo médio entre o início de uma atividade maliciosa e a sua detecção; a mediana costuma ser mais representativa. **p. 144**

MTTR. Mean Time To Remediate/Respond: tempo médio para corrigir uma vulnerabilidade ou responder a um incidente. **p. 20**

Multi-tenancy (multi-inquilino). Vários clientes compartilham a mesma aplicação; exige isolamento por tenant_id, RLS ou esquemas e bancos separados. **p. 53**

N

Namespaces e cgroups. Recursos do kernel que isolam o que o processo enxerga (namespaces) e limitam o que ele consome (cgroups); base dos contêineres. **p. 78**

Não repúdio. Impossibilidade de uma parte negar ter praticado uma ação, em geral por assinaturas digitais e registros íntegros. **p. 5**

NDR (Network Detection and Response). Detecção e resposta baseadas em tráfego e metadados de rede, útil para ver movimento lateral entre servidores e comunicação com C2. **p. 144**

Need-to-know. Princípio de dar acesso à informação somente a quem precisa dela para executar a tarefa. **p. 53**

Negar por padrão (fail-safe defaults). Sem regra explícita de permissão, o acesso é negado; falhas e exceções devem fechar o acesso (fail closed), nunca abri-lo. **p. 53**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

NetFlow e IPFIX. Metadados de fluxos de rede (origem, destino, portas, volume, tempo), úteis para detectar varreduras e exfiltração sem guardar o conteúdo. **p. 67**

NetworkPolicy. Objeto do Kubernetes que controla o tráfego entre pods; sem política, todos os pods se comunicam, então se começa com default deny. **p. 78**

NGFW (firewall de nova geração). Firewall stateful acrescido de identificação de aplicações e usuários, IPS, inteligência de ameaças e inspeção de TLS. **p. 67**

NIS2. Diretiva (UE) 2022/2555 sobre segurança de redes e sistemas de entidades essenciais e importantes, com notificação de incidentes significativos. **p. 27**

NIST AI RMF. Framework de gestão de risco de IA do NIST (1.0, de 2023), em quatro funções: Govern, Map, Measure e Manage; o perfil AI 600-1 trata de IA generativa. **p. 112**

NIST CSF 2.0. Cybersecurity Framework do NIST (2024): resultados organizados em seis funções (Govern, Identify, Protect, Detect, Respond, Recover). **p. 27**

NIST SP 800-115. Guia técnico do NIST para testes e avaliações de segurança: planejamento, descoberta, ataque e relato. **p. 166**

NIST SP 800-171. Requisitos para proteger informações não classificadas controladas (CUI) em sistemas de organizações não federais. **p. 27**

NIST SP 800-53. Catálogo detalhado de controles de segurança e privacidade do NIST, organizado em 20 famílias; base das autorizações do governo dos EUA. **p. 27**

NIST SP 800-61 Rev. 3. Guia do NIST (abril de 2025) que integra a resposta a incidentes à gestão de risco do CSF 2.0 e substitui a Rev. 2 (2012), de quatro fases. **p. 156**

NIST SSDF (SP 800-218). Secure Software Development Framework: práticas de alto nível em quatro grupos (PO, PS, PW e RV), usadas em contratos e atestados de desenvolvimento seguro. **p. 90**

Níveis do ASVS (L1, L2, L3). L1 é a defesa mínima para toda aplicação; L2 o padrão para as que tratam dados relevantes; L3 é para aplicações críticas, com defesa em profundidade. **p. 90**

Nível (CSF Tier). Grau de rigor e integração da gestão de riscos de cibersegurança: parcial, ciente de risco, repetível ou adaptativo. **p. 27**

Nmap. Ferramenta de varredura de redes que descobre hosts, portas e serviços; cada varredura deixa rastro nos logs do defensor. **p. 166**

Nonce. Número usado uma única vez com a mesma chave; não é segredo, mas repeti-lo quebra GCM, CTR e ChaCha20. **p. 34**

Norma (padrão interno). Documento obrigatório com requisitos objetivos e verificáveis que detalham a política, como regras de senha ou de classificação. **p. 27**

Normalização de logs. Conversão de eventos de formatos diferentes para um esquema comum de campos, requisito para correlacionar fontes e escrever regras reutilizáveis. **p. 144**

NTP (Network Time Protocol). Protocolo que sincroniza relógios de máquinas; sem hora confiável, eventos de fontes diferentes não se ordenam e a linha do tempo se perde. **p. 144**

NVD (National Vulnerability Database). Base de dados de vulnerabilidades do NIST que enriquece CVEs com pontuação CVSS, CWE e produtos afetados. **p. 20**

O

OAuth 2.0. Protocolo de delegação de acesso (RFC 6749): um app acessa uma API em nome do usuário, sem conhecer a senha. **p. 44**

OCSF (Open Cybersecurity Schema Framework). Esquema aberto e independente de fornecedor para eventos de segurança, que visa reduzir o custo de integrar ferramentas e fontes de dados. **p. 144**

OIDC federado (workload identity). Mecanismo em que o pipeline prova sua identidade com um token OIDC e a nuvem entrega credenciais temporárias, dispensando chaves de longa duração guardadas como segredo. **p. 135**

OPA e Rego. Open Policy Agent: motor de políticas como código (projeto graduado da CNCF), com a linguagem declarativa Rego. **p. 53**

OpenID Connect (OIDC). Camada de identidade sobre o OAuth 2.0, com ID token (JWT), UserInfo e descoberta em /.well-known/openid-configuration. **p. 44**

OpenSSF Scorecard. Ferramenta que pontua projetos de código aberto em verificações automáticas, como Maintained, Code-Review, Branch-Protection, Pinned-Dependencies, Signed-Releases e Token-Permissions. **p. 135**

OpenTelemetry. Padrão aberto de instrumentação (projeto da CNCF) para gerar e exportar logs, métricas e traços de aplicações de forma uniforme. **p. 144**

Operação atômica. Operação que se completa inteira ou não ocorre, sem estados intermediários visíveis a outras requisições, como um UPDATE condicional que decide e grava em uma só instrução. **p. 122**

Operador. Agente de tratamento que realiza o tratamento de dados pessoais em nome do controlador (LGPD). **p. 27**

Ordem de volatilidade. Princípio da RFC 3227: coletar primeiro as evidências que se perdem mais rápido (memória, conexões, processos), depois disco, logs remotos e mídias de arquivo. **p. 156**

Organismo de certificação. Entidade independente e acreditada que audita uma organização e emite o certificado de conformidade com uma norma. **p. 27**

Origem e site. Origem é o trio esquema, host e porta; site é o domínio registrável (por exemplo, exemplo.com.br). Subdomínios são origens diferentes dentro do mesmo site. **p. 98**

OSCP. Certificação prática de teste de intrusão da OffSec, com exame longo em laboratório e entrega de relatório. **p. 166**

OSSTMM. Open Source Security Testing Methodology Manual (ISECOM): metodologia aberta de testes operacionais, com métricas de superfície de ataque. **p. 166**

OT (tecnologia operacional). Sistemas que monitoram e controlam processos físicos (CLPs, SCADA); priorizam disponibilidade e segurança física sobre a confidencialidade. **p. 67**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

OWASP ASVS. Application Security Verification Standard: catálogo de requisitos verificáveis de segurança de aplicações (versão 5.0.0, de 2025, com 17 capítulos) usado em revisões, testes e contratos. **p. 90**

OWASP MASVS e MASTG. MASVS: padrão de verificação de segurança de apps móveis, em oito grupos (STORAGE, CRYPTO, AUTH, NETWORK, PLATFORM, CODE, RESILIENCE e PRIVACY). MASTG: o guia de testes correspondente. **p. 112**

OWASP SAMM. Software Assurance Maturity Model: modelo prescritivo de maturidade de segurança de software, com 5 funções de negócio e 15 práticas. **p. 27**

OWASP Top 10. Lista de conscientização do OWASP com as dez categorias de risco mais críticas de aplicações web (edição 2025, de A01 a A10); cada categoria agrupa várias CWEs. **p. 98**

OWASP Top 10 para LLM. Lista do OWASP GenAI Security Project (edição 2025) com os dez maiores riscos de aplicações com LLM, de LLM01 (injeção de prompt) a LLM10 (consumo ilimitado). **p. 112**

OWASP WSTG. Web Security Testing Guide: metodologia aberta de testes de segurança de aplicações web, organizada em categorias com objetivo, procedimento e correção. **p. 98**

P

Paciente zero (patient zero). Primeiro sistema ou conta comprometido em um incidente; encontrá-lo ajuda a descobrir a porta de entrada e o escopo. **p. 156**

Padding oracle. Ataque que usa respostas de erro sobre o preenchimento de um texto cifrado em CBC para decifrá-lo sem a chave. **p. 34**

Padrões seguros (secure defaults). Configuração inicial segura: o caminho mais fácil é o seguro, com recursos perigosos desativados e privilégios mínimos. **p. 90**

Palavra-código. Combinação secreta, definida antes e pessoalmente, que familiares ou equipes usam para confirmar a identidade em pedidos urgentes de dinheiro ou dados; protege contra voz clonada e deepfake. **p. 180**

PAM (Privileged Access Management). Gestão de acesso privilegiado: contas separadas, acesso just-in-time, cofre de credenciais e gravação de sessões. **p. 44**

Parse, don't validate. Princípio de converter a entrada em um tipo que só admite valores válidos, de modo que o restante do código trabalhe com o tipo, e não com texto solto. **p. 122**

Pass-the-hash. Técnica em que o hash de uma credencial (por exemplo NTLM) roubado é usado para autenticar sem conhecer a senha. **p. 11**

Passkey. Credencial FIDO descobrível que substitui a senha; pode ser sincronizada pela plataforma ou presa ao dispositivo. **p. 44**

Password spraying. Tentar poucas senhas comuns contra muitas contas, para evitar o bloqueio por tentativas excessivas em uma só conta. **p. 11**

PASTA. Process for Attack Simulation and Threat Analysis: metodologia de modelagem de ameaças em sete etapas, orientada ao risco de negócio. **p. 20**

Path traversal. Falha em que sequências como ../ em um nome de arquivo fazem o caminho sair da pasta permitida e acessar arquivos arbitrários (CWE-22). **p. 98**

PCI DSS. Payment Card Industry Data Security Standard: 12 requisitos para proteger dados de cartões de pagamento; versão vigente 4.0.1. **p. 27**

PDCA. Plan-Do-Check-Act: ciclo de melhoria contínua (planejar, fazer, verificar, agir) que estrutura os sistemas de gestão ISO. **p. 27**

PEP, PDP, PIP e PAP. Pontos de aplicação, decisão, informação e administração de políticas: arquitetura que separa a decisão da aplicação. **p. 53**

Pepper. Segredo comum a todas as senhas, guardado fora do banco (KMS ou HSM); inutiliza o ataque offline sobre um dump do banco. **p. 44**

Perfil (CSF Profile). Descrição da situação atual e da situação-alvo de uma organização segundo o CSF, com o plano para fechar a diferença. **p. 27**

Permissions-Policy. Cabeçalho que liga ou desliga recursos do navegador (câmera, microfone, geolocalização) para a página e para os quadros embutidos. **p. 98**

Persistência (persistence). Tática do ATT&CK: técnicas que mantêm o acesso após reinícios ou troca de senha, como tarefas agendadas, serviços, contas novas, chaves SSH e web shells. **p. 156**

Phishing. Fraude por mensagem (e-mail, site, SMS) que se passa por entidade confiável para induzir a pessoa a revelar dados ou executar algo. **p. 11**

PICERL. Modelo do SANS em seis etapas: Preparation, Identification, Containment, Eradication, Recovery e Lessons learned. **p. 156**

Pilha sombra (shadow stack). Cópia protegida dos endereços de retorno, mantida pelo hardware (Intel CET) e comparada antes de retornar; as CPUs Arm usam autenticação de ponteiros (PAC). **p. 122**

Pilot light (chama-piloto). Estratégia de DR com dados replicados e o mínimo da aplicação pronto, que é ampliado quando o plano é acionado; custo baixo e recuperação em dezenas de minutos. **p. 156**

PIN do SIM. Código que bloqueia o uso do chip em outro aparelho, útil em caso de roubo; não impede a troca fraudulenta de chip feita na operadora (SIM swap), por isso evite SMS como segundo fator. **p. 180**

PITR (restauração pontual). Point-in-time recovery: restaura o banco para um instante escolhido, combinando o backup completo com o log de transações; reduz o RPO a minutos. **p. 189**

Pivotar (investigação). Partir de um achado, como um IP, uma conta ou um hash, para procurar eventos relacionados em outras fontes e ampliar o quadro. **p. 144**

PKCE. Proof Key for Code Exchange (RFC 7636): vincula o authorization code ao cliente com code_verifier e code_challenge, impedindo seu uso se interceptado. **p. 44**

PKI (Public Key Infrastructure). Conjunto de autoridades certificadoras, certificados, políticas e processos que vincula chaves públicas a identidades. **p. 34**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Plano de controle e plano de dados. O plano de dados processa as requisições das pessoas usuárias; o de controle gerencia e configura o sistema (pipeline, identidades, alertas). Comprometê-lo costuma ser mais grave. **p. 189**

Plano de resposta a incidentes (IRP). Documento aprovado pela liderança que define o que é incidente, severidades, papéis, canais, procedimentos e obrigações de comunicação, e que é ensaiado e revisto. **p. 156**

PMF (802.11w). Protected Management Frames: proteção dos quadros de gerência do Wi-Fi contra deautenticação forjada. **p. 67**

Pod Security Standards. Três perfis de segurança de pods (privileged, baseline, restricted) aplicados pelo Pod Security Admission por rótulos de namespace. **p. 78**

Política de segurança da informação. Documento aprovado pela direção que declara princípios, papéis e regras de alto nível; é curto, estável e obrigatório. **p. 27**

Política de uso aceitável (AUP). Regras curtas sobre como equipamentos, contas e dados da organização podem ser usados (inclusive dispositivos pessoais, redes e IA); precisa ser clara, aplicável e conhecida desde a entrada. **p. 180**

Políticas como código. Regras de segurança escritas em linguagem executável e versionadas (OPA/Rego, Conftest, Kyverno), aplicadas no CI e na admissão do cluster. **p. 135**

Ponteiro pendente (dangling pointer). Ponteiro que guarda o endereço de memória já liberada ou de variável fora de escopo; é a origem do use-after-free. **p. 122**

Porto seguro (safe harbor). Compromisso da organização de não tomar medidas legais contra quem seguir a política de boa-fé; não vincula as autoridades. **p. 166**

Pós-exploração. Fase em que se mede o alcance de um acesso obtido (outros sistemas, dados, privilégios) para traduzir a falha em impacto no negócio. **p. 166**

Posse (ownership) em Rust. Cada valor tem um único dono, e as referências emprestadas têm tempo de vida e exclusividade verificados na compilação, o que impede ponteiros pendentes e corridas de dados. **p. 122**

Precisão e sensibilidade (recall). Precisão: fração dos alertas que eram reais, VP/(VP+FP). Sensibilidade: fração das ameaças que geraram alerta, VP/(VP+FN). **p. 144**

Prefixo __Host-. Prefixo de cookie que exige Secure, Path=/ e ausência de Domain, prendendo o cookie ao host exato. **p. 44**

Prefixos de cookie (__Host- e __Secure-). Convenção de nomes que o navegador faz cumprir: __Secure- exige Secure; __Host- exige também Path=/ e nenhum Domain, prendendo o cookie ao host exato. **p. 98**

Preflight. Requisição OPTIONS enviada pelo navegador antes de uma chamada entre origens "não simples", para perguntar ao servidor se o método e os cabeçalhos são permitidos. **p. 98**

Preservação legal (legal hold). Ordem de guardar dados e registros potencialmente relevantes para um litígio ou investigação, suspendendo o descarte rotineiro. **p. 156**

Pretexting. Criação de um cenário falso e plausível para convencer alguém a entregar informações ou acessos. **p. 11**

Princípio de Kerckhoffs. A segurança de um sistema criptográfico deve depender apenas do segredo da chave, não do sigilo do algoritmo. **p. 5**

Princípios de Cialdini. Seis alavancas de persuasão: reciprocidade, compromisso e coerência, prova social, autoridade, afinidade e escassez. **p. 11**

Privacidade. Uso adequado de dados pessoais conforme finalidade e consentimento ou base legal; depende da segurança, mas é um objetivo distinto. **p. 5**

Privacidade diferencial. Garantia matemática em que ruído calibrado nos resultados de consultas faz a presença ou a ausência de qualquer pessoa alterar pouco a saída. **p. 59**

Privacy by design. Projetar a privacidade desde a concepção, de forma proativa, como padrão e de ponta a ponta; sete princípios de Ann Cavoukian, refletidos no art. 46, §2º, da LGPD. **p. 59**

Privilege creep (acúmulo de privilégios). Acúmulo gradual de permissões quando a pessoa muda de função e mantém os acessos antigos; corrige-se removendo o acesso na mudança e revisando-o periodicamente. **p. 180**

Procedimento. Descrição passo a passo de como executar uma atividade de segurança de forma consistente. **p. 27**

Programa de conscientização (security awareness). Conjunto contínuo de comunicação, treinamento e simulações para mudar comportamentos de risco; deve ser medido por comportamento (reporte, tempo de resposta), não por presença em aula. **p. 180**

Project Zero. Equipe de pesquisa do Google dedicada a achar vulnerabilidades; popularizou a convenção de 90 dias entre o relato e a divulgação. **p. 166**

Prompt de sistema. Instruções fixas dadas pelo desenvolvedor ao modelo (papel, regras, formato); deve-se assumir que pode vazar, e nele não entram segredos (LLM07). **p. 112**

Prova de conceito (PoC). Demonstração mínima de que uma falha é real e explorável, sem causar dano nem acessar mais dados do que o necessário. **p. 166**

Proveniência (provenance). Declaração assinada sobre como um artefato foi produzido (repositório e commit, builder e parâmetros), que permite verificar que ele vem do código e do pipeline esperados. **p. 135**

Proxy de interceptação (ZAP e Burp Suite). Ferramenta posicionada entre o navegador e o servidor para inspecionar e alterar requisições e testar autorização, entrada e sessão; use só com autorização. **p. 98**

Pseudonimização. Tratamento em que o dado só se associa a uma pessoa por meio de informação adicional mantida à parte (LGPD, art. 13, §4º); continua sendo dado pessoal. **p. 59**

PTES. Penetration Testing Execution Standard: metodologia de testes de intrusão em sete fases, do pré-engajamento ao relatório. **p. 166**

— GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

pull_request_target. Gatilho do GitHub Actions que roda no contexto do repositório base, com acesso a segredos; é perigoso quando o workflow faz checkout e executa o código de um PR externo. **p. 135**

PURL (Package URL). Esquema padronizado de identificação de pacotes entre ecossistemas, como pkg:npm/lodash@4.17.21; serve de chave para cruzar um SBOM com bases de vulnerabilidades. **p. 135**

Push protection. Recurso do GitHub que bloqueia o envio (push) de código que contém um segredo reconhecido, antes que ele entre no histórico do repositório. **p. 135**

Pyramid of Pain. Modelo de David Bianco (2013) que ordena indicadores (hash, IP, domínio, artefatos, ferramentas, TTPs) pela dificuldade de o adversário trocá-los. **p. 11**

Q

Quadro de pilha (stack frame). Região da pilha reservada a uma chamada de função, com argumentos, endereço de retorno, frame pointer salvo e variáveis locais. **p. 122**

Quality gate. Critério automático que interrompe o pipeline quando não é atendido (por exemplo, achados críticos novos); precisa ser rápido, confiável e explicável para não ser contornado. **p. 135**

Quarentena de rede. Segmento isolado (VLAN ou regra de EDR) para onde vai um dispositivo suspeito, com acesso limitado ao necessário para a análise e a correção. **p. 156**

Quase-identificador. Atributo que sozinho não identifica, mas combinado com outros (CEP, data de nascimento, sexo) permite reidentificar uma pessoa. **p. 59**

Quase-incidente (near miss). Evento que poderia ter virado incidente, mas foi barrado a tempo; deve ser registrado, porque revela falhas de controle sem ter causado dano. **p. 156**

Quishing. Phishing por QR Code malicioso, que leva o usuário a um site fraudulento ou a baixar um aplicativo malicioso. **p. 11**

R

RaaS (Ransomware-as-a-Service). Modelo em que desenvolvedores alugam o ransomware e a infraestrutura a afiliados, dividindo o resgate obtido. **p. 11**

RACI. Matriz de responsabilidades: R (faz), A (aprova, um só responsável), C (consultado) e I (informado); evita atividades sem dono ou com donos demais. **p. 189**

RADIUS. Protocolo de autenticação, autorização e contabilização usado por Wi-Fi corporativo, VPN e comutadores para validar credenciais em um servidor central. **p. 67**

RAG (retrieval-augmented generation). Técnica que busca documentos em uma base, em geral vetorial, e os entrega ao modelo como contexto; exige controle de acesso na recuperação e proteção da base contra envenenamento. **p. 112**

RAII. Resource Acquisition Is Initialization: em C++, o recurso (memória, arquivo, trava) é adquirido na criação do objeto e liberado na destruição, o que liga o tempo de vida ao escopo. **p. 122**

Rainbow table. Tabela pré-calculada de hashes de senhas comuns; derrotada pelo uso de um sal único por senha. **p. 44**

Ransomware. Malware que cifra dados (e, cada vez mais, os exfiltra) e exige pagamento para restaurar o acesso ou evitar a divulgação. **p. 11**

RASP. Runtime Application Self-Protection: proteção que roda dentro da aplicação e bloqueia ataques em tempo de execução; é camada extra e não substitui a correção do defeito. **p. 135**

Rastreabilidade de requisitos. Ligação verificável entre requisito, ameaça, controle, teste e evidência; mostra se cada requisito tem prova e por que cada teste existe. **p. 189**

Rate limiting (limite de taxa). Controle que restringe quantas requisições uma chave (usuário, token, IP) pode fazer em um período; ao estourar, a API responde 429 com o cabeçalho Retry-After. **p. 112**

RBAC. Role-Based Access Control: permissões ligadas a papéis atribuídos a usuários; modelo NIST e ANSI INCITS 359, com hierarquia e SoD. **p. 53**

RBAC do Kubernetes. Controle de acesso por papéis na API do cluster (Role, ClusterRole e bindings); deve evitar cluster-admin e curingas. **p. 78**

Reautenticação (step-up). Pedido de nova prova de identidade, mais forte ou mais recente, antes de uma operação sensível (alterar nota, exportar dados), mesmo com a sessão já ativa. **p. 189**

ReBAC. Relationship-Based Access Control: decide por relações em um grafo (dono, membro, editor); modelo Zanzibar, do Google. **p. 53**

Recertificação de acessos. Revisão periódica em que donos de recurso confirmam quem deve manter cada acesso; o restante é removido. **p. 53**

Reconstrução (reimaging). Reinstalar o sistema a partir de uma imagem confiável, em vez de limpá-lo, porque não se garante remover toda a persistência de um host comprometido. **p. 156**

Recuperação (de incidente). Fase que restaura os serviços com segurança: backup limpo e validado, volta em etapas, monitoramento reforçado e confirmação de que a causa foi corrigida. **p. 156**

Recuperação de desastres (DR). Conjunto de técnicas e planos de TI (backups, réplicas, sites alternativos) que cumprem o RTO e o RPO após um desastre ou incidente grave. **p. 156**

Red teaming de IA. Teste adversarial de sistemas de IA, por pessoas ou ferramentas (como garak e PyRIT), com casos de injeção, vazamento e abuso, repetido a cada mudança de modelo ou de prompt. **p. 112**

Redirecionamento aberto (open redirect). Falha em que a aplicação redireciona para um destino informado pelo usuário sem validá-lo, permitindo phishing com o domínio confiável (CWE-601). **p. 98**

ReDoS (CWE-1333). Regular expression Denial of Service: expressão regular com backtracking exponencial ou polinomial em certas entradas, que prende a CPU. Evita-se com padrões sem ambiguidade, limite de tamanho e motores de tempo linear. **p. 122**

Redução da superfície de ataque. Eliminar funções, portas, dependências e dados desnecessários para diminuir o que pode ser atacado, corrigido e protegido. **p. 90**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Referrer-Policy. Cabeçalho que controla quanto da URL de origem o navegador envia no cabeçalho Referer ao navegar para outro site ou carregar recursos. **p. 98**

Registrato. Serviço do Banco Central para consultar, em seu CPF, relacionamentos com instituições financeiras, chaves Pix e outras informações; ajuda a descobrir contas ou produtos abertos em fraude. **p. 180**

Registro de riscos. Documento vivo que lista cada risco com ativo, ameaça, nível, tratamento, responsável, prazo e status. **p. 20**

Regra 3-2-1-1-0. Estratégia de backup: 3 cópias, em 2 mídias, 1 fora do local, 1 offline ou imutável e 0 erros na verificação de restauração. **p. 11**

Regras ASR (Attack Surface Reduction). Regras do Microsoft Defender que bloqueiam comportamentos de risco, como o Office criando processos filhos ou execução de scripts ofuscados. **p. 78**

Regras de engajamento (RoE). Documento que fixa escopo, janelas, técnicas permitidas e proibidas, contatos de emergência, critérios de parada e tratamento de dados de um teste. **p. 166**

Reidentificação. Ataque que associa registros supostamente anônimos a pessoas reais, cruzando-os com outras bases; ocorreu nos casos da AOL (2006) e do Netflix Prize. **p. 59**

RELRO e PIE. RELRO torna somente leitura as tabelas de ligação dinâmica (GOT) após o carregamento; PIE gera executáveis posicionáveis em qualquer endereço, o que habilita o ASLR do próprio programa. **p. 122**

Relying party (RP). Parte confiante: aplicação que aceita a autenticação feita por um IdP ou por um autenticador FIDO. **p. 44**

Replay (repetição de mensagem). Reenvio de uma mensagem legítima capturada para repetir seu efeito; combate-se com timestamp, nonce, janela de tolerância e idempotência. **p. 112**

Replicação. Cópia contínua dos dados para outro nó ou local, voltada à disponibilidade; replica também erros, exclusões e a cifragem do ransomware, e por isso não substitui o backup. **p. 59**

Repositório proxy interno. Espelho interno de registros públicos (Artifactory, Nexus, Verdaccio) por onde passam as dependências, com lista de permissão, varredura e quarentena de versões recém-publicadas. **p. 135**

Repudiation. Repúdio: negar ter praticado uma ação quando não há registro confiável que a comprove. **p. 20**

Requisito de segurança. Condição verificável que o sistema deve cumprir para resistir a abusos; precisa poder ser provada por um teste. **p. 90**

Requisitos como código. Registro dos requisitos de segurança em arquivo versionado e legível por máquina, ligado a testes, para que o CI verifique a cobertura a cada mudança. **p. 189**

Resolução CD/ANPD nº 15/2024. Regulamento de comunicação de incidente de segurança: prazo de 3 dias úteis e registro por, no mínimo, 5 anos. **p. 27**

Responsabilidade compartilhada. Divisão entre provedor de nuvem (segurança da nuvem) e cliente (segurança na nuvem: dados, identidades, configuração), que varia com IaaS, PaaS e SaaS. **p. 78**

Responsabilização (accountability). Capacidade de atribuir ações a uma identidade única, por meio de registros confiáveis, para auditoria e investigação. **p. 5**

REST, GraphQL e gRPC. Três estilos de API: REST (recursos e verbos HTTP, em JSON), GraphQL (consulta flexível sobre um único endpoint) e gRPC (chamadas binárias tipadas sobre HTTP/2, comuns entre serviços). **p. 112**

Resumo executivo. Parte inicial do relatório, em linguagem de negócio: o que foi testado, risco geral, principais achados e o que fazer primeiro. **p. 166**

Retenção de dados. Política que define por quanto tempo cada tipo de dado é mantido e quando deve ser excluído, conforme a finalidade e as obrigações legais. **p. 59**

Reteste. Nova verificação, depois da correção, de que o achado foi mesmo resolvido: corrigido, parcialmente corrigido ou persistente. **p. 166**

Reversão (rollback). Retorno rápido a uma versão anterior sabidamente boa quando uma liberação falha; precisa ser automatizado e testado, não improvisado. **p. 189**

Revisão de código por pares. Prática em que outra pessoa examina o código antes da integração, com roteiro de segurança (autorização, validação, segredos, erros). **p. 90**

Revisão de projeto (design review). Análise do desenho de um sistema, com modelo de ameaças e checklists, para achar falhas de arquitetura antes de implementá-las. **p. 90**

Revisão pós-incidente (postmortem). Análise feita em dias, com quem participou, que registra a linha do tempo, os fatores contribuintes, o que funcionou e as ações corretivas com dono e prazo. **p. 156**

RIPD (relatório de impacto). Documento que descreve os tratamentos de dados que podem gerar riscos às liberdades e aos direitos do titular e as medidas de mitigação (LGPD, art. 38). **p. 59**

Risco. Combinação da probabilidade de uma ameaça explorar uma vulnerabilidade e do impacto resultante sobre um ativo. **p. 5**

Risco de terceiros (TPRM). Gestão do risco criado por fornecedores e parceiros com acesso a dados ou sistemas: avaliar antes de contratar, limitar o acesso, exigir cláusulas de segurança e notificação, e revisar de tempos em tempos. **p. 180**

Risco inerente. Nível de risco antes da aplicação de qualquer controle. **p. 20**

Risco residual. Risco que permanece depois de aplicados os controles; deve ficar dentro da tolerância da organização. **p. 20**

Root e jailbreak. Remoção das restrições do sistema no Android (root) e no iOS (jailbreak), que dá acesso privilegiado ao aparelho e facilita a análise e a adulteração de apps. **p. 112**

Rootkit. Conjunto de técnicas que oculta a presença de um invasor no sistema, em geral atuando no kernel; o bootkit age na inicialização. **p. 11**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

ROP (return-oriented programming). Técnica de exploração que reaproveita trechos de código já existentes no programa, encadeados por endereços de retorno, para contornar o DEP e o NX; pilhas sombra e CFI a dificultam. **p. 122**

RoPA (registro das operações de tratamento). Registro que controlador e operador devem manter das operações de tratamento de dados pessoais (LGPD, art. 37). **p. 59**

ROSI. Return on Security Investment: retorno de um controle, calculado como (redução do ALE menos o custo) dividido pelo custo. **p. 20**

Roteiro de segurança de memória. Plano documentado de um fabricante para reduzir erros de memória (código novo em linguagem segura, migração gradual, endurecimento do legado); CISA e FBI pediram sua publicação até 1º/jan/2026. **p. 122**

Row-level security (RLS). Recurso do banco que filtra linhas por política conforme o usuário ou o contexto; última linha de defesa da autorização. **p. 53**

RPKI e ROV. Infraestrutura de chaves dos recursos da internet; o ROV valida, com ROAs assinados, qual AS pode originar um prefixo e com que tamanho máximo. **p. 67**

RPO (Recovery Point Objective). Quantidade máxima de dados, medida em tempo, que a organização aceita perder entre o último backup e o incidente. **p. 11**

RPZ e sinkholing. Zona de política de resposta do DNS e técnica de resolver domínios maliciosos para um endereço sob controle da organização, bloqueando-os e identificando máquinas infectadas. **p. 67**

RTO (Recovery Time Objective). Tempo máximo tolerável para restabelecer um serviço após uma interrupção. **p. 11**

Runbook e playbook. Roteiros de tratamento: o runbook guia a pessoa, passo a passo, em um tipo de alerta; o playbook costuma ser a versão orquestrada ou automatizada no SOAR. **p. 144**

Runner efêmero. Executor de pipeline criado para um único job e destruído em seguida, sem deixar resíduos de uma execução para a outra. **p. 135**

S

Safe Coding. Estratégia defendida pelo Google (2024): escrever o código novo em linguagens seguras e deixar o legado se estabilizar; no Android, a fatia de vulnerabilidades de memória caiu de 76% (2019) para 24% (2024). **p. 122**

Safetensors. Formato de arquivo de pesos de modelo que guarda apenas tensores e não executa código ao ser carregado, ao contrário do pickle. **p. 112**

Safety (segurança operacional). Evita que falhas de sistemas causem danos físicos a pessoas ou ao ambiente (sinalização, intertravamento); difere de security, que trata de adversários. **p. 5**

Sal (salt). Valor aleatório e único por senha, guardado junto do hash; impede tabelas pré-calculadas e hashes iguais para senhas iguais. **p. 44**

SAML 2.0. Padrão de SSO corporativo com asserções XML assinadas trocadas entre o IdP e o provedor de serviço. **p. 44**

Saneamento de mídia (Clear, Purge, Destroy). Métodos da NIST SP 800-88 para tornar dados irrecuperáveis: sobrescrever (Clear), tornar inviável a recuperação em laboratório (Purge) ou destruir a mídia (Destroy). **p. 59**

Sanitização de HTML. Remoção das partes perigosas de um HTML fornecido pelo usuário por uma biblioteca consolidada, como o DOMPurify, quando não é possível aceitar só texto. **p. 98**

Sanitizer. Instrumentação do compilador que detecta erros em tempo de execução, durante testes: ASan (memória), UBSan (comportamento indefinido), MSan (memória não inicializada) e TSan (corridas de dados). **p. 122**

SASE (Secure Access Service Edge). Serviços de rede (SD-WAN) e de segurança (gateway web, CASB, ZTNA, firewall como serviço) entregues na nuvem. **p. 67**

SAST. Static Application Security Testing: análise do código-fonte, sem executá-lo, para achar padrões inseguros (Semgrep, CodeQL, SonarQube); gera falsos positivos e não vê a execução. **p. 135**

SBOM (Software Bill of Materials). Inventário legível por máquina dos componentes de um software (nome, versão, fornecedor, identificadores, relações, licenças e hashes); responde rápido ao "onde usamos isso?" de uma nova CVE. **p. 135**

SCA (Software Composition Analysis). Análise que compara as dependências de um projeto com bases de vulnerabilidades e de licenças; exemplos: Dependabot, Renovate, OSV-Scanner, npm audit e pip-audit. **p. 135**

SCAP e OpenSCAP. Protocolo do NIST para automatizar a verificação de conformidade de configuração e ferramenta aberta que avalia sistemas contra perfis CIS e STIG. **p. 78**

SCIM. System for Cross-domain Identity Management (RFC 7644): protocolo para provisionar e desprovisionar contas entre sistemas. **p. 44**

SCP (Service Control Policy). Política de organização que define o máximo de permissões possíveis nas contas, mesmo para administradores; barreira contra desligar o registro, por exemplo. **p. 78**

Script injection em workflow. Uso de dados controlados por quem contribui (título de PR, nome de branch) dentro de comandos do pipeline, o que permite executar comandos; evita-se passando o dado por variável de ambiente. **p. 135**

Script kiddie. Atacante de baixa habilidade que usa ferramentas e scripts prontos criados por terceiros, sem compreender a fundo o que executa. **p. 11**

Scrubbing center. Centro de limpeza que recebe o tráfego desviado, descarta o malicioso e devolve o legítimo à origem. **p. 67**

seccomp. Filtro de chamadas de sistema que restringe o que um processo pode pedir ao kernel; o perfil padrão dos contêineres bloqueia chamadas perigosas. **p. 78**

Secret do Kubernetes. Objeto que guarda credenciais para pods; é codificado em base64, não cifrado, e requer cifragem do etcd e RBAC restrito. **p. 78**

Secure Boot. Recurso do UEFI que só inicia gerenciadores de boot e drivers assinados, impedindo a execução de código não confiável na partida. **p. 78**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Secure by design. Segurança incorporada desde o projeto, e não adicionada depois; no fornecedor, assume a responsabilidade por padrões seguros. **p. 5**

Secure by Design e Secure by Default. Princípios da CISA para fabricantes: assumir a responsabilidade pela segurança dos clientes, ser transparente e entregar produtos seguros já na configuração padrão. **p. 90**

Security champion. Pessoa da equipe de desenvolvimento com treinamento extra em segurança, que atua como ponte com a área de segurança e difunde boas práticas. **p. 90**

security.txt (RFC 9116). Arquivo publicado em /.well-known/security.txt que informa contato, política, idiomas e validade para quem quiser relatar uma vulnerabilidade. **p. 90**

Segmentação de rede. Divisão da rede em zonas com regras entre elas, para conter o movimento lateral de um invasor. **p. 67**

Segredo dinâmico. Credencial gerada sob demanda, de vida curta e escopo mínimo (por exemplo, um usuário de banco criado para uma sessão) e revogada ao fim do uso. **p. 135**

Segurança centrada no ser humano. Abordagem que projeta controles a partir de como as pessoas de fato se comportam, de modo que o caminho seguro seja também o mais fácil, em vez de depender de força de vontade. **p. 180**

Segurança da informação (SI). Proteção da informação, em qualquer meio, contra acesso, alteração, destruição ou indisponibilidade indevidos; abrange pessoas, processos e tecnologia. **p. 5**

Segurança de memória (memory safety). Garantia de que um programa só acessa memória que lhe pertence e em estado válido. Violá-la gera comportamento indefinido; C e C++ delegam essa garantia ao programador. **p. 122**

Segurança psicológica. Clima em que a pessoa se sente segura para admitir um erro, perguntar ou reportar um problema sem medo de humilhação ou punição; é pré-condição do reporte rápido de incidentes. **p. 180**

SEI CERT C. Padrão de codificação segura do SEI/Carnegie Mellon, com regras numeradas para C (como STR31-C, MEM30-C e INT32-C) e exemplos conformes e não conformes. **p. 122**

SELinux e AppArmor. Módulos de controle de acesso obrigatório (MAC) do Linux que confinam processos ao que a política permite, mesmo executando como root. **p. 78**

Senha-mestra. Única senha (de preferência uma frase longa) que protege o cofre do gerenciador de senhas; deve ser inédita, memorizada e acompanhada de MFA quando o serviço oferecer. **p. 180**

Separação de funções (SoD). Divisão de tarefas críticas entre pessoas diferentes (quem solicita não aprova), para evitar fraude e erro. **p. 53**

setuid e setgid. Bits de permissão que fazem um programa executar com os privilégios do dono ou do grupo do arquivo; binários vulneráveis viram escalada de privilégio. **p. 78**

Severidade do alerta. Classificação de impacto e urgência (crítica, alta, média, baixa) que define quem é acionado, em quanto tempo e com qual prioridade. **p. 144**

Severidade do incidente (SEV). Classificação de impacto e urgência do incidente (SEV 1 a SEV 4) que define quem é acionado, em quanto tempo e com que nível de decisão. **p. 156**

SGSI (ISMS). Sistema de Gestão de Segurança da Informação: conjunto de políticas, processos e controles gerido de forma contínua, como na ISO/IEC 27001. **p. 27**

Shadow IT. Uso de sistemas e serviços sem conhecimento ou aprovação da área de TI, geralmente para contornar controles incômodos. **p. 5**

Shift-left e shift-right. Shift-left é levar atividades de segurança às fases iniciais, onde corrigir é mais barato; shift-right é investir também em monitoramento e resposta após a liberação. **p. 90**

Shoulder surfing. Espiar a tela ou a digitação de alguém, diretamente ou por câmera, para capturar senhas, PINs ou dados; mitiga-se com filtro de privacidade e atenção ao ambiente. **p. 180**

SIEM (Security Information and Event Management). Plataforma que coleta, normaliza, correlaciona e guarda eventos de várias fontes, gera alertas e permite busca, painéis e relatórios. **p. 144**

Sigilo perfeito de encaminhamento (forward secrecy). Propriedade em que o vazamento da chave de longo prazo não expõe sessões passadas, pois cada sessão usa chaves efêmeras (ECDHE). **p. 34**

Sigma. Formato aberto de regras de detecção em YAML, independente do SIEM; conversores traduzem a mesma regra para a linguagem de cada plataforma. **p. 144**

Sigstore. Projeto de assinatura de software sem chave de longa duração: cosign assina e verifica, a Fulcio emite certificados de vida curta ligados a identidades OIDC e o Rekor registra tudo em um log de transparência. **p. 135**

SIM swap. Fraude que transfere o número de telefone da vítima para um chip do atacante, para interceptar SMS e códigos de verificação. **p. 11**

Simulação de phishing. Campanha interna com iscas falsas e controladas para medir e treinar a resposta das pessoas; exige política prévia e comunicada, resultados agregados e foco em aprender, não em expor. **p. 180**

SLE (Single Loss Expectancy). Perda esperada por ocorrência: valor do ativo multiplicado pelo fator de exposição. **p. 20**

Slopsquatting. Registro, por atacantes, de nomes de pacotes inventados (alucinados) por assistentes de IA; de 5% a 22% dos pacotes sugeridos por LLMs não existem (USENIX Security 2025). **p. 135**

Slowloris. Ataque de camada de aplicação que mantém muitas conexões HTTP abertas e lentas até esgotar a capacidade do servidor. **p. 11**

SLSA. Supply-chain Levels for Software Artifacts: especificação do OpenSSF (versão 1.2) com níveis de garantia sobre a origem e a integridade de artefatos; a trilha de Build vai de L0 a L3. **p. 135**

Sniffing. Captura passiva de tráfego de rede; só expõe conteúdo legível se o tráfego não estiver criptografado. **p. 11**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Snort, Suricata e Zeek. Ferramentas abertas de monitoramento: Snort e Suricata inspecionam por assinaturas e podem atuar como IPS; o Zeek analisa protocolos e gera registros detalhados. **p. 67**

SOAR (Security Orchestration, Automation and Response). Plataforma que automatiza a triagem e a resposta por playbooks integrados às ferramentas, com aprovação humana nas ações de maior impacto. **p. 144**

SOC (Security Operations Center). Função ou equipe que monitora, faz triagem, investiga e responde a eventos de segurança de forma contínua, em turnos definidos. **p. 144**

SOC 2. Relatório de auditoria independente (AICPA) sobre controles de um serviço segundo critérios de confiança; Tipo I (desenho) e Tipo II (eficácia). **p. 27**

Socket do Docker. Arquivo /var/run/docker.sock que dá acesso ao daemon; montá-lo em um contêiner equivale a dar root no host. **p. 78**

SOP (política de mesma origem). Regra do navegador que impede o script de uma origem de ler dados de outra; permite o envio de requisições entre origens, mas não a leitura da resposta. **p. 98**

SPDX e CycloneDX. Os dois principais formatos de SBOM: SPDX (Linux Foundation, ISO/IEC 5962) e CycloneDX (projeto do OWASP, também padronizado pela Ecma). **p. 135**

Spear phishing. Phishing direcionado e personalizado a uma pessoa ou grupo específico, com informações reais do alvo. **p. 11**

SPF (Sender Policy Framework). Registro DNS (RFC 7208) que lista os IPs autorizados a enviar e-mail por um domínio. **p. 67**

SPIFFE. Padrão de identidade de cargas de trabalho: cada serviço recebe um ID spiffe:// e um SVID (X.509 ou JWT) de vida curta. **p. 44**

Split tunneling. Configuração em que só o tráfego da organização entra na VPN e o restante sai direto pela internet; melhora o desempenho e amplia o risco do dispositivo. **p. 67**

Spoofing. Falsificação de identidade: fazer-se passar por outro usuário, serviço ou dispositivo. **p. 20**

SRI (Subresource Integrity). Atributo integrity que informa o hash esperado de um script ou estilo de terceiros; o navegador se recusa a executá-lo se o conteúdo mudar. **p. 98**

SSD e DSD. Restrições de separação de funções do RBAC: estática (papéis conflitantes na mesma pessoa) e dinâmica (na mesma sessão). **p. 53**

SSDLC (ciclo de vida seguro de software). SDLC com atividades, artefatos e responsáveis de segurança em cada fase (requisitos, projeto, implementação, verificação, liberação e operação). **p. 90**

SSRF (server-side request forgery). Falha em que o servidor faz uma requisição a um endereço escolhido ou influenciado por quem ataca, alcançando redes internas e metadados da nuvem (CWE-918). **p. 98**

SSVC. Stakeholder-Specific Vulnerability Categorization: árvore de decisão (CISA e SEI/CMU) que resulta em ações como acompanhar, atender ou agir. **p. 20**

Stalkerware. Aplicativo de monitoramento instalado às escondidas no aparelho de outra pessoa para acompanhar localização, mensagens e mídia; associado a perseguição e violência doméstica. **p. 180**

STIX. Structured Threat Information Expression: linguagem padronizada em JSON (OASIS) para descrever indicadores, adversários, campanhas e suas relações. **p. 144**

STRIDE. Mnemônico de seis ameaças: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service e Elevation of privilege. **p. 20**

sudo. Comando que concede privilégios de root por comando e registra o uso; deve ser limitado a comandos específicos e editado com visudo. **p. 78**

Superfície de ataque. Conjunto de todos os pontos de entrada e saída de dados que um adversário pode tentar usar; reduzi-la diminui as oportunidades de ataque. **p. 5**

SYN flood. DDoS de protocolo que envia muitos pedidos de conexão TCP incompletos para esgotar a tabela de conexões do servidor; mitigado com SYN cookies. **p. 11**

SYN flood e SYN cookies. O SYN flood esgota a tabela de conexões semiabertas; os SYN cookies permitem ao servidor não guardar estado até o ACK final. **p. 67**

Syslog. Protocolo e formato clássicos (RFC 5424) para enviar mensagens de log pela rede a um coletor central, com facilidade e severidade. **p. 144**

T

Tailgating. Entrada física não autorizada seguindo uma pessoa autorizada por uma porta controlada. **p. 11**

Tampering. Adulteração não autorizada de dados, código ou mensagens em repouso ou em trânsito. **p. 20**

Tática, técnica e subtécnica (ATT&CK). Tática é o objetivo do adversário (o porquê); técnica e subtécnica descrevem como ele o alcança (o como). **p. 11**

Taxa de clique e taxa de reporte. Percentuais de pessoas que clicam em uma isca simulada e que a reportam; o reporte e o tempo até reportar dizem mais sobre a defesa do que o clique isolado. **p. 180**

Taxa de escape. Fração das vulnerabilidades encontradas só em produção ou por terceiros, em vez de internamente; mede a eficácia das fases anteriores. **p. 90**

TAXII. Trusted Automated eXchange of Indicator Information: protocolo sobre HTTPS (OASIS) que transporta objetos STIX entre plataformas. **p. 144**

TDE (Transparent Data Encryption). Cifragem dos arquivos de dados e logs do banco, transparente para as consultas; protege contra roubo dos arquivos, não contra SQL injection nem administradores. **p. 59**

Telemetria. Dados de atividade de sistemas (logs, métricas, traços, fluxos de rede, eventos de endpoint) coletados para observar o ambiente e detectar anomalias. **p. 144**

Tempo de permanência (dwell time). Período entre o comprometimento inicial e a detecção ou a contenção; quanto maior, mais dano e mais rastros apagados. **p. 144**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Teste de balanceamento (LIA). Avaliação documentada do legítimo interesse: finalidade, necessidade, impacto sobre o titular e salvaguardas adotadas. **p. 59**

Teste de intrusão (pentest). Teste autorizado que encadeia falhas como faria um atacante, para mostrar o impacto real dentro de um escopo e um prazo definidos; termina em relatório com recomendações e reteste. **p. 166**

Teste de regressão. Teste automatizado que garante que uma falha já corrigida não reapareça em mudanças futuras do código. **p. 166**

Threat hunting (caça a ameaças). Busca proativa, guiada por hipótese, de atividade maliciosa que as detecções automáticas não pegaram; gera incidente, detecção nova ou correção de visibilidade. **p. 144**

Threat Modeling Manifesto. Documento de 2020 que reúne valores e princípios da modelagem de ameaças, como colaboração e refinamento contínuo. **p. 20**

Threat-informed defense. Estratégia de priorizar defesas com base em como os adversários relevantes realmente atuam, em geral usando o ATT&CK. **p. 11**

TIBER-EU. Quadro europeu de red team guiado por inteligência de ameaças para o setor financeiro, do Banco Central Europeu; o CBEST, do Banco da Inglaterra, é equivalente. **p. 166**

Time branco (white team). Célula de controle de um exercício: conhece as regras, coordena as equipes, faz o desconflito e decide quando parar. **p. 166**

Titular. Pessoa natural a quem se referem os dados pessoais tratados. **p. 27**

TLP (Traffic Light Protocol). Protocolo do FIRST com cinco rótulos (RED, AMBER+STRICT, AMBER, GREEN, CLEAR) que informa até onde uma informação pode ser repassada. **p. 144**

TLS 1.3. Versão atual do protocolo de transporte seguro (RFC 8446): handshake de 1 RTT, somente AEAD e sigilo perfeito obrigatório. **p. 34**

TOCTOU (CWE-367). Time-of-check to time-of-use: o recurso muda entre a verificação (existe? há permissão?) e o uso. Evita-se abrindo uma única vez e operando sobre o descritor já aberto. **p. 122**

Token anti-CSRF. Valor imprevisível, atrelado à sessão, enviado em campo oculto ou cabeçalho e conferido no servidor para provar que o pedido nasceu de uma tela legítima do próprio site. **p. 98**

Token bucket. Algoritmo de limite de taxa: um balde recebe fichas em ritmo constante, cada requisição consome uma e o tamanho do balde define a rajada permitida. **p. 112**

Tokenização. Troca do dado sensível por um token sem valor próprio, com o original guardado em um cofre separado; muito usada em cartões de pagamento (PCI DSS). **p. 59**

Tolerância a risco. Desvio máximo aceitável em relação ao apetite; limite a partir do qual é preciso reagir. **p. 20**

TOTP e HOTP. Códigos de uso único por tempo (RFC 6238) ou por contador (RFC 4226), calculados com HMAC a partir de uma semente compartilhada. **p. 44**

TPM (Trusted Platform Module). Chip que guarda chaves e medições de integridade do boot em hardware; base de BitLocker, boot medido e atestação do dispositivo. **p. 78**

Transferência internacional de dados. Envio de dados pessoais a outro país ou organismo internacional; só nas hipóteses dos arts. 33 a 36 da LGPD (adequação, cláusulas-padrão, consentimento específico etc.). **p. 59**

Tratamento do risco. Escolha da resposta a um risco: mitigar, transferir, evitar ou aceitar. **p. 20**

Treinamento por função (role-based). Conteúdo adaptado ao que cada papel enfrenta (financeiro, RH, TI, executivos, desenvolvedores), em vez de um curso igual para todos; aumenta a relevância das situações treinadas. **p. 180**

Triagem (triage). Primeira avaliação de um alerta: confirmar se é real, enriquecer com contexto, definir a severidade e decidir se fecha, escala ou investiga. **p. 144**

Trilha de auditoria (audit trail). Registro cronológico, protegido contra alteração, de quem fez o quê, quando, de onde e com qual resultado, usado em investigações e auditorias. **p. 144**

Trindade letal (lethal trifecta). Combinação de acesso a dados privados, exposição a conteúdo não confiável e capacidade de comunicação externa, que permite o vazamento de dados por agentes; retira-se ao menos uma das três. **p. 112**

Trojan (cavalo de troia). Malware disfarçado de programa legítimo que, uma vez executado pelo usuário, entrega outras funções maliciosas. **p. 11**

Trusted publishing. Publicação em registros de pacotes (como o PyPI) autenticada pela identidade OIDC do pipeline, sem token de longa duração guardado como segredo. **p. 135**

Trusted Types. Recurso do navegador que exige que os valores enviados a sinks perigosos do DOM passem por funções de política, eliminando classes de DOM XSS. **p. 98**

TTP. Táticas, Técnicas e Procedimentos: o modo de operar de um adversário; indicador de topo na Pyramid of Pain. **p. 11**

Tunelamento DNS. Uso de consultas e respostas DNS para esconder dados ou comandos (exfiltração e C2) em redes que só permitem esse tráfego. **p. 67**

Typosquatting. Publicação de pacote com nome quase igual ao de um popular, à espera de quem o instale por um erro de digitação. **p. 135**

U

UEBA (User and Entity Behavior Analytics). Análise que aprende o comportamento normal de contas e máquinas e sinaliza desvios de horário, volume, destino ou localização. **p. 144**

umask. Máscara que define quais permissões são retiradas de arquivos e diretórios novos; 022 gera arquivos 644 e diretórios 755. **p. 78**

Upload irrestrito de arquivos. Falha em que o servidor aceita e publica arquivos perigosos (executáveis, HTML com script) sem validar tipo, conteúdo, nome e tamanho (CWE-434). **p. 98**

Use-after-free (CWE-416). Uso de um ponteiro depois que o bloco foi liberado; o alocador pode ter entregue o bloco a outro objeto, e o acesso lê ou altera dados que não pertencem mais ao ponteiro. **p. 122**

GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

V

VDP e safe harbor. Política de divulgação de vulnerabilidades que define escopo e regras e se compromete a não processar quem age de boa-fé (safe harbor). **p. 90**

Vetor CVSS. Texto que codifica as métricas de uma vulnerabilidade, como CVSS:3.1/AV:N/AC:L/...; cada par métrica:valor responde a uma pergunta. **p. 20**

Vetor de ataque. Caminho usado para alcançar o alvo, como e-mail de phishing, serviço exposto, mídia removível ou fornecedor comprometido. **p. 5**

VEX (Vulnerability Exploitability eXchange). Declaração, complementar ao SBOM, que informa se um produto é afetado por uma CVE: afetado, não afetado (com justificativa), corrigido ou em investigação; reduz o ruído do SCA. **p. 135**

Viagem impossível (impossible travel). Detecção de logins da mesma conta em locais que não podem ser percorridos no intervalo entre eles; sugere credencial roubada. **p. 144**

Violação de dados (data breach). Incidente em que há acesso, divulgação, alteração ou perda confirmados de dados por quem não deveria; pode exigir comunicação à autoridade e aos titulares. **p. 156**

Vishing e smishing. Phishing por ligação de voz (vishing) e por SMS (smishing). **p. 11**

VLAN. Rede local virtual que separa domínios de broadcast no enlace; o tráfego entre VLANs passa por um roteador ou firewall, onde as regras se aplicam. **p. 67**

Volatility. Framework de código aberto para análise forense de memória RAM: lista processos, conexões, módulos carregados e artefatos de código malicioso. **p. 156**

VPN. Túnel cifrado sobre uma rede não confiável, usado para acesso remoto ou para ligar redes (site-to-site). **p. 67**

Vulnerabilidade. Fraqueza em um ativo, processo ou controle que pode ser explorada por uma ameaça: falha de código, configuração incorreta, política ausente. **p. 5**

W

WAF (Web Application Firewall). Firewall de aplicação que filtra HTTP e HTTPS por regras (como o OWASP Core Rule Set), com limite de taxa e controle de bots; não substitui a correção da aplicação. **p. 67**

Warm standby. Estratégia de DR com uma cópia reduzida e funcional da aplicação, sempre ligada, que escala ao acionar o plano; recuperação em minutos. **p. 156**

Web shell. Script malicioso instalado em um servidor web que dá a quem ataca um terminal remoto pelo navegador; costuma resultar de upload perigoso ou de injeção. **p. 98**

Webhook. Chamada que um provedor faz à sua API quando um evento ocorre; deve ser verificada por assinatura HMAC e janela de tempo, pois qualquer pessoa pode chamar o endereço. **p. 112**

Whaling. Phishing contra executivos e cargos de alto valor; muitas vezes associado a fraudes de pagamento (BEC). **p. 11**

Windows LAPS. Recurso que gera e rotaciona senhas únicas do administrador local de cada máquina, guardadas no diretório, evitando movimento lateral por senha repetida. **p. 78**

Wiper. Malware destrutivo que apaga ou corrompe dados de forma irrecuperável, às vezes se passando por ransomware. **p. 11**

WireGuard. VPN moderna e enxuta, com conjunto fixo de criptografia (Noise, Curve25519, ChaCha20-Poly1305, BLAKE2s), integrada ao kernel do Linux desde a versão 5.6. **p. 67**

Worm (verme). Malware que se propaga sozinho pela rede, explorando vulnerabilidades, sem precisar de ação do usuário nem de um arquivo hospedeiro. **p. 11**

WPA3 e SAE. Padrão Wi-Fi cuja autenticação SAE resiste a ataques de dicionário offline e oferece sigilo de encaminhamento; exige quadros de gerência protegidos. **p. 67**

WPS (Wi-Fi Protected Setup). Configuração por PIN de 8 dígitos, vulnerável a força bruta em cerca de 11.000 tentativas; deve ser desativada. **p. 67**

Write-up. Relato escrito de como um desafio ou uma falha foi resolvido: o que foi tentado, o que funcionou e o que se aprendeu. **p. 166**

X

XACML. eXtensible Access Control Markup Language: padrão OASIS de políticas ABAC e da arquitetura PEP, PDP, PIP e PAP. **p. 53**

XDR. Evolução do EDR que correlaciona telemetria de endpoints, e-mail, identidade, rede e nuvem em uma só plataforma de detecção e resposta. **p. 78**

XML signature wrapping. Ataque ao SAML que move o trecho assinado e injeta outro que a aplicação lê, burlando a validação da assinatura. **p. 44**

XSS (cross-site scripting). Falha em que dados não confiáveis são escritos em uma página sem tratamento para o contexto e o navegador os executa como script, com a identidade do site (CWE-79). **p. 98**

XSS refletido, armazenado e baseado em DOM. Refletido: o script vem na URL e volta na resposta. Armazenado: fica guardado no servidor e atinge todos os visitantes. DOM: é montado no navegador pelo JavaScript da própria página. **p. 98**

XXE (entidade externa XML). Falha em analisadores XML configurados para resolver entidades externas, que permite ler arquivos locais e provocar SSRF a partir de um XML enviado (CWE-611). **p. 98**

Y

YARA. Linguagem de regras que descreve padrões (cadeias, bytes, expressões regulares e condições) para identificar arquivos e memória, muito usada em análise de malware. **p. 144**

Z

Zanzibar. Sistema global de autorização do Google (2019) baseado em tuplas de relação; inspirou OpenFGA e SpiceDB. **p. 53**

Zero Trust. Modelo (NIST SP 800-207) em que nenhuma rede ou dispositivo é confiável por localização: todo acesso é decidido por política e verificado. **p. 53**

— GLOSSÁRIO TÉCNICO

CONTINUAÇÃO

Zip bomb e billion laughs. Entradas pequenas que se expandem enormemente: arquivo comprimido que explode ao descompactar (zip bomb) e expansão recursiva de entidades XML (billion laughs). Exigem tetos de saída e de profundidade. **p. 122**

ZTNA (Zero Trust Network Access). Acesso por aplicação, condicionado a identidade e estado do dispositivo, no lugar do acesso à rede inteira da VPN tradicional. **p. 67**

802.1X. Controle de acesso à porta (cabeada ou Wi-Fi) que exige autenticação EAP, validada por um servidor RADIUS, antes de liberar a rede. **p. 67**

#

5 porquês. Técnica de análise de causa que repete a pergunta "por quê?" para ir da falha visível até a causa organizacional. **p. 156**

REFERÊNCIAS

FONTES E LEITURAS

Onde conferir os dados, ler os documentos originais e continuar estudando.

As afirmações, os números e os exemplos deste guia se apoiam em padrões, normas, leis, relatórios e materiais de referência. As fontes abaixo seguem os blocos do guia, e a coluna do meio indica as páginas em que cada uma foi usada. **Normas, estudos e ferramentas mudam**: ao citar um número ou um prazo, abra a fonte e confira a data, o método e as ressalvas.

SE VOCÊ SÓ PUDER LER DEZ

Leituras essenciais

1. **NIST Cybersecurity Framework 2.0**: a linguagem comum para organizar um programa de segurança.
2. **OWASP Top 10:2025**: os riscos mais críticos de aplicações web e como preveni-los.
3. **OWASP ASVS**: requisitos verificáveis para projetar e testar aplicações.
4. **Verizon DBIR**: o retrato anual de como as violações realmente acontecem.
5. **MITRE ATT&CK**: o vocabulário de táticas e técnicas de quem ataca.
6. **NIST SP 800-63-4**: identidade, autenticação e MFA em detalhe.
7. **NIST SP 800-61 Rev. 3**: resposta a incidentes integrada à gestão de risco.
8. **Building Secure and Reliable Systems**: projetar e operar sistemas seguros, de graça.
9. **Cartilha de Segurança para Internet (CERT.br)**: proteção pessoal em português claro.
10. **Lei nº 13.709/2018 (LGPD)**: o texto da lei que rege os dados pessoais no Brasil.

Sobre os links e os dados: Os endereços são os canônicos de cada publicação, escolhidos por serem estáveis. Nem todas as páginas puderam ser abertas diretamente durante a redação; onde uma informação veio de resumo ou de conhecimento de base, o texto usa expressões como "costuma" e "cerca de". O guia reflete o estado das fontes em outubro e novembro de 2026 e é material de estudo: não substitui aconselhamento jurídico nem a leitura da fonte original.

ENTENDER: FUNDAMENTOS, AMEAÇAS, RISCOS E NORMAS

FONTE	USADA EM	LINK
The Protection of Information in Computer Systems · IEEE Proceedings · 1975 (Saltzer e Schroeder)	p. 5	doi.org/10.1109/PROC.1975.9939
NIST CSRC Glossary · NIST · contínuo	p. 5	csrc.nist.gov/glossary
Security Engineering (3ª ed.) · Ross Anderson · 2020	p. 5	cl.cam.ac.uk/~rja14/book.html
Building Secure and Reliable Systems · Google · 2020	p. 5 · p. 189	sre.google/books/building-secure-reliable-systems
Cartilha de Segurança para Internet · CERT.br/NIC.br · contínuo	p. 5 · p. 11 · p. 180	cartilha.cert.br
MITRE ATT&CK · MITRE · contínuo (v20, out/2026)	p. 11 · p. 144	attack.mitre.org
Data Breach Investigations Report (DBIR) · Verizon · 2026	p. 11 · p. 180	verizon.com/business/resources/reports/dbir
Cost of a Data Breach Report · IBM · 2026	p. 11	ibm.com/reports/data-breach

FONTES E LEITURAS

CONTINUAÇÃO

FONTE	USADA EM	LINK
#StopRansomware · CISA · contínuo	p. 11 · p. 59 · p. 156	cisa.gov/stopransomware
Alerta AA21-131A: DarkSide Ransomware · CISA/FBI · 2021	p. 11 · p. 156	cisa.gov/news-events/cybersecurity-advisories/aa21-131a
RFC 2827 (BCP 38): Network Ingress Filtering · IETF · 2000	p. 11	rfc-editor.org/rfc/rfc2827
NIST SP 800-30 Rev. 1: Guide for Conducting Risk Assessments · NIST · 2012	p. 20	csrc.nist.gov/pubs/sp/800/30/r1/final
NIST SP 800-37 Rev. 2: Risk Management Framework · NIST · 2018	p. 20	csrc.nist.gov/pubs/sp/800/37/r2/final
Common Vulnerability Scoring System (CVSS) · FIRST · v3.1 e v4.0	p. 20	first.org/cvss
Exploit Prediction Scoring System (EPSS) · FIRST · contínuo	p. 20	first.org/epss
Known Exploited Vulnerabilities Catalog · CISA · contínuo	p. 20	cisa.gov/known-exploited-vulnerabilities-catalog
CVE Program · MITRE/CISA · contínuo	p. 20	cve.org
National Vulnerability Database · NIST · contínuo	p. 20	nvd.nist.gov
Common Weakness Enumeration (CWE) · MITRE · contínuo	p. 20	cwe.mitre.org
Threat Modeling Manifesto · Comunidade · 2020	p. 20 · p. 189	threatmodelingmanifesto.org
OWASP Threat Dragon · OWASP · contínuo	p. 20	owasp.org/www-project-threat-dragon
ISO/IEC 27001:2022 · ISO · 2022	p. 27	iso.org/standard/27001
NIST Cybersecurity Framework (CSF) 2.0 · NIST · 2024	p. 27 · p. 156 · p. 189	nist.gov/cyberframework
NIST SP 800-53 Rev. 5 · NIST · 2020 (atualização 1)	p. 27	csrc.nist.gov/pubs/sp/800/53/r5/upd1/final
CIS Critical Security Controls v8.1 · CIS · 2024	p. 27	cisecurity.org/controls
CIS Benchmarks · CIS · contínuo	p. 27 · p. 78	cisecurity.org/cis-benchmarks
PCI Security Standards Council · PCI SSC · v4.0.1	p. 27	pcisecuritystandards.org
OWASP SAMM · OWASP · v2	p. 27 · p. 90 · p. 189	owaspsamm.org
Lei nº 13.709/2018 (LGPD) · Presidência da República · 2018	p. 27 · p. 59	planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm
Autoridade Nacional de Proteção de Dados · ANPD · contínuo	p. 27 · p. 59 · p. 180	gov.br/anpd
Lei nº 12.965/2014 (Marco Civil da Internet) · Presidência da República · 2014	p. 27	planalto.gov.br/ccivil_03/_ato2011-2014/2014/lei/l12965.htm
Lei nº 12.737/2012 · Presidência da República · 2012	p. 27 · p. 166	planalto.gov.br/ccivil_03/_ato2011-2014/2012/lei/l12737.htm
Lei nº 14.155/2021 · Presidência da República · 2021	p. 27	planalto.gov.br/ccivil_03/_ato2019-2022/2021/lei/L14155.htm
CERT.br · NIC.br · contínuo	p. 27	cert.br

PROTEGER: CRIPTOGRAFIA, IDENTIDADE, ACESSO E DADOS

FONTE	USADA EM	LINK
NIST SP 800-57 Parte 1 Rev. 5: Recommendation for Key Management · NIST · 2020	p. 34	csrc.nist.gov/pubs/sp/800/57/pt1/r5/final
RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3 · IETF · 2018	p. 34	rfc-editor.org/rfc/rfc8446

— FONTES E LEITURAS

CONTINUAÇÃO

FONTE	USADA EM	LINK
RFC 9325: Recommendations for Secure Use of TLS and DTLS · IETF · 2022	p. 34	rfc-editor.org/rfc/rfc9325
RFC 5280: Internet X.509 Public Key Infrastructure Certificate and CRL Profile · IETF · 2008	p. 34	rfc-editor.org/rfc/rfc5280
RFC 8555: Automatic Certificate Management Environment (ACME) · IETF · 2019	p. 34	rfc-editor.org/rfc/rfc8555
RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF) · IETF · 2010	p. 34	rfc-editor.org/rfc/rfc5869
RFC 8439: ChaCha20 and Poly1305 for IETF Protocols · IETF · 2018	p. 34	rfc-editor.org/rfc/rfc8439
NIST SP 800-38D: Recommendation for GCM and GMAC · NIST · 2007	p. 34	csrc.nist.gov/pubs/sp/800/38/d/final
Cryptographic Storage Cheat Sheet · OWASP · atual	p. 34 · p. 59	cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Chea…
SSL Configuration Generator · Mozilla · atual	p. 34	ssl-config.mozilla.org
Post-Quantum Cryptography (projeto) · NIST · atual	p. 34	csrc.nist.gov/projects/post-quantum-cryptography
CA/Browser Forum: Baseline Requirements e ballots · CA/Browser Forum · 2025	p. 34	cabforum.org
Cryptopals Crypto Challenges · Matasano/Cryptopals · atual	p. 34	cryptopals.com
NIST SP 800-63-4: Digital Identity Guidelines · NIST · 2025	p. 44 · p. 180	pages.nist.gov/800-63-4
Password Storage Cheat Sheet · OWASP · atual	p. 44	cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_She…
Authentication Cheat Sheet · OWASP · atual	p. 44	cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet…
Session Management Cheat Sheet · OWASP · atual	p. 44	cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_S…
RFC 9106: Argon2 Memory-Hard Function for Password Hashing · IETF · 2021	p. 44	rfc-editor.org/rfc/rfc9106
RFC 6238: TOTP: Time-Based One-Time Password Algorithm · IETF · 2011	p. 44	rfc-editor.org/rfc/rfc6238
RFC 7636: Proof Key for Code Exchange (PKCE) · IETF · 2015	p. 44	rfc-editor.org/rfc/rfc7636
RFC 9700: Best Current Practice for OAuth 2.0 Security · IETF · 2025	p. 44	rfc-editor.org/rfc/rfc9700
RFC 7519: JSON Web Token (JWT) · IETF · 2015	p. 44	rfc-editor.org/rfc/rfc7519
RFC 8725: JSON Web Token Best Current Practices · IETF · 2020	p. 44	rfc-editor.org/rfc/rfc8725
Passkeys · FIDO Alliance · atual	p. 44 · p. 180	fidoalliance.org/passkeys
Web Authentication (WebAuthn) Level 3 · W3C · atual	p. 44	w3.org/TR/webauthn-3
Pwned Passwords API (k-anonymity) · Have I Been Pwned · atual	p. 44	haveibeenpwned.com/API/v3#PwnedPasswords
OpenID Connect Core 1.0 · OpenID Foundation · 2014	p. 44	openid.net/specs/openid-connect-core-1_0.html
SPIFFE: Secure Production Identity Framework for Everyone · CNCF · atual	p. 44	spiffe.io
NIST SP 800-162: Guide to Attribute Based Access Control (ABAC) · NIST · 2014	p. 53	csrc.nist.gov/pubs/sp/800/162/final
NIST SP 800-207: Zero Trust Architecture · NIST · 2020	p. 53 · p. 67	csrc.nist.gov/pubs/sp/800/207/final
Authorization Cheat Sheet · OWASP · atual	p. 53	cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.…

— FONTES E LEITURAS

CONTINUAÇÃO

FONTE	USADA EM	LINK
OWASP Top 10:2025 · OWASP · 2025	p. 53 · p. 98 · p. 135 · p. 144	top10.owasp.org/2025
OWASP API Security Top 10 (2023) · OWASP · 2023	p. 53 · p. 112	owasp.org/API-Security
OWASP ASVS (capítulo de autorização) · OWASP · 2025	p. 53 · p. 189	owasp.org/www-project-application-security-verification-standard
CWE-639: Authorization Bypass Through User-Controlled Key · MITRE · atual	p. 53	cwe.mitre.org/data/definitions/639.html
Open Policy Agent: documentação · CNCF · atual	p. 53	openpolicyagent.org/docs/latest
Cedar Policy Language · AWS · atual	p. 53	cedarpolicy.com
OpenFGA: documentação · CNCF · atual	p. 53	openfga.dev/docs
Row Security Policies (PostgreSQL) · PostgreSQL · atual	p. 53	postgresql.org/docs/current/ddl-rowsecurity.html
NIST SP 800-88 Rev. 2: Guidelines for Media Sanitization · NIST · 2025	p. 59	csrc.nist.gov/pubs/sp/800/88/r2/final
How To Break Anonymity of the Netflix Prize Dataset (Narayanan e Shmatikov) · arXiv · 2006	p. 59	arxiv.org/abs/cs/0610105
AOL search log release · Wikipedia · atual	p. 59	en.wikipedia.org/wiki/AOL_search_log_release
The Algorithmic Foundations of Differential Privacy (Dwork e Roth) · Foundations and Trends in TCS · 2014	p. 59	cis.upenn.edu/~aaroth/privacybook.html
Guia Orientativo: Segurança da Informação para Agentes de Tratamento de Pequeno Porte · ANPD · 2021	p. 59	gov.br/anpd/pt-br/centrais-de-conteudo/materiais-educativos-e-pub…

INFRAESTRUTURA: REDES, SISTEMAS, NUVEM E CONTÊINERES

FONTE	USADA EM	LINK
NIST SP 800-41 Rev. 1: Guidelines on Firewalls and Firewall Policy · NIST · 2009	p. 67	csrc.nist.gov/pubs/sp/800/41/r1/final
NIST SP 800-94: Guide to Intrusion Detection and Prevention Systems (IDPS) · NIST · 2007	p. 67	csrc.nist.gov/pubs/sp/800/94/final
NIST SP 800-77 Rev. 1: Guide to IPsec VPNs · NIST · 2020	p. 67	csrc.nist.gov/pubs/sp/800/77/r1/final
RFC 9989: Domain-based Message Authentication, Reporting, and Conformance (DMARC) · IETF · 2026	p. 67	datatracker.ietf.org/doc/rfc9989
Email sender guidelines FAQ · Google · atual	p. 67	support.google.com/a/answer/14229414
Cloudflare 1.1.1.1 incident on June 27, 2024 · Cloudflare · 2024	p. 67	blog.cloudflare.com/cloudflare-1111-incident-on-june-27-2024
MANRS: Mutually Agreed Norms for Routing Security · Internet Society · atual	p. 67	manrs.org
Cloudflare 2025 Q3 DDoS threat report · Cloudflare · 2025	p. 67	blog.cloudflare.com/ddos-threat-report-2025-q3
Cloudflare says 1 Tbps+ DDoS attacks grew sixfold in the second quarter · SiliconANGLE · 2026	p. 67	siliconangle.com/2026/08/11/cloudflare-says-1-tbps-ddos-attacks-g…
What is a DDoS attack? · Cloudflare Learning Center · atual	p. 67	cloudflare.com/learning/ddos/what-is-a-ddos-attack
Suricata Documentation · OISF · atual	p. 67	docs.suricata.io
NIST SP 800-40 Rev. 4: Guide to Enterprise Patch Management Planning · NIST · 2022	p. 78	csrc.nist.gov/pubs/sp/800/40/r4/final
NIST SP 800-190: Application Container Security Guide · NIST · 2017	p. 78	csrc.nist.gov/pubs/sp/800/190/final

— FONTES E LEITURAS

CONTINUAÇÃO

FONTE	USADA EM	LINK
Pod Security Standards · Kubernetes · atual	p. 78	kubernetes.io/docs/concepts/security/pod-security-standards
Cloud Native Security (4C) · Kubernetes · atual	p. 78	kubernetes.io/docs/concepts/security/cloud-native-security
Secrets · Kubernetes · atual	p. 78	kubernetes.io/docs/concepts/configuration/secret
Docker Security Cheat Sheet · OWASP · atual	p. 78	cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Shee…
Shared Responsibility Model · AWS · atual	p. 78	aws.amazon.com/compliance/shared-responsibility-model
Configure the Instance Metadata Service (IMDSv2) · AWS Documentation · atual	p. 78	docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-…
Capital One to pay $80M penalty over 2019 data breach · CIO Dive · 2020	p. 78	ciodive.com/news/capital-one-breach-penalty/583063
Microsoft: Faulty CrowdStrike update affected 8.5M devices · TechTarget · 2024	p. 78	techtarget.com/cybersecurity/news/366596532/Microsoft-Faulty-Crow…

DESENVOLVER: CICLO DE VIDA, WEB, APIS, CÓDIGO E CADEIA DE SUPRIMENTOS

FONTE	USADA EM	LINK
NIST SP 800-218: Secure Software Development Framework (SSDF) v1.1 · NIST · 2022	p. 90 · p. 135 · p. 189	csrc.nist.gov/pubs/sp/800/218/final
OWASP Application Security Verification Standard (ASVS) 5.0 · OWASP · 2025	p. 90	asvs.owasp.org
BSIMM · Black Duck · atual	p. 90	bsimm.com
Secure by Design · CISA · 2023	p. 90	cisa.gov/securebydesign
RFC 9116: A File Format to Aid in Security Vulnerability Disclosure · IETF · 2022	p. 90	rfc-editor.org/rfc/rfc9116
OWASP Cheat Sheet Series · OWASP · atual	p. 98 · p. 189	cheatsheetseries.owasp.org
SQL Injection Prevention Cheat Sheet · OWASP · atual	p. 98	cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_C…
Cross Site Scripting Prevention Cheat Sheet · OWASP · atual	p. 98	cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Preve…
Cross-Site Request Forgery Prevention Cheat Sheet · OWASP · atual	p. 98	cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery…
Server Side Request Forgery Prevention Cheat Sheet · OWASP · atual	p. 98	cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forger…
Content Security Policy Cheat Sheet · OWASP · atual	p. 98	cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Ch…
OWASP Web Security Testing Guide (WSTG) · OWASP · atual	p. 98 · p. 166	owasp.org/www-project-web-security-testing-guide
CWE Top 25 Most Dangerous Software Weaknesses (2025) · MITRE · 2025	p. 98 · p. 122	cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html
Segurança na Web (MDN Web Docs) · Mozilla · atual	p. 98	developer.mozilla.org/en-US/docs/Web/Security
Web Security Academy · PortSwigger · atual	p. 98 · p. 166	portswigger.net/web-security
OWASP ZAP · ZAP · atual	p. 98	zaproxy.org
HSTS Preload List · hstspreload.org · atual	p. 98	hstspreload.org
RFC 6265: HTTP State Management Mechanism · IETF · 2011	p. 98	rfc-editor.org/rfc/rfc6265
#StopRansomware: CL0P Ransomware Gang Exploits MOVEit Vulnerability (AA23-158A) · CISA e FBI · 2023	p. 98	cisa.gov/news-events/cybersecurity-advisories/aa23-158a

— FONTES E LEITURAS

CONTINUAÇÃO

FONTE	USADA EM	LINK
REST Security Cheat Sheet · OWASP · atual	p. 112	cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.…
GraphQL Cheat Sheet · OWASP · atual	p. 112	cheatsheetseries.owasp.org/cheatsheets/GraphQL_Cheat_Sheet.html
OWASP Mobile Application Security (MASVS e MASTG) · OWASP · atual	p. 112	mas.owasp.org
OWASP MASVS · OWASP · v2	p. 112	mas.owasp.org/MASVS
OWASP Mobile Top 10 · OWASP · 2024	p. 112	owasp.org/www-project-mobile-top-10
OWASP Top 10 for LLM Applications 2025 · OWASP GenAI Security Project · 2025	p. 112	genai.owasp.org/llm-top-10
NIST AI Risk Management Framework · NIST · 2023	p. 112	nist.gov/itl/ai-risk-management-framework
NIST AI 100-2 E2025: Adversarial Machine Learning · NIST · 2025	p. 112	csrc.nist.gov/news/2025/nist-ai-100-2-adversarial-machine-learnin…
MITRE ATLAS · MITRE · atual	p. 112	atlas.mitre.org
Secure AI Framework (SAIF) · Google · atual	p. 112	saif.google
The lethal trifecta for AI agents · Simon Willison · 2025	p. 112	simonwillison.net/2025/Jun/16/the-lethal-trifecta
Play Integrity API · Google · atual	p. 112	developer.android.com/google/play/integrity
SEI CERT C Coding Standard · SEI/Carnegie Mellon · atual	p. 122	wiki.sei.cmu.edu/confluence/display/c/SEI+CERT+C+Coding+Standard
OWASP Secure Coding Practices: Quick Reference Guide · OWASP · atual	p. 122	owasp.org/www-project-secure-coding-practices-quick-reference-gui…
Input Validation Cheat Sheet · OWASP · atual	p. 122	cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_She…
Deserialization Cheat Sheet · OWASP · atual	p. 122	cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Shee…
Logging Cheat Sheet · OWASP · atual	p. 122 · p. 144	cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
AddressSanitizer · LLVM/Clang · atual	p. 122	clang.llvm.org/docs/AddressSanitizer.html
libFuzzer: a library for coverage-guided fuzz testing · LLVM · atual	p. 122	llvm.org/docs/LibFuzzer.html
Compiler Options Hardening Guide for C and C++ · OpenSSF · atual	p. 122	best.openssf.org/Compiler-Hardening-Guides/Compiler-Options-Harde…
Memory Safe Languages: Reducing Vulnerabilities in Modern Software Development · NSA e CISA · jun/2025	p. 122	nsa.gov/Press-Room/Press-Releases-Statements/Press-Release-View/A…
Product Security Bad Practices · CISA e FBI · out/2024	p. 122	ic3.gov/CSA/2024/241016-2.pdf
Eliminating Memory Safety Vulnerabilities at the Source · Google Security Blog · 2024	p. 122	security.googleblog.com/2024/09/eliminating-memory-safety-vulnera…
The Rust Programming Language: Understanding Ownership · Rust Project · atual	p. 122	doc.rust-lang.org/book/ch04-00-understanding-ownership.html
C++ Core Guidelines · Stroustrup e Sutter · atual	p. 122	isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
Details of the Cloudflare outage on July 2, 2019 · Cloudflare · 2019	p. 122	blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-20…
pickle: Python object serialization · Python Software Foundation · atual	p. 122	docs.python.org/3/library/pickle.html
secrets: generate secure random numbers · Python Software Foundation · atual	p. 122	docs.python.org/3/library/secrets.html
Heartbleed Bug · Codenomicon · 2014	p. 122	heartbleed.com

FONTES E LEITURAS

CONTINUAÇÃO

FONTE	USADA EM	LINK
SLSA: Supply-chain Levels for Software Artifacts · OpenSSF · v1.2	p. 135	slsa.dev
SLSA v1.2: Build track basics · OpenSSF · v1.2	p. 135	slsa.dev/spec/v1.2/build-track-basics
Sigstore · OpenSSF e Linux Foundation · atual	p. 135	sigstore.dev
OpenSSF Scorecard · OpenSSF · atual	p. 135	scorecard.dev
Widespread Supply Chain Compromise Impacting npm Ecosystem · CISA · set/2025	p. 135	cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-co…
Supply Chain Compromise of Third-Party GitHub Action (CVE-2025-30066) · CISA · mar/2025	p. 135	cisa.gov/news-events/alerts/2025/03/18/supply-chain-compromise-th…
SPDX · Linux Foundation · atual	p. 135	spdx.dev
CycloneDX · OWASP · atual	p. 135	cyclonedx.org
Software Bill of Materials (SBOM) · CISA · atual	p. 135	cisa.gov/sbom
OSV: Open Source Vulnerabilities · Google e OpenSSF · atual	p. 135	osv.dev
in-toto · CNCF · atual	p. 135	in-toto.io
Reproducible Builds · Projeto Reproducible Builds · atual	p. 135	reproducible-builds.org
CVE-2024-3094 (xz-utils) · NIST NVD · 2024	p. 135	nvd.nist.gov/vuln/detail/CVE-2024-3094
CVE-2021-44228 (Log4Shell) · NIST NVD · 2021	p. 135	nvd.nist.gov/vuln/detail/CVE-2021-44228
CI/CD Security Cheat Sheet · OWASP · atual	p. 135	cheatsheetseries.owasp.org/cheatsheets/CI_CD_Security_Cheat_Sheet…
Gitleaks · Gitleaks · atual	p. 135	github.com/gitleaks/gitleaks

OPERAR: DETECÇÃO, RESPOSTA E TESTES DE SEGURANÇA

FONTE	USADA EM	LINK
NIST SP 800-92: Guide to Computer Security Log Management · NIST, 2006	p. 144	csrc.nist.gov/pubs/sp/800/92/final
Sigma: formato genérico de regras de detecção · SigmaHQ	p. 144	sigmahq.io
YARA · VirusTotal	p. 144	virustotal.github.io/yara
The Pyramid of Pain · David Bianco, 2013	p. 144	detect-respond.blogspot.com/2013/03/the-pyramid-of-pain.html
Traffic Light Protocol (TLP) 2.0 · FIRST	p. 144	first.org/tlp
MISP Project · MISP	p. 144	misp-project.org
STIX e TAXII: documentação CTI · OASIS Open	p. 144	oasis-open.github.io/cti-documentation
CAIS/RNP: Centro de Atendimento a Incidentes de Segurança · RNP	p. 144	rnp.br/en/node/7890
Staff report: Target Missed Opportunities (Senado dos EUA) · U.S. Senate Commerce Committee, 2014	p. 144 · p. 156	commerce.senate.gov/2014/3/rockefeller-staff-report-details-targe…
NIST SP 800-61 Rev. 3: Incident Response Recommendations and Considerations for Cybersecurity Risk Management · NIST, abr. 2025	p. 156	csrc.nist.gov/pubs/sp/800/61/r3/final
RFC 3227: Guidelines for Evidence Collection and Archiving · IETF, 2002	p. 156	rfc-editor.org/rfc/rfc3227
Channel File 291 Incident Root Cause Analysis · CrowdStrike, ago. 2024	p. 156	crowdstrike.com/wp-content/uploads/2024/08/Channel-File-291-Incid…

— FONTES E LEITURAS

CONTINUAÇÃO

FONTE	USADA EM	LINK
Helping our customers through the CrowdStrike outage · Microsoft, jul. 2024	p. 156	news.microsoft.com/source/asia/2024/07/21/helping-our-customers-t…
Postmortem Culture: Learning from Failure · Google SRE Book	p. 156	sre.google/sre-book/postmortem-culture
ANPD: Autoridade Nacional de Proteção de Dados · ANPD	p. 156	gov.br/anpd/pt-br
NIST SP 800-115: Technical Guide to Information Security Testing and Assessment · NIST · 2008	p. 166	csrc.nist.gov/pubs/sp/800/115/final
PTES: Penetration Testing Execution Standard · Comunidade PTES	p. 166	pentest-standard.org
CERT Guide to Coordinated Vulnerability Disclosure · CERT/CC, Carnegie Mellon SEI	p. 166	certcc.github.io/CERT-Guide-to-CVD
OWASP Juice Shop · OWASP	p. 166	owasp.org/www-project-juice-shop
Atomic Red Team · Red Canary	p. 166	github.com/redcanaryco/atomic-red-team

PRATICAR: PESSOAS, CULTURA E PROJETO-GUIA

FONTE	USADA EM	LINK
Understanding the Efficacy of Phishing Training in Practice · UC San Diego · IEEE S&P 2025	p. 180	sysnet.ucsd.edu/~voelker/pubs/phishtrain-oakland25.pdf
Secure Our World · CISA · atual	p. 180	cisa.gov/secure-our-world
MED 2.0 do Pix (apresentação) · Banco Central do Brasil · 2025	p. 180	bcb.gov.br/content/estabilidadefinanceira/pix/MED/MED_2-0_Circuit…
Registrato · Banco Central do Brasil · atual	p. 180	registrato.bcb.gov.br
SaferNet Brasil · SaferNet · atual	p. 180	new.safernet.org.br

BONS ESTUDOS!



ESTUDAR · PRATICAR · PROTEGER · EVOLUIR

Segurança não é um produto: é um processo. Quem entende as ameaças, projeta com cuidado e verifica sempre constrói sistemas que merecem confiança. Continue estudando e praticando — de forma ética e autorizada.